



PhD thesis

On Private Continual Counting

Designing efficient correlated noise mechanisms via matrix factorization

Joel Daniel Andersson

Advisor: Rasmus Pagh

November 20, 2025

Abstract

This thesis studies the problem of maintaining a sum over a bitstream under *differential privacy* (DP), called *DP continual counting*. Informally, a randomized algorithm is DP if the output distribution is robust to flipping a bit in the input. Any algorithm for DP continual counting necessarily incurs an error. The algorithmic question becomes *how small an error can we achieve, while still guaranteeing DP?* Motivated by practical use cases in e.g., private machine learning, we are also interested in algorithms with small resource footprint.

Almost all practical algorithms for DP continual counting are based on adding correlated noise, sampled independently of the input, to the exact counts. The task of choosing a noise distribution can be expressed as factoring a lower-triangular matrix of all-ones. This thesis investigates DP continual counting primarily from this lens: identifying factorizations with low error that also admit efficient computation. Our main contributions are:

- Efficient constant-factor improvements for approximate and pure DP. To do this, we leverage unused structure in tree-based factorization, such as non-uniform sensitivity and sub-optimal decoding into prefixes. We also use techniques from linear algebra to perturb the leading-constant optimal square-root factorization into using sublinear memory and time per step, at a small multiplicative blow-up in error.
- Improved trade-offs for DP continual counting with *privacy expiration*. Here the privacy guarantee of an input is allowed to degrade with time. We give an efficient tree-based algorithm with polylogarithmic privacy expiration. By proving a new lower bound in this model, we also show that our expiration/error trade-off is optimal for some parameter ranges.
- Space complexity lower bounds for online convolution. We show lower bounds on the number of real values needed to be stored by any online algorithm exactly computing inner products on a real input stream. When the sequence of inner products defines a lower-triangular Toeplitz matrix, we show that the space complexity is tied to the rationality of a corresponding generating function. In particular, we prove that any implementation of the square-root factorization needs linear space in this model.

Resumé

Denne afhandling studerer *DP-kontinuerlig optælling*, dvs. at vedligeholde en sum af en strøm af bits under såkaldt *differential privacy* (DP). Uformelt sagt er en randomiseret algoritme DP, hvis output-fordelingen er robust over for små ændringer i input. Vores algoritmiske spørgsmål er: *hvor lille en fejl kan vi opnå, samtidig med at vi garanterer DP?* Motiveret af praktiske anvendelser, f.eks. i privat maskinlæring, så kræver vi også algoritmer med lavt ressourceforbrug.

Næsten alle praktiske algoritmer til DP-kontinuerlig optælling er baseret på at tilføje korreleret støj, som er samplet uafhængigt af inputtet, til de eksakte optællinger. Opgaven med at vælge en støjfordeling kan udtrykkes som en faktorisering af en nedre trekantsmatrix bestående udelukkende af ettaller. Denne afhandling undersøger DP-kontinuerlig optælling primært fra dette perspektiv: at identificere faktoriseringer med lav fejl, som samtidig muliggør effektiv beregning. Vores hovedbidrag er:

- Forbedringer i konstante faktorer for både approksimativ og ren DP. For at opnå dette udnytter vi ubrugt struktur i træbaserede faktoriseringer, såsom ikke-uniform følsomhed og suboptimal afkodning til præfikssummer. Vi bruger også teknikker fra lineær algebra til at perturbere kvadratrodsfaktoriseringen med optimal ledende konstant, så den har sublineært plads- og tidsforbrug pr. trin, mod en lille multiplikativ forøgelse af fejlen.
- Forbedrede afvejninger for DP-kontinuerlig optælling med *privacitetsudløb*. Her tillades det, at DP-garantien for et input forringes over tid. Vi giver en effektiv træbaseret algoritme med polylogaritmisk privacitetsudløb. Ved at bevise en ny nedre grænse i denne model viser vi desuden, at vores afvejning mellem udløb og fejl er optimal for visse parametervalg.
- Nedre grænser for pladskompleksitet for online foldning. Vi viser nedre grænser for antallet af relle tal, som enhver online algoritme skal lagre for nøjagtigt at beregne indre produkter på en reel inputstrøm. Når rækken af indre produkter danner en Toeplitz-matrix, viser vi, at pladskompleksiteten er knyttet til rationaliteten af en tilsvarende frembringerfunktion. Specifikt beviser vi, at enhver implementering af kvadratrodsfaktoriseringen kræver lineær plads i denne model.

Acknowledgements

No man is an island; the same goes for PhD degrees. This work has been a collective endeavor, and I am grateful to all who supported me along the way. First and foremost, I am thankful for my supervisor Rasmus. On interviewing for the position, he advertised it to me as an apprenticeship. There is theory involved, but you learn a trade (doing research) by applying yourself to it. For the past three years I have been happily hammering away under his guidance. Thanks Rasmus for taking a chance on an applied physicist, for generously sharing your time and for not only being a great supervisor, but also a great person.

I also owe thanks to Boel Nelson for her encouragement, insight and informal supervision over my PhD. To my co-authors—Monika, Palak, Boel, Rasmus, Lukas, Satchit, Teresa, Sahel, Jalaj and Amir—thank you for your time, ideas and collaboration. To Nikita, thanks for reading an early draft of this thesis and for interesting (and ongoing) research discussions. I also want to thank Graham Cormode, Sofya Raskhodnikova and Mikkel Thorup for serving on my thesis committee.

My PhD has mostly been completed at BARC and the AC section. The ambition and energy that permeates the office, but also its warmth, has been a great influence on me. In particular I direct thanks to the cryptic crossword club and its chief instigator Srikanth. To the BARC running club, and its running-plus-swimming offshoot, for encouraging me to leave my desk. To the usual suspects you encounter on an afternoon weekend visit to the office. To Anne for keeping the place together.

I am also fortunate to have been a member of the Providentia group. Differential privacy is not a huge field, especially not in Europe, and so working in it can easily become a solitary project. Not so in my case. Thanks to all the privacy heads I have shared the Providentia office with: Christian, Boel, Lukas, Hao, Jacob and Sia.

I also want to direct thanks to the wider DP community for its warm welcome. To Adam Smith for hosting me at Boston University in the Spring semester of 2025, and the nice people I got to meet over there. To Thomas Steinke for taking an interest in my work and for related discussions. To the organizers of TPD and the OpenDP community meetings, for creating events where the community can meet in person. To the unsung hero of the anonymous reviewer, for setting aside time to review my work—even when they end up rejecting it.

I am also thankful for the colleagues I have gotten to know and who have supported me over the PhD—the following is a non-exhaustive list. I distinctly recall starting my PhD after an industry stint, and feeling a bit overwhelmed initially. Thank you Ioana for early on giving me the perspective and encouragement that I needed. In a similar vein, thanks to Jonas for not only being good company, but also for acting a competent guide through the obstacle course that is completing a PhD. I also send gratitude to Martin for cheering me on, whether it be from the sidelines or in the race course. More broadly, I am very thankful for all the colleagues who insisted on occasionally leaving the desk for e.g., having a beer. This one goes out especially to Christian, Jonas, Lukas, Duri and Jacob—thanks for the sanity boost.

I thank my family back in Sweden—Karina, Dan-Erik and Alexander—for their unconditional support in all my endeavors, even if they would probably prefer if I pursued my interests closer to Göteborg. A big thanks to all my friends from the other side of the bridge who, in spite of my treacherous crossing over to Copenhagen, stay in touch with me.

Lastly, a shoutout to GPT-5 for producing the figure on the frontpage.

Contents

1	Introduction	1
2	Differential Privacy	5
2.1	Key properties of differential privacy	6
2.2	Privately releasing real queries	6
2.3	Alternative definitions	7
3	Private Continual Counting	9
3.1	Factorization mechanisms	9
3.2	Formalizing error metrics	10
3.3	Efficient factorizations	12
3.4	Brief desiderata for good factorizations	13
3.5	Upper bounds	14
	Pure differential privacy	14
	Approximate differential privacy	17
3.6	Lower bounds	19
	Pure differential privacy	19
	Approximate differential privacy	20
3.7	State of the art	21
4	Overview of contributions	23
4.1	Paper A: A Smooth Binary Mechanism	24
4.2	Paper B: Continual Counting via Binning	25
4.3	Paper C: Count on Your Elders	28
4.4	Paper D: Counting with Privacy Expiration	30
4.5	Paper E: Complexity of Online Convolution	33
A	A Smooth Binary Mechanism	37
A.1	Introduction	38
	Our contributions	38
	Related work	40
A.2	Preliminaries	41
	Continual observation of bit stream	41
	Differential privacy	42

Differentially private continual counting	43
A.3 Our mechanism	44
Closer analysis of the binary mechanism	44
A smooth binary mechanism	45
A.4 Comparison of mechanisms	50
A.5 Conclusion and discussion	52
Appendix	53
B Streaming Continual Counting via Binning	55
B.1 Introduction	56
B.2 Introducing the problem	60
Preliminaries	60
Factorization mechanisms.	61
B.3 Approximation by binning	63
Small perturbations preserve our norms of interest	63
Binned matrices are space (and time) efficient	67
Perturbing matrices into binned matrices	69
Applications to square-root factorizations for counting	74
B.4 Empirical evaluation	79
Results for the Bennett matrix	79
Results with momentum and weight decay	80
B.5 Conclusion and discussion	83
C Count on Your Elders	85
C.1 Introduction	86
Overview of technical ideas	88
Related work	90
C.2 Preliminaries	91
C.3 Mechanisms	92
Binary Tree Mechanisms	92
k -ary Tree Mechanisms	93
k -ary trees with subtraction	95
C.4 Proving Theorem C.1	96
C.5 Laplace vs Gaussian noise	100
Laplace noise scaled to ℓ_2 sensitivity	101
C.6 Lower bound over random inputs	103
C.7 Discussion	107
Appendix	109
D Counting with Gradual Privacy Expiration	111
D.1 Introduction	112
Our Contributions	114
Technical Overview	116
D.2 Preliminaries	117

CONTENTS

D.3	Warmup	118
	A Simple Algorithm with Linear Privacy Expiration	118
	An Algorithm with Logarithmic Privacy Expiration	119
D.4	Proof of Theorem D.2 and Corollary D.3	121
D.5	Lower Bound on the Privacy Expiration	123
D.6	Conclusion	125
	Appendix	127
E	Space Complexity of Online Convolution	133
E.1	Introduction	134
	The model	134
	Generating functions	136
	Results	138
	Applications	139
E.2	Basic properties	141
	Algebraic decompositions do not help	141
	Algebraic algorithms are linear	141
E.3	A lower bound mechanism	143
E.4	Asymptotic complexity	144
E.5	Finite time horizons	145
	Bibliography	151

Chapter 1

Introduction

Any claim as to the massive scale of data generation is bound to ring both trite and true. We adorn ourselves with microphones, location sensors and health trackers on a daily basis. Our internet search history and shopping habits are continuously logged. Many different entities get access to this data and are able to extract insights from it. This is not necessarily in conflict with our interests. A traffic monitoring service with access to my vehicle location may offer more accurate traffic predictions. Sharing the information that I buy orange juice with my local grocer may enable them to offer more targeted discounts. A health monitoring app may use my data to improve its predictions of e.g., sleep apnea. These are arguably good outcomes.

On the other hand, personal data can be sensitive. I may not wish to share widely that I am driving towards a specialized healthcare clinic. A job seeker might not want to share with a future employer that they have recently been stocking up on diapers. My health insurance premium might go up if the insurance company gets access to my recent health markers. Beyond these clear-cut cases, there is also the issue of auxiliary data sources. A particular data point may seem innocuous today, but future data sets may facilitate linkage attacks. Visiting a certain store on a given day may not seem sensitive, but if it afterwards becomes public knowledge that a crime ring was operating from the store, then that judgment may change.

Informed by this, we would like to release *statistics* on personal datasets, without revealing “too much” information about individual data points. For example, when predicting traffic flows, we should avoid disclosing the exact location of individual cars in the process. In estimating the incidence of heart disease in a population, no individual should have their health record made public. Here, we do not consider *statistical inference* to be a privacy concern. If we can learn that property A is highly prevalent in population B , without compromising information about any individual in B , then this does not constitute a privacy violation on behalf of population B .

Differential privacy (DP) [DMN⁺06] is a framework that addresses the concerns we have raised, while still facilitating the release of statistics on datasets. On an intuitive level, DP can be viewed as a robustness property of algorithms. It requires that an algorithm for computing a statistic cannot have an output distribution that depends too much on any individual data point in the input. Of course, it follows that an algorithm that disregards its input is DP, but not interesting. The interesting algorithmic question for a given statistic is rather:

How accurate can an algorithm be, while still satisfying DP?

A particularly challenging setting is called DP *under continual observation* [DNP⁺10b; CSS11]. Here the input dataset is gradually revealed to us, with a new data record $x_t \in \mathcal{X}$ arriving at time t , and we have to immediately output a statistic on $\{x_1, \dots, x_t\}$. This models scenarios in which data evolves over time, e.g., such as in monitoring problems: How many cars are passing through a particular checkpoint? Has there been a recent surge in page visits? How is the connectivity in a social network graph evolving over time? The challenge derives from having to answer many queries about the dataset, and answering each with only a partial view of the full dataset.

An important problem in this space is referred to as *DP counting under continual observation* [DNP⁺10b; CSS11], or *DP continual counting* for short. Here $x_t \in \{0, 1\}$, and the statistic released at time t is $\sum_{i=1}^t x_i$. In other words, we wish to release prefix sums on a binary input stream. Besides being a fundamental problem, DP continual counting is also an important primitive. It can for example be found in private algorithms for releasing graph statistics [SLM⁺18; FHO21; ELM⁺24; JSW24; RS25], histograms [CLS⁺12; Upa19; CR22; HQY⁺22], and not the least private machine learning [KMS⁺21; DMR⁺22; CDP⁺23; CMR⁺23; DMP⁺24; KL24; KMU⁺25].

Private machine learning. The following application to private machine learning has especially driven recent research into improved algorithms for DP continual counting.¹ On a (very) high level, the translation from continual counting to machine learning proceeds in the following manner. Assume we are training a machine learning model with m dimensions, with weight vector $\theta^{(t)} \in \mathbb{R}^m$ after t steps of gradient descent, using constant learning rate η . To perform a step, we first compute a gradient $g^{(t+1)} \in \mathbb{R}^m$ of the loss of the model defined by $\theta^{(t)}$ (at some sensitive example x_{t+1}). Then we perform the update:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \cdot g^{(t+1)} = \theta^{(0)} - \eta \sum_{i=1}^{t+1} g^{(i)},$$

¹For an in-depth treatment of this connection, see the excellent monograph [PUC⁺25].

where $\theta^{(0)} \in \mathbb{R}^m$ is the initialization of the model weights. The intuition here is that, if we can privately compute prefixes on the gradients, then we have a means of privately training a machine learning model.

State-of-the-art for privately training machine learning models is to take an algorithm for DP continual counting and run m copies of it, one per model dimension. As such, improved privacy-utility trade-offs for this problem allows for injecting less noise into the training process. In large-scale deployments, it is also often the case that the number of gradient descent steps T is much smaller than m [DMP⁺24]. This promotes space and time efficient DP continual counting algorithms, which aligns with the scope of this thesis.

Thesis scope. This thesis studies the problem of differentially private continual counting in depth. It develops algorithms for both *pure* and *approximate* differential privacy,² with the goal of improving error bounds while retaining computational efficiency. It includes lower bounds on the error of general algorithms, and also on the space complexity of certain classes of algorithms. It also studies a relaxation of differential privacy under continual observation, where privacy guarantees are allowed to evolve over time, and derives upper and lower bounds in this framework.

Assumed background. The thesis assumes knowledge of basic linear algebra, such as vector norms, matrix inverses and rank. More advanced concepts such as operator norms are introduced as they are used; see [Str23] for a good general reference on linear algebra. A basic knowledge of algorithm analysis is also assumed, including concepts such as big-O notation and amortized analysis, with [CL22] serving as a possible reference. No background in differential privacy is expected from the reader; see [DR14] for an introduction to DP.

Thesis organization. Chapter 2 introduces the concepts in differential privacy necessary for reading this thesis. A reader already familiar with differential privacy can skip this section, as the concepts and notation are standard. Chapter 3 formally defines differentially private continual counting, introduces the class of mechanisms studied, and summarizes previous (and concurrent) work. Chapter 4 gives an overview of the contributions in this thesis, including a short technical overview for each paper included. The papers that make up the work in this thesis are included as appendices: Paper A–E. As this thesis focuses on theoretical contributions, some empirical evaluation has been excluded from these works. We also note that each work introduces additional notation.

²See Chapter 2 for definitions.

Chapter 2

Differential Privacy

Differential privacy [DMN⁺06] is the *gold standard* for private data analysis. It formalizes, and parametrizes, the intuition that a private algorithm should produce “similar” outputs on “similar” input data sets. To this end, we use D to denote a dataset of records $x_1, \dots, x_n$ where each x_i is taken from a universe $\mathcal{X}$. Differential privacy is a property of an algorithm, which we usually refer to as a *mechanism*. A mechanism is a randomized map $\mathcal{M} : \mathcal{X}^* \rightarrow \mathcal{Y}$, so that a statistical query on a dataset $D \in \mathcal{X}^*$ is $\mathcal{M}(D) \in \mathcal{Y}$.

Towards a formalism, for two datasets $D, D' \in \mathcal{X}^*$ we use the notation $D \simeq D'$ to signify that they are *neighboring*. There are multiple ways of defining a neighboring dataset—for most of this thesis we are going to use the “replace-one-entry” version.

Definition 2.1 (Neighboring dataset). Let $D, D' \in \mathcal{X}^*$ be two datasets. They are *1-neighboring*, denoted $D \simeq D'$, if $|D \setminus D'| = |D' \setminus D| = 1$.

A mechanism is differentially private if $\mathcal{M}(D)$ and $\mathcal{M}(D')$ “look indistinguishable”. Different notions of differential privacy correspond to different approaches for measuring this indistinguishability. We state the most commonly used notion next.

Definition 2.2 (Differential privacy [DMN⁺06]). Let $\mathcal{M} : \mathcal{X}^* \rightarrow \mathcal{Y}$ be a randomized algorithm. For fixed $\varepsilon > 0$ and $\delta \in [0, 1)$, we say that $\mathcal{M}$ is (ε, δ) -*differentially private* if for all $D \simeq D'$ and measurable subsets $S \subset \mathcal{Y}$, we have that

$$\Pr[\mathcal{M}(D) \in S] \leq e^\varepsilon \Pr[\mathcal{M}(D') \in S] + \delta.$$

We additionally refer to $(\varepsilon, 0)$ -DP as ε -DP, and more broadly as *pure* DP. By contrast, (ε, δ) -DP, with $\delta > 0$, is referred to as *approximate* DP. Intuitively, pure DP requires that the probability of any algorithm output differs by no more than a multiplicative constant over neighboring inputs. Approximate DP relaxes this assumption by introducing a slack term—outputs with sufficiently small probability are allowed to violate this multiplicative bound.

2.1 Key properties of differential privacy

Differential privacy has two nice properties that are useful for algorithm design: robustness to *postprocessing* and *composition*. We state them next.

Fact 2.3 (Postprocessing [DR14]). *Let $\mathcal{M} : \mathcal{X} \rightarrow \mathcal{Y}$ be an (ε, δ) -DP algorithm. Let $f : \mathcal{Y} \rightarrow \mathcal{Y}'$ be a randomized mapping. Then $f \circ \mathcal{M} : \mathcal{X} \rightarrow \mathcal{Y}'$ is (ε, δ) -DP.*

This postprocessing property says that once a private release has been made, there exists no procedure, randomized or deterministic, which can break the privacy guarantee. Postprocessing can also be algorithmically useful, as we will see later.

Fact 2.4 (Basic composition [DR14]). *Let $\mathcal{M}_i : \mathcal{X}^* \rightarrow \mathcal{Y}_i$ be an $(\varepsilon_i, \delta_i)$ -DP algorithm for $i \in [k]$. Then if $\mathcal{M}_{[k]} : \mathcal{X}^* \rightarrow \prod_{i=1}^k \mathcal{Y}_i$ is defined to be $\mathcal{M}_{[k]}(x) = (\mathcal{M}_1(x), \dots, \mathcal{M}_k(x))$, then $\mathcal{M}_{[k]}$ is $(\sum_{i=1}^k \varepsilon_i, \sum_{i=1}^k \delta_i)$ -DP.*

Intuitively, basic composition tells us that if we execute a differentially private algorithm multiple times on the same data set, then we get a privacy guarantee for the set of all outputs by “adding up” the corresponding privacy parameters. This composition is tight for pure DP ($\delta = 0$), but for approximate DP ($\delta > 0$), one can do better.

Fact 2.5 (Advanced composition (simplified) [KOV15; Ste22]). *Consider the same setup as in Theorem 2.4 and $(\varepsilon_i, \delta_i) = (\varepsilon, \delta)$ for all $i \in [k]$. Then $\mathcal{M}_{[k]}$ satisfies (ε', δ') -differential privacy for any $\delta' \geq k\delta$ where*

$$\varepsilon' = k\varepsilon^2/2 + \sqrt{2k\varepsilon^2 \ln(1/\delta')}.$$

Composition is important from both an algorithmic and a systems point of view. Algorithmically because it enables a modular approach to algorithm design. We are allowed to combine multiple private algorithms into a single one; we do not have to holistically study such an algorithm. From a systems perspective, it is also unlikely that there will be a single monolithic entity publishing statistics about any given user. More realistically, there will be a mix of entities publishing statistics. As long as each of these entities publishes statistics with differential privacy, we can still derive a formal privacy guarantee for the aggregate release.

2.2 Privately releasing real queries

Throughout this thesis, our main concern will be to release vector-valued functions. To this end, consider the case of releasing the m -dimensional function $f : \mathcal{X}^* \rightarrow \mathbb{R}^m$ with differential privacy. We will use the following two key primitives. They both rely on the ℓ_p sensitivity of the function, defined next.

Definition 2.6 (Sensitivity). For $p \geq 1$, we define the ℓ_p sensitivity of a vector-valued function $f : \mathcal{X}^* \rightarrow \mathbb{R}^m$ as $\Delta_p := \max_{D \simeq D'} \|f(D) - f(D')\|_p$.

Adding appropriately scaled i.i.d. noise to each coordinate of $f(D)$ results in a private mechanism. In particular, one can add Gaussian or Laplace-distributed noise scaled by the ℓ_2 or ℓ_1 sensitivity, respectively. We use the notation $\text{Lap}(b)$ for the zero-mean Laplace distribution with probability density function $f(y) \propto \exp(-|y|/b)$ and variance $2b^2$. Let $\mathcal{N}(\mu, \sigma^2)$ denote a Gaussian distribution with mean μ and variance σ^2 .

Definition 2.7 (Laplace mechanism [DMN⁺06]). Let $\mathcal{M}(D) = f(D) + z$ be the mechanism that adds i.i.d. zero-mean m -dimensional noise to $f(D)$ where $z_i \sim \text{Lap}(\Delta_1(f)/\varepsilon)$ for $i \in [m]$. Then $\mathcal{M}(D)$ satisfies ε -differential privacy.

Definition 2.8 (Gaussian mechanism [DR14]). Let $\varepsilon, \delta \in (0, 1)$. Let $\mathcal{M}(D) = f(D) + z$ be the mechanism that adds i.i.d. zero-mean m -dimensional noise to $f(D)$ where $z_i \sim \mathcal{N}(0, \frac{2 \ln(1.25/\delta)}{\varepsilon^2} \cdot \Delta_2^2)$ for $i \in [m]$. Then $\mathcal{M}(D)$ satisfies (ε, δ) -differential privacy.

We note that the term $\frac{2 \ln(1.25/\delta)}{\varepsilon^2}$ in the variance of the Gaussian mechanism is not tight, in the sense that a smaller constant can be derived for fixed (ε, δ) [BW18].

2.3 Alternative definitions

Other variants of DP also exist, such as zero-concentrated [BS16], Rényi [Mir17], and Gaussian DP [DRS22]. Some of the work in this thesis will use ρ -zero-concentrated DP in particular, but these results could equally well have been stated in terms of (ε, δ) -DP (there is a standard conversion from ρ -zCDP to (ε, δ) -DP). The necessary background is given in the papers in question.

Chapter 3

Private Continual Counting

Differential privacy under continual observation [DNP⁺10b; CSS11] refers to the setting where the sensitive data set evolves over time, and a private release must be performed immediately after each update. Formally, let $D^{(t)} = (x_1, \dots, x_t)$ denote the data set at time t , where $x_t \in \mathcal{X}$ is streamed as input at time t , and we have a bound T on the stream length. We consider two streams $x_1, x_2, \dots, x_T$ and $x'_1, x'_2, \dots, x'_T$ neighboring inputs if there exists a unique time step t' for which $x_{t'} \neq x'_{t'}$, and we assume that we have a bound T on the stream length. Privately releasing $f : \mathcal{X}^* \rightarrow \mathcal{Y}$ under continual observation is the following online problem:

At each time $t \leq T$, output $y_t \approx f(D^{(t)})$

where the distribution of the sequence of outputs $(y_1, \dots, y_T)$ satisfies DP.

There is also the stronger notion of *adaptive* continual observation [DMR⁺22; JRS⁺23]. Here an adversary can select the next input x_{t+1} based on observing the mechanism's outputs $(y_1, \dots, y_t)$ (but *not* the internal randomness of the mechanism). This is an important setting for e.g., machine learning applications where input gradients are derived from previously output private models. All mechanisms discussed in this thesis satisfy this stronger notion.

Private continual counting corresponds to the case where the inputs are bits, $\mathcal{X} = \{0, 1\}$, and we wish to release sums: $f : \{0, 1\}^* \rightarrow \mathbb{Z}$ where $f(D) = \sum_{x \in D} x$. Simply put, it concerns releasing all prefixes of a bit stream. The intuitive privacy challenge here is that, for two neighboring streams differing at time t' , the output will differ for all time steps $t \geq t'$. Any private mechanism for the problem needs to introduce enough randomness into the output to obfuscate this difference.

3.1 Factorization mechanisms

Consider the simpler *batch model* for the problem, where $x_1, \dots, x_T$ are available up-front and all T prefixes are to be output at once. To this end, we use matrix-

vector notation and write $x \in \{0, 1\}^T$ for the input and define the *counting matrix* $A \in \{0, 1\}^{T \times T}$ as the lower-triangular matrix of all-ones. Here, we are consequently interested in designing a DP mechanism $\mathcal{M} : \{0, 1\}^T \rightarrow \mathbb{R}^T$ where $\mathcal{M}(x) \approx Ax$, and two inputs are neighboring, written $x \simeq x'$, if $\|x - x'\|_1 = 1$. Given this formulation of the problem, we can apply the *factorization mechanism* (also known as the *matrix mechanism*) framework [LMH⁺15]. The idea is simple:

1. Factor A into matrices $L \in \mathbb{R}^{T \times m}$, $R \in \mathbb{R}^{m \times T}$ such that $A = LR$.
2. Privately release Rx , using either the Laplace (for ε -DP) or Gaussian mechanism (for (ε, δ) -DP), as $Rx + z$, with corresponding noise $z \in \mathbb{R}^m$.
3. Output $\mathcal{M}(x) = L(Rx + z) = Ax + Lz \in \mathbb{R}^T$ as postprocessing.

Essentially, any factorization of A into a matrix product translates into a mechanism for privately releasing Ax in the batch model. From this point onward, when we write (L, R) without further context, it relates to the factorization mechanism for A where $A = LR$. As the noise is generated independently of the input, it is straightforward to argue that the online mechanism:

$$\text{“On receiving } x_t, \text{ output } \mathcal{M}(x)_t = (Ax)_t + (Lz)_t\text{”}$$

in fact is (*nonadaptively*) DP under continual observation. Using known results [DMR⁺22, Proposition 2.1, Theorem 2.1], we can strengthen the guarantee to being *adaptive*. In summary, we have reduced the search over algorithms for private continual counting, to a search over factorizations of the counting matrix A .

3.2 Formalizing error metrics

Before stating more refined choices of factorizations, we state desiderata for attractive choices of factorizations. Two error metrics used for private continual counting are *mean (expected) squared error* and *maximum (expected) squared error*:¹

$$\begin{aligned} \text{MeanSE}(\mathcal{M}) &= \max_{x \in \{0, 1\}^T} \mathbb{E}[\|\mathcal{M}(x) - Ax\|_2^2] / T, \\ \text{MaxSE}(\mathcal{M}) &= \max_{x \in \{0, 1\}^T} \max_{1 \leq t \leq T} \mathbb{E}[(\mathcal{M}(x) - Ax)_t^2]. \end{aligned}$$

Since factorization mechanisms are unbiased, the corresponding errors can be computed exactly and efficiently for a factorization. In the following, $\|A\|_F$ is the Frobenius norm of a matrix (the ℓ_2 norm taken over all its entries) and $\|A\|_{p \rightarrow q}$ is the induced operator norm from ℓ_p to ℓ_q , defined as

$$\|A\|_{p \rightarrow q} = \max_{v \in \mathbb{R}^T : \|v\|_p = 1} \|Av\|_q.$$

¹The literature sometimes uses the root of these same errors.

Of particular interest are the norms $1 \rightarrow 1$, $1 \rightarrow 2$ and $2 \rightarrow \infty$, which correspond to the *maximum ℓ_1 column norm*, *maximum ℓ_2 column norm*, and the *maximum ℓ_2 row norm* respectively. Overloading the notation such that $\text{MeanSE}(L, R)$ describes the error of (L, R) , we have that:

$$\begin{aligned} \text{MeanSE}(L, R) &= \mathbb{E}[\|Lz\|_2^2]/T = \|L\|_F^2 \cdot \text{Var}[z_1]/T \\ &= C_{\varepsilon, \delta} \cdot \|L\|_F^2 \cdot \Delta_p(R)^2/T \\ &= C_{\varepsilon, \delta} \cdot \|L\|_F^2 \cdot \|R\|_{1 \rightarrow p}^2/T. \end{aligned}$$

The second and third step follows from z being isotropic noise with variance $C_{\varepsilon, \delta} \Delta_p^2$ in each coordinate. For ε -DP, Rx is released with the Laplace mechanism, where $p = 1$ and $C_{\varepsilon, 0} = 2/\varepsilon^2$; for (ε, δ) -DP and the Gaussian mechanism, $p = 2$ and $C_{\varepsilon, \delta} = O(\log(1/\delta)/\varepsilon^2)$.²

Repeating the exercise for the maximum squared error, we get

$$\text{MaxSE}(L, R) = C_{\varepsilon, \delta} \cdot \|L\|_{2 \rightarrow \infty}^2 \|R\|_{1 \rightarrow p}^2.$$

Due to factorization mechanisms being unbiased, $\text{MaxSE}(L, R)$ and $\text{MeanSE}(L, R)$ can be thought of as the *maximum* and *average* variance per time step. It immediately follows that $\text{MeanSE}(L, R) \leq \text{MaxSE}(L, R)$.

There is also a third error class, that while not admitting an exact closed form for factorization mechanisms, is still of interest: *ℓ_∞ error with constant probability* (taken over the randomness of $\mathcal{M}$):

$$\text{MaxErr}(\mathcal{M}) = \inf \{ \alpha > 0 \mid \forall x \in \{0, 1\}^T : \|\mathcal{M}(x) - Ax\|_\infty \leq \alpha \text{ w.p. } \geq 2/3 \}.$$

The appeal of $\text{MaxErr}(\mathcal{M})$ is that its promised guarantee is (arguably) more easily conveyed. With probability at least $2/3$, the running count is off by no more than $\text{MaxErr}(\mathcal{M})$ at any given time step.

Employing the same notational shorthand, we can express it as:

$$\text{MaxErr}(L, R) = \inf \{ \alpha > 0 : \|Lz\|_\infty \leq \alpha \text{ w.p. } \geq 2/3 \},$$

since $\mathcal{M}(x) - Ax = Lz$. Here we can not give an exact expression, but for (ε, δ) -DP, we can always bound $\text{MaxErr}(L, R)$ using $\text{MaxSE}(L, R)$. Note that the noise in each coordinate of Lz is Gaussian and has variance at most $\text{MaxSE}(L, R)$. Applying a tail bound, we can say that for any given t , $(Lz)_t$ has error bounded by $O(\sqrt{\text{MaxSE}(L, R) \log(1/\beta)})$ with probability at least $1 - \beta$. Setting $\beta = \frac{2}{3T}$ and applying a union bound over all T steps immediately yields

$$\text{MaxErr}(L, R) = O(\sqrt{\text{MaxSE}(L, R) \log T}).$$

For ε -DP, a similar statement can often be derived for factorization mechanisms, but it instead relies on using concentration of sums of independent Laplace random variables (see [CSS11, Lemma 2.8]).

²We denote the base k logarithm by $\log_k(\cdot)$ and let $\log(\cdot) := \log_2(\cdot)$.

As $C_{\varepsilon,\delta}$ only pertains to the analysis of the Laplace and Gaussian mechanisms, and is independent of the factorization, we will suppress it from now on: parameters are set such that $C_{\varepsilon,\delta} = 1$.

3.3 Efficient factorizations

In addition to having accurate mechanisms, we would also like for them to be computationally *efficient*. Here efficiency means efficiently running a factorization mechanism, rather than efficiently constructing/finding a factorization (L, R) itself.

Our model of computation treats real computation as atomic, meaning that a real number takes up one unit of memory, and we consider addition and multiplication of two real numbers as constant time operations. We also assume that an entry of $z \in \mathbb{R}^m$ (distributed as either Laplace or Gaussian) can be drawn in constant time and with constant space overhead.

Of course, physical computers work with bit representations, not real numbers. Floating point implementations are known to potentially compromise privacy [Mir12], but there are techniques to mitigate such issues [CKS20; KLS21]. Another drawback to working with real numbers is that bit manipulation is not supported. Nevertheless, this is a common computational model (at least implicitly) in works on private continual counting [CSS11; DMR⁺22; DMP⁺24].

We previously noted that the factorization mechanism for private continual counting operates as:

$$\text{“On receiving } x_t, \text{ output } (Ax)_t + (Lz)_t\text{”}$$

but equivalently we could express it as

$$\text{“On receiving } x_t, \text{ output } (A(x + R^{-1}z))_t\text{”}$$

Distributionally they are the same as $Ax + Lz = A(x + R^{-1}z)$, where the latter implicitly adds noise to the input being counted over.

As computing A applied to any real vector can be performed in constant time and space per step, the challenge can be reduced to the following question.

Question: *What computational resources, measured in time and space, do we need for computing Lz (or $R^{-1}z$), entry-by-entry?*

All papers in this thesis will consider space and time usage of Lz rather than $R^{-1}z$, but we note that the latter is always an option (most factorizations are not optimized towards making only $R^{-1}z$ efficient, but some are [KMU⁺25]).

We use the following definition, adapted from [AP25], for expressing the space usage of a factorization mechanism.

Definition 3.1 (Space complexity). Let $L \in \mathbb{R}^{T \times m}$, $z \in \mathbb{R}^m$ and consider the problem of outputting $(Lz)_1, \dots, (Lz)_T$ in sequence. An algorithm is defined

by a state update function that maps the step number t , current state and a subset of entries in z to a new state and an output $(Lz)_t$. If each entry of z is accessed at most once, we call the algorithm *one-pass*. We consider linear algorithms where the state is a vector in $\mathbb{R}^s$, and the update function at each step is linear in z and the state. The *space complexity* of L is the smallest s for which there exists a one-pass algorithm for outputting Lz for arbitrary z .

A *streaming algorithm* is a special case of a one-pass algorithm (only defined for lower-triangular L) where at step number t it reads z_t , then immediately outputs $(Lz)_t$. The restriction on the algorithm being *one-pass* is needed to simulate that any noise term z_i is generated *once* and then has to be stored in memory if it is to be used for future outputs. Without such a restriction, the space complexity would be constant for any matrix L (store the output in a single register and read z on-demand). Also note that this notion of space complexity does not track the memory cost of storing L itself. This is motivated by applications in which the same matrix L can be re-used over many parallel problems (e.g., private machine learning), and also by the fact that many matrices L used in practice have a succinct representation (e.g., Toeplitz, tree structure). In summary, our definition tracks *how much information about z does one need to store, to output Lz ?*

For time complexity, our definition is more standard.

Definition 3.2 (Time complexity). Let $L \in \mathbb{R}^{T \times m}$, $z \in \mathbb{R}^m$ and consider the problem of outputting $(Lz)_1, \dots, (Lz)_T$ in sequence. We consider linear algorithms in z , where we denote by μ the sum of the number of additions and multiplications the algorithm performs over all T outputs. The *time complexity* of L is the smallest μ for which there exists a (linear) algorithm for outputting Lz for arbitrary z .

Common across both definitions is that we restrict ourselves to algorithms linear in z for computing Lz . This is natural in the sense that the objective is linear, but of course also restrictive. In practice, all algorithms in past work are linear. Later in Paper E, we show that for lower-triangular matrices, strengthening the model to allow for arbitrary continuous computation buys you nothing in terms of space complexity. We also show that strengthening the model to non-continuous update maps in z renders the problem trivial: any lower-triangular L can be solved in constant space. We emphasize however that this last construction relies on space-filling curves, and is more of theoretical than practical interest.

3.4 Brief desiderata for good factorizations

In short, we are investigating factorizations (L, R) such that $LR = A$ where A is the lower-triangular matrix of all-ones. We are interested in minimizing the

errors:

$$\text{MaxSE}(L, R) = \|L\|_{2 \rightarrow \infty}^2 \|R\|_{1 \rightarrow p}^2, \quad \text{MeanSE}(L, R) = \|L\|_F^2 \|R\|_{1 \rightarrow p}^2 / T,$$

where $p = 1$ for ε -DP and $p = 2$ for (ε, δ) -DP. The ℓ_∞ error with constant probability, $\text{MaxErr}(L, R)$, is also of interest, but a low such error is implied by low $\text{MaxSE}(L, R)$. Simultaneously, we want the structure of $L \in \mathbb{R}^{T \times m}$ to admit an *efficient, linear* algorithm for computing Lz for any $z \in \mathbb{R}^m$. Here an efficient algorithm means:

1. It uses few additions and multiplications.
2. It reads z in one-pass, storing a minimal number of real values internally.

3.5 Upper bounds

It is worth emphasizing that all known mechanisms with good utility for private continual counting are factorization mechanisms.³ We begin with discussing factorizations designed for ε -DP.

Pure differential privacy

Two naive choices of factorizations readily present themselves: (I, A) (output perturbation) and (A, I) (input perturbation). The first choice suggests that we directly release all prefixes with the Laplace mechanism; the second that we instead release the input stream, and sum up the private (noised) “bits” as postprocessing. We get

$$\begin{aligned} \text{MaxSE}(I, A) &= \|I\|_{2 \rightarrow \infty}^2 \|A\|_1^2 = T^2, \\ \text{MaxSE}(A, I) &= \|A\|_{2 \rightarrow \infty}^2 \|I\|_1^2 = T, \end{aligned}$$

implying that, at least for ε -DP, one can do better than adding independent noise to each output. It turns out that polylogarithmic error is possible.

The first (factorization) mechanism to achieve non-trivial error was the *binary tree mechanism*, typically accredited to Dwork, Naor, Pitassi, and Rothblum [DNP⁺10b] and Chan, Shi, and Song [CSS11], although similar constructions were proposed independently by others [GWX10; HRM⁺10]. The version given here is due to Chan, Shi, and Song, which deviates slightly from [DNP⁺10b]; we will return to the latter variant in Paper D. The main idea is conveyed in Figure 3.1.

³A technique in [DNR⁺15] allows for improved utility on *sparse* inputs. It privately (and online) tries to divide the input into blocks containing multiple 1s, and runs a continual counting mechanism on these blocks. While constructing a block, it keeps outputting the prefix up to the latest completed block. Any factorization mechanism (pure or approximate) can be composed with this technique. As our errors are taken over worst case inputs, this technique does not lead to improved error.

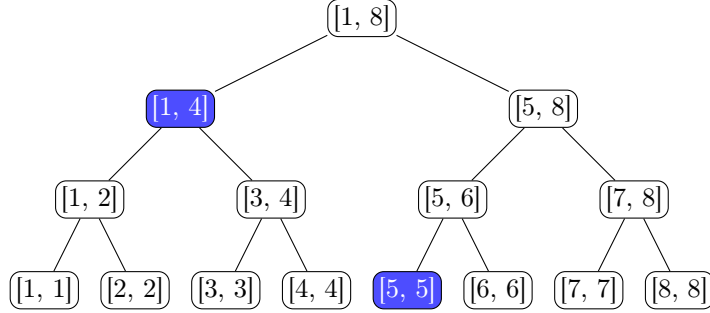


Figure 3.1: Structure used for the binary tree mechanism [CSS11] for $T = 8$. A node $[i, j]$ stores the interval sum $\sum_{k=i}^j x_k$ plus some noise. Blue nodes represent the noisy interval sums added together to form the prefix at time $t = 5$.

For simplicity, assume $T = 2^h$ for some integer $h \geq 1$. The binary tree mechanism uses the dyadic interval decomposition of $[1, T]$ defined as

$$\mathcal{T}_h = \bigcup_{0 \leq \ell \leq h} \mathcal{T}_h^\ell = \bigcup_{0 \leq \ell \leq h} \left\{ [1 + i \cdot 2^\ell, (i+1) \cdot 2^\ell] : i \in \mathbb{N}, 0 \leq i < 2^{h-\ell} \right\},$$

where $|\mathcal{T}_h| = 2T - 1$. It then defines the right factor $R_2 \in \{0, 1\}^{(2T-1) \times T}$, where each row of $R_2 x$ encodes an interval sum on x , as defined from a unique interval $I \in \mathcal{T}_h$. The tree in Figure 3.1 shows all the dyadic interval sums for $T = 8$, and in particular how a prefix up to time $t = 5$ can be constructed by adding together nodes in the released noisy tree (i.e., $R_2 x + z$). More generally, the construction in [CSS11] at time t sums a set of disjoint intervals, the union of which forms $[1, t]$. The construction only uses nodes which are the *left child* of a parent, or the root, which uniquely determines the left factor $L_2 \in \{0, 1\}^{T \times (2T-1)}$.

We discuss the binary tree mechanism again in the overview of Paper A. Here we settle for the following observations:

1. The number of nodes summed at time t to produce a prefix equals the number of 1s in the binary encoding of t .
2. Any given node in the tree is only used for an interval $\subseteq [1, T]$ of time steps.

With these observations in hand, we can establish that

$$\text{MaxSE}(L_2, R_2) = \|L_2\|_{2 \rightarrow \infty}^2 \|R_2\|_{1 \rightarrow 1}^2 = h(h+1)^2 = O(\log^3 T).$$

Here $\|L_2\|_{2 \rightarrow \infty} = \sqrt{h}$ due to one node per level in the tree (barring the root) being used for $t = 2^h - 1$; $\|R_2\|_{1 \rightarrow 1} = h + 1$ due to each column of R_2 having a 1 for each time the corresponding index participates in an interval. By analyzing

the average ℓ_1 weight of a binary counter, it can be shown that $\text{MeanSE}(L_2, R_2)$ is asymptotically a factor 2 smaller.

For efficiency, $L_2 z$ can be implemented with space h , since at most one node per level is used at any time, and once an interval stops being used it can be erased from memory. By a similar argument, the matrix-vector product $L_2 z$ can also be computed in time $O(T)$ (and amortized $O(1)$ time per step), by using a lazy computation: any node used is added and removed at most once over the execution over T steps.

Improvements over the binary tree mechanism. For ε -DP, there are essentially two classes of improved factorizations. One strand of improvement centers on identifying that the noisy binary tree $R_2 x + z$ stores redundant information. The left factor matrix L_2 chosen in [CSS11] never uses right children, and so misses out on information from roughly half the nodes in the tree—information that could be used to improve the error. Honaker [Hon15] identified that the noise of nodes could be (non-uniformly across levels) reduced by constants if the redundant information in the tree is used. The same observation was also made earlier in the range query literature [HRM⁺10]. For factorization mechanisms more broadly, Li, Miklau, Hay, McGregor, and Rastogi [LMH⁺15] identified that, once you fix the query matrix A and the right factor R , then the choice of left factor L minimizing $\text{MeanSE}(L, R)$ is exactly $L^* = AR^\dagger$, where “ $\dagger$ ” denotes the Moore-Penrose pseudoinverse.

While an appealing choice, choosing $L_2^* = AR_2^\dagger$ for the binary tree encoding leads to a dense matrix with no known efficient implementation. However, a subset of the techniques in [Hon15] can be employed to the binary tree mechanism to reduce the variance per node in the tree by up to a factor 2 while retaining the efficiency of the binary tree mechanism, as done by Kairouz, McMahan, Song, Thakkar, Thakurta, and Xu [KMS⁺21] and referred to as “Honaker Online”. That mechanism was designed for (ε, δ) -DP, but the idea extends to ε -DP.

A separate strand of research has considered greater branching factors in the tree, primarily in the range query literature [QYL13; CKS19; CR22]. One can generalize from a binary tree mechanism (L_2, R_2) to a k -ary tree mechanism (L_k, R_k) by instead at each step summing $\leq k - 1$ nodes per step (ignoring the right-most child on each level). Treating k as a constant, one can recover the same asymptotic time and space complexity as for the binary tree. Optimizing over k , both the mean and maximum squared error is minimized for $k = 17$, and yields roughly a factor 4 improvement over the binary tree mechanism. In Paper C we show that one can reduce the error by roughly another factor 2, at essentially no cost in efficiency, by choosing a different left factor $\tilde{L}_k$.

Lastly we comment on a natural question: why not optimize directly over (L, R) to minimize the error? For the mean expected squared error, the optimization problem can be posed as a semidefinite program with a rank

constraint [LMH⁺15, Theorem 3]. While the program might be solvable in polynomial time, they argue that known methods require time $\Omega(T^8)$. Follow-up work has improved the scalability by restricting the search space [MMH⁺23], but necessarily without guarantees of finding a global optimum. The hardness of the problem is unique to ε -DP and owes to its dependence on the ℓ_1 sensitivity. The corresponding optimization problem for (ε, δ) -DP with Gaussian noise is convex and can be efficiently solved in practice for $T \leq 2048$, as shown by Denisov, McMahan, Rush, Smith, and Guha Thakurta [DMR⁺22]. As a last point, we note that even if we identify an optimal factorization for ε -DP via direct optimization, we have no reason to believe that the resulting factorization would be efficient with respect to either time or space.

Approximate differential privacy

For (ε, δ) -DP and factorizations based on Gaussian noise, many results carry over. The only change is that the sensitivity is evaluated with respect to the ℓ_2 norm rather than ℓ_1 . Since a low ℓ_1 sensitivity implies a low ℓ_2 sensitivity, any “decent” factorization for ε -DP will translate into a “decent” factorization for (ε, δ) -DP. As an example, the binary tree mechanism (L_2, R_2) for approximate DP attains an error of $O(\log^2 T)$ instead of $O(\log^3 T)$, due to $\|R_2\|_{1 \rightarrow 2} = \sqrt{\|R_2\|_{1 \rightarrow 1}} = O(\sqrt{\log T})$. More generally, we can observe that factorizations for ε -DP will promote *sparsity* in the right factor matrix, whereas the switch to ℓ_2 norm for (ε, δ) -DP opens up the possibility of using a *dense* right factor. Indeed, the optimal factorizations derived from optimizing over $\text{MeanSE}(L, R)$ are in fact dense [DMR⁺22].

The first explicit dense construction with formal error guarantees was the *square-root factorization* due to Fichtenberger, Henzinger, and Upadhyay [FHU23] and Henzinger, Upadhyay, and Upadhyay [HUU23]. The same factorization was first studied outside of the DP literature by Bennett [Ben77]. Here the factors are chosen to be equal: $L = R = \sqrt{A} \in \mathbb{R}^{T \times T}$ where

$$\sqrt{A} = \begin{bmatrix} a_0 & 0 & 0 & \cdots & 0 \\ a_1 & a_0 & 0 & \cdots & 0 \\ a_2 & a_1 & a_0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{T-1} & a_{T-2} & a_{T-3} & \cdots & a_0 \end{bmatrix}, \quad a_k = \left| \binom{-1/2}{k} \right| = \frac{\binom{2k}{k}}{4^k} = \Theta\left(\frac{1}{\sqrt{k}}\right)$$

with $a_0 = 1$. It satisfies the following error bound [DMP⁺24]:

$$\max \left\{ \text{MaxSE}(\sqrt{A}, \sqrt{A}), \text{MeanSE}(\sqrt{A}, \sqrt{A}) \right\} \leq \left(1 + \frac{\gamma}{\pi} + \frac{\ln(T)}{\pi} \right)^2.$$

where $\gamma \leq 0.57722$ is the Euler-Mascheroni constant. The leading constant in the error is a factor $(\pi \log e)^2/2 \approx 10.3$ and $(\pi \log e)^2 \approx 20.5$ smaller than

the binary tree mechanism, for the mean and maximum squared error respectively. This factorization is in fact tight (up to the leading constant) over all factorizations for both maximum and mean squared error, and exactly optimal over all lower-triangular Toeplitz factors with respect to the maximum squared error [DMP⁺24]. We will discuss lower bounds more in the next section.

While the square-root factorization achieves this near-optimal error, no efficient implementation of $\sqrt{A}z$ (or $A^{-1/2}z$) is known. It is possible to implement the matrix-vector computation using the fast Fourier transform in time $O(T \log T)$, but no sublinear space algorithm is known. Motivated by this, Paper A improves the maximum squared error of the binary tree mechanism by a leading factor of 4, while retaining its efficiency.

Another direction of research into factorizations has been to design factorizations that *structurally approximate* the square-root factorization. Paper B opts for tackling the problem head-on. Taking an existing factorization $LR = A$, it directly perturbs the left factor L into a new matrix $\hat{L}$ supporting efficient computation, and shows that the induced factorization $\hat{L}\hat{R} = A$ has error close to the original. More precisely, it achieves a $(1+\xi)$ -multiplicative approximation of the error in $\tilde{O}_\xi(\sqrt{n})$ space and time per output.

Dvijotham, McMahan, Pillutla, Steinke, and Thakurta [DMP⁺24] approached the same approximation problem by instead taking a generating function view of matrix factorization. They use the observation that there are three functionally identical views of the same mathematical object:

- A sequence $(f_k)_{k=0}^\infty$ where $f_k \in \mathbb{R}$
- An ordinary generating function $f : \mathbb{C} \rightarrow \mathbb{C}$ where

$$f(x) = \sum_{k=0}^{\infty} f_k x^k = \sum_{k=0}^{\infty} \frac{f^{(k)}(0)}{k!} x^k.$$

- A family of lower-triangular Toeplitz matrices $T[f, n] \in \mathbb{R}^{n \times n}$ for $n \in \mathbb{N}$ generated by f

$$\forall i, j \in [n] \quad T[f, n]_{i,j} = \begin{cases} f_{i-j} & \text{if } i \geq j, \\ 0 & \text{if } i < j. \end{cases}$$

The counting matrix A in this view can be identified with the constant sequence $(1, 1, 1, \dots)$ and the generating function $a(x) = \frac{1}{1-x}$. Finding a factorization of the counting matrix into two lower-triangular Toeplitz matrices can thus be expressed as finding generating functions $l(x), r(x)$ such that $a(x) = l(x)r(x)$. The square-root factorization corresponds to setting $l(x) = r(x) = \sqrt{a(x)} = \frac{1}{\sqrt{1-x}}$. The main contribution of [DMP⁺24] was to identify that choosing $l(x), r(x)$ as appropriate rational function approximations of $\frac{1}{\sqrt{1-x}}$ allows for an arbitrarily

close approximation of the $\text{MaxSE}(L, R)$ error with good efficiency. More precisely, they identified a factorization (L, R) that approximates $\text{MaxSE}(\sqrt{A}, \sqrt{A})$ up to a lower order term, and does so in space and time per step $\Theta(\log^2 T)$.

In Paper E, we rule out a large and natural class of algorithms for computing $\sqrt{A}z$ (and $A^{-1/2}z$) in low space— $\Omega(T)$ space is indeed necessary for any algorithm performing continuous operations. For the same computational model, we also show that the algorithm used for implementing Lz , or $R^{-1}z$, in [DMP⁺24] is exactly optimal in terms of its memory cost.

3.6 Lower bounds

Having accounted for past constructive upper bounds, we switch focus to briefly cover lower bounds.

Pure differential privacy

There is essentially one distinct lower bound for ε -DP continual counting, which is the original packing lower bound of [DNP⁺10b] (the proof of which is given in [DR14]). We re-cap the result and give a proof sketch for it.

Theorem 3.3 ([DNP⁺10b, Theorem 4.2]). *Any ε -DP mechanism $\mathcal{M}$ for continual counting has $\text{MaxErr}(\mathcal{M}) = \Omega(\varepsilon^{-1} \log T)$.*

Proof sketch. The proof works by constructing a family of inputs which are pair-wise close, and for which disjoint distinguishing events exist. To this end, let $k \leq \log(T)/4$ and k divide T . Additionally, assume that $\mathcal{M}$ incurs an error of at most k over all T time steps with probability at least $2/3$ (that is, $\text{MaxErr}(\mathcal{M}) = k$). Consider the family of inputs $x^{(0)}, x^{(1)}, \dots, x^{(T/k)} \in \{0, 1\}^T$ where $x^{(0)}$ is the all-zeros input, and $x^{(i)}$ is zero everywhere except for indices in the block $B_i = [1 + (i - 1) \cdot k, i \cdot k]$.

Define E_i as the event where $\mathcal{M}(x)$ outputs a value $\geq k/2$ at the end of B_i , and reporting a value $< k/2$ at the end of all B_j for $j < i$. By ε -DP, we have that

$$1 \geq \sum_{1 \leq i \leq T/k} \Pr[\mathcal{M}(x^{(0)}) \in E_i] \geq \sum_{1 \leq i \leq T/k} e^{-k\varepsilon} \Pr[\mathcal{M}(x^{(i)}) \in E_i] \geq \frac{2Te^{-k\varepsilon}}{3k},$$

where the first step uses disjointedness of the events and the second step uses group privacy.⁴ Taking a logarithm and simplifying yields $k = \Omega(\varepsilon^{-1} \log T)$. $\square$

The lower bound fundamentally identifies a (small) family of inputs for which no mechanism can be accurate. In Paper C we show that this same lower bound extends to *random* inputs: any mechanism accurate on inputs sampled uniformly at random from $\{0, 1\}^T$ will necessarily incur the same error.

⁴If $\mathcal{M}$ is (ε, δ) -DP for inputs differing in 1 record, then $\mathcal{M}$ is $(k\varepsilon, ke^{(k-1)\varepsilon}\delta)$ -DP for inputs differing in $k > 0$ records.

Approximate differential privacy

The ε -DP lower bound in Theorem 3.3 also extends to (ε, δ) -DP, but for a restricted range of δ . See also [Vad17, Theorem 5.14].

Corollary 3.4. *Any (ε, δ) -DP mechanism $\mathcal{M}$ for continual counting has $\text{MaxErr}(\mathcal{M}) = \Omega(\varepsilon^{-1} \log T)$ if $\delta = e^\varepsilon / T^{\Omega(1)}$.*

Proof idea. Repeat the proof of Theorem 3.3, but instead use group privacy for (ε, δ) -DP. This yields the inequality

$$\frac{2Te^{-k\varepsilon}}{3k} + Te^{-\varepsilon}\delta \leq 1,$$

which recovers the previous lower bound if $\delta = O(e^\varepsilon / T)$. $\square$

For the mean and maximum squared error, we actually have asymptotically tight lower bounds for (ε, δ) -DP.

Theorem 3.5 ([HUU23, Theorem 1.3] (simplified)). *For any $\varepsilon > 0$ and $0 \leq \delta \leq \frac{c}{2e^\varepsilon}$ for some absolute constant $c > 0$, let $\mathcal{M}$ be a (ε, δ) -DP mechanism for continual counting. Then for all T ,*

$$\min \{ \text{MaxSE}(\mathcal{M}), \text{MeanSE}(\mathcal{M}) \} = \Omega_\varepsilon(\log^2 T).$$

It essentially establishes that factorization mechanisms are asymptotically optimal for (ε, δ) -DP with respect to these errors, which motivates the enduring attention paid to them. Also note that the theorem supports $\delta = 0$, so the same lower bound also applies to pure ε -DP. Here however, there is a logarithmic factor separating the lower bound from the upper bound of $O(\log^3 T)$.

Factorization lower bounds. Before ending our discussion of lower bounds, we briefly comment on lower bounds specific to factorization mechanisms. While for ε -DP the corresponding combination of norms for maximum and mean squared error do not easily admit any lower bound, there is a richer literature for (ε, δ) -DP. For a complex matrix $M \in \mathbb{C}^{m \times n}$, define $\gamma_F(M)$ and $\gamma_2(M)$ as follows:

$$\begin{aligned} \gamma_F(M) &= \min \{ \|L\|_F \|R\|_{1 \rightarrow 2} / \sqrt{T} : LR = M \}, \\ \gamma_2(M) &= \min \{ \|L\|_{2 \rightarrow \infty} \|R\|_{1 \rightarrow 2} : LR = M \}. \end{aligned}$$

Here $\gamma_F(A)$ and $\gamma_2(A)$ are norms, and exactly characterize the smallest mean and maximum expected squared error over all factorization mechanisms (up to taking squares) for (ε, δ) -DP continual counting. By using a dual characterization of the γ_2 norm [LSS08, Theorem 9], Matoušek, Nikolov, and Talwar [MNT20] expressed a lower bound of the form:

$$\gamma_2(A) \geq \frac{\|A\|_*}{T}$$

		MaxSE($\mathcal{M}$)	MeanSE($\mathcal{M}$)	MaxErr($\mathcal{M}$)
ε -DP	Upper	$O(\log^3 T)$	$O(\log^3 T)$	$O(\log^2 T)$
	Lower	$\Omega(\log^2 T)$	$\Omega(\log^2 T)$	$\Omega(\log T)$
(ε, δ) -DP	Upper	$O(\log^2 T)$	$O(\log^2 T)$	$O(\log^{3/2} T)$
	Lower	$\Omega(\log^2 T)$	$\Omega(\log^2 T)$	$\Omega(\log T)$

Table 3.1: Summary of upper and lower bounds for private continual counting for constant $\varepsilon < 1$ and sufficiently small $\delta = e^\varepsilon / T^{\Omega(1)}$. All upper bounds are attained for the binary tree mechanism [DNP⁺10b; CSS11]. $\text{MaxErr}(\mathcal{M}) = O(\log^2 T)$ for ε -DP follows from the concentration of sums of Laplace random variables [CSS11, Lemma 2.8]. Lower bounds on $\text{MaxSE}(\mathcal{M})$ and $\text{MeanSE}(\mathcal{M})$ are from [HUU23, Theorem 1.3], and the lower bounds on $\text{MaxErr}(\mathcal{M})$ are from [DNP⁺10b, Theorem 4.2].

where $\|\cdot\|_*$ denotes the *nuclear* norm and equals the sum of singular values for the matrix. By lower-bounding $\|A\|_*$ they were able to show

$$\gamma_2(A) \geq \frac{\ln(2T+1)}{\pi}.$$

For the γ_F norm, Henzinger, Upadhyay, and Upadhyay [HUU23] showed that

$$\gamma_F(A) \geq \frac{\|A\|_*}{T},$$

yielding an identical lower bound for $\gamma_F(A)$. Taken together:

$$\inf_{LR=A} \min \{\text{MaxSE}(L, R), \text{MeanSE}(L, R)\} \geq \left(\frac{\ln(2T+1)}{\pi} \right)^2,$$

showing that $(\sqrt{A}, \sqrt{A})$ is an optimal factorization, up to lower order terms.

3.7 State of the art

We end this overview of private continual counting by briefly summarizing the state of the art for the problem. Table 3.1 summarizes the asymptotics. For (ε, δ) -DP there is a gap of $\sqrt{\log T}$ between upper and lower bound for $\text{MaxErr}(\mathcal{M})$, but for the remaining error notions we have tight characterizations. In the case of ε -DP, there is a full factor of $\log(T)$ between upper and lower bound across all three error notions. That said, as $\text{MaxErr}(\mathcal{M})$ is not measuring squared error, its $\log(T)$ gap is “larger”.

When going beyond asymptotics, $(\sqrt{A}, \sqrt{A})$ [FHU23; HUU23] is as we have discussed optimal over factorizations, up to lower order terms, for (ε, δ) -DP and maximum/mean squared error. Dvijotham, McMahan, Pillutla, Steinke, and Thakurta [DMP⁺24] approximate its maximum squared error up to lower

order terms in space $\Theta(\log^2 T)$. Our work in [AP25] (Paper B) achieves a $(1 + \xi)$ -multiplicative approximation of $(\sqrt{A}, \sqrt{A})$ for both maximum and mean squared error in space $\tilde{O}_\xi(\sqrt{T})$. Empirically it achieves a *lower* error than the square-root factorization (and consequently [DMP⁺24]) in sublinear space for T up to $\approx 10^4$. Recently Henzinger and Upadhyay [HU25] proposed a new explicit factorization, further improving lower order terms in the maximum squared error for (ε, δ) -DP. No efficient implementation of this algorithm is currently known.

For ε -DP, the best upper bound for mean squared error is, to the best of our knowledge, given in [APS⁺25] (Paper C). It uses a k -ary tree (for odd k) with node subtraction, achieving a bound of

$$\text{MeanSE}(\tilde{L}_k, R_k) = \left(\frac{k(1 - 1/k^2)}{2 \log(k)^3} + o(1) \right) \frac{\log(T)^3}{\varepsilon^2}$$

which is minimized for $k = 19$ yielding a leading constant of 0.1236. Treating k as a constant, $\tilde{L}_k z$ can be computed in time $O(T)$ and space $O(\log T)$. We note that, by definition, the factorization $(L_k^*, R_k) = (AR_k^\dagger, R_k)$ will attain a smaller mean squared error. That said, (1) we are not aware of any analysis showing a tighter upper bound for $\text{MeanSE}(L_k^*, R_k)$ and (2) $L_k^* z$ would be dense, and so likely not computationally efficient.

Although not explicitly argued in [APS⁺25], the same construction $(\tilde{L}_k, R_k)$ also yields a factor 2 improvement for maximum squared error compared to (L_k, R_k) . Indeed, we go from adding at most $k - 1$ nodes per level to combining at most $(k - 1)/2$ nodes, and so:

$$\text{MaxSE}(\tilde{L}_k, R_k) = \left(\frac{k - 1}{\log(k)^3} + o(1) \right) \frac{\log(T)^3}{\varepsilon^2}.$$

Here the error is minimized for $k = 17$, yielding a leading constant of 0.2343. Each of these error bounds offer more than a factor 8 improvement over the binary tree mechanism.

Chapter 4

Overview of contributions

After having discussed differential privacy and its application to continual counting, we are ready to present the contributions of this thesis. The following is a list of works produced during the PhD program, the first five of which are included in the thesis.

- Paper **A**: Joel Daniel Andersson and Rasmus Pagh. A Smooth Binary Mechanism for Efficient Private Continual Observation [AP23]. NeurIPS 2023.
- Paper **B**: Joel Daniel Andersson and Rasmus Pagh. Streaming Private Continual Counting via Binning [AP25]. SaTML 2025.
- Paper **C**: Joel Daniel Andersson, Rasmus Pagh, Teresa Anna Steiner, Sahel Torkamani. Count on Your Elders: Laplace vs Gaussian Noise [APS⁺25]. FORC 2025.
- Paper **D**: Joel Daniel Andersson, Monika Henzinger, Rasmus Pagh, Teresa Anna Steiner, Jalaj Upadhyay. Continual Counting with Gradual Privacy Expiration [AHP⁺24]. NeurIPS 2024.
- Paper **E**: Joel Daniel Andersson and Amir Yehudayoff. On the Space Complexity of Online Convolution [AY25]. Manuscript.
- Not included: Joel Daniel Andersson, Lukas Retschmeier, Boel Nelson, Rasmus Pagh. Private Lossless Multiple Release [ARN⁺25]. ICML 2025.
- Not included: Joel Daniel Andersson, Palak Jain, Satchit Sivakumar. *Working title*: Fine-Grained Error Bounds for Private Continual Cardinality Estimation in Fully Dynamic Streams. Manuscript.

All papers included in this thesis are on, some variation of, private continual counting. They are listed in an order that tries to keep papers with more similar themes adjacent. Two works were excluded from the thesis.

The paper [ARN⁺25] is excluded on the basis of it not being related to continual counting, and so not fitting in with the framing of the thesis. It deals with the question of performing multiple DP releases of a single query, at different privacy levels received in arbitrary order, and at no loss in utility. This mechanism property is termed *private lossless multiple release*, closely related to *private gradual release* [KHP16] from past work. The main result is that private lossless multiple release is possible for any additive noise mechanism, as long as the underlying noise distribution satisfies a certain *decomposability* property.

The second work excluded is a recent contribution with Palak Jain and Satchit Sivakumar. Topically it fits well into the thesis, but it is excluded as the material has yet to undergo peer-review. This work deals with the related problem of *private cardinality estimation on fully dynamic streams*. On a high level, we study the private release of the size of a set $S_t \subseteq \mathcal{U}$, which is dynamically updated with multiple additions and deletions of items from $\mathcal{U}$ per step t . Here privacy is measured *per item*, so two neighboring streams can differ at potentially all time steps. Past work by Jain, Kalemaj, Raskhodnikova, Sivakumar, and Smith [JKR⁺23] reduced the problem to private continual counting, but for a distinct neighboring relation, and showed that the binary tree mechanism could be deployed out-of-the-box. In our recent work, we show (1) how this reduction is an instantiation of the factorization mechanism for a special class of neighboring inputs, (2) give tighter analysis of tree-based factorization in this framework, and (3) demonstrate how to employ dense factorizations for a leading constant improvement over [JKR⁺23].

We give a short summary of the contributions and the techniques used in each of the included works next.

4.1 Paper A: A Smooth Binary Mechanism

At the time of writing this paper, the square-root factorization [FHU23; HUU23], with leading constant optimal error for (ϵ, δ) -DP, had recently been discovered. That said, no efficient implementation of it was known. $\sqrt{A}z$ can be implemented as a convolution in $O(\log T)$ time per step, but requires $\Omega(T)$ space. As such, the space of *efficient* leading constant improvements was still open. In addition to the improved error bound, another argument in favor of $(\sqrt{A}, \sqrt{A})$ over the (approximate DP) binary tree mechanism, is the stability of its error. The variance of $(\sqrt{A}, \sqrt{A})$ at time $t \leq T$ is $\Theta(\log(t) \log(T))$, whereas the binary tree mechanism has variance per step swinging between $\Theta(\log(t) \log(T))$ and $\Theta(\log T)$ (as the number of nodes needed for any given prefix varies).

Our contribution here is a tree-based factorization mechanism (L, R) that reduces the maximum expected squared error (equivalently: maximum variance) of the binary tree mechanism by a factor of 4, while also having constant variance per step, and retains the efficiency.

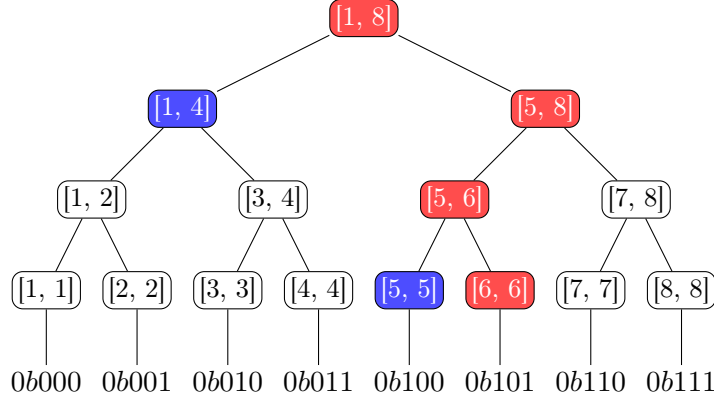


Figure 4.1: Each leaf labeled by $\text{bin}(t)$ for a tree supporting $T = 7$. Nodes marked in red show the path from the root to x_6 , and the blue nodes show the prefix up to x_5 .

Assume $T = 2^h - 1$ for some positive integer h , and let $\text{bin}(q)$ denote the binary encoding of an integer $q \leq T$ with h digits. Consider a labeling of the leaves of the complete tree with $T + 1$ leaves, where the first leaf is labeled $\text{bin}(0)$, next is $\text{bin}(1)$ etc., see Figure 4.1 for an illustration.

Our improvement is built on two observations:

1. For $t > 1$, the number of 1s in the label of x_t decides the number of nodes added together to produce the prefix at time $t - 1$.
2. The path from the root to x_t passes through a number of left children which equals the number of 0s in its label.

Intuitively, a label with many 1s implies that the prefix up to the preceding leaf uses many nodes. Conversely, a label with many 0s implies that the leaf has many ancestors which are used for prefixes, and so any input stored in that leaf has high sensitivity. Our improvement is quite simple: restrict to even h , and only store inputs in leaves with balanced labels (remaining leaves are set to store zero). When releasing a prefix for x_t , compute the prefix up to the leaf in the tree preceding x_{t+1} . This reduces the ℓ_2 sensitivity from $\sqrt{h}$ to $\sqrt{h/2}$, and makes each prefix a sum of $h/2$ nodes.

Naturally, we need a deeper tree to support T balanced labels. We show that a tree height of $h \geq \lceil \log T \rceil + \log \log T$ suffices. This allows us to argue for the claimed factor 4 improvement in maximum squared error. The space and time efficiency are mostly inherited from the binary tree structure.

4.2 Paper B: Continual Counting via Binning

When we started working on this paper we had already published [AP23] and there had been no follow-up work to it. We were still interested in coming up

with efficient factorization mechanisms for (ε, δ) -DP. There was still no efficient implementation for $(\sqrt{A}, \sqrt{A})$, and efficient tree-based mechanisms (such as our smooth binary mechanism) were suboptimal by constants. The question was still open if we could get arbitrarily close to the error of $(\sqrt{A}, \sqrt{A})$ while still admitting efficient computation. As we were preparing this paper for submission to NeurIPS'24, [DMP⁺24] was uploaded to arXiv. They used completely distinct techniques to ours to show a stronger result (space complexity of $\Theta(\log^2 T)$ for same error up to lower order terms), but only for mean squared error.

Overview of results. In this work we show how to construct dense factorizations $(\hat{L}, \hat{R})$ that approximate the error of $(\sqrt{A}, \sqrt{A})$ up to a multiplicative constant $(1+\xi)$, in $O_\xi(\sqrt{T} \log T)$ space and time per step. We also give empirical results which suggest that our factorizations may in fact run in polylogarithmic space (and time per step). Our factorizations also achieve lower mean and maximum squared error than $(\sqrt{A}, \sqrt{A})$ while still admitting sublinear space. This is perhaps surprising, but not inconsistent with the theory. We know that $(\sqrt{A}, \sqrt{A})$ is exactly optimal over lower-triangular Toeplitz factorizations for maximum squared error, but our approximation approach destroys any such structure, allowing us to slightly improve lower order terms. This also implies a lower error than what [DMP⁺24] achieve, as their factorization is also Toeplitz.

Technical overview. Compared to the approach in [DMP⁺24] built on rational function approximation, ours is more direct. Given an $n \times n$ lower-triangular matrix L we introduce the notion of a *binning*. We call $\mathcal{B} = (\mathcal{B}^1, \mathcal{B}^2, \dots, \mathcal{B}^n)$ a binning on $[n]$ if (1) every $\mathcal{B}^i$ is a set of non-overlapping intervals, the union of which is $[i]$, and (2) each interval $I \in \mathcal{B}^{i+1}$ can be expressed as the result of merging intervals in $\mathcal{B}^i \cup \{[i+1, i+1]\}$. A matrix L is said to be $\mathcal{B}$ -binned if on row i it is constant on column indices given by $\mathcal{B}^i$. This is visually demonstrated in Figure 4.2. Define $|\mathcal{B}| = \max_{i \in [n]} |\mathcal{B}^i|$ as the size of the binning. Our observation is that a $\mathcal{B}$ -binned matrix L can compute a matrix-vector product Lz in $O(|\mathcal{B}|)$ space and time per step. In short, matrices admitting small binnings are efficient (note that every lower-triangular matrix admits the large and trivial binning where $\mathcal{B}^i = \{[1, 1], [2, 2], \dots, [i, i]\}$).

We proceed to argue that starting from an interesting factorization, e.g., $(L, R) = (\sqrt{A}, \sqrt{A})$, we can perturb the left factor L into an (invertible) matrix $\hat{L}$ admitting a sublinear binning, inducing a new factorization $(\hat{L}, \hat{R})$ with error close to that of (L, R) . To this end we introduce the notion of a (η, μ) -perturbation. For $\eta, \mu \geq 0$ we say that $\hat{L}$ is a (η, μ) -perturbation of L if for all $i, j \in [n]$:

$$|(\hat{L} - L)_{i,j}| \leq \eta |L_{i,j}| + \mu.$$

Starting from a generic factorization (L, R) , we show that if we derive a (η, μ) -perturbation $\hat{L}$ for sufficiently small $\eta, \mu \geq 0$ (as functions in norms on L), then

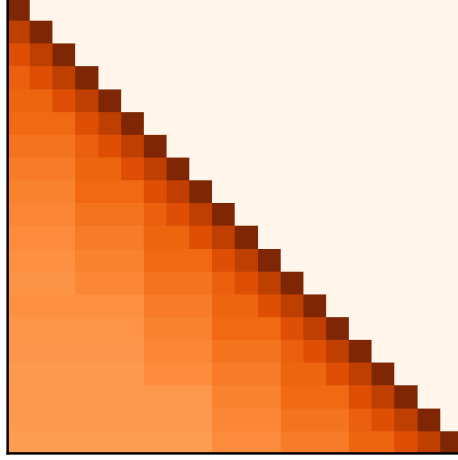


Figure 4.2: Colormap of a 20×20 $\mathcal{B}$ -binned matrix with $|\mathcal{B}| = 7$.

we can arrive at bounds for the induced factorization $(\hat{L}, \hat{R})$:

$$\begin{aligned} \text{MaxSE}(\hat{L}, \hat{R}) &\leq (1 + \xi) \text{MaxSE}(L, R), \\ \text{MeanSE}(\hat{L}, \hat{R}) &\leq (1 + \xi) \text{MeanSE}(L, R), \end{aligned}$$

for arbitrarily small ξ . That such bounds exist is not necessarily surprising (for $\eta, \mu \rightarrow 0$ such a statement must be true), but the bounds turn out to be tight enough to be useful.

Our second step is to give an algorithm that, starting from a matrix L , derives a $\mathcal{B}$ -binned matrix $\hat{L}$. The algorithm passes over the matrix L row-by-row, and maintains a set of intervals on each row corresponding to $\mathcal{B}^i$. When the algorithm goes from row i to $i + 1$, it copies over the intervals from $\mathcal{B}^i$, and then greedily passes over them in column-descending order, greedily merging two intervals based on how close the underlying entries are. After having finished the single pass, it adds the singleton interval $[i + 1, i + 1]$ to the result, defining $\mathcal{B}^{i+1}$, and then proceeds to the next row and repeats. To get the matrix $\hat{L}$, we take the binning $\mathcal{B}$, superimpose it on L , and then set $\hat{L}_{i,j}$ to the *average* of the corresponding values in L , i.e.,

$$\hat{L}_{i,j} = \frac{1}{|I|} \sum_{j' \in I} L_{i,j'} \quad \text{where} \quad j \in I \in \mathcal{B}^i.$$

The merge criterion is parameterized with $0 \leq c, \tau < 1$, which control (η, μ) . An important point is that we merge intervals so that the underlying values are within a factor of $[c^2, c]$ of one another, which allows us to reduce the number of distinct values per row (gives a small binning), and also guarantees that the multiplicative approximation is bounded. The parameter τ instead serves a threshold under which entries are small enough that they can be approximated

by a single interval. For this greedy merge approach to work we do need certain monotonicity conditions on the matrix L , but we show that such conditions are met by a broad class of matrices, including $L = \sqrt{A}$.

Open questions. There are a few questions left open by our work.

- **Open Question 1:** Can the space-error trade-off be tightened for binned factorizations? We give $\tilde{O}_\varepsilon(\sqrt{n})$ space for a multiplicative constant approximation, but the empirical evidence seems to support polylog space. A tighter analysis might not be possible for arbitrary factorizations, but if we restrict our attention to the square-root factorization, and use more of its properties, it could be.
- **Open Question 2:** Our algorithm for approximating L by $\hat{L}$ is a greedy algorithm proceeding row-by-row. Could a more *global* search over binnings lead to better guarantees?
- **Open Question 3:** The technique of binning has the benefit that it does not need the starting factorization (L, R) to be lower-triangular Toeplitz, unlike the technique of rational function approximation [DMP⁺24]. Are there examples of interesting factorizations where the technique of binning is the only recourse? [HKU25] uses our technique in this direction.
- **Open Question 4:** Our streaming algorithm for efficiently computing $\hat{L}z$ is simple. The state update rule only copies over information from z and merges internal states by summing them together. The output map sums all internal real values to produce an output. These are numerically stable operations. Are there practical benefits to such computations, compared to the approach in e.g., [DMP⁺24] that updates its internal state using matrix-vector multiplication?

4.3 Paper C: Count on Your Elders

This paper consists of essentially two results: an efficient constant factor improvement for ε -DP continual counting, and a qualitative strengthening of the classic packing lower bound (Theorem 3.3). In improving the upper bound, we note that our ε -DP algorithm is surprisingly competitive with the square-root factorization for (ε, δ) -DP. In particular, comparing our error to the leading-constant optimal error bound on mean expected squared error from [HUU23], we get that our bound is lower as long as $\delta = O(T^{-0.92})$. Inspired by this, we also derive a (ε, δ) -DP guarantee for the Laplace mechanism. The cheeky title is a play on: (1) using Laplace noise for DP is an older technique compared to using Gaussian noise, and (2) Pierre-Simon Laplace was born in 1749; Carl Friedrich Gauss was born in 1777.

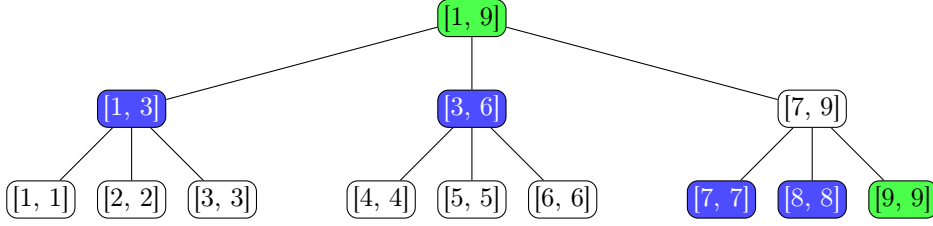


Figure 4.3: A complete ternary tree for $T = 9$ with nodes used to produce a prefix up to $t = 8$ marked out. Blue nodes depict the “standard” decoding of the tree; green nodes depict our decoding. We get the same prefix by adding $[1, 9]$ and subtracting $[9, 9]$.

An upper bound The upper bound starts from the natural extension of the binary tree mechanism to a k -ary tree mechanism. For simplicity, assume $T = k^h$ for integer $h \geq 1$ and a complete tree with branching factor k where the nodes stores the corresponding (noisy) interval sums on $x \in \{0, 1\}^T$. Given the complete k -ary tree, the natural generalization instead adds $\leq k - 1$ nodes per level to produce a prefix at each time step. This mechanism retains the logarithmic space and linear time complexity of the binary tree mechanism. Our improvement here is simple: you really only need to add, *or subtract*, $\lfloor k/2 \rfloor$ nodes per level to express any prefix; see Figure 4.3 for an example. Focusing on the case where k is odd, we create a prefix by on each level either adding the $(k - 1)/2$ left-most children or subtracting the right-most $(k - 1)/2$ children, and potentially also adding the root. We show that this yields a unique mapping of prefix to a set of nodes.

As in the case of the binary tree, one can analyze the k -ary tree mechanism by looking at the encoding of each time step in base k with digits in $\{0, \dots, k - 1\}$. The ℓ_1 weight of the digits in the encoding yields the number of nodes added for the prefix. We make the observation that the same holds for our constructed prefixes, but for numbers encoded in base k with *offset digits* taken from $\{-(k - 1)/2, \dots, (k - 1)/2\}$. This treatment allows for arguing about the mean squared error (average number of nodes combined per prefix is equal to average ℓ_1 weight of digits), and to argue about time/space efficiency. In the end we show that when choosing k to minimize the leading constant for the mean squared error ($k = 19$), we get roughly a factor 2 improvement over only using addition for a k -ary tree.

A lower bound. The lower bound we give in the paper takes the classic packing lower bound (Theorem 3.3) and extends it to a different family of inputs. The original lower bound says that any mechanism $\mathcal{M}$ for ϵ -DP continual counting necessarily has $\text{MaxErr}(\mathcal{M}) = \Omega(\log T)$ on a particular set of sparse inputs. Here, each input has a single block with a logarithmic number of 1s, and these blocks are non-overlapping across inputs. A natural question

to ask is if this set of inputs is particularly hard, and if in fact most inputs are easier.

Our contribution here is a negative result: at least for random inputs, things are no easier. We show that any ε -DP mechanism $\mathcal{M}$ necessarily incurs the same error on inputs drawn uniformly at random from $\{0, 1\}^T$. The proof idea is somewhat technical, but mirrors the structure of the previous lower bound. On a high level, the idea is to again create a set of inputs $x^{(0)}, x^{(1)}, \dots$, but this time around they are random. They are individually close in total variation distance to the uniform distribution, but *pair-wise highly correlated*. Having small TV distance means that any accuracy guarantee on the uniform distribution also extends to our sets of inputs, and we choose to correlate our inputs such that $x^{(0)} - x^{(i)}$ has 1s in a distinct region of the input. This allows an accurate mechanism to distinguish $(x^{(0)}, x^{(i)})$ from $(x^{(0)}, x^{(j)})$ (by running $\mathcal{M}(x^{(0)}) - \mathcal{M}(x^{(i)})$ and detecting the location of the 1s). Summing over disjoint distinguishing events and invoking group privacy (after carefully setting parameters) recovers the lower bound.

Open questions. There are a few natural questions.

- **Open Question 1:** For ε -DP continual counting, there are gaps of a full $\log(T)$ in each of the errors $\text{MaxSE}(\cdot)$, $\text{MeanSE}(\cdot)$ and $\text{MaxErr}(\cdot)$. Which of the bounds, if any, is tight? My conjecture is that the upper bounds based on factorization mechanisms are tight for that class of mechanisms.
- **Open Question 2:** Our extension of the packing lower bound from a deterministic set of inputs to uniformly random inputs might extend beyond the case of prefix queries. Can the same technique be applied more broadly?

4.4 Paper D: Counting with Privacy Expiration

This paper investigates improved algorithms continual counting for the model of *privacy expiration* introduced by Bolot, Fawaz, Muthukrishnan, Nikolov, and Taft [BFM⁺13]. It is a variant of ε -DP for problems under continual observation.

Definition (Privacy expiration [BFM⁺13, Definition 3]). *Let $g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ be a non-decreasing¹ function. Let $\mathcal{A}$ be a randomized online algorithm that takes as an input a stream $x_1, x_2, \dots$ and at every time step t outputs $\mathcal{A}(x_1, \dots, x_t)$. $\mathcal{A}$ satisfies ε -differential privacy with expiration (function) g if for all $\tau \geq 1$, for all measurable $S \subseteq \text{range}(\mathcal{A})^*$, all possible inputs $x_1, \dots, x_\tau$, all $j \leq \tau$ and all x'_j with $|x_j - x'_j| \in [0, 1]$*

$$\Pr[(\mathcal{A}(x_1, \dots, x_j, \dots, x_t))_{t=1}^\tau \in S] \leq e^{g(\tau-j)\varepsilon} \Pr[(\mathcal{A}(x_1, \dots, x'_j, \dots, x_t))_{t=1}^\tau \in S]$$

¹Original definition in [BFM⁺13] states “non-increasing”, which we believe is a typo.

where the probability is over the coin throws of $\mathcal{A}$. We refer to $g(\tau - j) \cdot \varepsilon$ as the privacy loss.

The definition is trying to capture a time-dependence on the privacy guarantee. Conceptually, when running private continual counting for T time steps, it would seem plausible that the input x_1 is less private than x_T , due to the greater number of prefixes that x_1 participates in. Here this intuition is expressed using an expiration function g , where the privacy guarantee afforded by the input x_t , evaluated at a future time step $\tau \geq t$, is exactly $\varepsilon \cdot g(\tau - t)$ -DP. To translate this definition to conventional ε -DP under continual observation, an algorithm satisfying ε -DP with privacy expiration g is also $\varepsilon \cdot g(T - 1)$ -DP when run for T steps. For constant expiration function $g(d) = 1$ the two definitions overlap.

Past work in [BFM⁺13] gave an algorithm for the special case when data only has to satisfy privacy for a sliding window, i.e.,

$$g(d) = \begin{cases} 1 & \text{if } d < W, \\ \infty & \text{if } d \geq W. \end{cases}$$

Our contribution is to show that smoother privacy expiration is possible, in particular polylogarithmic g , and with good utility. As an example of parameter settings, we show that a privacy expiration of $g(d) = O(\log^{1.51} d)$ allows for a constant expected error per step. The algorithm we give is also efficient in terms of space and time, and runs on *unbounded streams*. We additionally adapt the classic packing lower bound (Theorem 3.3) to privacy expiration, and show that our error-expiration trade-off is tight in some parameter regimes.

Technical overview. The main technical work in the paper revolves around modifying a less commonly used version of the binary tree mechanism [DNP⁺10b]. For now, assume $T = 2^h$ for integer $h \geq 1$ and consider the dyadic decomposition $\mathcal{T}_h$. This variant still builds a complete binary tree with T leaves, but here we do not store partial sums in the nodes. Instead, each node only stores independent noise drawn from $\text{Lap}(\log(T)/\varepsilon)$. Labeling the leaves by $1, \dots, T$, the output we produce at time t is the exact prefix $(Ax)_t$, plus one noise term for each node on the path from leaf t to the root. They prove privacy for this mechanism in a different way than the factorization framework. Instead, they argue via a shifting argument on the Laplace random variables.

Let $x \simeq x'$ be two neighboring inputs differing at time j by 1, and let $\mathcal{M}(x) = Ax + Lz \in R^T$ be the private mechanism output evaluated on x . Now, fixing the random values z used for $\mathcal{M}(x)$, we can argue that it is possible to identify a set of $\leq \log T$ values in z to shift by 1, producing z' , such that $\mathcal{M}(x') = Ax + Lz' = \mathcal{M}(x)$. Using the density of a Laplacian random variable, we can argue directly for the relative probabilities of seeing $\mathcal{M}(x)$ and $\mathcal{M}(x')$ assuming the same output is within a factor e^ε , allowing us to show ε -DP.

For privacy expiration we re-use the same idea, but slightly modify the algorithm. First we remove all nodes on the path to the first leaf (including

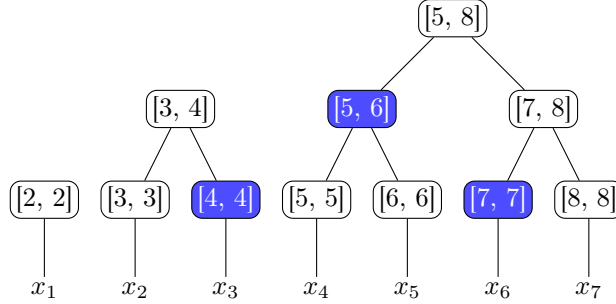


Figure 4.4: The reduced tree used for the main algorithm in Paper D. The blue nodes denote the noise variables that would get shifted by 1 for $j = 3$ and $\tau = 6$ in the argument for proving logarithmic privacy expiration.

the first leaf itself) from the tree, starting the labeling from the second leaf instead; see Figure 4.4 for an illustration of the reduced tree. This change is an improvement to the original algorithm, and implies that the leaf labeled by t has $\leq \log(t)$ ancestors, and so implies that the noise gradually increases. The second change we do is to alter the scale of the noise: noise is now drawn from $\text{Lap}(1/\varepsilon)$. After these amendments the resulting mechanism runs on unbounded streams. To show privacy expiration $g(d)$ for this new mechanism, we argue the following. Given $x \simeq x'$ differing by 1 at step j , for any $\tau \geq j$, we can identify a set of $\leq g(\tau - j + 1) = g(d)$ nodes (noise variables) to shift so that the set of outputs at time steps $t \in [j, \tau]$ are equal across $\mathcal{M}(x)$ and $\mathcal{M}(x')$. See Figure 4.4 for an example. By the same reasoning, this yields probabilities within a factor $\leq e^{\varepsilon \cdot g(d)}$, proving the claim. In particular we show that $g(d) = 2 \log(d + 1) + 2$ is possible.

To get a richer trade-off over expiration functions, we make one change to the algorithm: nodes at level ℓ store noise $\text{Lap}((\ell + 1)^{\lambda-1}/\varepsilon)$, where $\lambda = 1$ corresponds to our previous case. For $\lambda > 1$ we get that nodes higher up in the tree store less noise, which implies improved utility at some cost in privacy. The argument to show $g(d) = O(\log^\lambda d)$ privacy expiration is similar to what was argued before, except that nodes do not uniformly contribute to the factor $e^{\varepsilon \cdot g(d)}$. In the final algorithm we also allow for a time lag $B \geq 0$, meaning the algorithm at time $t > B$ releases a private prefix on the input up to x_{t-B} . This implies that $g(d) = 0$ for $d < B$ (any input experiences perfect privacy B steps after it is streamed), and is used for arguing tightness with respect to our lower bound.

Open questions. There are a few natural questions.

- *Open Question 1:* Is it possible give tight privacy expiration for generic

factorization mechanisms? Any work in this direction could also consider extending the definition to other variants of DP, such as e.g., ρ -zCDP.

- **Open Question 2:** Privacy expiration is inherently defined for *event-level* DP; it does not make much sense if a user owns multiple data points. That said, is there a generalization of privacy expiration for user-level DP that gives meaningful guarantees?
- **Open Question 3:** In the same spirit as this work, are there other *weaker* (and, preferably, defensible) notions of DP under continual observation that allow for improved utility? E.g., a computationally bounded adversary?

4.5 Paper E: Complexity of Online Convolution

Lastly there is the one paper on computational complexity. While not immediately related to privacy at a first glance, the question is informed by the space complexity for (ε, δ) -DP continual counting. As already recounted, after the leading-constant optimal factorization $(\sqrt{A}, \sqrt{A})$ was discovered [FHU23; HUU23], there was nevertheless follow-up work offering up *worse* error guarantees with the motivation that no space efficient implementation of $(\sqrt{A}, \sqrt{A})$ was known [AP23; DMP⁺24; AP25]. Any attempt at implementing $\sqrt{A}z$ efficiently seemed to require $\Omega(T)$ space, but there was no lower bound to back up such a claim. Additionally it was not clear what the techniques needed for proving such a claim would be. This paper addresses this situation: we give lower bounds for the space complexity of computing $\sqrt{A}z$ in a streaming model, and in fact for any lower-triangular Toeplitz matrix.

Overview of results. Before getting into any details, we introduce our streaming model.² Here and throughout, L is a lower-triangular matrix.

Definition (streaming model (simplified)). *A streaming algorithm is a sequence of maps $m^{(t)}$ and $u^{(t)}$ as follows. Let $\beta : \{-1\} \cup \mathbb{N} \rightarrow \mathbb{N}$ be a buffer size function with $\beta(-1) = 1$. Denote the state at time t by $s^{(t)} \in \mathbb{R}^{\beta(t)}$. Initially, the state is $s^{(-1)} = 0 \in \mathbb{R}$. Upon receiving z_t , the algorithm updates its internal state:*

$$s^{(t)} = u^{(t)}(s^{(t-1)}, z_t),$$

where $u^{(t)} : \mathbb{R}^{\beta(t-1)} \times \mathbb{R} \rightarrow \mathbb{R}^{\beta(t)}$. The algorithms then outputs

$$m^{(t)}(s^{(t)})$$

²Unlike in the previous papers, we zero-index vectors and matrices here, and treat x as the argument to a generating function. This overview will use the same conventions.

where $m^{(t)} : \mathbb{R}^{\beta(t)} \rightarrow \mathbb{R}$. We assume that the update rules are continuous; for each t , the map $u^{(t)}$ is a continuous map. The algorithm solves the streaming problem defined by $L \in \mathbb{R}^{\mathbb{N} \times \mathbb{N}}$ if for every z and every t ,

$$m^{(t)}(s^{(t)}) = (Lz)_t.$$

First of all, we treat real computation as atomic. An algorithm is allowed to store a set of real values and the memory cost of an algorithm is exactly described by $\beta(t)$: how many real values are needed at time t to answer all future queries. Our model has restrictions. Firstly, the update maps are restricted to continuous computation. We motivate this restriction by observing that without such a restriction, all matrices L have $O(1)$ space complexity. Another restriction is that both update and output maps are chosen independently of the input. On seeing z_t , I am for example not allowed to choose between two different update maps, $u^{(t)}$ and $\hat{u}^{(t)}$, depending on the value of z_t —unless the composite update map is continuous in z_t .

There is a subtlety in how the streaming model corresponds to the reality of factorization mechanisms, in which we need to compute Lz to privatize the true prefixes. Namely, even if L is lower-triangular, and so $(Lz)_t$ only depends on $z_{t' \leq t}$, we can still sample z in arbitrary order. As pointed out in a remark in the paper, our lower bounds extend to a setting where in addition to receiving z_t at time t , an algorithm also possesses perfect predictions for future $z_{t' > t}$. This implies that our lower bounds also hold in the *one-pass* model.

Moving on, we define $\text{space}_L(t; \tau)$ as the minimum value b such that there is a streaming algorithm with buffer size β that computes Lz , correct up to time τ and $\max_{t' \leq t} \beta(t') \leq b$. Our main results are stated for real, lower-triangular Toeplitz matrices. Using notation similar to [DMP⁺24], we denote a lower-triangular Toeplitz matrix by $T[G]$ where $G(x) = \sum_{k=0}^{\infty} a_k x^k$ and

$$T[G]_{i,j} = T[a]_{i,j} = \begin{cases} a_{i-j} & \text{if } i \geq j, \\ 0 & \text{if } i < j. \end{cases}$$

The concept of rational function is key to our results. We restate the following definition from the paper for convenience.

Definition. A polynomial $Q(x)$ of degree d is an expression of the form $Q(x) = \sum_{k=0}^d q_k x^k$ where $q_d \neq 0$. The polynomial $Q(x)$ is invertible if $q_0 \neq 0$. The generating function G is a rational function of degree at most d if there are polynomials P, Q with Q invertible so that $Q(x) \cdot G(x) = P(x)$ where $\deg(Q) \leq d$ and $\deg(P) \leq d - 1$. We say that G has degree d if it is of degree at most d but not of degree at most $d - 1$. We say that G is irrational if it is not rational (of any degree).

As an example, we have the generating function for the continual counting matrix: $G_1 = \frac{1}{1-x}$ is a degree 1 rational function. Its square-root $\sqrt{G_1} = G_{1/2} =$

$\frac{1}{\sqrt{1-x}}$, the generating function for the factors of the square-root factorization, is an example of an irrational function.

For infinite time horizons, our results can be summarized as

$$\begin{aligned} G \text{ is an irrational function} &\implies \text{space}_G(t; \infty) = t + 1, \\ G \text{ is a rational function of degree } d &\implies \text{space}_G(t; \infty) = \min\{t + 1, d\}. \end{aligned}$$

This is to say that rationality of the generating function *exactly* characterizes when the corresponding streaming problem is solvable in finite space. For finite time horizons (e.g., when we have a bound on the length of the stream as in continual counting), the result for rational G extends. For irrational generating functions, however, we are not so lucky—it is possible to construct irrational generating functions that “look rational” in the early coefficients, but where the irrational structure eventually kicks in. Instead, for finite time horizons and irrational generating functions, we have to work with generating functions on a case-by-case basis. The highlight here is the following result:

$$\text{space}_{G_{1/2}}(t; 2t) = t + 1.$$

This implies that the square-root factorization for (ε, δ) -DP continual counting over T steps requires $\Omega(T)$ space in our model.

Technical overview. The key lemma in the paper shows that:

$$\text{space}_L(t; t + q) \geq \text{rank}(M).$$

where $M \in \mathbb{R}^{(q+1) \times (t+1)}$ is the submatrix formed from L using row indices $t, t + 1, \dots, t + q$ and the first $t + 1$ columns.

The idea for the proof is to consider a fixed input z which is zero for all indices after t (equivalently let $z \in \mathbb{R}^{t+1}$), and only study the set of outputs at times $t, t + 1, \dots, t + q$. Denote by $g(z) \in \mathbb{R}^{q+1}$ the output of the algorithm at these time steps, and let $f(z) \in \mathbb{R}^{\beta(t)}$ denote the algorithm state at time t . Our streaming model enforces that g can be factored as $g = h \circ f$, for some map h . For the algorithm to be correct, we need that $g(z) = Mz$.

We assume towards a contradiction that $\beta(t) < \text{rank}(M)$. To finish the proof we argue (via the Borsuk-Ulam theorem using the continuity of f) that there must exist an input $z' \in \mathbb{R}^{t+1}$ for which $f(z') = f(-z')$, but where $Mz' \neq 0$. The contradiction follows from

$$Mz' = g(z') = h(f(z')) = h(f(-z')) = g(-z') = -Mz',$$

proving $\beta(t) \geq \text{rank}(M)$.

The proof does not enforce any structure on L besides being lower-triangular, and so the matrix M does not necessarily have any structure. However, when we restrict our attention to Toeplitz $L = T[G]$, then we can say a good deal about

M . We denote the *Hankel matrix* with generating function $G = \sum_{k=0}^{\infty} a_k x^k$ by $H[G]$, and define it as

$$H[G]_{i,j} = a_{i+j}.$$

M can be identified as a column permutation of a submatrix of $H[G]$, formed by taking the first $t + 1$ columns and first $q + 1$ rows. We use the shorthand $H[G]^{(q,t)}$ for this matrix.

From this point on in the paper, proving space lower bounds on Toeplitz matrices reduces to studying the rank of Hankel matrices. A key theorem by Kronecker allows us to relate the rank of Hankel matrices and their principal submatrices to the rational function degree of the underlying generating function. This allows us to determine the space complexity of $T[G]$ for rational G over arbitrary time horizons, and to establish results for irrational G under infinite time horizons. For irrational G and finite time horizons, the question of lower bounding $\text{space}_G(t; 2t)$ reduces to lower bounding $\text{rank}(H[G]^{(t,t)})$.

Open questions. There are a few questions left open by our work.

- **Open Question 1:** One natural follow-up question is: what can be shown once we relax to approximate computation of Lz ? We believe our results can be extended to allow for computing $(Lz)_t$ with an error tolerance.
- **Open Question 2:** Not so much an open question as potential future work, but the lower bound mechanism is expressed for arbitrary lower-triangular matrices. By investigating non-Toeplitz factorization mechanisms (e.g., the binary tree mechanism), one can investigate if existing implementations for the noise addition are tight. How much less space can they be implemented in?

Paper A

A Smooth Binary Mechanism

Abstract. In privacy under continual observation we study how to release differentially private estimates based on a dataset that evolves over time. The problem of releasing private prefix sums of $x_1, x_2, x_3, \dots \in \{0, 1\}$ (where the value of each x_i is to be private) is particularly well-studied, and a generalized form is used in state-of-the-art methods for private stochastic gradient descent (SGD). The seminal *binary mechanism* privately releases the first t prefix sums with noise of variance polylogarithmic in t . Recently, Henzinger et al. and Denisov et al. showed that it is possible to improve on the binary mechanism in two ways: The variance of the noise can be reduced by a (large) constant factor, and also made more even across time steps. However, their algorithms for generating the noise distribution are not as efficient as one would like in terms of computation time and (in particular) space. We address the efficiency problem by presenting a simple alternative to the binary mechanism in which 1) generating the noise takes constant average time per value, 2) the variance is reduced by a factor about 4 compared to the binary mechanism, and 3) the noise distribution at each step is *identical*. Empirically, a simple Python implementation of our approach outperforms the running time of the approach of Henzinger et al., as well as an attempt to improve their algorithm using high-performance algorithms for multiplication with Toeplitz matrices.

*Full title: *A Smooth Binary Mechanism for Efficient Private Continual Observation*, Joel Daniel Andersson, Rasmus Pagh (BARC, University of Copenhagen), NeurIPS'23.

A.1 Introduction

There are many actors in society that wish to publish aggregate statistics about individuals, be it for financial or social utility. Netflix might recommend movies based on other users' preferences, and policy might be driven by information on average incomes across groups. Whatever utility one has in mind however, it should be balanced against the potential release of sensitive information. While it may seem anodyne to publish aggregate statistics about users, doing it without consideration to privacy may expose sensitive information of individuals [DN03]. Differential privacy offers a framework for dealing with these problems in a mathematically rigorous way.

A particular setting is when statistics are updated and released continually, for example a website releasing its number of visitors over time. Studying differential privacy in this setup is referred to as *differential privacy under continual observation* [DNP⁺10b; DR14]. A central problem in this domain is referred to as *differentially private counting under continual observation* [DNP⁺10b; CSS11], *continual counting* for short. It covers the following problem: a binary stream $x_1, x_2, x_3, \dots$ is received one element at a time such that x_t is received in round t . The objective is to continually output the number of 1s encountered up to each time step while maintaining differential privacy. We consider two streams x and x' as neighboring if they are identical except for a single index i where $x_i \neq x'_i$. It suffices to study the setting in which there is a known upper bound T on the number of prefix sums to release — algorithms for the case of unbounded streams then follow by a general reduction [CSS11].

Aside from the natural interpretation of continual counting as the differentially private release of user statistics over time, mechanisms for continual counting (and more generally for releasing prefix sums) are used as a subroutine in many applications. Such a mechanism is for example used in Google's privacy-preserving federated next word prediction model [KMS⁺21; MT22; CGM⁺24], in non-interactive local learning [STU17], in stochastic convex optimization [HLL⁺22] and in histogram estimation [CLS⁺12; Upa19; CR22; HQY⁺22] among others.

Given the broad adoption of continual counting as a primitive, designing algorithms for continual counting that improve constants in the error while scaling well in time and space is of practical interest.

Our contributions

In this paper we introduce the *Smooth Binary Mechanism*, a differentially private algorithm for the continual counting problem that improves upon the original binary mechanism by Dwork, Naor, Pitassi, and Rothblum [DNP⁺10b] and Chan, Shi, and Song [CSS11] in several respects, formalized in Theorem A.1 and compared to in Table A.1.

Table A.1: Comparison between different ρ -zCDP mechanisms for continual counting.

Mechanism $\mathcal{M}$	$\text{Var}[\mathcal{M}(t)] \cdot \frac{2\rho}{\log_2^2 T}$	Time to produce all T outputs	Space	Matrix type
Binary mech.	1	$O(T)$	$O(\log T)$	sparse
Honaker Online	$0.5 + o(1)$	$O(T \log T)$	$O(\log T)$	sparse
[DMR ⁺ 22]	$0.0487 \dots + o(1)^*$	$O(T^2)$	$O(T^2)$	dense
[HUU23]	$0.0487 \dots + o(1)$	$O(T \log T)^{**}$	$O(T)$	Toeplitz
Our mechanism	$0.25 + o(1)$	$O(T)$	$O(\log T)$	sparse

“Honaker Online” refers to the “Estimation from below” variant in Honaker [Hon15] as implemented in Kairouz, McMahan, Song, Thakkar, Thakurta, and Xu [KMS⁺21]. There is no explicit bound on variance in [DMR⁺22], but the method finds an optimal matrix factorization so it should achieve same variance^{*} as [HUU23] up to lower order terms. As to their efficiency, [DMR⁺22] contains empirical work for reducing the time and space usage by approximating the matrices involved as a sum of a banded matrix and a low-rank approximation, but without formal guarantees. The time usage in^{**} assumes implementing the matrix-vector product using FFT. Leveraging FFT for continual observation is not a novel approach [CMR⁺23]. All sparse matrices have $O(\log T)$ nonzero entries per row or column, and all dense matrices have $\Omega(T^2)$ nonzero entries — a Toeplitz matrix can be seen as intermediate in the sense of having $O(T)$ unique diagonals, allowing for efficient storage and multiplication.

Theorem A.1 (Smooth Binary Mechanism). *For any $\rho > 0$, $T > 1$, there is an efficient ρ -zCDP continual counting mechanism $\mathcal{M}$, that on receiving a binary stream of length T satisfies*

$$\text{Var}[\mathcal{M}(t)] = \frac{1 + o(1)}{8\rho} \log_2(T)^2$$

where $\mathcal{M}(t)$ is the output prefix sum at time t , while only requiring $O(\log T)$ space, $O(T)$ time to output all T prefix sums, and where the error is identically distributed for all $1 \leq t \leq T$.

Our mechanism retains the scalability in time and space of the binary mechanism while offering an improvement in variance by a factor of $4 - o(1)$. It also has the *same* error distribution in every step by design, which could make downstream applications easier to analyze.

Sketch of technical ideas. Our starting point is the binary mechanism which, in a nutshell, uses a complete binary tree with $\geq T + 1$ leaves (first T leaves corresponding to $x_1, \dots, x_T$) in which each node contains the sum of

the leaves below, made private by adding random noise (e.g. from a Gaussian distribution). To estimate a prefix sum $\sum_{i=1}^t x_i$ we follow the path from the root to the leaf storing x_{t+1} . Each time we go to a right child the sum stored in its sibling node is added to a counter. An observation, probably folklore, is that it suffices to store sums for nodes that are *left children*, so suppose we do not store any sum in nodes that are right children. The number of terms added when computing the prefix sum is the number of 1s in the binary representation $\text{bin}(t)$ of t , which encodes the path to the leaf storing x_{t+1} . The *sensitivity* of the tree with respect to x_t , i.e., the number of node counts that change by 1 if x_t changes, is the number of 0s in $\text{bin}(t-1)$. Our idea is to only use leaves that have *balanced* binary representation, i.e. same number of 0s and 1s (assuming the height h of the tree is an even integer). To obtain T useful leaves we need to make the tree slightly deeper — it turns out that height h slightly more than $\log_2(T)$ suffices. This has the effect of making the sensitivity of every leaf $h/2$, and the noise in every prefix sum a sum of $h/2$ independent noise terms.

Limitations. As shown in Table A.1 and Section A.4, the smooth binary mechanism, while improving on the original binary mechanism, cannot achieve as low variance as matrix-based mechanisms such as [HUU23]. However, given that the scaling of such methods can keep them from being used in practice, our variant of the binary mechanism has practical utility in large-scale settings.

Related work

All good methods for continual counting that we are aware of can be seen as instantiations of the *factorization mechanism* [LMH⁺15], sometimes also referred to as the *matrix mechanism*. These methods perform a linear transformation of the data, given by a matrix R , then add noise according to the sensitivity of Rx , and finally obtain the sequence of estimates by another linear transformation given by a matrix L . To obtain unbiased estimates, the product LRx needs to be equal to the vector of prefix sums, that is, LR must be the lower triangular all 1s matrix.

Seminal works introducing the first variants of the binary mechanism are due to Chan, Shi, and Song [CSS11] and Dwork, Naor, Pitassi, and Rothblum [DNP⁺10b], but similar ideas were proposed independently in Hay, Rastogi, Miklau, and Suciú [HRM⁺10] and Gehrke, Wang, and Xiao [GWX10]. Honaker [Hon15] noticed that better estimators are possible by making use of *all* information in the tree associated with the binary tree method. A subset of their techniques can be leveraged to reduce the variance of the binary mechanism by up to a factor 2, at some cost of efficiency, as shown from their implementation in Kairouz, McMahan, Song, Thakkar, Thakurta, and Xu [KMS⁺21].

A number of recent papers have studied improved choices for the matrices L , R . Denisov, McMahan, Rush, Smith, and Guha Thakurta [DMR⁺22] treated

the problem of finding matrices leading to minimum largest variance on the estimates as a convex optimization problem, where (at least for T up to 2048) it was feasible to solve it. To handle a larger number of time steps they consider a similar setting where a restriction to *banded* matrices makes the method scale better, empirically with good error, but no theoretical guarantees are provided.

Fichtenberger, Henzinger, and Upadhyay [FHU23] gave an explicit decomposition into lower triangular matrices, and analyzed its error in the ℓ_∞ metric. The matrices employed are Toeplitz (banded) matrices. Henzinger, Upadhyay, and Upadhyay [HUU23] analyzed the same decomposition with respect to mean squared error of the noise, and showed that it obtains the best possible error among matrix decompositions where L and R are square matrices, up to a factor $1 + o(1)$ where the $o(1)$ term vanishes when T goes to infinity.

A breakdown of how our mechanism compares to existing ones is shown in Table A.1. While not achieving as small an error as the factorization mechanism of Henzinger, Upadhyay, and Upadhyay [HUU23], its runtime and small memory footprint allows for better scaling for longer streams. For concreteness we consider privacy under ρ -zero-Concentrated Differential Privacy (zCDP) [BS16], but all results can be expressed in terms of other notions of differential privacy.

A.2 Preliminaries

Binary representation of numbers. We will use the notation $\text{bin}(n)$ to refer to the binary representation of a number $n \in [2^h]$, where $h > 0$ is an integer, and let $\text{bin}(n)$ be padded with leading zeros to h digits. For example, $\text{bin}(2) = 0b010$ for a tree of height $h = 3$. When indexing such a number we let index i refer to the i^{th} least significant bit, e.g. $\text{bin}(2)_1 = 1$. We will also refer to prefixes and suffixes of binary strings, and we use the convention that a prefix of a binary string includes its most significant bits.

Partial sums (p-sums). To clear up the notation we use the concept of p-sums $\Sigma[i, j]$ where $\Sigma[i, j] = \sum_{t=i}^j x_t$. We will furthermore use the concept of noisy p-sums

$$\widehat{\Sigma}[i, j] = \Sigma[i, j] + X[i, j], \quad X[i, j] \sim \mathcal{F},$$

where $\mathcal{F}$ is a suitable distribution for the DP paradigm, e.g. Laplacian or Gaussian. For convenience we also define $\hat{x}_t = \widehat{\Sigma}[t, t]$, i.e. $\hat{x}_t$ is the single stream element with noise added.

Continual observation of bit stream

Given an integer $T > 1$ we consider a finite length binary stream $x = (x_1, x_2, \dots, x_T)$, where $x_t \in \{0, 1\}$, $1 \leq t \leq T$, denotes the bit appearing in the stream at time t .

Definition A.2 (Continual Counting Query). Given a stream $x \in \{0, 1\}^T$, the *count* for the stream is a mapping $c : \{1, \dots, T\} \rightarrow \mathbb{Z}$ such that $c(t) = \sum_{i=1}^t x_i$.

Definition A.3 (Continual Counting Mechanism). A *continual counting mechanism* $\mathcal{M}$ takes a stream $x \in \{0, 1\}^T$ and produces a (possibly random) vector $\mathcal{M}(x) \in \mathbb{R}^T$ where $\mathcal{M}(x)_t$ is a function of the first t elements of the stream. For convenience we will abuse notation and write $\mathcal{M}(t)$ for $\mathcal{M}(x)_t$ when there is little chance for ambiguity.

To analyze a counting mechanism from the perspective of differential privacy, we also need a notion of neighboring streams.

Definition A.4 (Neighboring Streams). Streams $x, x' \in \{0, 1\}^T$ are said to be *neighboring*, denoted $x \sim x'$, if $|\{i \mid x_i \neq x'_i\}| = 1$.

Intuitively, for a counting mechanism to be useful at a given time t , we want it to minimize $|\mathcal{M}(t) - c(t)|$. We consider unbiased mechanisms that return the true counts in expectation and we focus on minimizing $\text{Var}[\mathcal{M}(t) - c(t)]$.

Differential privacy

For a mechanism to be considered differentially private, we require that the outputs for any two neighboring inputs are indistinguishable. We will state our results in terms of ρ -zCDP:

Definition A.5 (Concentrated Differential Privacy (zCDP) [BS16]). For $\rho > 0$, a randomized algorithm $\mathcal{A} : \mathcal{X}^n \rightarrow \mathcal{Y}$ is ρ -zCDP if for any $\mathcal{D} \sim \mathcal{D}'$, $D_\alpha(\mathcal{A}(\mathcal{D}) \parallel \mathcal{A}(\mathcal{D}')) \leq \rho\alpha$ for all $\alpha > 1$, where $D_\alpha(\mathcal{A}(\mathcal{D}) \parallel \mathcal{A}(\mathcal{D}'))$ is the α -Rényi divergence between $\mathcal{A}(\mathcal{D})$ and $\mathcal{A}(\mathcal{D}')$.

In the scenario where we are looking to release a real-valued function $f(\mathcal{D})$ taking values in $\mathbb{R}^d$, we can achieve zCDP by adding Gaussian noise calibrated to the ℓ_2 sensitivity of f .

Lemma A.6 (Gaussian Mechanism [BS16]). Let $f : \mathcal{X}^n \rightarrow \mathbb{R}^d$ be a function with global ℓ_2 -sensitivity $\Delta := \max_{\mathcal{D} \sim \mathcal{D}'} \|f(\mathcal{D}) - f(\mathcal{D}')\|_2$. For a given data set $\mathcal{D} \in \mathcal{X}^n$, the mechanism that releases $f(\mathcal{D}) + \mathcal{N}(0, \frac{\Delta^2}{2\rho})^d$ satisfies ρ -zCDP.

It is known that ρ -zCDP implies $(\rho + 2\sqrt{\rho \ln(1/\delta)}, \delta)$ -differential privacy for every $\delta > 0$ [BS16].

Lastly, when comparing counting mechanisms based on Gaussian noise, we note that it is sufficient to look at the variance. For a single output $\mathcal{M}(t)$, the variance $\text{Var}[\mathcal{M}(t)]$ is the only relevant metric, as it completely describes the output distribution. For a series of outputs $\mathcal{M}_x$, and related norms $\|\mathcal{M}_x\|_p$, we note that a mechanism with lower variance will be more concentrated in each coordinate and have lower p^{th} moment, allowing for tighter bounds on the norm.

Differentially private continual counting

We next describe two approaches to continual counting.

Binary mechanism. The binary mechanism [DNP⁺10b; GWX10; HRM⁺10; CSS11] can be considered the canonical mechanism for continual counting. In this section we present a variant of it where only left subtrees are used. The mechanism derives its name from the fact that a binary tree is built from the input stream. Each element from the stream is assigned a leaf in the binary tree, and each non-leaf node in the tree represents a p-sum of all elements in descendant leaves. All values are stored noisily in nodes, and nodes are added together to produce a given prefix sum. Such a binary tree is illustrated in Figure A.1a.

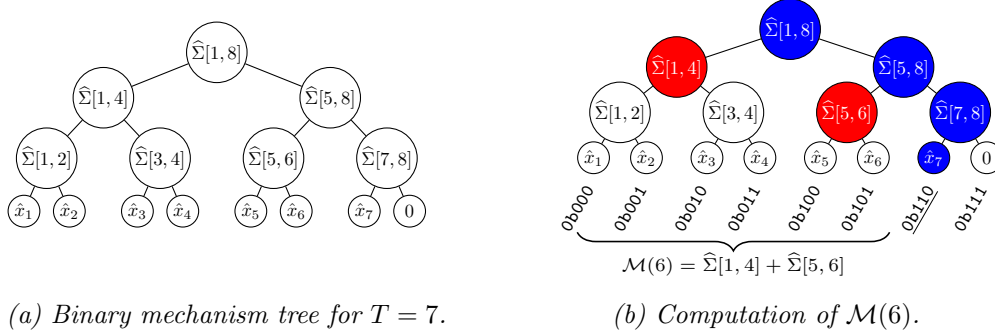


Figure A.1: Binary trees for a sequence of length $T = 7$. In Figure A.1b each leaf is labeled by $\text{bin}(t - 1)$, and it illustrates how the prefix sum up to $t = 6$ can be computed from $\text{bin}(t)$. Blue nodes describe the path taken by Algorithm 1, and the sum of red nodes form the desired output $\mathcal{M}(6)$.

We will return for a closer analysis of the binary mechanism in Section A.3. For now we settle for stating that the ρ -zCDP binary mechanism achieves $\text{Var}[\mathcal{M}(t)] = O(\log(T)^2)$ and is known to be computationally efficient: to release all prefix sums up to a given time $t \leq T$ requires only $O(\log(T))$ space and $O(t)$ time.

Factorization mechanism. The binary mechanism belongs to a more general class of mechanisms called *factorization mechanisms* [LMH⁺15], sometimes also referenced as *matrix mechanisms*. Computing a prefix sum is a linear operation on the input stream $x \in \mathbb{R}^T$, and computing all prefix sums up to a given time T can therefore be represented by a matrix $A \in \mathbb{R}^{T \times T}$, where $c(t) = (Ax)_t$. A is here a lower-triangular matrix of all 1s. The factorization mechanism characterizes solutions to the continual counting problem by factorizing A as $A = LR$ with corresponding mechanism $\mathcal{M}(x) = L(Rx + z)$, $z \sim \mathcal{F}^n$ where n is the dimension of Rx .

Algorithm 1 Prefix Sum for Binary Mechanism

```

1: Input: binary tree of height  $h$  storing  $\widehat{\Sigma}[a, b]$  for  $b \leq t$ , time  $t \leq 2^h - 1$ 
2:  $output \leftarrow 0$ 
3:  $s \leftarrow \text{bin}(t)$  {padded to  $h$  bits by adding zeros}
4:  $a \leftarrow 1; b \leftarrow 2^h$ 
5: for  $i = h - 1$  to  $0$  do
6:    $d = \lfloor \frac{a+b}{2} \rfloor$ 
7:   if  $s_i = 1$  then
8:      $output \leftarrow output + \widehat{\Sigma}[a, d]$ 
9:      $a \leftarrow d + 1$ 
10:  else
11:     $b \leftarrow d$ 
12:  end if
13: end for
14: return  $output$ 

```

Intuitively Rx represents linear combinations of the stream, which are made private by adding noise z , and which then are aggregated by L . To achieve ρ -zCDP for this mechanism, we let $z \sim \mathcal{N}(0, \frac{\Delta^2}{2\rho})^n$ where $\Delta = \max_i \|Re_i\|_2$, e_i being the i^{th} unit vector. It follows that the corresponding output noise becomes Lz with coordinates $(Lz)_i \sim \mathcal{N}(0, \frac{\Delta^2}{2\rho} \|L_i\|_2^2)$ where L_i is the i^{th} row of L .

Extension to multidimensional input. While we have so far assumed one-dimensional input, any one-dimensional ρ -zCDP factorization mechanism naturally generalizes to higher dimensions. This fact enables factorization mechanisms to be applied to the problem of private learning where gradients are summed. We sketch an argument based on linear algebra in Section A.A.

A.3 Our mechanism

To introduce our variant of the binary mechanism, we need to return to the original mechanism.

Closer analysis of the binary mechanism

As has been pointed out in earlier work [DMR⁺22; HUU23], a naive implementation of the binary mechanism will lead to variance that is non-uniform with respect to time: the number of 1s in the bitwise representation of t determines the variance. To underline this, consider the pseudo-code in Algorithm 1 for computing a prefix sum given a binary mechanism tree structure. The code assumes that the tree has $\geq t + 1$ leaves, a detail that only matters when t is a power of 2 and allows us to never use the root node. To illustrate a simple case, see Figure A.1b.

We can make the following two observations:

- $\text{bin}(t - 1)$ encodes a path from the root to the leaf where x_t is stored.
- To compute prefix sums using Algorithm 1, we only need to store values in “left children”.

Combining these observations, we get the following result:

Proposition A.7. *The squared ℓ_2 -sensitivity Δ^2 of x_t is equal to the number of 0s in $\text{bin}(t - 1)$.*

To see this, note that the number of 0s in $\text{bin}(t - 1)$ is equal to the number of left-children that are passed through to reach the given node. Changing the value of x_t impacts all its ancestors, and since only left-children are used for prefix sums, the result immediately follows. This does not address the volatility of variance with respect to time. However, studying Algorithm 1 gives the reason:

Proposition A.8. *$\text{Var}[\mathcal{M}(t)]$ is proportional to the number of 1s in $\text{bin}(t)$.*

The result follows from the fact that the number of terms added together for a given prefix sum $c(t)$ is equal to the number of 1s in $\text{bin}(t)$, since Line 8 is executed for each such 1. In this view, each node that is used for the prefix sum at time t can be identified as a prefix string of $\text{bin}(t)$ that ends with a 1.

We will return to Theorem A.7 and Theorem A.8 when constructing our smooth mechanism, but for now we settle for stating that combined they give the exact variance at each time step for the binary mechanism. Following Li, Miklau, Hay, McGregor, and Rastogi [LMH⁺15], to make the mechanism private we have to accommodate for the worst sensitivity across all leaves, which yields Theorem A.9.

Theorem A.9 (Exact Variance for Binary Mechanism [DNP⁺10b; CSS11]). *For any $\rho > 0$, $T > 1$, the ρ -zCDP binary mechanism $\mathcal{M}$ based on Algorithm 1 achieves variance*

$$\text{Var}[\mathcal{M}(t)] = \frac{\lceil \log(T + 1) \rceil}{2\rho} \|\text{bin}(t)\|_1$$

for all $1 \leq t \leq T$, where $\|\text{bin}(t)\|_1$ is equal to the number of 1s in $\text{bin}(t)$.

A smooth binary mechanism

Based on the analysis, a naive idea to improve the binary mechanism would be to only consider leaves with “favorable” time indices. To make this a bit more precise, we ask the following question: could there be a better mechanism in which we store elements in only a subset of the leaves in the original binary tree,

Algorithm 2 Prefix Sum for Smooth Binary Mechanism

```

1: Input: binary tree of height  $h$  storing  $\widehat{\Sigma}[a, b]$  for  $b \leq m(t)$ , time  $t$  where  $m(t+1) \leq 2^h$ 
2:  $output \leftarrow 0$ 
3:  $s \leftarrow \text{bin}(m(t+1))$  {padded to  $h$  bits by adding zeros}
4:  $a \leftarrow 1; b \leftarrow 2^h$ 
5: for  $i = h - 1$  to  $0$  do
6:    $d = \lfloor \frac{a+b}{2} \rfloor$ 
7:   if  $s_i = 1$  then
8:      $output \leftarrow output + \widehat{\Sigma}[a, d]$ 
9:      $a \leftarrow d + 1$ 
10:  else
11:     $b \leftarrow d$ 
12:  end if
13: end for
14: return  $output$ 

```

and then use Algorithm 1 to compute the prefix sums? We give an affirmative answer.

Consider a full binary tree of height h where we let the leaves be indexed by i (1-indexed). Given a sequence of elements $x \in \{0, 1\}^T$ (we assume a great enough height h to accommodate for all elements) and an integer $0 \leq k \leq h$, we conceptually do the following for each leaf with index i :

- If $\text{bin}(i - 1)$ has k 0s, store the next element of the stream in the leaf.
- Otherwise store a token 0 in the leaf.

More rigorously stated, we are introducing an injective mapping from time to leaf-indices $m : \{1, \dots, T + 1\} \rightarrow [2^h + 1]$, such that $m(t)$ is the $(t - 1)^{\text{st}}$ smallest h -bit integer with k 0s in its binary representation. x_t gets stored in the leaf with index $m(t)$, and to compute $\mathcal{M}(t)$, we would add p-sums based on $\text{bin}(m(t + 1))$. The resulting algorithm is listed as Algorithm 2, where $\widehat{\Sigma}[i, j]$ is the noisy count of leaf i through j .

It follows that the ℓ_2 -sensitivity is equal to $\sqrt{k}$, and that any prefix sum will be a sum of $h - k$ nodes. Importantly, the latter fact removes the dependence on t for the variance.

Choosing a k . The optimal choice of k depends on the differential privacy paradigm. Here we only consider ρ -zCDP where $\Delta = \sqrt{k}$. Since each prefix sum is computed as a sum of $h - k$ nodes, the variance for a given prefix sum becomes $\text{Var}[\mathcal{M}(t)] = (h - k) \cdot \frac{k}{2\rho}$, which for $k = h/2$ gives a leading constant of $1/4$ compared to the maximum variance of the binary mechanism. This choice of k is valid if the tree has an even height h , and it maximizes the number of available leaves. Such a tree together with a computation is shown in Figure A.2.

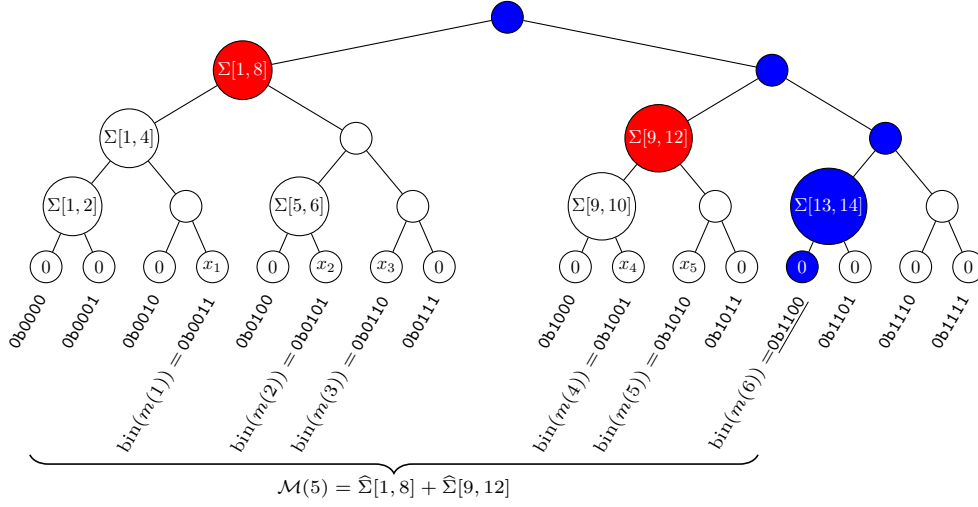


Figure A.2: Computation of $\mathcal{M}(5)$ using the smooth binary mechanism where $T = 5$. The figure illustrates how the prefix sum up to t can be computed from $\text{bin}(m(t+1))$. Blue nodes describe the path taken by Algorithm 2, and the sum of red nodes form the desired output $\mathcal{M}(t)$. All values shown in nodes are stored noisily, and $\Sigma[i, j]$ is here defined as the sum of leaves i through j . Observe that the noise in each node is drawn from the same distribution and that each query is formed by adding together $h/2 = 2$ noisy nodes, implying an identical distribution of the error at each step.

A penalty in height. The analysis above assumes that we have a tree of sufficient height. If we before had a tree of height $\lceil \log(T+1) \rceil$, we now need a tree of height $h \geq \lceil \log(T+1) \rceil$ to have enough leaves with the right ratio of 1s in their index. To account for this, we let h be the smallest even integer such that $h \geq \lceil \log(T) \rceil + a \log \log T$, where a is a constant. For our new tree to support as many prefix sums, we need that $\binom{h}{h/2} \geq T+1$. This holds for $a > 1/2$ and sufficiently large T , but we show it for $a = 1$ next. Using Stirling's approximation in the first step, we can establish that

$$\binom{h}{h/2} \geq \frac{2^h}{\sqrt{2h}} \geq \frac{2^{\log T + \log \log T}}{\sqrt{2} \sqrt{\lceil \log T \rceil + \log \log T}} = \frac{\log T}{\sqrt{2} \sqrt{\lceil \log T \rceil + \log \log T}} \cdot T,$$

which is at least $T+1$ for $T \geq 13$.

Resulting variance. This height penalty makes $\text{Var}[\mathcal{M}(t)]$ no longer scale as $\log(T+1)^2$, but $(\log(T) + a \log \log(T))^2$. Nevertheless, we can state the following:

Lemma A.10. *For any $\rho > 0$, $T \geq 13$, the ρ -zCDP smooth binary mechanism $\mathcal{M}$ achieves variance*

$$\text{Var}[\mathcal{M}(t)] = \frac{1 + o(1)}{8\rho} \log(T)^2$$

where $1 \leq t \leq T$, and the $o(1)$ term is at most $\frac{2 \log \log(T)}{\log(T)} + \left(\frac{\log \log(T)}{\log(T)}\right)^2$.

which is an improvement over the original binary mechanism by a factor of $1/4$ with regard to the leading term. This improvement is shown empirically in Section A.4.

Constant average time per output. When outputting T prefix sums continuously while reading a stream, we only have to store the noise of the nodes, not the p-sums themselves. To make this more explicit, let S_t describe the set of nodes (p-sum indices) that the smooth mechanism adds together to produce the output at time t . To produce $\mathcal{M}(t+1)$ given $\mathcal{M}(t)$, we effectively do:

$$\mathcal{M}(t+1) = \mathcal{M}(t) + x_{t+1} + \sum_{(i,j) \in S_{t+1} \setminus S_t} X[i, j] - \sum_{(i,j) \in S_t \setminus S_{t+1}} X[i, j].$$

To quantify the cost, we need to deduce how many nodes are replaced from t to $t+1$, which means reasoning about S_t and S_{t+1} . Recalling that each element in S_t can be identified by a prefix string of $\text{bin}(m(t+1))$ terminating with a 1, consider Figure A.3. Based on the pattern shown in Figure A.3, where the leaf indices only differ in their least significant bits, we get that $|S_{t+1} \setminus S_t| = |S_t \setminus S_{t+1}| = n$, where n is the number of 1s in the least significant block of 1s. We formalize this observation next to give a bound on the average cost when outputting a sequence of prefix sums.

Lemma A.11. *Assuming the cost of replacing a node in a prefix sum is 1, then the cost to release all $\binom{2k}{k} - 1$ prefix sums in the tree of height $2k$ using the smooth binary mechanism is at most $2\binom{2k}{k}$.*

$$\begin{aligned} \text{bin}(m(t+1)) &= \boxed{\dots} \boxed{0} \underbrace{\boxed{1 \dots 1}}_n \boxed{0 \dots 0} ; \\ \text{bin}(m(t+2)) &= \boxed{\dots} \boxed{1} \boxed{0 \dots 0} \underbrace{\boxed{1 \dots 1}}_{n-1} \end{aligned}$$

Figure A.3: The least significant bits of two leaf indices in a full binary tree that are neighboring with respect to time when used in the smooth binary mechanism. If the first cluster of 1s in $\text{bin}(m(t+1))$, counted from the least significant bit, has n 1s then n nodes in total will be replaced from t to $t+1$.

Proof. As argued before, to compute $\mathcal{M}(t+1)$ given $\mathcal{M}(t)$ we need to replace a number of nodes equal to the size n of the least significant block of 1s in $\text{bin}(m(t+1))$. We can therefore directly compute the total cost by enumerating all valid indices with different block sizes as

$$\begin{aligned} \text{cost} &= -k + \sum_{n=1}^k \sum_{i=k-n}^{2k-n-1} n \binom{i}{k-n} = -k + \sum_{n=1}^k n \binom{2k-n}{k-n+1} \\ &= -k + \sum_{i=1}^k \sum_{j=1}^i \binom{k-1+j}{k-1} \\ &= -2k + \sum_{i=1}^k \binom{k+i}{k} = \binom{2k+1}{k+1} - (2k+1) \leq 2 \binom{2k}{k} \end{aligned}$$

where the initial $-k$ term comes from excluding the last balanced leaf index in the tree. $\square$

In particular, this implies that the average cost when releasing all prefix sums in a full tree is ≤ 2 . For T that does not use all leaves of a tree, comparing to the closest T' where $T \leq T' = \binom{2k}{k} - 1$ also implies an average cost of ≤ 4 for arbitrary T . For a single output the cost is $O(\log(T))$. It is not hard to check that computing $\text{bin}(m(t+1))$ from $\text{bin}(m(t))$ can be done in constant time using standard arithmetic and bitwise operations on machine words.

Logarithmic space. The argument for the binary mechanism only needing $O(\log T)$ space extends to our smooth variant directly.

Lemma A.12. *The smooth binary mechanism runs in $O(\log T)$ space.*

Proof. The key observation is that once a given p-sum is removed from a prefix sum, then it will never re-appear. Let its associated prefix string at time t be s of length l . If we look at the same l bit positions in $\text{bin}(m(t))$ as t increases and interpret it as a number, then it is monotonically increasing with t . This implies that once a given prefix-string s in $\text{bin}(m(t))$ disappears at $t' > t$, then it will never be encountered again. We can therefore free up the memory used for storing any p-sum the moment it is no longer used, implying we only need to store the $O(\log T)$ active at any given point in time. $\square$

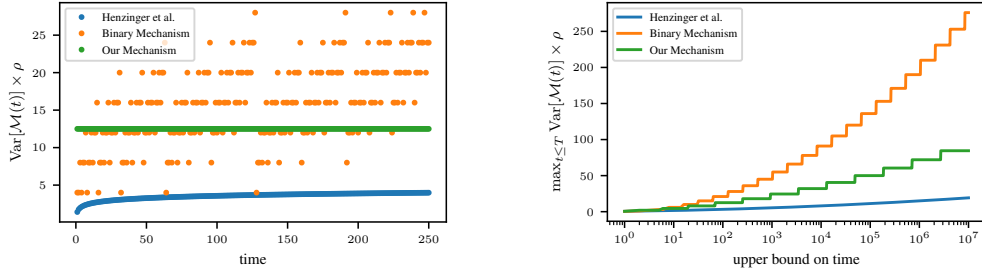
Finishing the proof for Theorem A.1. The last, and more subtle, property of our mechanism is that the error at each time step not only has constant variance, but it is identically distributed. By fixing the number of 1s in each leaf index to k , the output error at each step becomes the k -fold convolution of the noise distribution used in each node, independently of what that distribution is. Our mechanism achieves this by design, but note that this can be achieved for any other mechanism (e.g. the regular binary mechanism) by adding fresh

noise in excess of what gives privacy (at the expense of utility). Combining Lemmas A.10, A.11 and A.12 together with this last property, we arrive at Theorem A.1.

A.4 Comparison of mechanisms

In this section we review how the smooth binary mechanism compares to the original binary mechanism, and the factorization mechanism of Henzinger, Upadhyay, and Upadhyay [HUU23]. A Python implementation of our smooth binary mechanism (and the classic binary mechanism) can be found on <https://github.com/jodander/smooth-binary-mechanism>. We do not compare to Denisov, McMahan, Rush, Smith, and Guha Thakurta [DMR⁺22] since their method is similar to that of Henzinger, Upadhyay, and Upadhyay [HUU23] in terms of error, and less efficient in terms of time and space usage. We also do not compare to the online version of Honaker [Hon15] as it is less efficient in time, and its improvement in variance over the binary mechanism is at most a factor of 2, and can therefore be estimated from the figures.

Variance comparison. To demonstrate how the variance behaves over time for our smooth binary mechanism, see Figure A.4. Given a fix T , the tree-based



(a) Running variance for $T = 250$.

(b) Maximum variance vs. bound on T .

Figure A.4: Comparison of variance between the mechanism in Henzinger, Upadhyay, and Upadhyay [HUU23], the standard binary mechanism and our mechanism. Figure A.4a shows $\text{Var}[\mathcal{M}(t)]$ for $1 \leq t \leq T$ for $T = 250$, whereas Figure A.4b shows the maximum variance that each mechanism would attain for a given upper bound on time. At the last time step in Figure A.4b, our mechanism reduces the variance by a factor of 3.27 versus the binary mechanism.

mechanisms compute the required tree height to support all elements, and the factorization mechanism a sufficiently large matrix. The volatility of the error in the regular binary mechanism is contrasted by the stable noise distribution of our smooth binary mechanism, as demonstrated in Figure A.4a. In terms of achieving the lowest kariance, the factorization mechanism in Henzinger, Upadhyay, and Upadhyay [HUU23] is superior, as expected. This result is

replicated in Figure A.4b where our mechanism offers a substantial improvement in terms of maximum variance over the original binary mechanism, but does not improve on Henzinger, Upadhyay, and Upadhyay [HUU23].

Computational efficiency comparison. While our mechanism does not achieve as low noise as the mechanism of Henzinger, Upadhyay, and Upadhyay [HUU23], it scales well in time and space, and with respect to the dimensionality of the stream elements. This is empirically demonstrated in Figure A.5. Since

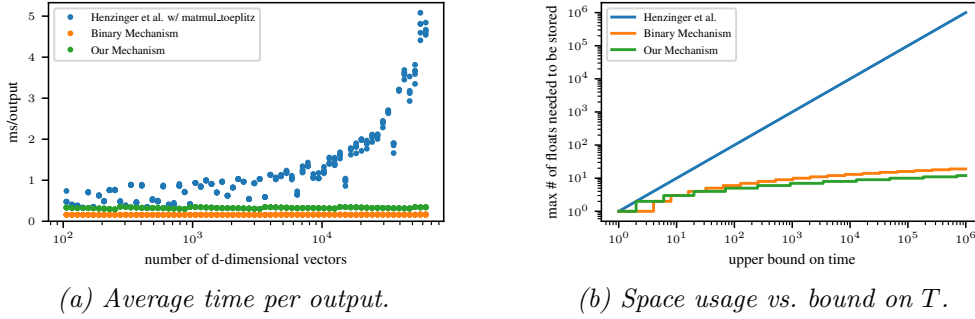


Figure A.5: Comparison of computational efficiency between the mechanism in Henzinger, Upadhyay, and Upadhyay [HUU23], the binary mechanism and our mechanism. Figure A.5a shows the computation time spent per d -dimensional input. The simulation was run 5 times for each method, meaning each method has 5 data points in the plot per time step. The computation was performed for elements of dimension $d = 10^4$, was run on a Macbook Pro 2021 with Apple M1 Pro chip and 16 GB memory using Python 3.9.6, scipy version 1.9.2, and numpy version 1.23.3. Figure A.5b shows the maximum number of floats that has to be stored in memory when outputting all prefix sums up to a given time, assuming binary input.

the matrix used in Henzinger, Upadhyay, and Upadhyay [HUU23] is a Toeplitz matrix, the scipy method “matmul_toeplitz” (based on FFT and running in time $O(dT \log T)$ to produce a d -dimensional output) was used to speed up the matrix multiplication generating the noise in Figure A.5a.

The discrepancy in the computation time scaling can largely be attributed to space: the needed space for these tree-based methods scales logarithmically with T , and linearly with T for matrix-multiplication based methods. This is demonstrated in Figure A.5b. As to the difference in computation time between the tree-based methods, the smooth binary mechanism generates twice as much fresh noise per time step on average, which likely is the dominating time sink in this setup.

A.5 Conclusion and discussion

We presented an improved “smooth” binary mechanism that retains the low time complexity and space usage while improving the additive error and achieving stable noise at each time step. Our mechanism was derived by performing a careful analysis of the original binary mechanism, and specifically the influence of the binary representation of leaf indices in the induced binary tree. Our empirical results demonstrate the stability of the noise and its improved variance compared to the binary mechanism. The factorization mechanism of Henzinger, Upadhyay, and Upadhyay [HUU23] offers better variance, but is difficult to scale to a large number of time steps, especially if we need high-dimensional noise.

We note that the smooth binary mechanism can be extended to ε -DP. The optimal fraction of 1s in the leaves of the binary tree would no longer be $1/2$. An interesting problem is to find mechanisms that have lower variance and attractive computational properties. It is possible that the dependence on T can be improved by leaving the factorization mechanism framework, but in absence of such a result the best we can hope is to match the variance obtained by Henzinger, Upadhyay, and Upadhyay [HUU23].

Acknowledgments and disclosure of funding

We would like to thank Jalaj Upadhyay for useful discussions about our results and their relation to previous work. We would also like to thank the reviewers for their comments and suggestions which have contributed to a clearer paper. The authors are affiliated with Basic Algorithms Research Copenhagen (BARC), supported by the VILLUM Foundation grant 16582, and are also supported by Providentia, a Data Science Distinguished Investigator grant from Novo Nordisk Fonden.

Appendix A.A Counting on multidimensional inputs

Assuming a d -dimensional input stream with elements $x_i \in \mathbb{R}^d$, let two streams x, x' be neighboring if they differ at exactly one time step t where $\|x_t - x'_t\|_2 \leq 1$. Given a one-dimensional ρ -zCDP factorization mechanism for continual counting with factorization $A = LR$, we want to argue that running it along each dimension separately gives a ρ -zCDP factorization mechanism for releasing prefix sums on x .

To see this, consider the new, flattened vector input $\tilde{x} \in \mathbb{R}^{d \cdot T}$ where $\tilde{x} = [(x_1)_1, (x_1)_2, \dots, (x_1)_d, (x_2)_1, \dots, (x_T)_d]$, i.e., $(x_t)_j = \tilde{x}_{(t-1) \cdot d + j}$. Analogously for these new inputs, we can define a counting matrix $\tilde{A} = A \oplus I_{d \times d} \in \mathbb{R}^{d \cdot T \times d \cdot T}$ that sums each dimension separately, where $\oplus$ is the Kronecker product and $I_{d \times d}$ is the d -dimensional identity matrix. We get a corresponding factorization $\tilde{A} = \tilde{L}\tilde{R}$ where $\tilde{L} = L \oplus I_{d \times d}$ and $\tilde{R} = R \oplus I_{d \times d}$.

It follows immediately that, $\tilde{L}, \tilde{R}$ gives a factorization mechanism for privately releasing $\tilde{A}\tilde{x}$, and from it we can extract all private prefix sums on the original stream x . Letting $\tilde{x}'$ be the vector representation of the stream $x' \sim x$, we will reason about the ℓ_2 sensitivity Δ next. First observe that for neighbouring inputs x and x' that differ at time t , we have:

$$\|\tilde{R}(\tilde{x} - \tilde{x}')\|_2 = \|Re_t\|_2 \cdot \|\tilde{x} - \tilde{x}'\|_2$$

and therefore

$$\Delta = \sup_{\tilde{x} \sim \tilde{x}'} \|\tilde{R}(\tilde{x} - \tilde{x}')\|_2 = \max_i \|Re_i\|_2 \cdot \sup_{\tilde{x} \sim \tilde{x}'} \|\tilde{x} - \tilde{x}'\|_2 = \max_i \|Re_i\|_2,$$

which equates the ℓ_2 sensitivity of the one-dimensional case. Recapitulating, any one-dimensional ρ -zCDP factorization mechanism for continual counting is also a ρ -zCDP counting mechanism for inputs of higher dimension, as long as the neighboring relation is appropriately extended.

Paper B

Streaming Private Continual Counting via Binning

Abstract. In differential privacy, *continual observation* refers to problems in which we wish to continuously release a function of a dataset that is revealed one element at a time. The challenge is to maintain a good approximation while keeping the combined output over all time steps differentially private. In the special case of *continual counting* we seek to approximate a sum of binary input elements. This problem has received considerable attention lately, in part due to its relevance in implementations of differentially private stochastic gradient descent. *Factorization mechanisms* are the leading approach to continual counting, but the best such mechanisms do not work well in *streaming* settings since they require space proportional to the size of the input. In this paper, we present a simple approach to approximating factorization mechanisms in low space via *binning*, where adjacent matrix entries with similar values are changed to be identical in such a way that a matrix-vector product can be maintained in sublinear space. Our approach has provable sublinear space guarantees for a class of lower triangular matrices whose entries are monotonically decreasing away from the diagonal. We show empirically that even with very low space usage we are able to closely match, and sometimes surpass, the performance of asymptotically optimal factorization mechanisms. Recently, and independently of our work, Dvijotham et al. have also suggested an approach to implementing factorization mechanisms in a streaming setting. Their work differs from ours in several respects: It only addresses factorization into *Toeplitz* matrices, only considers *maximum* error, and uses a different technique based on rational function approximation that seems less versatile than our binning approach.

* Full title: *Streaming Private Continual Counting via Binning*, Joel Daniel Andersson, Rasmus Pagh (BARC, University of Copenhagen), SaTML'25.

B.1 Introduction

Analyzing private data involves creating methods to query datasets while protecting the privacy of the individuals within them. A leading framework in this field is *differential privacy* [DMN⁺06], which provides precise probabilistic assurances regarding individual privacy. The balance between the usefulness of the data and the privacy protections offered by differentially private algorithms is significantly influenced by the nature of the queries being made.

Continual observation [DNP⁺10b; DR14] constitutes a particularly interesting scenario. In this setting, data entries are received sequentially and each new entry requires an immediate query response based on the entire dataset accumulated up to that point. The challenge comes from the fact that each data entry can affect all future outputs. A fundamental problem in this context is *counting under continual observation* [DNP⁺10b; CSS11], *continual counting* for short. The input stream is $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T \in \{0, 1\}$, arriving one item at a time, where $\mathbf{x}_t$ is received at time t , and the task is to output the number of 1s observed so far. Two input streams $\mathbf{x}$ and $\mathbf{x}'$ are considered neighboring if they differ at exactly one index. Although this is one of the most basic problems in the area, there are still gaps between the lower and upper limits for both approximate and pure differential privacy [DNP⁺10b; DNR⁺15; FHU23; HUU23; CLN⁺24; DMP⁺24].

In recent years, the continual counting problem has gained renewed attention due to its use as a subroutine for private learning [KMS⁺21; DMR⁺22; CMR⁺23; CGS⁺24]. For some private learning settings, the useful quantity to make private turns out to be *prefix sums on sequences of gradient vectors* [TS13; KMS⁺21], a problem that can be reduced to continual counting. In deep learning settings in particular, the regime of parameters is such that only space-efficient mechanisms for continual counting are viable, since there is a blow-up in the space proportional to the dimension of the gradients (i.e., the numbers of parameters in the model). Dvijotham, McMahan, Pillutla, Steinke, and Thakurta [DMP⁺24] provide an example of a learning task where, given 32-bit precision, over 40 petabytes of memory would be needed to run the mechanism with space complexity $\Omega(T)$.

For approximate differential privacy, the setting in which deep private learning typically takes place, all known mechanisms, excluding special cases such as sparse inputs, have an expected error of $\Theta(\log T)$ per step, and no improvement in the asymptotic dependence on T has been achieved since the first non-trivial algorithms in [DNP⁺10b; CSS11]. The best known continual counting mechanisms belong to the class of *factorization mechanisms* [LMH⁺15]. Let A be the matrix such that $A\mathbf{x}$ is the prefix sum vector for $\mathbf{x}$. A factorization mechanism is characterized by a factorization of A into a product of matrices, $A = LR$. Recent work [HUU23] showed that the factorization where $L = R = \sqrt{A}$ (a matrix also studied in [Ben77]) achieves optimal mean squared error, up to lower order vanishing terms, across all factorizations. Unfortunately,

all existing implementations of this algorithm have space complexity $\Omega(T)$.¹ Thus, alternative factorization mechanisms running in sublinear space have been proposed [DMR⁺22; AP23; DMP⁺24].

Our contributions. We propose a framework for factorization mechanisms using a class of lower-triangular matrices $L \in \mathbb{R}^{T \times T}$ that yield low space complexity, as defined next.

Definition B.1 (Space complexity of L for streaming). Let $L \in \mathbb{R}^{T \times T}$ be a lower-triangular matrix and let $\mathbf{z} \in \mathbb{R}^T$ arrive as a stream where $\mathbf{z}_i$ arrives at time i and we immediately need to output $(L\mathbf{z})_i$. A streaming algorithm is defined by a state update function that maps the step number, current state, and input $\mathbf{z}_i$ to a new state and an output $(L\mathbf{z})_i$. We consider *linear* streaming algorithms where the state is a vector in $\mathbb{R}^q$ and the update function for each step number i is a linear function of the state and the input $\mathbf{z}_i$. The *space complexity* of L is the smallest q for which there exists a (linear) streaming algorithm outputting $L\mathbf{z}$.

Note that the state update function may depend on L and the output is not recorded in the state, so neither L nor the output $L\mathbf{z}$ are counted in the space usage. The class of matrices we consider define, for each row, a partition of the set of nonzero entries into intervals such that the entries in each interval of the row are identical. Furthermore the intervals align between consecutive rows such that each interval of row i is either the union of intervals in row $i-1$ or the singleton interval $\{i\}$. This allows us to compute $(L\mathbf{z})_i$ in small space by keeping track of the sum of entries in $\mathbf{z}$ corresponding to each interval. We call this class of matrices *binned* matrices, and the structure of the binning decides the space complexity.

Our main contribution is an algorithm (Algorithm 3) that approximates a given matrix L by a binned matrix $\hat{L}$. The concept is demonstrated in Figure B.1. When applied to an invertible matrix L of a factorization LR , we get a new factorization $\hat{L}\hat{R}$ where $\hat{R} = \hat{L}^{-1}LR$. We show that if L comes from a particular class of matrices, namely those satisfying Theorem B.21, the binned factorization offers good utility at low space complexity. In the particular case of continual counting and the (almost optimal) factorization $L = R = B = \sqrt{A}$ [Ben77; FHU23; HUU23], we show the following result.

Theorem B.2. Let $B \in \mathbb{R}^{T \times T}$ be the Bennett matrix. Then for constant $\xi \in (0, 1)$ there is a factorization $\hat{L}\hat{R} = B^2$ where:

- $\hat{L}\hat{R}$ satisfies $\text{MeanSE}(\hat{L}, \hat{R}) \leq (1+\xi) \text{MeanSE}(B, B)$ and $\text{MaxSE}(\hat{L}, \hat{R}) \leq (1+\xi) \text{MaxSE}(B, B)$.
- $\hat{L}$ has space complexity $O_\xi(\sqrt{T} \log T)$.

¹As we will show in Paper E, $\Omega(T)$ space is needed for any “natural” implementation.

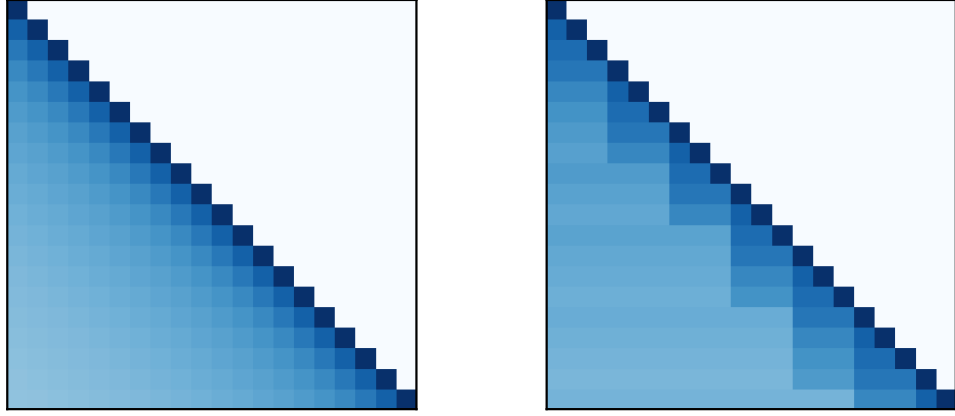
(a) Lower triangular matrix L .(b) L after binning.

Figure B.1: Illustration of the concept of binning on a 20-by-20 matrix. Each square corresponds to an entry in the matrix, with a darker shade illustrating a larger value. The binned matrix in Figure B.1b uses at most 4 intervals per row, as illustrated by each row of the matrix having at most 4 shades.

- Given $\mathbf{z} \in \mathbb{R}^T$ and random access to B , $\hat{L}\mathbf{z}$ can be computed incrementally in time $O_\xi(\sqrt{T} \log T)$ per output.

That is, we achieve the same *mean (expected) squared error* and *maximum (expected) squared error* up to a factor of $(1 + \xi)$ in sublinear space and time per output. We supplement our theoretical result with empirical results up to $T \approx 10^4$ that confirm the space- and error efficiency of our method, even at space usage much lower than what our theoretical results require. Surprisingly, we do not only nearly match the error of the asymptotically optimal factorization, but often *improve* the error. We also provide theoretical and empirical results for the more general square-root factorization of the counting matrix with momentum and weight decay [KL24], where our approach is equally suitable.

Sketch of technical ideas. We start out by considering an arbitrary factorization LR and investigate how the norms $\|L\|_F$, $\|L\|_{2 \rightarrow \infty}$ and $\|R\|_{1 \rightarrow 2}$ change under a perturbation of L (adjusting R such that the matrix product is preserved). For this we need $L \in \mathbb{R}^{T \times T}$ to be invertible, and consider perturbations $P \in \mathbb{R}^{T \times T}$ where for $\eta, \mu \geq 0$, we have that $|P_{i,j}| \leq \eta |L_{i,j}| + \mu$. If P is chosen so that $\hat{L} = L + P$ is invertible, we can then define $\hat{R} = \hat{L}^{-1}LR$, and get a new factorization $\hat{L}\hat{R} = LR$. We show that the error of $\hat{L}\hat{R}$ exceeds that of LR by no more than a multiplicative factor $(1 + \xi)$, where ξ depends on μ, η and L , and can be made arbitrarily close to zero. We then proceed to describe a class of lower-triangular matrices, referred to as *binned* matrices where row i is assigned a partition $\mathcal{B}^i$ of $[i] = \{1, 2, \dots, i\}$ and where the matrix is constant

on each interval of the partition. Furthermore, we require that $\mathcal{B}^{i+1}$ can be constructed by merging intervals in $\mathcal{B}^i$ and then adding $\{i+1\}$ to it. Such matrices have space complexity equal to the maximum number of intervals in any of the partitions $\mathcal{B}^1, \dots, \mathcal{B}^T$, which can be $o(T)$. We then give an algorithm for approximating a matrix L by a binned matrix $\hat{L}$, where the central idea is to group elements based on them being within a multiplicative constant c of each other, up to a small additive constant. For a class of matrices, which in particular includes the square-root factorization of the counting matrix, we show that the resulting perturbation $P = \hat{L} - L$ can be bounded entry-wise, implying good error, with a binning of sublinear size, resulting in small space usage. (Empirically, logarithmic space seems to suffice.)

Limitations. Our technique is only known to apply to a class of factorization mechanisms in which the left matrix (defining the noise distribution) satisfies a certain monotonicity condition (Definition B.21). Without conditions on this matrix, our binning might lead to an increase in the sensitivity of the factorization that would significantly increase the amount of noise added by the mechanism. The experimental results for counting use the exact sensitivity of the factorization rather than the upper bound from our theoretical analysis — using this sharper bound requires a quadratic time preprocessing algorithm to be run. We stress that this computation only depends on T and can therefore be done “offline” before any data is received. If the exact sensitivity is stored, it can be reused for any stream with length at most T .

Our bound on space usage does *not* include the space for storing the matrix L , and we furthermore assume that an entry of L can be accessed in constant time. If the elements in L can be computed efficiently on the fly (like in our key examples), then there is no need for storing L . If L has no such structure, we note that for many applications, the attention paid to space usage is derived from use cases where we are interested in running the mechanism on d -dimensional vectors where d may be larger than the number of outputs, e.g., prefixes of gradients. In such applications, the space needed for storing L explicitly may be dominated by the space needed for storing the set of d -dimensional noise vectors. In particular, for deep learning the regime of parameters is often such that $d \gg T$.

Related work. Central to our work is the *factorization mechanism* [LMH⁺15], also known as *matrix mechanism*, framework to which all mechanisms discussed here belong. It addresses the problem of releasing linear queries $A \in \mathbb{R}^{q \times T}$ when applied to a dataset $\mathbf{x} \in \mathbb{R}^T$, and solves it by factoring A into matrices L, R where $A = LR$, privately releasing $R\mathbf{x} + \mathbf{z}$ where the noise $\mathbf{z}$ is scaled to the sensitivity of R , and then outputting $L(R\mathbf{x} + \mathbf{z})$.

While our techniques derive from factorization mechanisms, and our results extend to more general query matrices, the problem that motivates us is

continual counting. The first mechanism to achieve polylogarithmic error for this problem was the *binary mechanism* [DNP⁺10b; CSS11], and it did so in space complexity $O(\log T)$. Since then no asymptotic improvements in the error have been achieved without further assumptions on the input (e.g., sparsity [DNR⁺15]).

Recent work identified that the Toeplitz matrix factorization $L = R = \sqrt{A}$ for continual counting achieved optimal, up to lower-order vanishing terms, mean squared error over all factorizations [HUU23] and state-of-the-art maximum squared error [FHU23]. However, the structure of L admits no algorithm sublinear space complexity, rendering it impractical for large-scale applications. The same is true of other factorizations corresponding to aggregation matrices used in data analysis and machine learning applications [HUU24; KL24].

More recently, and independently of our work, Dvijotham, McMahan, Pillutla, Steinke, and Thakurta [DMP⁺24], identified the same problem and designed a near-optimal solution with space complexity $\Theta(\log^2 T)$ and the same maximum squared error up to a $(1 + o(1))$ multiplicative factor. They prove near-optimality with respect to the maximum squared error but do not state any results for mean squared error. Their solution is based on approximating L and R by Toeplitz matrices $\hat{L}$ and $\hat{R}$ where the values on subdiagonals of $\hat{L}$ satisfy a recursion that facilitates computing Lz in low space. Their technique could potentially be extended to factorizations of different A , but their existing analysis only holds for the lower-triangular all-1s matrix and relies on all matrices involved being Toeplitz. Our approach does not need any assumption of Toeplitz structure and is arguably simpler. It has a weaker theoretical space bound, but our empirical work shows that it works well with much less space, suggesting that our analysis is not tight.

Our main algorithm for producing binned matrices is similar in structure to *weight-based merging histograms* of Cohen and Strauss [CS03, Section 5]. They considered (approximately) outputting decaying prefix sums of a stream $x_1, \dots, x_T$, i.e., releasing $\sum_{i=1}^t g(t-i)x_i$ where g is a non-increasing function, while using little space for storing the stream. This can be translated into our setting as approximating a Toeplitz left factor matrices L by a matrix $\hat{L}$ with low space complexity. Though our binned matrices are constructed in a way similar to weight-based merging histograms, we stress that our objectives are very different and require an entirely different and more sophisticated analysis. We use the term “binning” that we feel better expresses what happens to the rows of a matrix.

B.2 Introducing the problem

Preliminaries

We let $\mathbb{N} = \{1, 2, \dots\}$ denote the set of all positive integers, use the notation $[a, b]$ for $a, b \in \mathbb{N}$, $a \leq b$ to denote the set of integers $\{a, a+1, \dots, b\}$, and in

particular let $[a] = [1, a]$.

Linear algebra. For a matrix $A \in \mathbb{R}^{m \times n}$, let $A_{i,j} \in \mathbb{R}$ for $i \in [m], j \in [n]$ denote an entry. In the case where $[a, b] \subseteq [n]$ and $A_{i,j}$ assumes the same value for all $j \in [a, b]$, we will abuse notation and let $A_{i,[a,b]}$ represent this scalar. We reserve the notation $|A|$ for the matrix with entries $|A|_{i,j} = |A_{i,j}|$. $\|A\|_2$ denotes the operator norm of a matrix and if A is invertible then $\kappa(A) = \|A\|_2 \|A^{-1}\|_2$ defines the condition number of A . Whenever a matrix product AB is given without specifying the dimensions of $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{m' \times n'}$, we are implicitly imposing the restriction $n = m'$.

Computational model. Our computational model assumes that any real number can be stored in constant space, and that the multiplication or addition of two reals can be performed in constant time. When a data structure is said to be *random access*, we mean that an arbitrary entry from it can be retrieved in constant time.

Differential privacy. For a mechanism to be considered differentially private, we require that the output distributions of any two neighboring inputs, $D \sim D'$, are indistinguishable.

Definition B.3 ((ϵ, δ) -Differential Privacy [DR14]). A randomized algorithm $\mathcal{M} : \mathcal{X}^n \rightarrow \mathcal{Y}$ is (ϵ, δ) -differentially private if for all $S \subseteq \text{Range}(\mathcal{M})$ and all neighboring inputs $\mathcal{D}, \mathcal{D}' \in \mathcal{X}^n$, the neighboring relation written as $\mathcal{D} \sim \mathcal{D}'$, we have that:

$$\Pr[\mathcal{M}(\mathcal{D}) \in S] \leq \exp(\epsilon) \Pr[\mathcal{M}(\mathcal{D}') \in S] + \delta.$$

We refer to $(\epsilon, 0)$ -differential privacy as ϵ -differential privacy, or *pure* differential privacy.

When releasing a real-valued function $f(\mathcal{D})$ taking values in $\mathbb{R}^d$, we can achieve (ϵ, δ) -DP by adding Gaussian noise scaled to the ℓ_2 -sensitivity of f .

Lemma B.4 (Gaussian Mechanism [DR14]). Let $\epsilon \in (0, 1), \delta \in (0, 1)$, and $f : \mathcal{X}^n \rightarrow \mathbb{R}^d$ be a function with ℓ_2 -sensitivity $\Delta := \max_{\mathcal{D} \sim \mathcal{D}'} \|f(\mathcal{D}) - f(\mathcal{D}')\|_2$. For a given data set $\mathcal{D} \in \mathcal{X}^n$, the mechanism that releases $f(\mathcal{D}) + \mathcal{N}(0, C_{\epsilon, \delta} \Delta^2)^d$ satisfies (ϵ, δ) -DP where $C_{\epsilon, \delta} = 2 \ln(1.25/\delta)/\epsilon^2$.

Factorization mechanisms.

Most known mechanisms for continual counting (at least all such that produce unbiased outputs) can be expressed in the *factorization mechanism* framework [LMH⁺15], also known as the *matrix mechanism* framework. The general formulation describes the setting where the input is $\mathbf{x} \in \mathbb{R}^n$, for some neighbouring relation “ $\sim$ ”, and we want to privately release linear queries $f(\mathbf{x}) = A\mathbf{x} \in \mathbb{R}^q$ on $\mathbf{x}$, where $A \in \mathbb{R}^{q \times n}$.

Given a factorization $L \in \mathbb{R}^{q \times m}$, $R \in \mathbb{R}^{m \times n}$ such that $A = LR$, the framework states that the mechanism $\mathcal{M}(\mathbf{x}) = A\mathbf{x} + L\mathbf{z}$ where $\mathbf{z}$ is drawn from $\mathcal{N}(0, C_{\delta, \varepsilon} \Delta^2)^m$, $\Delta = \max_{\mathbf{x} \sim \mathbf{x}'} \|R(\mathbf{x} - \mathbf{x}')\|_2$, and $C_{\varepsilon, \delta}$ is a constant depending on ε, δ , satisfies (ε, δ) -DP. The intuition behind the mechanism is that we encode our input $\mathbf{x}$ as $R\mathbf{x}$, release it privately as $R\mathbf{x} + \mathbf{z}$ using the Gaussian mechanism, and then output $\mathcal{M}(\mathbf{x}) = L(R\mathbf{x} + \mathbf{z}) = A\mathbf{x} + L\mathbf{z}$ as post-processing. Notably, each coordinate of the output $\mathcal{M}(\mathbf{x})$ is in expectation equal to $A\mathbf{x}$, i.e., the mechanism is unbiased.

Applied to our setting, let $A \in \mathbb{R}^{T \times T}$ be the lower-triangular all-1s matrix and let the input vectors $\mathbf{x}, \mathbf{x}' \in \{0, 1\}^T$ be considered neighboring if $\mathbf{x}$ is equal to $\mathbf{x}'$ up to a bit flip at one unique coordinate. Any factorization L, R where $A = LR$ thus constitutes a mechanism for continual counting with $\Delta = \|R\|_{1 \rightarrow 2}$, where $\|\cdot\|_{1 \rightarrow 2}$ denotes the maximum ℓ_2 -norm of all columns in a matrix.

There are two common error metrics for continual counting: *mean squared error* (MSE) and *maximum squared error* (MaxSE). The mean squared error, is given as

$$\begin{aligned} \text{MeanSE}(\mathcal{M}) &= \frac{1}{T} \mathbb{E}_{\mathcal{M}} [\|\mathcal{M}(\mathbf{x}) - A\mathbf{x}\|_2^2] \\ &= \frac{1}{T} \mathbb{E}_{\mathbf{z}} [\|L\mathbf{z}\|_2^2] = \frac{C_{\varepsilon, \delta}}{T} \|L\|_F^2 \|R\|_{1 \rightarrow 2}^2 \end{aligned}$$

where $\|\cdot\|_F$ is the Frobenius norm. Analogously for the maximum (expected) squared error, we get

$$\begin{aligned} \text{MaxSE}(\mathcal{M}) &= \max_i \mathbb{E}_{\mathcal{M}} [(\mathcal{M}(\mathbf{x}) - A\mathbf{x})_i^2] \\ &= \max_i \mathbb{E}_{\mathbf{z}} [(L\mathbf{z})_i^2] = C_{\varepsilon, \delta} \|L\|_{2 \rightarrow \infty}^2 \|R\|_{1 \rightarrow 2}^2 \end{aligned}$$

where $\|\cdot\|_{2 \rightarrow \infty}$ denotes the maximum ℓ_2 -norm of all *rows* in the matrix. Intuitively, since $\mathcal{M}$ is unbiased, these two error metrics correspond to the *average variance* and *maximum variance* over T outputs. For notational convenience, we will overload this notation and use $\text{MeanSE}(L, R)$ and $\text{MaxSE}(L, R)$ for the error of a matrix mechanism built on a factorization L, R .

(Almost) optimal factorizations. For (ε, δ) -DP, it is known [HUU23] that choosing $L = R = \sqrt{A}$ gives a continual counting mechanism with optimal mean squared error up to lower-order vanishing terms across all factorizations. The same factorization also improved the best known *maximum expected squared error* [FHU23]. For this error, it was recently shown to be optimal across all *lower-triangular Toeplitz* factorizations [DMP⁺24]. We call the corresponding matrix $B = \sqrt{A}$ the *Bennett matrix* [Ben77]. Given these strong error bounds, choosing the factorization $L = R = B$ would seem to be the obvious choice in practical settings.

Space matters. As identified in [DMR⁺22; DMP⁺24] and elsewhere, the space efficiency of the counting mechanisms has a large impact for large-scale applications in private learning. For a given stream $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$ we need to add $(L\mathbf{z})_t$ at time t to the output to achieve privacy. Importantly, the memory needed to compute $\mathcal{M}(\mathbf{x})_t$ will therefore directly depend on the structure of L . If $L = I \in \mathbb{R}^{T \times T}$, the identity matrix, then the memory needed is constant, as each entry $\mathbf{z}_t$ is used once at time t and then never again. If L and R correspond to the factorization of the *binary mechanism* [DNP⁺10b; CSS11], then the needed memory will be $O(\log T)$. Unfortunately, in the particular case of $L = R = B$, the structure of L seems to require $\Omega(T)$ space.

This begs the question: can we find a factorization where $L \approx B$, $R \approx B$ and which runs in sublinear space? In particular, can we approximate its mean and maximum squared error?

B.3 Approximation by binning

Given a factorization $LR = A$ where L is lower-triangular and invertible, we are looking to approximate it, producing a new factorization $\hat{L}\hat{R} = A$ where $\text{MeanSE}(\hat{L}, \hat{R}) \approx \text{MeanSE}(L, R)$, $\text{MaxSE}(\hat{L}, \hat{R}) \approx \text{MaxSE}(L, R)$, and $\hat{L}$ has space complexity $o(T)$. We will approach the problem by expressing $\hat{L}$ as the sum of L , and a perturbation to each entry P . The big picture idea is that, for some class of L , perturbing its elements a little might be sufficient to achieve sublinear space. Furthermore, if the perturbation P is small enough and preserves the invertibility of L , then the requisite norms of $\hat{L}$ and $\hat{R} = \hat{L}^{-1}A$ will be close to those of L and R . We address this problem next.

Small perturbations preserve our norms of interest

We start by formalizing the intuition that sufficiently small perturbations preserve norms.

Definition B.5. For matrices $L, P \in \mathbb{R}^{n \times n}$, we call $L+P$ an (η, μ) -perturbation of L if given $\eta, \mu \geq 0$, we have that $|P_{i,j}| \leq \eta |L_{i,j}| + \mu$ for all $i, j \in [n]$.

Lemma B.6. Let $L, P \in \mathbb{R}^{n \times n}$ and $L+P$ be an (η, μ) -perturbation of L . Then $\|L+P\|_F \leq (1+\eta)\|L\|_F + \mu n$ and $\|L+P\|_{2 \rightarrow \infty} \leq (1+\eta)\|L\|_{2 \rightarrow \infty} + \mu\sqrt{n}$.

Proof. We have that

$$\begin{aligned} \|L+P\|_F &\leq \|L\|_F + \|P\|_F \leq \|L\|_F + \|\eta |L| + \mu J\|_F \\ &\leq (1+\eta)\|L\|_F + \mu\|J\|_F = (1+\eta)\|L\|_F + \mu n, \end{aligned}$$

where $J \in \mathbb{R}^{n \times n}$ is the all-1s matrix, proving the statement. The first and last inequalities are cases of the triangle inequality, whereas the second step uses the definition of a (η, μ) -perturbation and the fact that Frobenius norm only

depends on the absolute value of each entry. The statement for $\|L + P\|_{2 \rightarrow \infty}$ follows from an identical argument, except that $\|J\|_{2 \rightarrow \infty} = \sqrt{n}$. $\square$

Unsurprisingly, a small perturbation of L does not affect the norms we are interested in by much. To show that $\hat{L} = L + P$ induces a factorization $\hat{L}\hat{R} = LR$ with error comparable to LR requires more work. We start by bounding $\|\hat{R}\|_{1 \rightarrow 2}$.

Lemma B.7. *Let L, R, P be matrices where L and $L + P$ are invertible, and $\|L^{-1}\|_2 \cdot \|P\|_2 \leq 1/2$. Suppose that $\hat{R} = (L + P)^{-1}LR$. Then $\|\hat{R}\|_{1 \rightarrow 2} \leq (1 + 2\|P\|_2 \cdot \|L^{-1}\|_2)\|R\|_{1 \rightarrow 2}$.*

Proof. We begin by upper and lower bounding $\|L(\hat{R} - R)\|_{1 \rightarrow 2}$. The following fact will be used for both bounds.

Fact B.8. *For any matrix product AB , we have that $\|AB\|_{1 \rightarrow 2} \leq \|A\|_2 \|B\|_{1 \rightarrow 2}$. If furthermore A is invertible, then $\|AB\|_{1 \rightarrow 2} \geq \|B\|_{1 \rightarrow 2} / \|A^{-1}\|_2$.*

To see why Fact B.8 holds we begin with the lower bound for invertible A . Note that the standard inequality $\|A\mathbf{x}\|_2 \leq \|A\|_2 \|\mathbf{x}\|_2$ also implies $\|A\mathbf{x}\|_2 \geq \|\mathbf{x}\|_2 / \|A^{-1}\|_2$ via

$$\|\mathbf{x}\|_2 = \|A^{-1}A\mathbf{x}\|_2 \leq \|A^{-1}\|_2 \|A\mathbf{x}\|_2.$$

We can thus derive a lower bound as follows

$$\|AB\|_{1 \rightarrow 2} = \sup_{\|\mathbf{x}\|_1=1} \|AB\mathbf{x}\|_2 \geq \sup_{\|\mathbf{x}\|_1=1} \frac{\|B\mathbf{x}\|_2}{\|A^{-1}\|_2} = \frac{\|B\|_{1 \rightarrow 2}}{\|A^{-1}\|_2},$$

The upper bound is shown in an analogous way, concluding the proof of Fact B.8.

For the upper bound on $\|L(\hat{R} - R)\|_{1 \rightarrow 2}$, note that

$$L(\hat{R} - R) = L\hat{R} - A = (L - \hat{L})\hat{R} = -P\hat{R},$$

and thus $\|L(\hat{R} - R)\|_{1 \rightarrow 2} \leq \|P\|_2 \|\hat{R}\|_{1 \rightarrow 2}$. For the lower bound, we directly derive

$$\|L(\hat{R} - R)\|_{1 \rightarrow 2} \geq \|\hat{R} - R\|_{1 \rightarrow 2} / \|L^{-1}\|_2.$$

Combining the two, we get $\|\hat{R} - R\|_{1 \rightarrow 2} \leq \|P\|_2 \cdot \|L^{-1}\|_2 \cdot \|\hat{R}\|_{1 \rightarrow 2}$. Applying the triangle inequality we can further get $\|\hat{R} - R\|_{1 \rightarrow 2} \geq \|\hat{R}\|_{1 \rightarrow 2} - \|R\|_{1 \rightarrow 2}$, which combined with the preceding inequality and some re-arranging yields

$$(1 - \|P\|_2 \cdot \|L^{-1}\|_2) \|\hat{R}\|_{1 \rightarrow 2} \leq \|R\|_{1 \rightarrow 2}.$$

By our assumption $(1 - \|P\|_2 \cdot \|L^{-1}\|_2) \geq 1/2$, and so we can divide by it, resulting in

$$\begin{aligned} \|\hat{R}\|_{1 \rightarrow 2} &\leq \|R\|_{1 \rightarrow 2} / (1 - \|P\|_2 \cdot \|L^{-1}\|_2) \\ &\leq (1 + 2\|P\|_2 \cdot \|L^{-1}\|_2) \|R\|_{1 \rightarrow 2}, \end{aligned}$$

where the last step uses that $\frac{1}{1-x} \leq 1 + 2x$ for $0 \leq x \leq 1/2$. This concludes the argument. $\square$

Intuitively, if the perturbation P is “sufficiently small” compared to L , then the sensitivity of our new factorization $\hat{L}\hat{R}$ is bounded by that of LR . To this end, our next lemma shows that, if we restrict the left factor matrix L to be non-negative, then we can bound $\|P\|_2$ using $\|L\|_2$.

Lemma B.9. *Let $L \in \mathbb{R}_{\geq 0}^{n \times n}$, $P \in \mathbb{R}^{n \times n}$ and $L + P$ be an (η, μ) -perturbation of L . Then $\|P\|_2 \leq \eta\|L\|_2 + \mu n$.*

Proof. Let $U = \eta L + \mu J$ where $J \in \mathbb{R}^{n \times n}$ is the all-1s matrix. We have that $\|P\|_2 \leq \| |P| \|_2$, since for any $\mathbf{v} \in \mathbb{R}^n$, $|P|$ applied to $|\mathbf{v}|$ will always yield an equal or greater norm than $P\mathbf{v}$. Similarly, we also have that $\| |P| \|_2 \leq \|U\|_2$, since (1) any unit vector $\mathbf{v}$ maximizing $\| |P| \mathbf{v} \|_2$, will have the same sign in each entry, and (2) $|P_{i,j}| \leq U_{i,j}$ by definition of the (η, μ) -perturbation. Taken together this gives us:

$$\|P\|_2 \leq \|U\|_2 \leq \eta\|L\|_2 + \mu\|J\|_2 = \eta\|L\|_2 + \mu n. \quad \square$$

We are now ready to state our main lemma for perturbed factorizations.

Lemma B.10. *Let LR be a factorization, and let $\hat{L}$ be an (η, μ) -perturbation of $L \in \mathbb{R}_{\geq 0}^{n \times n}$ for $\eta = \frac{\xi}{144\kappa(L)}$ and $\mu = \frac{\xi\|L\|_2}{144n\kappa(L)}$ where $0 \leq \xi \leq 24$. Also let L and $\hat{L}$ be invertible, and define $\hat{R} = \hat{L}^{-1}LR$. Then $\text{MeanSE}(\hat{L}, \hat{R}) \leq (1 + \xi) \text{MeanSE}(L, R)$ and $\text{MaxSE}(\hat{L}, \hat{R}) \leq (1 + \xi) \text{MaxSE}(L, R)$.*

Proof. We start by proving the statement for the mean squared error. Assume $\eta \leq \frac{1}{6\kappa(L)}$ and $\mu \leq \frac{\|L\|_2}{6n\kappa(L)}$. First, note that Theorem B.6 gives a bound on $\|\hat{L}\|_F$. Secondly, invoking Theorem B.9 we have that $\|P\|_2 \leq \eta\|L\|_2 + \mu n \leq \frac{1}{3\|L^{-1}\|_2}$ by Theorem B.5 and our assumptions on η, μ . This implies $\|P\|_2\|L^{-1}\|_2 \leq 1/2$ and allows us to use Theorem B.7 for a bound on $\|\hat{R}\|_{1 \rightarrow 2}$. For the mean squared error, we thus have

$$\begin{aligned} \frac{\text{MeanSE}(\hat{L}, \hat{R})}{\text{MeanSE}(L, R)} &= \frac{\|\hat{L}\|_F^2 \|\hat{R}\|_{1 \rightarrow 2}^2}{\|L\|_F^2 \|R\|_{1 \rightarrow 2}^2} \leq \left(1 + \eta + \frac{\mu n}{\|L\|_F}\right)^2 \left(1 + 2\|P\|_2\|L^{-1}\|_2\right)^2 \\ &\leq \left(1 + \eta + \frac{\mu n}{\|L\|_F}\right)^2 (1 + 2(\eta\|L\|_2 + \mu n)\|L^{-1}\|_2)^2 \\ &= (1 + \eta)^2 \left(1 + \frac{\mu n}{(1 + \eta)\|L\|_F}\right)^2 (1 + 2\eta\kappa(L))^2 \left(1 + \frac{2\mu n\|L^{-1}\|_2}{1 + 2\eta\kappa(L)}\right)^2 \\ &\leq (1 + \eta)^2 (1 + 2\eta\kappa(L))^2 (1 + 2\mu n\|L^{-1}\|_2)^2 \left(1 + \frac{\mu n}{\|L\|_F}\right)^2 \\ &\leq 1 + 24 \left(\eta(1 + 2\kappa(L)) + \frac{\mu n(1 + 2\|L\|_F\|L^{-1}\|_2)}{\|L\|_F} \right) \\ &\leq 1 + 72 \left(\eta\kappa(L) + \mu n\|L^{-1}\|_2 \right) = 1 + \xi \leq 25. \end{aligned}$$

The first inequality is using Theorem B.6 and Theorem B.7, the second inequality Theorem B.9. The fourth inequality is using the fact that each factor is of the form $(1 + \omega)^2$ where $\omega \leq 1/3$, in which case $(1 + \omega)^2 \leq 1 + 3\omega$, and in general that $\prod_{i \in [n]} (1 + \omega_i)^2 \leq \prod_{i \in [n]} (1 + 3\omega_i) \leq 1 + 3 \cdot 2^{n-1} \sum_{i \in [n]} \omega_i$ if all $\omega_i \leq 1/3$. Each value ω_i can be confirmed to be less than or equal to $1/3$ by inspection based on our assumptions on η, μ . For the two non-immediate cases, note that $1 = \|I\|_2 = \|L^{-1}L\|_2 \leq \|L^{-1}\|_2 \|L\|_2 = \kappa(L) \leq \|L^{-1}\|_2 \|L\|_F$ and thus our assumptions imply $\eta \leq 1/6$ and $\mu \leq \frac{\|L\|_F}{6n}$. The same matrix norm inequality is used in the second to last inequality. It follows that choosing $\eta = \frac{\xi}{144\kappa(L)}, \mu = \frac{\xi}{144n\|L^{-1}\|_2} = \frac{\xi\|L\|_2}{144n\kappa(L)}$ for any $\xi \leq 24$ gives a $(1 + \xi)$ -approximation of the error.

For the maximum squared error, the same argument can be used with the exception that we are bounding $\|L\|_{2 \rightarrow \infty}$ at the first step.

$$\begin{aligned} \frac{\text{MaxSE}(\hat{L}, \hat{R})}{\text{MaxSE}(L, R)} &= \frac{\|\hat{L}\|_{2 \rightarrow \infty}^2 \|\hat{R}\|_{1 \rightarrow 2}^2}{\|L\|_{2 \rightarrow \infty}^2 \|R\|_{1 \rightarrow 2}^2} \\ &\leq \left(1 + \eta + \frac{\mu\sqrt{n}}{\|L\|_{2 \rightarrow \infty}}\right)^2 \left(1 + 2\|P\|_2 \|L^{-1}\|_2\right)^2 \\ &\leq \left(1 + \eta + \frac{\mu n}{\|L\|_F}\right)^2 \left(1 + 2\|P\|_2 \|L^{-1}\|_2\right)^2, \end{aligned}$$

where the last step uses that $\|L\|_F \leq \sqrt{n}\|L\|_{2 \rightarrow \infty}$. From this point all manipulations are identical to the previous case, and so we can derive the same guarantee. $\square$

If we allow for the matrix L to have negative values, then we can get a slightly weaker statement. Although we do not consider this class of matrices L for our applications, we state it as a separate lemma as it may be of independent interest, but only sketch a proof.

Lemma B.11. *Let LR be a factorization, and let $\hat{L}$ be an (η, μ) -perturbation of $L \in \mathbb{R}^{n \times n}$ for $\eta = \frac{\xi}{144\|L\|_F\|L^{-1}\|_2}$ and $\mu = \frac{\xi\|L\|_2}{144n\kappa(L)}$ where $0 \leq \xi \leq 24$. Also let L and $\hat{L}$ be invertible, and define $\hat{R} = \hat{L}^{-1}LR$. Then $\text{MeanSE}(\hat{L}, \hat{R}) \leq (1 + \xi) \text{MeanSE}(L, R)$ and $\text{MaxSE}(\hat{L}, \hat{R}) \leq (1 + \xi) \text{MaxSE}(L, R)$.*

Proof sketch. The proof is almost identical to that of Theorem B.10. We need an equivalent version of Theorem B.9 where instead $\|P\|_2 \leq \eta\|L\|_F + \mu n$, where the relaxation of norm is a result of the operator norm being sensitive to the sign of matrix entries (unlike the Frobenius norm). We can derive such a bound from the proof of Theorem B.6 and noting that $\|L + P\|_2 \leq \|L\|_F + \|P\|_2 \leq \|L\|_F + \|P\|_F$. This looser bound on $\|P\|_2$ translates into a stricter bound on η , but otherwise the structure of the proof is the same.

Similarly to the other proof, we assume $\eta \leq \frac{1}{6\|L\|_F\|L^{-1}\|_2}$ and $\mu \leq \frac{\|L\|_2}{6n\kappa(L)}$. We have that $\|P\|_2 \leq \|P\|_F \leq \eta\|L\|_F + \mu n \leq \frac{1}{3\|L^{-1}\|_2} < \frac{1}{2\|L^{-1}\|_2}$. From this

point on, we can repeat the steps in the proof of Theorem B.10 to bound the multiplicative blow-up in error. $\square$

At this point we have derived conditions for when a small perturbation of L into $\hat{L}$ induces a factorization $\hat{L}\hat{R}$ with good error. A separate, perhaps surprising point, is that the lemmas we have derived make no assumptions on the structure of R . No bounds on any norm of R is necessary, and R need not even be a square matrix. Only the left factor L determines the error bounds.

Our next point is to characterize when a left factor matrix is amenable to running in low space for streaming.

Binned matrices are space (and time) efficient

To the best of our knowledge, there is no universal definition for space efficiency in the context of real-valued vector inputs. In the case where L is a Toeplitz matrix with entries on its subdiagonals $(d_0, d_1, \dots, d_n)$, [DMP⁺24] identified that if these entries satisfy a recursion $d_k = \sum_{j=1}^s \lambda_j \cdot d_{k-j}$, for $k \geq s$, then $L\mathbf{z}$ can be computed in space $O(s)$. They called matrices meeting this definition *Buffered Linear Toeplitz matrices*. Our class of space-efficient matrices, *binned matrices* is distinct from theirs—focusing instead on matrices with few unique entries per row—and in particular it includes non-Toeplitz matrices.

To introduce our class of matrices we will rely on *partitions of intervals*.

Definition B.12. Let $\mathcal{I}$ be the set of all intervals on $[1, \infty)$ and $a \leq b$ be positive integers. We then refer to $C_{[a,b]} \subset \mathcal{I}$ as a *partition of the interval* $[a, b]$ if (1) $\bigcup_{I \in C_{[a,b]}} I = [a, b]$, and, (2) there are no overlapping intervals in $C_{[a,b]}$, i.e., $\forall I, I' \in C_{[a,b]}, I \neq I' : I \cap I' = \emptyset$. We let $(C_{[a,b]})_i$ represent the i^{th} interval in the partition when ordered by their left end-point.

In particular, we will be interested in sequences of partitions, *binning*s, formally defined next.

Definition B.13. Let n be a positive integer and consider the sequence of partitions $\mathcal{B} = (C_{[1]}, C_{[2]}, \dots, C_{[n]})$ where $\mathcal{B}^i = C_{[i]}$. We call $\mathcal{B}$ a *binning of* $[n]$ if, for each $i \in [n-1]$, we have that for each $I \in \mathcal{B}^i$, there exists an $I' \in \mathcal{B}^{i+1}$ such that $I \subseteq I'$. We define the size of a binning, $|\mathcal{B}|$, to be the largest size of all its partitions, i.e. $|\mathcal{B}| = \max_{i \in [n]} |\mathcal{B}^i|$.

An important property of binning is that, given a binning $\mathcal{B}$ of $[n]$, then for every $i \in [n-1]$, $\mathcal{B}^{i+1}$ can be constructed by merging the intervals in $\mathcal{B}^i \cup [i+1, i+1]$. It turns out that this property, enables space-efficiency when applied to lower-triangular matrices.

Definition B.14 ($\mathcal{B}$ -binned matrices). Let $L \in \mathbb{R}^{n \times n}$ be a lower-triangular matrix, and let $\mathcal{B}$ be a binning of $[n]$. If for each $i \in [n]$, we have that for all $I \in \mathcal{B}^i$, $L_{i,j}$ assumes the same value for all $j \in I$, then we say L is a $\mathcal{B}$ -binned matrix.

Intuitively, a lower-triangular matrix L is $\mathcal{B}$ -binned if on any row i it is constant on intervals as described by $\mathcal{B}^i$. This fact also implies that a $\mathcal{B}$ -binned matrix can always be stored in space $O(n \cdot |\mathcal{B}|)$. By definition, being a binned matrix is not a strong condition, as the trivial binning $\mathcal{B}$ where $|\mathcal{B}^i| = i$ holds for any lower-triangular matrix. It turns out to be a meaningful concept nonetheless, as the size of the binning is linked to space and time complexity.

Lemma B.15. *A $\mathcal{B}$ -binned matrix L has space complexity $\leq |\mathcal{B}|$.*

Lemma B.16. *Consider a $\mathcal{B}$ -binned matrix $L \in \mathbb{R}^{T \times T}$ and $\mathbf{z} \in \mathbb{R}^n$. Then $L\mathbf{z}$ can be computed incrementally in time $O(|\mathcal{B}|)$ per output if, at the time of outputting $(L\mathbf{z})_i$, we have that:*

1. *The partitions $\mathcal{B}^{i-1}$ (for $i > 1$) and $\mathcal{B}^i$ are accessible and stored in sorted order.*
2. *We have random access to the i^{th} row of L .*

To prove Lemmas B.15 and B.16, we begin by proving intermediate results for how the computation $L\mathbf{z}$ can be performed efficiently using an internal state of size $|\mathcal{B}|$.

Remark B.17. Let $\mathcal{B}$ be a binning of $[n]$ and $\mathbf{z} \in \mathbb{R}^n$. For any $i \in [n-1]$, define $\mathbf{b}^i \in \mathbb{R}^{|\mathcal{B}|}$ where $\mathbf{b}_k^i = \sum_{j \in \mathcal{B}_k^i} \mathbf{z}_j$. Then, given only $\mathcal{B}^i$ and $\mathcal{B}^{i+1}$ in sorted order, vector $\mathbf{b}^i$ and scalar $\mathbf{z}_{i+1}$, we can compute $\mathbf{b}^{i+1}$ in time $O(|\mathcal{B}|)$. Furthermore, $\mathbf{b}^{i+1}$ can be computed in-place using the memory buffer of $\mathbf{b}^i$.

Proof. Since there is a bijection between $\mathbf{b}_k^i$ and a unique interval $I = \mathcal{B}_k^i$, we use the notation $\mathbf{b}_I^i$ when convenient. For every $I \in \mathcal{B}^{i+1}$ where $i+1 \notin I$, we have by the definition of binnings that there exists a unique set of disjoint intervals $S \subseteq \mathcal{B}^i$ such that $\bigcup_{I' \in S} I' = I$. In particular this allows us to write $\mathbf{b}_I^{i+1} = \sum_{j \in I} \mathbf{z}_j = \sum_{I' \in S} \mathbf{b}_{I'}^i$. By the same argument, there must exist a set $S \subseteq \mathcal{B}^i$ such that for the unique interval $I \in \mathcal{B}^{i+1}$ containing $i+1$, we can express it as $\mathbf{b}_I^{i+1} = \mathbf{z}_{i+1} + \sum_{I' \in S} \mathbf{b}_{I'}^i$. The computation is therefore possible.

For the efficiency of this computation, note further that any entry of $\mathbf{b}^{i+1}$ is a range sum on $\mathbf{b}^i$, with the exception of the entry $\mathbf{b}_{|\mathcal{B}^{i+1}|}^{i+1}$ that is a (potentially empty) range sum on $\mathbf{b}^i$ plus $\mathbf{z}_{i+1}$. These ranges are disjoint by the definition of a binning. It is also the case that $\mathbf{b}_k^{i+1}$ will only be updated using entries $\mathbf{b}_{k'}^i$, where $k' \geq k$, due to how intervals in $\mathcal{B}^{i+1}$ are the result of merging intervals in $\mathcal{B}^i$. It follows that we can update $\mathbf{b}^i$ in-place into $\mathbf{b}^{i+1}$ by updating the entries in ascending order. It can also be done efficiently by maintaining two pointers for $\mathbf{b}^i$, one for reading and one for writing (the reading pointer always behind or at the same position as the writing pointer), and two reading pointers for $\mathcal{B}^i$ and $\mathcal{B}^{i+1}$. The update requires scanning each of these structures once and performing $O(1)$ constant time operations each time a pointer is moved. Each structure containing $O(|\mathcal{B}|)$ elements thus implies the time complexity. $\square$

Remark B.18. Using the setup in Theorem B.17, consider the output sequence $\mathbf{b}^1, \dots, \mathbf{b}^n \in \mathbb{R}^{|\mathcal{B}|}$. If at the time of updating $\mathbf{b}^i$ into $\mathbf{b}^{i+1}$ we have access to $\mathcal{B}^i$ and $\mathcal{B}^{i+1}$ in sorted order, we can compute the sequence in-place and in time $O(|\mathcal{B}|)$ per output.

Proof. The statement follows almost immediately from Theorem B.17 and induction. For the base case, $\mathbf{b}^1 \in \mathbb{R}^{|\mathcal{B}|}$ has a single non-zero entry equaling $\mathbf{z}_1$, and so can be computed in time $O(|\mathcal{B}|)$. Next, for the induction step, assume we have already computed $\mathbf{b}^i$ and have it stored in memory. By our assumption we have access to $\mathcal{B}^i$ and $\mathcal{B}^{i+1}$, allowing us to invoke Theorem B.17 to compute $\mathbf{b}^{i+1}$ using $\mathbf{b}^i$ in-place and in time $O(|\mathcal{B}|)$. Invoking induction thus proves the claim. $\square$

We are now ready to prove Lemmas B.15 and B.16.

Proof of Theorem B.15. Given $\mathbf{b}^i$, as defined in Theorem B.17, and L , we have enough information to output $(L\mathbf{z})_i$. This follows from L being constant on the intervals given in $\mathcal{B}$, meaning we can write $(L\mathbf{z})_i = \sum_{I \in \mathcal{B}^i} L_{i,I} \mathbf{b}_I^i$. Since $\mathcal{B}$ can be read directly from L , we can therefore invoke Theorem B.18, proving that an internal state of size $|\mathcal{B}|$ is sufficient for computing $L\mathbf{z}$. $\square$

Proof of Theorem B.16. Analogously to the proof of Theorem B.16, note that the i^{th} row of L and $\mathbf{b}^i$ is sufficient to output $(L\mathbf{z})_i$. This computation, from $\mathbf{b}^i$ to $(L\mathbf{z})_i$, requires $O(|\mathcal{B}|)$ additions, multiplications, and random accesses into the i^{th} row of L —all constant time operations. Since the necessary partitions in $\mathcal{B}$ are provided to update $\mathbf{b}^i$ into $\mathbf{b}^{i+1}$ at each step, by the assumption in the lemma, we can also invoke Theorem B.18. We thus have that the internal state can be updated in time $O(|\mathcal{B}|)$, and that the state can be mapped to an output in time $O(|\mathcal{B}|)$, proving our lemma. $\square$

Perturbing matrices into binned matrices

While binned matrices are nice, the issue is that most matrices do not admit very interesting binnings. The Bennett matrix for example only admits the trivial binning. Our intention is therefore to approximate a matrix L by a matrix $\hat{L}$ that has a non-trivial binning, but is still “close” to L .

Definition B.19. A lower-triangular matrix $L \in \mathbb{R}^{n \times n}$ is said to be $\mathcal{B}$ -approximated by a $\mathcal{B}$ -binned matrix $\hat{L} \in \mathbb{R}^{n \times n}$ if for every $1 \leq j \leq i \leq n$, we have that $\hat{L}_{i,j} = \frac{L_{i,a} + L_{i,b}}{2}$ where $[a, b] \in \mathcal{B}^i$ and $j \in [a, b]$.

Intuitively, $\hat{L}$ is the result of superimposing the binning on the matrix L , and setting the entries of $\hat{L}$ to be equal to the average of the entries in L at the end of each interval. Note that any other rule for how to set the entries (including mixing rules across intervals) would also result in a $\mathcal{B}$ -binned matrix. A more natural rule than what is proposed in Theorem B.19 would have been

Algorithm 3 Our binning procedure

```

1: Input: Matrix  $L \in \mathbb{R}_{\geq 0}^{n \times n}$ , positive scalars  $c, \tau < 1$ 
2: Initialize sets  $\mathcal{B}^0, \mathcal{B}^1, \dots, \mathcal{B}^n \leftarrow \emptyset$ 
3: for  $i = 1$  to  $n$  do
4:    $k \leftarrow |\mathcal{B}^{i-1}|$ 
5:   while  $k > 0$  do
6:      $[a, b] \leftarrow \mathcal{B}_k^{i-1}, k' \leftarrow k$ 
7:     if  $k > 1$  and  $L_{i,b+1} > 0$  and  $L_{i,a}/L_{i,b+1} > c$  then
8:        $[a', b'] \leftarrow \mathcal{B}_{k'-1}^{i-1}$ 
9:       while  $k' > 1$  and  $L_{i,a'}/L_{i,b+1} \geq c^2$  do
10:         $a \leftarrow a', k' \leftarrow k' - 1$ 
11:         $[a', b'] \leftarrow \mathcal{B}_{k'-1}^{i-1}$ 
12:       end while
13:       if  $k' = 1$  and  $L_{i,a'}/L_{i,b+1} \geq c^2$  then
14:         $a \leftarrow a'$ 
15:       end if
16:     end if
17:     if  $L_{i,b} \leq \tau$  then
18:       Add  $[1, b]$  to  $\mathcal{B}^i, k \leftarrow 0$ 
19:     else
20:       Add  $[a, b]$  to  $\mathcal{B}^i, k \leftarrow k' - 1$ 
21:     end if
22:   end while
23:   Add  $[i, i]$  to  $\mathcal{B}^i$ 
24:   output  $\mathcal{B}^i$ 
25: end for

```

to choose $\hat{L}$ to be the *average over all entries in each interval*, since this would minimize $\|\hat{L} - L\|_F$. This choice would however necessitate querying $\Omega(n)$ entries in L to produce a row in $\hat{L}$. Our chosen rule approximates the average instead using $O(|\mathcal{B}|)$ entries. This will prove important later when considering the time complexity of computing $\hat{L}\mathbf{z}$.

Note that Theorem B.19 only specifies how a matrix can be approximated given a binning $\mathcal{B}$, but it says nothing about how to find a $\mathcal{B}$ that performs well. Our next result is an algorithm, given as Algorithm 3, that produces a binning with good guarantees for a certain class of matrices. We describe the core idea of the algorithm next. Given a lower-triangular matrix L , and positive constants $c, \tau < 1$, it produces a binning $\mathcal{B}$ parameterized in c, τ . It tries to maintain that for any $i \in [n]$ and $[a, b] \in \mathcal{B}^i$, $b < i$, we have that $c^2 \leq \frac{L_{i,a}}{L_{i,b+1}} \leq c$, with the possible exception of a single (possibly large) interval containing only entries less than or equal to τ . It achieves this by greedily merging intervals from one row to the next.

Specifically, given the partition of row i , $\mathcal{B}^i$, it constructs the next row's partition, $\mathcal{B}^{i+1}$ by greedily merging intervals in the partition $\mathcal{B}^i$, and then adding the interval $[i+1, i+1]$ to the result. Iterating over the intervals in $\mathcal{B}^i$, in order of decreasing column indices, it greedily merges two intervals $[a, b]$, $[b+1, d]$ whenever $\frac{L_{i+1,b+1}}{L_{i+1,d+1}} > c$ and the resulting interval $[a, d]$ satisfies $\frac{L_{i+1,a}}{L_{i+1,d+1}} \geq c^2$. As long as the result is still $\geq c^2$, it continues merging with new intervals. Once a candidate interval $[a, b]$ with $L_{i,b} \leq \tau$ is encountered, it merges all remaining intervals in $\mathcal{B}^i$ into a single interval $[1, b]$. Since the definition of a binning can be expressed in terms of merging intervals in $\mathcal{B}^i \cup \{[i+1, i+1]\}$ to produce $\mathcal{B}^{i+1}$, the procedure will always produce a binning, potentially the trivial binning.

We begin by showing that, under an assumption of random access to entries in L , the $\mathcal{B}$ -approximation of L derived from running Algorithm 3, $\hat{L}$, has good time complexity.

Lemma B.20. *Consider a lower-triangular matrix and let $\hat{L}$ be the $\mathcal{B}$ -approximation of L where $\mathcal{B}$ is the output from running Algorithm 3 on L . Given $\mathbf{z} \in \mathbb{R}^n$ and random access to entries in L , then $\hat{L}\mathbf{z}$ can be computed row-by-row, reading one element of $\mathbf{z}$ at a time, in time $O(|\mathcal{B}|)$ per output.*

Proof. Our approach will rely on computing $\mathcal{B}$ and $\hat{L}\mathbf{z}$ in parallel. First observe that the for-loop in Algorithm 3 runs in time $O(|\mathcal{B}|)$. This can be seen by noting that any given interval in $\mathcal{B}^{i-1}$ appears at most twice inside the outermost while-loop: once as a possible candidate for merging into on line 11, and/or as the interval to start merging from on line 8. Since all operations inside the for-loop can be made constant time operations (accessing an entry of L is constant time due to the random access assumption) the time to output $\mathcal{B}^i$ given $\mathcal{B}^{i-1}$ is $O(|\mathcal{B}|)$.

Keeping both $\mathcal{B}^{i-1}$ and $\mathcal{B}^i$ in memory when outputting $(\hat{L}\mathbf{z})_i$ almost allows us to invoke Theorem B.16 to prove the statement, but we still need random access to the i^{th} row of $\hat{L}$. We can simulate this random access by choosing our approximation rule in Theorem B.19 to only depend on a constant number of elements in L . In particular, picking the stated rule where for $[a, b] \in \mathcal{B}^i$: $\hat{L}_{i,[a,b]} = \frac{L_{i,a} + L_{i,b}}{2}$ works, concluding the proof. $\square$

At this point, we have shown that $\mathcal{B}$ -binned matrices derived from running Algorithm 3 have space and time complexity linear in the binning size. We show next that, for a certain class of matrices, Algorithm 3 can produce binnings achieving a good trade-off between binning size and error when applied to the left factor in a factorization.

Definition B.21 (MRMs). We say that an invertible lower-triangular matrix $L \in \mathbb{R}^{n \times n}$ is a *monotone ratio* matrix (*MRM* for short) if it satisfies that

1. All entries $0 < L_{i,j} \leq 1$ for all $i, j \in [n]$ with $i \leq j$.²
2. For all $i \in [n]$ the function $g(x) = L_{i,x}$ is non-decreasing for $x \in [i]$.
3. For all $j, j' \in [n]$ where $j \leq j'$ the function $h(x) = \frac{L_{x,j}}{L_{x,j'}}$ is non-decreasing for $x \in [j', n]$.

When an MRM L is the left factor of a factorization LR , we call the factorization an *MRM factorization*.

Before stating how our main algorithm performs on these matrices, we briefly note that the requirement that the entries of the matrix are less than or equal to 1 is without loss of generality when applied to factorizations: For every scalar $a > 0$, $LR = (\frac{1}{a}L)(aR) = L'R'$ yields a new factorization with all entries re-scaled.

Lemma B.22. *Executing Algorithm 3 on an MRM $L \in \mathbb{R}^{n \times n}$ with positive $c, \tau < 1$ returns a $\mathcal{B}$ with $|\mathcal{B}| = O\left(\frac{\log(1/\tau)}{\log(1/c)}\right)$.*

Proof. We will argue for an arbitrary row index $i \in [n]$. Note that the two left-most intervals in $\mathcal{B}^i$ are the only possible intervals $I \in \mathcal{B}^i$ where there could be $j \in I : L_{i,j} \leq \tau$ due to the condition on line 17. We will argue that the remaining intervals cannot be too many. Define $S \subseteq \mathcal{B}^i$ as $\mathcal{B}^i$ with these unique intervals removed if they exist, where now for all $[a, b] \in S : \tau < L_{i,a} \leq 1$. For any two contiguous intervals $[a, b], [b+1, \ell] \in S$, at least one of the two ratios $\frac{L_{i,a}}{L_{i,b+1}}$ and $\frac{L_{i,b+1}}{L_{i,\ell+1}}$ must be $\leq c$. To prove this, assume to the contrary that both are $> c$. Then the merged interval $[a, \ell]$ would have a ratio of $\frac{L_{i,a}}{L_{i,\ell+1}} = \frac{L_{i,a}}{L_{i,b+1}} \cdot \frac{L_{i,b+1}}{L_{i,\ell+1}} \geq c^2$. Since both intervals have a ratio $> c$ and merging them would yield a ratio $\geq c^2$, these intervals would have gotten merged during the execution of Algorithm 3, proving the claim by contradiction. We thus have the property that for any row index i , at least $1/2 - o(1)$ of these intervals in S satisfy having a ratio $\leq c$. Since L is an MRM, we have that entries on each row of L are non-decreasing with the column index, therefore the ratio can only decrease by a factor c at most $O(\log_{1/c}(1/\tau)) = O(\frac{\log(1/\tau)}{\log(1/c)})$ times, implying $|\mathcal{B}| \leq |S| + 2 = O(\frac{\log(1/\tau)}{\log(1/c)})$. $\square$

Having shown that the size of the binning can be parameterized in c, τ , we prove next that the induced matrix approximation is a $(1/c^2 - 1, \tau)$ -perturbation of the original one.

Lemma B.23. *Consider an MRM L and let $\hat{L}$ be the $\mathcal{B}$ -approximation of L where $\mathcal{B}$ is the output from running Algorithm 3 on L with positive $c, \tau < 1$. Then $\hat{L}$ is invertible and a $(1/c^2 - 1, \tau)$ -perturbation of L .*

²All results for MRMs would still hold if entries are allowed to be zero, but at the cost of a more convoluted property (3).

Proof. We start by arguing for the case where $\tau < \min_{i \geq j} L_{i,j}$. In this case the condition on line 17 is never true, and so all intervals produced are solely based on c . $\mathcal{B}$ then satisfies that: for all $i \in [n]$ and $[a, b] \in \mathcal{B}^i$, $\frac{L_{i,a}}{L_{i,b}} \geq c^2$. For $a = b$, the statement is immediate, whereas for $a < b$ we note that the interval must have been the result of a merge at some row index $i' \leq i$. At the time of the merge, it must have satisfied $c^2 \leq \frac{L_{i',a}}{L_{i',b+1}}$. Combining the second and third property of Theorem B.21, we therefore have that $c^2 \leq \frac{L_{i',a}}{L_{i',b+1}} \leq \frac{L_{i,a}}{L_{i,b+1}} \leq \frac{L_{i,a}}{L_{i,b}}$, proving the statement in general.

We next wish to prove that for all $i, j \in [n]$ we have $c^2 L_{i,j} \leq \hat{L}_{i,j} \leq \frac{1}{c^2} L_{i,j}$. For $i < j$, the statement is trivial as $L_{i,j} = \hat{L}_{i,j} = 0$, so consider $i \geq j$. Assuming $j \in [a, b] \in \mathcal{B}^i$, then we have that $\hat{L}_{i,j} = \frac{L_{i,a} + L_{i,b}}{2}$ and thus $c^2 \leq \frac{L_{i,a}}{L_{i,b}} \leq \frac{\hat{L}_{i,j}}{L_{i,j}}$ due to the monotonicity of the rows of L (Property 2 in Theorem B.21). For the second inequality we get $\frac{1}{c^2} \geq \frac{L_{i,b}}{L_{i,a}} \geq \frac{\hat{L}_{i,j}}{L_{i,j}}$ by an identical argument. Letting $P = \hat{L} - L$, we therefore have that for every $i, j \in [n]$ that $P_{i,j} \leq (1/c^2 - 1)L_{i,j}$ and $P_{i,j} \geq (c^2 - 1)L_{i,j}$, thus we get that $|P_{i,j}| \leq \max(1/c^2 - 1, 1 - c^2)|L_{i,j}| = (1/c^2 - 1)|L_{i,j}|$.

Now consider the case where $\tau \geq \min_{i \geq j} L_{i,j}$ and line 17 evaluates to true for a row $i \in [n]$. In this case every interval except the left-most one of the form $[1, b]$ is guaranteed to satisfy the same condition. However, since $\hat{L}_{i,[1,b]}$ will be set to the average of two positive values less than or equal to τ , we instead get that $|P_{i,j}| = |\hat{L}_{i,j} - L_{i,j}| \leq \tau$ for $j \in [1, b]$. Thus P satisfies $|P_{i,j}| \leq (1/c^2 - 1)|L_{i,j}| + \tau$ for every $i, j \in [n]$.

Lastly, as invertibility of a lower-triangular matrix is decided by its diagonal, we also note that $\hat{L}$ is invertible, since the $\mathcal{B}$ produced by Algorithm 3 satisfies $[i, i] \in \mathcal{B}^i$, and therefore $\hat{L}_{i,i} = L_{i,i}$. $\square$

Lemma B.24. *Consider the MRM factorization LR and let $\hat{L}$ be the $\mathcal{B}$ -approximation of $L \in \mathbb{R}^{n \times n}$ where $\mathcal{B}$ is the output from running Algorithm 3 on L for $c = \exp(-\frac{\xi}{576\kappa(L)})$ and $\tau = \frac{\xi\|L\|_2}{144n\kappa(L)}$ for $0 < \xi \leq 24$. Then $\hat{L}$ induces a factorization $\hat{L}\hat{R} = LR$ with error guarantees $\text{MeanSE}(\hat{L}, \hat{R}) \leq (1 + \xi) \text{MeanSE}(L, R)$ and $\text{MaxSE}(\hat{L}, \hat{R}) \leq (1 + \xi) \text{MaxSE}(L, R)$.*

Proof. From Theorem B.23, we have that $\hat{L}$ is a $(1/c^2 - 1, \tau)$ -perturbation of L . The statement to prove follows immediately from Theorem B.10 if we show that the conditions imposed on $1/c^2 - 1$ and τ are stricter or equal those from Theorem B.10. μ is set here to the same value as in that lemma, so the only consideration is η . Letting $c = \exp(-1/s)$, we would like to set s such that $1/c^2 - 1 = \exp(2/s) - 1 \leq 4/s = \eta = \frac{\xi}{144\kappa(L)}$, where the inequality is true if $s \geq 2$. Solving for s in the last step, we get $s = \frac{576\kappa(L)}{\xi} \geq \frac{576 \cdot 1}{24} = 24$, and so the inequality is valid. Since we set τ equal to μ , and $1/c^2 - 1$ smaller than or

equal to the η in Theorem B.10, we are guaranteed the same approximation factor. $\square$

We are now ready to state the guarantees of our binning procedure when applied to MRMs.

Theorem B.25. *Consider the MRM factorization LR and let $\hat{L}$ be the $\mathcal{B}$ -approximation of $L \in \mathbb{R}^{n \times n}$ where $\mathcal{B}$ is the output from running Algorithm 3 on L for $c = \exp(-\frac{\xi}{576\kappa(L)})$ and $\tau = \frac{\xi\|L\|_2}{144n\kappa(L)}$ for $0 < \xi \leq 24$. Then $|\mathcal{B}| = O_\xi(\kappa(L) \log(n\|L^{-1}\|_2))$ and $\hat{L}$ induces a factorization $\hat{L}\hat{R} = LR$ where:*

- $\hat{L}\hat{R}$ satisfies $\text{MeanSE}(\hat{L}, \hat{R}) \leq (1 + \xi) \text{MeanSE}(L, R)$ and $\text{MaxSE}(\hat{L}, \hat{R}) \leq (1 + \xi) \text{MaxSE}(L, R)$.
- $\hat{L}$ has space complexity at most $|\mathcal{B}|$.
- Given $\mathbf{z} \in \mathbb{R}^n$ and random access to L , $\hat{L}\mathbf{z}$ can be computed incrementally in time $O(|\mathcal{B}|)$ per output.

Proof. Using the setup in Theorem B.24 and then invoking Theorem B.22 for the given values of c and τ gives $|\mathcal{B}| = O(\kappa(L) \log(n\|L^{-1}\|_2/\xi)/\xi)$ for the error guarantees. This can then be translated into the statements on space and time complexity by invoking Theorem B.15 and Theorem B.20 respectively. $\square$

We reiterate a previous point from Theorem B.10: the guarantees in Theorem B.25 are *independent* of R . The right factor R can be chosen arbitrarily as long as it has n rows. It is only properties of the left factor L that decide the approximation's guarantees.

Applications to square-root factorizations for counting

While we have now shown that our approach for binning works well for certain factorizations, we show next that this class includes the square-root factorization for the counting matrix. Calling back to applications, the counting matrix receives as much attention as it does, in part, due to its role in facilitating private learning. We note that n steps of weight updates in gradient descent with *momentum* and *weight decay* can be expressed as

$$\mathbf{w}^{(i)} = \alpha \mathbf{w}^{(i-1)} - \eta \mathbf{m}^{(i)} \quad \text{given} \quad \mathbf{m}^{(i)} = \beta \mathbf{m}^{(i-1)} + \mathbf{g}^{(i)},$$

where $\mathbf{w}^{(0)} = \mathbf{0}, \mathbf{w}^{(1)}, \dots, \mathbf{w}^{(n)} \in \mathbb{R}^d$ are weight vectors, $\mathbf{g}^{(1)}, \dots, \mathbf{g}^{(n)} \in \mathbb{R}^d$ are gradient updates, $\eta > 0$ is the learning rate, $0 \leq \beta < 1$ is the momentum strength, and $0 < \alpha \leq 1$ is the weight decay. Unrolling the recursion, we can express the weight updates in matrix form as:

$$W = -\eta \cdot A_{\alpha, \beta} G \quad \text{where} \quad W = \begin{bmatrix} (\mathbf{w}^{(1)})^T \\ \vdots \\ (\mathbf{w}^{(n)})^T \end{bmatrix}, \quad G = \begin{bmatrix} (\mathbf{g}^{(1)})^T \\ \vdots \\ (\mathbf{g}^{(n)})^T \end{bmatrix},$$

and $A_{\alpha,\beta} \in \mathbb{R}^{n \times n}$ is a lower-triangular Toeplitz matrix with subdiagonals $a_0, \dots, a_{n-1}$ of the form $a_k = \frac{\alpha^{k+1} - \beta^{k+1}}{\alpha - \beta}$. A factorization mechanisms for $A_{\alpha,\beta}$ thus implies a primitive for private learning (privatizing weight updates). We restrict α and β to satisfy $0 \leq \beta < \alpha \leq 1$ with the motivation that these ranges make sense when α, β are interpreted as weight decay and momentum respectively. In the particular case of $A_{\alpha=1, \beta=0} = A$, we recover the standard all-1s lower-triangular counting matrix.

It turns out that $A_{\alpha,\beta}$ has a closed form expression for its square-root for general α, β :

Fact B.26 (Theorem 1 and Lemma 6 in [KL24]). *Let $B_{\alpha,\beta} \in \mathbb{R}^{n \times n}$ where $A_{\alpha,\beta} = B_{\alpha,\beta}^2$. Then $B_{\alpha,\beta}$ is a lower-triangular Toeplitz matrix with nonincreasing subdiagonals $b_0, \dots, b_{n-1}$ where $b_j = \sum_{i=0}^j \alpha^{j-i} \gamma(j-i) \gamma(i) \beta^i$, $\gamma(k) = \frac{1}{4^k} \binom{2k}{k}$, and*

$$\frac{\alpha^j}{2\sqrt{j+1}} \leq b_j \leq \frac{\alpha^j}{(1 - \beta/\alpha)\sqrt{j+1}}.$$

The square-root factorization is interesting to study as it known to give asymptotically optimal error [KL24, Theorem 3 and 4]. In the particular case of the Bennett matrix $B = \sqrt{A} = B_{\alpha=1, \beta=0}$, it is well-studied [Ben77; FHU23; HUU23; DMP⁺24], and gives the optimal leading constant over all factorizations for both mean and maximum squared error. The only issue with the factorization $L = R = B_{\alpha,\beta}$ is that the best known bound on the space complexity of $B_{\alpha,\beta}$ is $\Omega(n)$, for all choices of α, β . We show next that our binning approach can approximate the square-root factorization with space complexity $o(n)$, and begin by proving that $B_{\alpha,\beta}$ is an MRM.

Fact B.27. $B_{\alpha,\beta} \in \mathbb{R}^{n \times n}$ is an MRM.

Proof. $B_{\alpha,\beta}$ trivially satisfies the first two properties in Theorem B.21 by noticing that $b_j = \Theta_{\alpha,\beta}(\alpha^j / \sqrt{j})$ according to Theorem B.26. What remains to show is that it has a monotone ratio. For all $j, j' \in [n]$, where $j < j'$, we consider the function $h(x) = \frac{(B_{\alpha,\beta})_{x,j}}{(B_{\alpha,\beta})_{x,j'}} = \frac{b_{x-j}}{b_{x-j'}} = \Theta_{\alpha,\beta}(\alpha^{j-j'} \sqrt{\frac{x-j'}{x-j}}) = \Theta_{\alpha,\beta}(\alpha^{j-j'} \sqrt{1 + \frac{j-j'}{x-j}})$ on $x \in [j', n]$. Since $j < j'$, $j - j'$ is negative and thus $h(x)$ is increasing with x , proving the last property. $\square$

To derive the quality of our approximation, we need to upper bound $\|B_{\alpha,\beta}\|_2$ and $\|B_{\alpha,\beta}^{-1}\|_2$. To this end, we will use the following lemma for the operator norm of lower-triangular Toeplitz matrices.

Lemma B.28. *Let $Q \in \mathbb{R}^{n \times n}$ be a lower-triangular Toeplitz matrix with subdiagonals $q_0, \dots, q_{n-1}$. Then $\|Q\|_2 \leq \sum_{i=0}^{n-1} |q_i|$.*

Proof. Let $Q^{(i)}$ denote the matrix which is zero everywhere except on the i^{th} sub-diagonal where it equals q_i . We therefore have that, $\|Q\|_2 = \|\sum_{i=0}^{n-1} Q^{(i)}\|_2 \leq$

$\sum_{i=0}^{n-1} \|Q^{(i)}\|_2 = \sum_{i=0}^{n-1} |q_i|$, where the first inequality is repeatedly using the triangle inequality, and the last equality comes from identifying that a matrix consisting of only a subdiagonal of 1s has an operator norm of 1. $\square$

For the bound on $\|B_{\alpha,\beta}^{-1}\|_2$, we will also need the following bound.

Lemma B.29. *For integer $k \geq 0$, define $\gamma'(k)$ as the function where $\gamma'(0) = 1$ and $\gamma'(k) = \gamma(k) - \gamma(k-1)$ for $k \geq 1$. Then for $k \geq 1$,*

$$-\frac{1}{\sqrt{\pi k}(2k-1)} \leq \gamma'(k) \leq -\frac{1}{2\sqrt{k}(2k-1)}.$$

Proof. Note that $\gamma(k) - \gamma(k-1) = \frac{1}{4^k} \left(\binom{2k}{k} - \binom{2k-2}{k-1} \right) = \frac{\binom{2k}{k}}{4^k} \left(1 - \frac{4k^2}{2k(2k-1)} \right) = -\frac{1}{2k-1} \gamma(k)$. Invoking known bounds on $\gamma : \frac{1}{2\sqrt{k}} \leq \gamma(k) \leq \frac{1}{\sqrt{\pi k}}$ [KL24, Lemma 5], proves the claim. $\square$

We are now ready to prove the operator norm bounds.

Lemma B.30. *For $0 \leq \beta < \alpha \leq 1$ and integer $n > 1$, $B_{\alpha,\beta} \in \mathbb{R}^{n \times n}$ has an inverse satisfying $\|B_{\alpha,\beta}^{-1}\|_2 \leq \frac{1}{1-\beta/\alpha} C$, where C is a constant independent of α, β and n .*

Proof. Define $E_\omega \in \mathbb{R}^{n \times n}$, $0 < \omega \leq 1$ to be the lower-triangular Toeplitz matrix with entries $(E_\omega)_{i,j} = \omega^{i-j}$ for $i \geq j$, with inverse E_ω^{-1} that is a matrix with 1s on the main diagonal and $-\omega$ on the subdiagonal immediately below. Since $A_{\alpha,\beta} = E_\alpha E_\beta$ (by inspection), we can write $\|B_{\alpha,\beta}^{-1}\|_2 = \|A_{\alpha,\beta}^{-1} B_{\alpha,\beta}\|_2 = \|E_\beta^{-1} E_\alpha^{-1} B_{\alpha,\beta}\|_2$. Noting that $E_\alpha^{-1} B_{\alpha,\beta} = Q$ is a Toeplitz matrix with 1s on the main diagonal and subdiagonals $q_k = b_k - \alpha b_{k-1}$, we can write

$$\begin{aligned} q_k &= b_k - \alpha b_{k-1} \\ &= \sum_{i=0}^k \alpha^{k-i} \gamma(k-i) \gamma(i) \beta^i - \sum_{i=0}^{k-1} \alpha^{k-i} \gamma(k-1-i) \gamma(i) \beta^i \\ &= \beta^k \gamma(k) + \sum_{i=0}^{k-1} \alpha^{k-i} \gamma(i) (\gamma(k-i) - \gamma(k-i-1)) \beta^i \\ &= \beta^k \gamma(k) + \sum_{i=0}^{k-1} \alpha^{k-i} \gamma(i) \gamma'(k-i) \beta^i \\ &= \sum_{i=0}^k \alpha^{k-i} \gamma(i) \gamma'(k-i) \beta^i. \end{aligned}$$

Repeating the manipulation, we have that $S = E_\beta^{-1}Q$ is another Toeplitz matrix with 1s on the main diagonal and subdiagonals $s_k = q_k - \beta q_{k-1}$, where

$$\begin{aligned}
s_k &= q_k - \beta q_{k-1} \\
&= \sum_{i=0}^k \alpha^{k-i} \gamma(i) \gamma'(k-i) \beta^i \\
&\quad - \sum_{i=0}^{k-1} \alpha^{k-i-1} \gamma(i) \gamma'(k-i-1) \beta^{i+1} \\
&= (\alpha^k + \beta^k) \gamma'(k) + \sum_{i=1}^{k-1} \alpha^{k-i} \gamma'(i) \gamma'(k-i) \beta^i \\
&= \alpha^k \sum_{i=0}^k \left(\frac{\beta}{\alpha} \right)^i \gamma'(i) \gamma'(k-i) .
\end{aligned}$$

In the end we are interested in bounding $|s_k|$, and so we want to upper bound $f(i) = |\gamma'(i) \gamma'(k-i)|$ for $i = 0, \dots, k$. Noting that $f(i) = f(k-i)$ and $|\gamma'(i)| = \Theta(i^{-3/2})$ (Theorem B.29), we can bound $f(i)$ as $f(i) \leq f(0) = |\gamma'(0) \gamma'(k)| = |\gamma'(k)|$. Continuing, we therefore have

$$|s_k| \leq \alpha^k |\gamma'(k)| \sum_{i=0}^k \left(\frac{\beta}{\alpha} \right)^i \leq \frac{\alpha^k |\gamma'(k)|}{1 - \beta/\alpha}.$$

Noting that $S = B_{\alpha,\beta}^{-1}$, we apply Theorem B.28 to S :

$$\|S\|_2 \leq \sum_{k=0}^{n-1} |s_k| \leq \frac{1}{1 - \beta/\alpha} \left(1 + \sum_{k=1}^{n-1} \frac{\alpha^k}{\sqrt{\pi k(2k-1)}} \right),$$

where the second inequality uses Theorem B.29. For all valid choices of α, β the final sum can be shown to converge to a constant via e.g. bounding $\alpha \leq 1$ and using an integral approximation, proving the statement. $\square$

Lemma B.31. *For $0 \leq \beta < \alpha \leq 1$ and integer $n > 1$, $B_{\alpha,\beta} \in \mathbb{R}^{n \times n}$ satisfies $\|B_{\alpha,\beta}\|_2 \leq \frac{2\sqrt{n-1}}{1-\beta}$ for $\alpha = 1$, and otherwise $\|B_{\alpha,\beta}\|_2 \leq \frac{1}{(1-\beta/\alpha)(1-\alpha)}$.*

Proof. Invoking Theorem B.28 we get that

$$\|B_{\alpha,\beta}\|_2 \leq \sum_{k=0}^{n-1} |b_k| \leq \frac{1}{1 - \beta/\alpha} \sum_{k=0}^{n-1} \frac{\alpha^k}{\sqrt{k+1}}$$

where the bound on b_k comes from Theorem B.26. For $\alpha = 1$ we have

$$\frac{1}{\sqrt{k+1}} \leq \int_k^{k+1} \frac{1}{\sqrt{x}} dx$$

for $k > 0$ and thus

$$\begin{aligned}\|B_{\alpha,\beta}\|_2 &\leq \frac{1}{1-\beta} \left(1 + \int_1^n \frac{1}{\sqrt{x}} dx\right) \\ &= \frac{1}{1-\beta} (1 + 2\sqrt{n} - 2) = \frac{2\sqrt{n} - 1}{1-\beta}.\end{aligned}$$

In the case where $\alpha < 1$ we can write

$$\|B_{\alpha,\beta}\|_2 \leq \frac{1}{1-\beta/\alpha} \sum_{k=0}^{n-1} \alpha^k \leq \frac{1}{(1-\beta/\alpha)(1-\alpha)} . \quad \square$$

We can now state our approximation guarantee.

Theorem B.32. *Let $0 \leq \beta < \alpha \leq 1$ and $\xi \in (0, 1)$. Further, let $\hat{L}$ be the $\mathcal{B}$ -approximation of $B_{\alpha,\beta} \in \mathbb{R}^{n \times n}$ derived from running Algorithm 3 with parameters c and τ . Then there exists c, τ for which*

$$|\mathcal{B}| = \begin{cases} O_{\alpha,\beta,\xi}(\sqrt{n} \log n) & \text{if } \alpha = 1, \\ O_{\alpha,\beta,\xi}(\log n) & \text{otherwise,} \end{cases}$$

and $\hat{L}$ induces a factorization $\hat{L}\hat{R} = B_{\alpha,\beta}^2$ where:

- $\text{MeanSE}(\hat{L}, \hat{R}) \leq (1 + \xi) \text{MeanSE}(B_{\alpha,\beta}, B_{\alpha,\beta})$ and $\text{MaxSE}(\hat{L}, \hat{R}) \leq (1 + \xi) \text{MaxSE}(B_{\alpha,\beta}, B_{\alpha,\beta})$.
- $\hat{L}$ has space complexity at most $|\mathcal{B}|$.
- Given random access to $B_{\alpha,\beta}$, and $z \in \mathbb{R}^n$, $\hat{L}z$ can be computed sequentially in time $O(|\mathcal{B}|)$ per output.

Proof. Noting that $B_{\alpha,\beta}$ is an MRM (Theorem B.27), the theorem follows immediately from invoking Theorem B.25 applied to the factorization $L = R = B_{\alpha,\beta}$, and using Theorem B.30 and B.31 for the bounds on the operator norms. For $\alpha = 1$ we arrive at a space complexity of $O\left(\frac{1}{(1-\beta)^2} \sqrt{n} \log\left(\frac{n}{\xi(1-\beta)}\right)/\xi\right)$ ³ and $O\left(\frac{1}{(1-\beta/\alpha)^2(1-\alpha)} \log\left(\frac{n}{\xi(1-\beta/\alpha)}\right)/\xi\right)$ for $\alpha < 1$. $\square$

Theorem B.2 is a special case of Theorem B.32 for $\alpha = 1, \beta = 0$.

³If $\alpha = 1$, then it is possible to remove the ξ inside of the logarithm by choosing τ smaller than the smallest entry in $B_{\alpha,\beta}$ and doing a finer analysis.

B.4 Empirical evaluation

While we have explicit bounds on the space and error of our approach when applied to counting matrices (with and without momentum and/or weight decay), the strength of our approach is most easily visible in our empirical results. For the canonical problem of continual counting (corresponding to $\alpha = 1, \beta = 0$) where the square-root factorization is asymptotically optimal up to lower order terms, *we outperform it in sublinear space for practical regimes of n for both mean squared and maximum squared error*. This is, to the best of our knowledge, the first result explicitly making this point. For the generalized counting problem including momentum and/or weight-decay, we approximate the square-root factorization of $A_{\alpha,\beta}$ with equal success.

Notes on implementation and hardware. We have implemented Algorithm 3 together with helper methods in Python which allows for efficient computation of factorizations $\hat{L}, \hat{R}$, the exact mean and maximum squared error of which is shown. Our implementation is available on <https://github.com/jodander/streaming-via-binning>. All plots shown have been produced on a Macbook Pro 2021 with Apple M1 Pro chip and 16 GB memory, running Python 3.9.6, with scipy version 1.9.2, pandas version 1.5.3, and numpy version 1.23.3. Although further optimization is possible, it can produce factorizations of the counting matrix $A \in \mathbb{R}^{T \times T}$ for T up to $\sim 10^4$ in a few seconds. The bottleneck of the experiments lies in storing all matrices involved as dense matrices. As pointed out earlier in connection with Theorem B.14, a $T \times T$ $\mathcal{B}$ -binned matrix can be stored in $O(T \cdot |\mathcal{B}|)$ memory, and the square-root factorizations we compare to can be stored in $O(n)$ memory due to their Toeplitz structure.

For all plots shown, we start from a factorization $L, R \in \mathbb{R}^{n \times n}$. We $\mathcal{B}$ -approximate L by $\hat{L}$ using the binning $\mathcal{B}$ from running Algorithm 3 on L for positive parameters $c, \tau < 1$, then compute $\hat{R} = \hat{L}^{-1}LR$. The $\mathcal{B}$ -approximation we use is the one from Theorem B.19, averaging based on the end points of the intervals. The space complexity of $\hat{L}$ will be equal to $|\mathcal{B}|$ and the error we report will be an exact computation based on $\hat{L}$ and $\hat{R}$, either $\text{MeanSE}(\hat{L}, \hat{R})$ or $\text{MaxSE}(\hat{L}, \hat{R})$ depending on the experiment. We make no use of any theoretical bounds.

Results for the Bennett matrix

To visualize the result, and showcase that our approach is meaningful already for small time horizons T , we give example factorizations in Figure B.2. The space usage of $\hat{L}$ is clearly visible in Figure B.2b, since each row can be identified to have at most 8 unique colors, i.e. intervals. As our technique relies on heuristics for perturbing a left-factor matrix into becoming space-efficient, and then bounding the blow-up of $\|\hat{R}\|_{1 \rightarrow 2}$, there is less to say about the structure in Figure B.2c. At most we can note qualitatively that $\hat{R}$ looks similar to B .

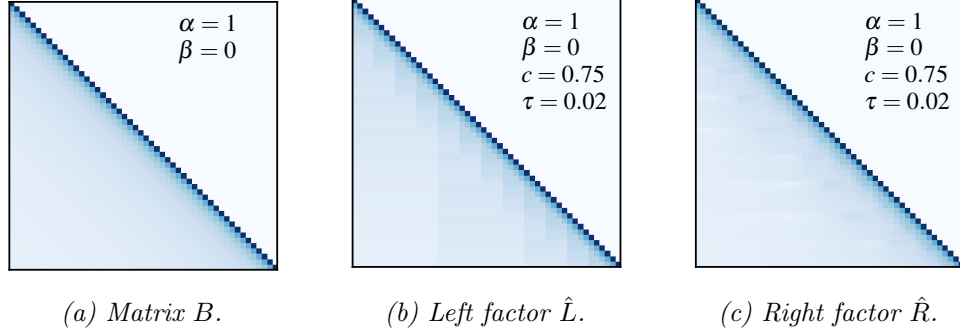


Figure B.2: Plot of entries of different factorizations of the counting matrix $A = A_{\alpha=1, \beta=0}$ for $T = 50$. Darker shades imply matrix entries closer to 1; white implies 0. Figure B.2a corresponds to the factorization $A = B^2$. $\hat{L}$ is the $\mathcal{B}$ -approximation of B generated from Algorithm 3 when run with $c = 0.75, \tau = 0.02$ on B , and $\hat{R} = \hat{L}^{-1}A$. For the matrices shown we have that $\text{MeanSE}(\hat{L}, \hat{R}) / \text{MeanSE}(B, B) = 0.9965$, $\text{MaxSE}(\hat{L}, \hat{R}) / \text{MaxSE}(B, B) = 0.9951$ and $|\mathcal{B}| = 8$.

What we can note quantitatively, and which is of greater interest, is the fact that our more space efficient construction has a *lower* mean and maximum squared error. We seem to be able to get the best of both worlds: lower error *and* space usage.

This is not a computational fluke. Simply put, the optimality results in [HUU23; DMP⁺24] show that $B^2 = A$ is *asymptotically optimal*, but on fixing T one can hope to improve lower order terms for the error, and, as we show, *even in sublinear space*. In the particular case of the maximum squared error, $B^2 = A$ is exactly optimal over Toeplitz factorizations [DMP⁺24], implying that our binned factorizations can achieve a lower error than any Toeplitz factorization of A . We go on to show that this fact is demonstrably true up to at least $T = 10^4$. The main takeaway from Figure B.3 is that our method indeed scales well for the region of T shown. If a relatively weak assumption on the blow-up in error is acceptable, then the empirical data in Fig. B.3a and B.3b is compatible with our method running in $O(\log T)$ space. We also wish to underline the fact that the plots do not change much whether one considers maximum or mean squared error. While this is also true for our bounds, it is not obvious that it should be carried out in practice given that our analysis is not believed to be tight.

Results with momentum and weight decay

To showcase the generality of our approach we also demonstrate it for the square-root factorization of the counting matrix $L = R = \sqrt{A_{\alpha, \beta}}$ that incorporates weight decay ($\alpha < 1$) and/or momentum ($\beta > 0$). First, in Figure B.4,

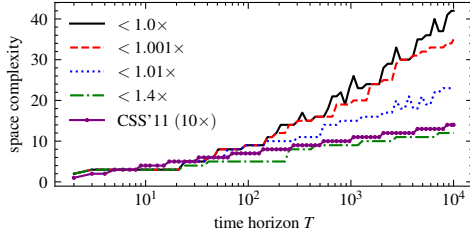
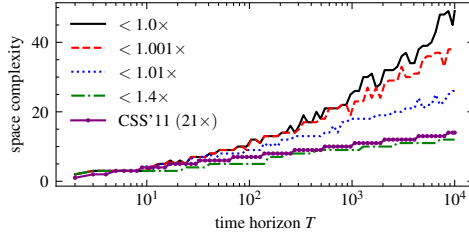
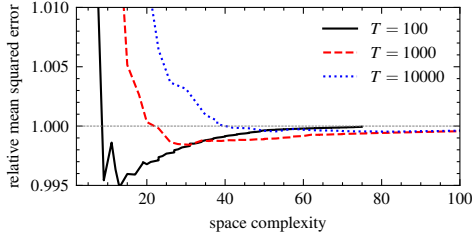
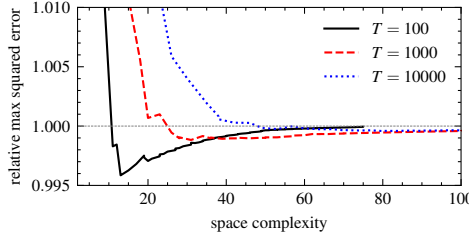
(a) Space complexity needed for fixed relative MeanSE vs. T .(b) Space complexity needed for fixed relative MaxSE vs. T .(c) Relative MeanSE vs. space complexity for different T .(d) Relative MaxSE vs. space complexity for different T .

Figure B.3: Plots showing the trade-off between space complexity and multiplicative blow-up in the mean and maximum squared error for our factorizations relative to [HUU23]. All factorizations shown here were produced by running Algorithm 3 with $c = 1 - 1/d$ for integers $d \geq 2$ and $\tau = 1/T$. In Figure B.3a and Figure B.3b, d was initialized to 2 for each T , and then incremented until the error was sufficiently small for each curve. In Figure B.3c and Figure B.3d the points are generated from using $1.1 \leq d \leq 100$. Figure B.3a and Figure B.3b shows the space complexity (binning size) needed by the approximation $\hat{L}$, as a function of T , to be within a given multiple of this error. The dashed lines show the space complexity needed to be no more than some small fraction above the error, whereas the black line shows space needed for our method to achieve a smaller error. The line with dots shows the space complexity of the binary mechanism in [CSS11], whose error is asymptotically worse by a factor of 10 or 21, for mean and maximum squared error respectively. Figure B.3c and Figure B.3d shows the trade-off between the blow-up in error versus space for different values of T .

we showcase what solutions for small instances look like when we introduce momentum and weight decay.

Qualitatively one can note that introducing momentum ($\beta > 0$) results in a matrix $B_{\alpha,\beta}$ with a mass that is more evenly distributed. Introducing weight decay ($\alpha < 1$) should intuitively have the opposite effect, where mass becomes more concentrated closer to the diagonal. For both cases we are able to reduce the space requirement from 50 to 8 buffers, but we are only able to

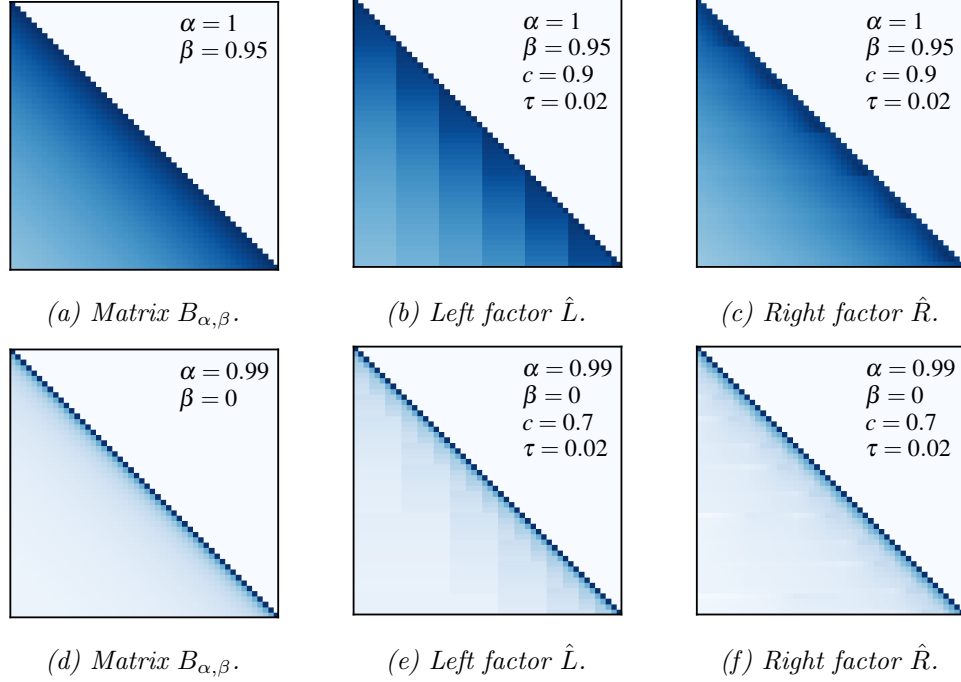


Figure B.4: Plot of entries of different factorizations of $A_{\alpha,\beta}$ for $n = 50$. Darker shades imply matrix entries closer to 1; white implies 0. Figure B.4a and Figure B.4d correspond to the square-root factorization $A_{\alpha,\beta} = B_{\alpha,\beta}^2$. $\hat{L}$ is the $\mathcal{B}$ -approximation of $B_{\alpha,\beta}$ generated from Algorithm 3 when run with c and τ as specified in the figures, on $B_{\alpha,\beta}$, and where $\hat{R} = \hat{L}^{-1}A_{\alpha,\beta}$. The factorization shown in Fig. B.4b and B.4c achieves $\text{MeanSE}(\hat{L}, \hat{R}) / \text{MeanSE}(B_{\alpha,\beta}, B_{\alpha,\beta}) = 0.9945$ and $\text{MaxSE}(\hat{L}, \hat{R}) / \text{MaxSE}(B_{\alpha,\beta}, B_{\alpha,\beta}) = 0.9947$, compared to a relative error of 1.015 and 1.026 respectively for the factorization in Fig. B.4e and B.4f. Both our factorizations have a binning size $|\mathcal{B}|$ of 8.

achieve an improvement in error for the case with momentum. This trend is replicated in Figure B.5, where we more systematically investigate the influence of α and β , and $\alpha < 1$ proves to be the harder case. The main points to make here is that our technique works well across these parameter choices of α, β , allowing us to substantially reduce the space at the cost of a constant blow-up in error. We note that, contrary to the case of the Bennett matrix where $\alpha = 1$, setting $\tau = 1/T$ has an impact when $\alpha < 1$. Since the entries of $B_{\alpha,\beta}$ decay exponentially away from the diagonal when $\alpha < 1$ (see Theorem B.26), there can be entries smaller than $\tau = 1/T$, all of which will be grouped into at most two different intervals per row. As in the case of $\alpha = 1, \beta = 0$, whether it is maximum or mean squared error has little impact – our approach is well-suited to both error variants.

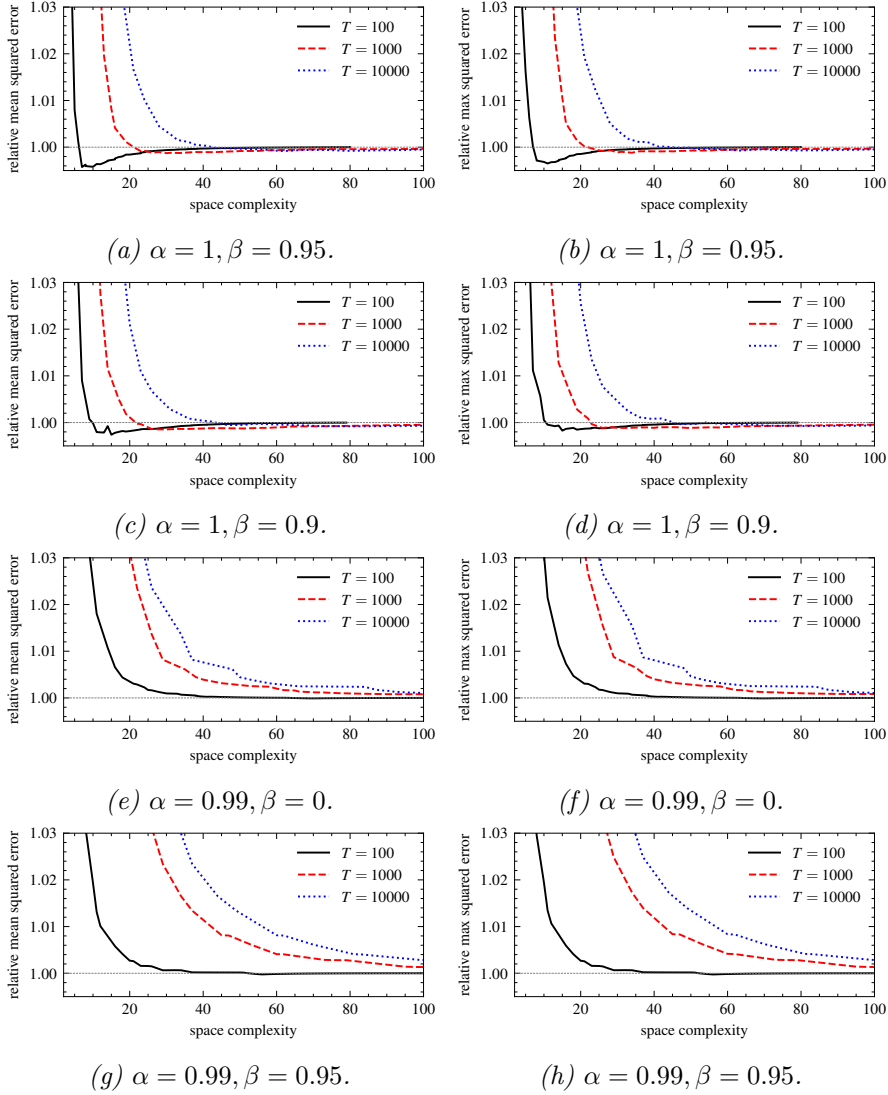


Figure B.5: Plots showing the trade-off between space complexity and multiplicative blow-up in the mean and maximum squared error for our factorizations relative to the square-root factorization of $A_{\alpha,\beta}$, for different α, β . All factorizations shown here were produced by running Algorithm 3 with $c = 1 - 1/d$ for $1.1 \leq d \leq 100$, and $\tau = 1/n$.

B.5 Conclusion and discussion

We have introduced a new framework for approximating factorization mechanisms based on a technique we call “binning”. We give an algorithm that always produces a valid binning, and we show that, for a class of factorizations LR , calling it with suitable parameters produces a new factorization $\hat{L}\hat{R}$ running in

sublinear space at the cost of a small multiplicative increase in error. This class includes the asymptotically optimal, up to lower order terms, factorization of the counting matrix $L = R = \sqrt{A}$ in [Ben77; FHU23; HUU23]. Empirically we show something stronger. For T at least up to 10^4 , we produce factorizations with *lower error in sublinear space*. For weaker multiplicative guarantees, our empirical results suggest that our approach leads to logarithmic space complexity. Beyond this, we also show that our approach works once momentum and weight decay are introduced to the counting matrix.

There are many possible directions for future work. The main question we leave is the space efficiency possible by binning: our theoretical bounds on space and error do not appear to be tight in general, as demonstrated by our empirical work. An approach relying on more global properties of the perturbation $P = \hat{L} - L$, rather than bounding each entry, could be a way forward. Another question worth pursuing is what other linear query matrices (and their factorizations) can be worth studying under binning. This is in particular relevant to non-Toeplitz matrices, where the techniques in [DMP⁺24] are not applicable. It would also be of interest to investigate whether the binnings produced by Algorithm 3 can be substantially improved.

Finally, we mention the *bit complexity* of space-efficient mechanisms that rely on real-valued noise. It seems straightforward to replace Gaussian noise by discrete Gaussian noise [CKS20] in our mechanism: Since we only add random samples together there are no numerical issues and we can get high accuracy with only $O(\log T)$ bits per value, assuming that the exact prefix sums can be represented in $O(\log T)$ bits. For other methods such as [DMP⁺24] it is less clear how many bits of precision are needed to maintain the privacy/utility trade-off.

Acknowledgments. The authors are affiliated with Basic Algorithms Research Copenhagen (BARC), supported by the VILLUM Foundation grant 54451, and are also supported by Providentia, a Data Science Distinguished Investigator grant from Novo Nordisk Fonden.

Paper C

Count on Your Elders

Abstract. In recent years, Gaussian noise has become a popular tool in differentially private algorithms, often replacing Laplace noise which dominated the early literature on differential privacy. Gaussian noise is the standard approach to *approximate* differential privacy, often resulting in much higher utility than traditional (pure) differential privacy mechanisms. In this paper we argue that Laplace noise may in fact be preferable to Gaussian noise in many settings, in particular when we seek to achieve (ϵ, δ) -differential privacy for small values of δ . We consider two scenarios:

First, we consider the problem of counting under continual observation and present a new generalization of the binary tree mechanism that uses a k -ary number system with *negative digits* to improve the privacy-accuracy trade-off. Our mechanism uses Laplace noise and whenever δ is sufficiently small it improves the mean squared error over the best possible (ϵ, δ) -differentially private factorization mechanisms based on Gaussian noise. Specifically, using $k = 19$ we get an asymptotic improvement over the bound given in the work by Henzinger, Upadhyay and Upadhyay (SODA 2023) when $\delta = O(T^{-0.92})$.

Second, we show that the noise added by the Gaussian mechanism can always be replaced by Laplace noise of comparable variance for the same (ϵ, δ) -differential privacy guarantee, and in fact for sufficiently small δ the variance of the Laplace noise becomes strictly better. This challenges the conventional wisdom that Gaussian noise should be used for high-dimensional noise.

Finally, we study if counting under continual observation is easier in an average-case sense than in a worst-case sense. We show that, under pure differential privacy, the worst-case error for a random input must be $\Omega(\log(T)/\epsilon)$, matching the known lower bound for worst-case inputs.

* Full title: *Count on Your Elders: Laplace vs Gaussian Noise*, Joel Daniel Andersson, Rasmus Pagh (BARC, University of Copenhagen), Teresa Anna Steiner (University of Southern Denmark), Sahel Torkamani (University of Edinburgh), FORC'25.

C.1 Introduction

In private data analysis the goal is to release the result of computations on datasets while safeguarding the privacy of the individuals whose data make up the dataset. A popular framework for achieving this is *differential privacy* [DMN⁺06], which gives probabilistic guarantees on how much can be learned about an individual’s data from the result of a computation. Algorithms providing differential privacy (DP) guarantees conceptually introduce *noise*, randomly perturbing the result of a corresponding exact, non-private computation. The trade-off between utility and privacy guarantees depends on the nature of the computation.

Let $\mathbf{x}, \mathbf{x}' \in \{0, 1\}^T$ be input datasets. We say $\mathbf{x}$ and $\mathbf{x}'$ are neighboring if they are equal except for a single bit. We want to compute all prefix sums of $\mathbf{x}$ with differential privacy under this neighboring relation, i.e., we want to privately release $A\mathbf{x}$ where A is the lower-triangular all 1s matrix. If we add the restriction that the bits $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$ are received one by one as a stream, and we are to output $(A\mathbf{x})_i$ on receiving $\mathbf{x}_i$, then this problem is referred to as *counting under continual observation* or *continual counting* for short [DNP⁺10b; CSS11].

The standard solution to this problem is the *factorization mechanism* [LMH⁺15], and it involves (1) factorizing A into matrices L, R such that $A = LR$, (2) privately releasing $R\mathbf{x}$ by adding noise $\mathbf{z}$ and then (3) multiplying the private release by L from the left: $L(R\mathbf{x} + \mathbf{z}) = A\mathbf{x} + L\mathbf{z}$. Defining the ℓ_p sensitivity as $\Delta_p = \max_{i \in [T]} \|R\mathbf{e}_i\|_p$, where $\mathbf{e}_i$ is the i^{th} unit basis vector, classical results on DP say that adding Laplace noise scaled to Δ_1 yields ε -DP (also called *pure DP*), and adding Gaussian noise scaled to Δ_2 yields (ε, δ) -DP (also called *approximate DP*). The scheme rests on the observation that while the sensitivity of A may be large, the sensitivity of R can be much smaller, and so adding noise $L\mathbf{z}$ to the final result may result in better accuracy than adding noise directly to $A\mathbf{x}$.

While continual counting is a deceptively simple problem, we note that it still has significant gaps between upper and lower bounds [DNP⁺10b; DNR⁺15; FHU23; HUU23; CLN⁺24]. The problem is used as a subroutine in various applications ranging from private learning [KMS⁺21; DMR⁺22; MT22; CMR⁺23; CGM⁺24] to histogram estimation [CLS⁺12; Upa19; CR22; HQY⁺22], so any improvement in upper bounds affect these applications.

Our contributions. Our first contribution is an easy-to-implement tree-aggregation-based mechanism for continual counting under ε -DP, and based on Laplace noise, with properties stated in Theorem C.1.

Theorem C.1. *Given a constant odd integer $k \geq 3$ and integer $T \geq 2$, there exists an ε -DP algorithm for continual counting that for a stream $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$*

achieves mean squared error $\frac{k(1-1/k^2)}{2\varepsilon^2 \log(k)^3} \cdot \log(T)^3 + o(\log(T)^3)$, computing all outputs in time $O(T)$ and space $O(\log T)$.

We denote the base k logarithm by $\log_k$, and define $\log(\cdot) = \log_2(\cdot)$. To the best of our knowledge, this is the best bound on the mean squared error known for any ε -DP mechanism, up to lower-order asymptotically vanishing terms. It improves the error achieved by applying techniques in [Hon15] to the binary tree mechanism (as implemented in [KMS⁺21] and referred to elsewhere as “Honaker Online”) by a factor 4 when $k = 19$. While this is interesting in its own right, we note that this improvement in the leading constant has implications for how the error compares to (ε, δ) -DP mechanisms. Henzinger, Upadhyay and Upadhyay [HUU23] produce a factorization with a bound on the mean squared error of $C_{\varepsilon, \delta}^2 \left(1 + \frac{\ln(4T/5)}{\pi}\right)^2$, where $C_{\varepsilon, \delta} = \frac{2}{\varepsilon} \sqrt{\frac{4}{9} + \ln\left(\frac{1}{\delta} \sqrt{\frac{2}{\pi}}\right)}$, and show that no factorization mechanism based on Gaussian noise can asymptotically improve on this error.¹ We improve on their error using Laplace noise for small enough δ .

Theorem C.2. *Given an integer $T \geq 2$, there exists an ε -DP algorithm for continual counting, based on Laplace noise, that improves on the mean squared error bound given in [HUU23], up to lower-order asymptotically vanishing terms, whenever $\delta = O(T^{-0.92})$.*

Given that δ is commonly set to be $o(1/T)$, this result is surprising. The phenomenon is discussed in Section C.5, but in a nutshell a sufficiently small δ must cause the error to greatly increase if the mechanism is based on Gaussian noise. While improving constants for ε -DP is of independent interest, Theorem C.2 implies that further improvements also have implications for (ε, δ) -DP.

Motivated by the dominance of Gaussian noise in applications, and the moral in Theorem C.2 for Laplace noise, our second contribution is (ε, δ) -DP guarantees for the Laplace mechanism for given ℓ_1 and ℓ_2 sensitivities.

Theorem C.3 (Approximate DP for the Laplace Mechanism). *Let $f : \mathcal{X}^n \rightarrow \mathbb{R}^d$ be a function with ℓ_p sensitivity $\Delta_p := \max_{\mathcal{D} \sim \mathcal{D}'} \|f(\mathcal{D}) - f(\mathcal{D}')\|_p$. For a given dataset $\mathcal{D} \in \mathcal{X}^n$, $\delta \in (0, 1)$ and $\lambda > \Delta_1$, the output $f(\mathcal{D}) + \text{Lap}(\lambda)^d$, satisfies (ε, δ) -differential privacy where*

$$\varepsilon = \min \left\{ \frac{\Delta_1}{\lambda}, \frac{\Delta_2}{\lambda} \left(\frac{\Delta_2}{2\lambda} + \sqrt{2 \ln(1/\delta)} \right) \right\}.$$

Essentially we can implement (ε, δ) -DP for the Laplace mechanism where the resulting ε is never worse than what simple composition gives, while also

¹As the constant $C_{\varepsilon, \delta}$ is valid for all $\varepsilon, \delta \in (0, 1)$, it seems plausible that it can be reduced via a tighter analysis of the Gaussian mechanism, as done in for example [BW18].

being competitive with the Gaussian mechanism in the regime where it is advantageous.

Finally, in Section C.6 we show the following lower bound, which matches the classical packing lower bound for continual observation under pure differential privacy [DNP⁺10b; DR14] in the case where the input is random.

Theorem C.4. *Let $\mathcal{U}$ be the distribution of strings of length T where every element in the string is drawn from $B(1/2)$ (Bernoulli distribution with $p = 1/2$). Let $\mathcal{M}$ be an ε -differentially private algorithm that solves the continual counting problem for strings from $\mathcal{U}$ with error at most α with probability at least $3/4$ (probability over random input and random choices of the algorithm). Then $\alpha = \Omega(\varepsilon^{-1} \log T)$.*

Overview of technical ideas

Improved tree-based mechanism. We start out by building a complete k -ary tree of height $h = \lceil \log_k(T + 1) \rceil$ on top of the input $\mathbf{x} \in \{0, 1\}^T$, see Figure C.1. To analyze this, it turns out that expressing a time step t as a k -ary number, i.e., as h digits in $\{0, \dots, k - 1\}$, i.e. $\mathbf{t} \in \{0, \dots, k - 1\}^h$, gives a direct means of computing which vertices contribute to a given prefix sum. Increasing the arity of the tree reduces the height, which reduces the ℓ_1 -sensitivity and levels of vertices that might get added, but increases the number of vertices to add per level. To mitigate the increase in number of vertices to add, we leverage *subtraction*, see Figure C.2.

To analyze the structure of the resulting prefix sums when subtraction is introduced, it turns out that rather than expressing t in digits from 0 to $k - 1$, we should instead allow for *negative digits*. Focusing on the case of odd k , we thus represent t as $\mathbf{t} \in \{-\frac{k-1}{2}, \dots, \frac{k-1}{2}\}^h$. The interpretation of $\mathbf{t}$ becomes: if $\mathbf{t}_\ell \geq 0$, add $\mathbf{t}_\ell$ leftmost children on level ℓ , otherwise, *subtract* $|\mathbf{t}_\ell|$ rightmost children on level ℓ . Identifying that $\|\mathbf{t}\|_1$ is equal to the number of vertices used to produce the estimate at time t , we can analyze the average number of vertices added for an estimate from this representation by studying $\mathbf{t}$ for $t \in [1, T]$. This gives, roughly, a factor-2 reduction in the mean squared error versus not using subtraction.

Approximate DP with Laplace noise. To derive the (ε, δ) -DP guarantee of the Laplace mechanism (Theorem C.3), we make use of a heterogeneous composition theorem [KOV15]. Given two arbitrary multidimensional neighboring inputs, we argue for the privacy loss in each coordinate when one of them is released with respect to its neighbor. To get the aggregate privacy guarantee for the entire vector, we apply the composition theorem over the individual coordinates being released, and show that the guarantee must hold for any two neighboring outputs. It turns out that, as in the case of the Gaussian

mechanism, the (ε, δ) -DP guarantee for Laplace noise naturally depends on the ℓ_2 -sensitivity of the function being released.

Lower bound for random inputs. Finally, the lower bound of Theorem C.4 is shown by a variation of the packing argument that has been used to show an $\Omega(\log(T)/\varepsilon)$ lower bound for worst-case inputs. The core idea is to construct a set of *correlated* input strings that are individually close to uniformly random, but a continual observation mechanism that has low maximum error on all of them would be able to reliably distinguish these input strings. More precisely, for integers T , B , $m = T/B$ and $k \leq B/4$, we construct our collection of strings $\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}$ by first decomposing $[T]$ into blocks of size B : $B_1 =$

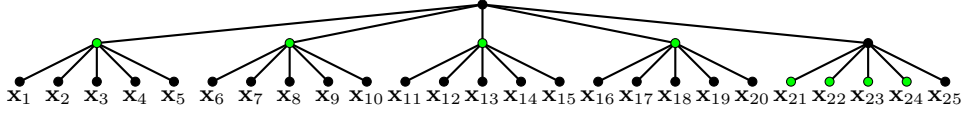
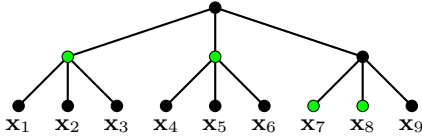
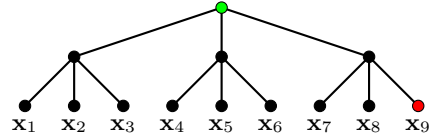


Figure C.1: Complete 5-ary tree with 25 leaves containing inputs $\mathbf{x}_1, \dots, \mathbf{x}_{25}$ and inner vertices containing subtree sums. To compute the sum of the first 24 inputs we can add 4 inner vertices and 4 leaves (shown in green) — in general, the worst-case number of vertices for k -ary trees is $k - 1$ per level. When subtree sums are made private by random (Laplace) noise the variance of a prefix sum estimator is proportional to the number of terms added. In Section C.3 we analyze the privacy and utility of this natural generalization of the binary tree mechanism to k -ary trees, among other things showing that the mean number of terms is about half of the worst case.



(a) Prefix sum with only vertex addition



(b) Prefix sum with vertex subtraction

Figure C.2: Complete ternary trees, each with 9 leaves containing inputs $x_1, \dots, x_9$ and inner vertices containing subtree sums. The trees illustrate two different possibilities for computing the sum of the first 8 inputs: (C.2a) add two inner vertices and two leaf vertices, or (C.2b) subtract one vertex (shown in red) from the sum stored in the root vertex (shown in green). The variance of using the latter method, with subtraction, is half of the former method, assuming the noise distribution for each vertex is the same. In Section C.3 we analyze the privacy and utility of our new generalization of the binary tree mechanism to k -ary trees that makes use of subtraction. Among other things we show that the worst-case number of terms needed per level is $\lfloor k/2 \rfloor$, and the mean number of terms is about half of the worst case.

$[1, B], B_2 = [B + 1, 2B], \dots, B_m = [T - B + 1, T]$ and then doing the following sampling process: $\mathbf{x}^{(0)}$ is sampled uniformly from $\{0, 1\}^T$ conditioned on each block B_i containing between $B/4$ and $3B/4$ 1s, and for $i \in [m]$, $\mathbf{x}^{(i)}$ is derived from $\mathbf{x}^{(0)}$ by flipping k 0s to 1s in block B_i . We show that each individual string has $o(1)$ total variation distance to the uniform distribution over $\{0, 1\}^T$, and thus a mechanism accurate on the uniform distribution will also be accurate on our random strings. Also, pairs of our random strings will be $2k$ -neighboring with probability 1. The key insight is that the difference of the true count of $\mathbf{x}^{(i)}$ minus $\mathbf{x}^{(0)}$ is k at the end of B_i , and zero at the end of each preceding B_j for $j < i$, and this event is detectable if the mechanism is sufficiently accurate on both inputs. Choosing parameters carefully, a packing-like argument in the end shows that a too low error on uniformly distributed inputs contradicts differential privacy.

Related work

Throughout this section, we focus on *unbiased* mechanisms for continual counting with “good error”. Here “error” is a shorthand for mean squared error over T outputs, and “good” is an error of $O(\log(T)^3)$ for ϵ -DP mechanisms and $O(\log(T)^2 \log(1/\delta))$ for (ϵ, δ) -DP mechanisms. We note that there exists some biased mechanisms [DNR⁺15], but they focus on reducing the error for *sparse* inputs and give no improvement for dense streams. Unless stated otherwise, constant factor improvements are always for ϵ -DP.

Historically, the first mechanisms with good error are typically credited to Chan, Shi, and Song [CSS11] and Dwork, Naor, Pitassi, and Rothblum [DNP⁺10b], though similar constructions were proposed independently by others [GWX10; HRM⁺10]. These results are commonly referred to as *binary tree mechanisms*. Honaker [Hon15] observed that the full tree used by the binary tree mechanism can be used to produce more efficient estimates since each prefix query can be computed using different combinations of vertices. A subset of Honaker’s techniques can be used to reduce the mean squared error by up to a factor 2, at some cost in efficiency. Rather than leveraging the density of the tree to improve the error, Andersson and Pagh [AP23] identified that making the tree sparser—only using a subset of the leaves to store inputs—allowed for a factor 2 reduction in the error while keeping the variance of outputs constant, for (ϵ, δ) -DP. Although they do not state a result for it, using their technique for ϵ -DP yields a factor 4 improvement over the binary tree mechanism, which is still more than a factor 2 away from our improvement. We also mention the work of Qardaji, Yang, and Li [QYL13] which, while focusing on range queries, did investigate reducing the mean squared error by means of considering trees of higher arity. They claim an optimal arity of $k = 16$, but average over a different set of queries, and do not leverage the subtraction of vertices. Cardoso and Rogers [CR22] used k -ary trees for prefix sums as a subroutine for (ϵ, δ) -DP and investigated choosing k to minimize the maximum variance.

Many recent papers [DMR⁺22; FHU23; HUU23; DMP⁺24; AP25; HKU25; HU25] have focused on improving the error for continual counting for (ε, δ) -DP by directly searching among factorizations of A , the lower-triangular all-1s matrix. Most notably, Henzinger, Upadhyay, and Upadhyay [HUU23] showed that there exists a near-optimal factorization $L = R = \sqrt{A}$ (also identified by Bennett [Ben77]) whose error is *optimal up to the leading constant* across all factorizations. But, importantly, this is for (ε, δ) -DP and assumes using Gaussian noise. Due to the high ℓ_1 sensitivity of $R = \sqrt{A}$, the noise needed to release $R\mathbf{x}$ with the (conventional) Laplace mechanism results in an ε -DP matrix mechanism with worse error than that of the binary mechanism.

The second part of our paper explores the use of Laplace noise for (ε, δ) -DP, especially in the small δ regime—a regime we are not the first to investigate [SU16]. We claim no novelty in pointing out that the Gaussian mechanism scales poorly for small δ (see e.g. [DSZ21]). We do note that there is a rich literature investigating alternative (non-Gaussian) noise distribution for approximate DP [GDG⁺19; Liu19; GDG⁺20; HAB⁺20; DK22; Vin22]. In [SMM19] they compare how Laplace and Gaussian noise behaves over many compositions, and especially observe that eventually they will both converge to Gaussian privacy loss distributions.

Finally, we turn to lower bounds. While [HUU23] recently established a $\Omega(\log(T)^2)$ lower bound on the mean squared error, the strongest known lower bound for ℓ_∞ error remains the $\Omega(\log T)$ result of [DNP⁺10b]. The hard inputs in [DNP⁺10b] (see the proof in [DR14] for details) are very sparse vectors with a single block of 1s that can be reliably distinguished by any algorithm that outputs prefix sums with noise significantly smaller than the block size, yet are close to each other in neighbor distance. This structure means that a “packing argument” can be used to show a lower bound on the amount of noise needed. One might wonder if *most* inputs are easier and allow a smaller error—our lower bound shows that this is not the case. Some algorithmic problems have the property that worst-case problem instances can be reduced to random instances (for example [HLS22]). Though we are not aware of such a reduction for continual counting, our lower bound can be seen as an instantiation of this general principle.

C.2 Preliminaries

Before introducing any mechanism, we first introduce some notation and background. We use $\mathbb{N}$ to denote all positive integers, and for $a, b \in \mathbb{N}$ let $[a, b] \subset \mathbb{N}$ refer to the set of integers $\{a, a + 1, \dots, b\}$. A vector-valued quantity $\mathbf{y} \in \mathbb{R}^n$ is expressed in bold notation and its scalar entries are written as $\mathbf{y}_i$ for $i \in [1, n]$. Standard definitions for ε -DP and (ε, δ) -DP are given in Section C.A.

As the main goal of this paper is the study and design of differentially private algorithms for counting queries, we will first formalize the problem. Let

vector $\mathbf{x} \in \{0, 1\}^T$ be an input stream, where $\mathbf{x}_t$ is received at time t . Then, two input streams $\mathbf{x}, \mathbf{x}'$ are *neighboring inputs*, denoted $\mathbf{x} \sim \mathbf{x}'$, if they take on the same value at every time step except one. The target function we wish to privately release is $f : \{0, 1\}^T \rightarrow \mathbb{R}^T$ where $f(\mathbf{x})_t = \sum_{i=1}^t \mathbf{x}_i$. We say that a random algorithm $\mathcal{M} : \{0, 1\}^T \rightarrow \mathbb{R}^T$ is a differentially private mechanism for continual counting if it satisfies either ε -DP or (ε, δ) -DP under this neighboring relation.

For the entirety of this paper, we will express the utility of an algorithm in terms of its *mean squared error* defined as: $\text{MeanSE}(\mathcal{M}, \mathbf{x}) = \frac{1}{T} \mathbb{E} [\|\mathcal{M}(\mathbf{x}) - f(\mathbf{x})\|_2^2] = \frac{1}{T} \sum_{t=1}^T \text{Var}[\mathcal{M}(\mathbf{x})_t]$, where the second equality only holds if $\forall \mathbf{x} \in \{0, 1\}^T : \mathbb{E}[\mathcal{M}(\mathbf{x})] = f(\mathbf{x})$, which is the case for all factorization mechanisms in general, and all mechanisms covered in this work in particular. That is to say, for unbiased mechanisms $\mathcal{M}$, their mean squared error will be equal to the average variance. Moving forward, any mention of error without further qualification will refer to the mean squared error.

C.3 Mechanisms

In this section, we will gradually build up towards the algorithm outlined in Theorem C.1. We begin by describing the binary tree mechanism, which will be our starting point. Then we will consider how the utility improves when we extend to k -ary trees, and we give an algorithm and results for this case. Finally, our main contribution will be to incorporate *subtraction* of vertices in k -ary trees to further improve the utility. The formal proofs leading to Theorem C.1 are given in the next section.

Binary Tree Mechanisms

The first algorithms [DNP⁺10b; GWX10; HRM⁺10; CSS11] with good utility are based on binary trees and are referred to as the *binary tree mechanism*. Implementation details vary, but we will consider a formulation of it where only the left subtrees are used, and in particular not the root vertex, structurally most similar to Algorithm 2 in [CSS11]. The main idea for the algorithm is to build a binary tree of height $h = \lceil \log(T + 1) \rceil$ on top of the input stream $\mathbf{x}$ where each leaf stores an input $\mathbf{x}_t$, and each vertex higher up in the tree stores the sum of its children. Having such a tree, it is possible to add together vertices in the tree in such a way that at most 1 vertex per level in the tree is used, up to h in total, where the sum corresponds to a prefix sum up to $t \in [1, T]$. In particular, which vertices that get summed to produce a given prefix sum is described exactly by the h -bit *binary representation* of the current time step, where each 1 corresponds to adding a vertex in the tree to the output. To make this private, we add i.i.d. Laplace noise $\text{Lap}(\Delta_1/\varepsilon)$ to each vertex in the tree, where $\Delta_1 = h$ since the trees for two neighboring inputs will differ in one

vertex per level, excluding the root which is not used in computations. We defer making exact statements about the corresponding utility to Section C.3, where this description of the binary tree mechanisms is a special case of Algorithm 4 for $k = 2$.

k -ary Tree Mechanisms

We will next consider trees of greater arity $k \geq 2$ to improve constants for the error. We note that using trees of higher arity has already been studied [QYL13; CKS19; CR22]. With a greater arity tree, a lesser height is needed to accommodate a given number of leaves. However, it also requires adding more vertices per level to generate a given prefix sum estimate. To explore this trade-off, we introduce notation for k -ary trees next.

Definition C.5 (k -ary trees). For integers $k \geq 2$ and $h \geq 1$, we define a k -ary tree of height h with k^h leaves as $\mathcal{T}_{k,h}$. Then for integer $1 \leq \ell \leq h + 1$, let $\mathcal{T}_{k,h}^\ell = \{[1 + j \cdot k^{\ell-1}, (j+1) \cdot k^{\ell-1}] : j \in \mathbb{N}, 0 \leq j < k^{h-\ell+1}\}$ be the set of vertices on level ℓ , where a vertex $I \in \mathcal{T}_{k,h}^\ell$ is a child of $I' \in \mathcal{T}_{k,h}^{\ell+1}$ if $I \subset I'$, and where $\mathcal{T}_{k,h} = \bigcup_{1 \leq \ell \leq h+1} \mathcal{T}_{k,h}^\ell$.

Additionally, let $\Sigma(I) = \sum_{i \in I} \mathbf{x}_i$ for $I \in \mathcal{T}_k$ when such a tree is used to store partial sums on $\mathbf{x}$, $\hat{\Sigma}$ for the case where a vertex in the tree stores a partial sum, plus independent noise. Also, we will find it expedient to express integers using k -ary digits, which we define next.

Definition C.6 (k -ary representation of integer). Let k, w and $t \leq k^w - 1$ be positive integers. We then define $\mathbf{t} = \text{rep}_k(t, w) \in [0, k-1]^w$ as the unique vector in $[0, k-1]^w$ satisfying $\sum_{i=1}^w k^{i-1} \mathbf{t}_i = t$.

The intuition behind using k -ary to analyze k -ary tree mechanisms is the same as in the case where $k = 2$ — there is a clean analogue between the two. The h -digit k -ary representation of a time step t , $\mathbf{t} = \text{rep}_k(t, h)$, encodes a set of vertices in $\mathcal{T}_{k,h}$ whose sum (when interpreted as partial sums) equals the prefix sum up to t . In particular, $\mathbf{t}_\ell$ gives the number of vertices that get added at level ℓ of the tree. With this observation in mind, we introduce Algorithm 4.

Now we state the properties of Algorithm 4. We begin by discussing the privacy of the output of the algorithm. Subsequently, we examine the variance and average number of used vertices to analyze the mean squared error. Finally, we analyze the time and space complexity of the algorithm.

Lemma C.7. *Algorithm 4 satisfies ε -DP.*

Proof. Note that line 11 always adds the noisy count from a vertex in the tree at level $\ell \in [1, h]$ since $p = p_n \stackrel{(\text{mod } k^{\ell-1})}{=} 0$, and since $\mathbf{t}$ is a valid representation

Algorithm 4 k -ary Tree Mechanism

```

1: Input:  $\mathbf{x} \in \{0, 1\}^T$ ,  $k \geq 2$ , privacy parameter  $\varepsilon$ 
2:  $h \leftarrow \lceil \log_k(T + 1) \rceil$ 
3: Construct  $k$ -ary tree  $\mathcal{T}_{k,h}$ .
4: For all  $I \in \mathcal{T}_{k,h}$  compute  $\widehat{\Sigma}(I) = \Sigma(I) + z$  for i.i.d.  $z \sim \text{Lap}(h/\varepsilon)$ .
5: for  $t = 1$  to  $T$  do
6:    $count \leftarrow 0$ 
7:    $\mathbf{t} \leftarrow \text{rep}_k(t, h)$ 
8:    $p \leftarrow 0$ 
9:   for  $\ell = h$  to  $1$  do
10:    for  $j = 1$  to  $\mathbf{t}_\ell$  do
11:       $count \leftarrow count + \widehat{\Sigma}([p + 1, p + k^{\ell-1}])$ 
12:       $p \leftarrow p + k^{\ell-1}$ 
13:    end for
14:  end for
15:  At time  $t$ , output  $count$ 
16: end for

```

of a time step $t \leq T$, we also have $p \leq t$, and so the vertex must be contained in the tree. Note further that everything after line 4 can be considered post-processing of privately releasing the tree $\mathcal{T}_{k,h}$ storing partial sums. Since we never use the root vertex of the tree at level $h + 1$, any two neighboring inputs will differ at exactly h vertices which we use, therefore $\Delta_1 = h$, implying that releasing the individual counts in the tree with $\text{Lap}(h/\varepsilon)$ gives privacy. $\square$

For the utility, note that the output from Algorithm 4 at time t has variance $(2h^2/\varepsilon^2) \cdot \|\text{rep}_k(t, h)\|_1$, since $\|\text{rep}_k(t, h)\|_1$ vertices with variance $(2h^2/\varepsilon^2)$ are added up. Consequently, we can conclude this subsection with the following Lemma and Theorem.

Lemma C.8. *Algorithm 4 achieves a mean squared error of $\frac{(k-1)h^3}{\varepsilon^2(1-1/k^h)}$ over $T = k^h - 1$ outputs.*

Proof. Identifying that the mean squared error over T steps is equal to the average number of vertices added together to produce an output, call this number $a_{[1,T]}$, times the variance of each vertex, $2h^2/\varepsilon^2$. Observe that $a_{[1,T]} = \frac{1}{T} \sum_{t=1}^T \|\text{rep}_k(t, h)\|_1$ and that T is the greatest integer possible to represent with h digits from $\{0, 1, \dots, k-1\}$. This implies that we can compute the average ℓ_1 -weight over $\{0, 1, \dots, k-1\}^h$, $a_{[0,T]} = h \cdot \frac{1}{k} \sum_{d=0}^{k-1} |d| = \frac{h(k-1)}{2}$. Since $T \cdot a_{[1,T]} = (T+1) \cdot a_{[0,T]}$, we have that $a_{[1,T]} = (1 + 1/T) \cdot a_{[0,T]} = \frac{h(k-1)}{2(1-1/k^h)}$. Plugging this in gives a mean squared error of $\frac{(k-1)h}{2(1-1/k^h)} \cdot \frac{2h^2}{\varepsilon^2} = \frac{(k-1)h^3}{\varepsilon^2(1-1/k^h)}$. $\square$

Theorem C.9. *Given integer $T \geq 2$ and constant integer $k \geq 2$, there exists an ε -DP mechanism for continual counting that on receiving a stream $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$ achieves a mean squared error of $\frac{k-1}{\varepsilon^2 \log(k)^3} \cdot \log(T)^3 + o(\log(T)^3)$, computes all outputs in time $O(T)$ and $O(\log T)$ space.*

As Theorem C.9 is not our main contribution, we provide a proof sketch in Section C.B. Before continuing and including subtraction as a technique, we note that Theorem C.9 describes the standard binary tree mechanism for $k = 2$, and that the leading constant in the error is minimized for $k = 17$, in which case it equals ≈ 0.234 .

k -ary trees with subtraction

Next, we will investigate what further improvements are possible if we allow for the subtraction of vertices to produce outputs. The reason for doing so is rather simple: if you need to add $\geq \lceil k/2 \rceil$ left-children for an estimate, you might as well add the parent and subtract the $\leq \lfloor k/2 \rfloor$ right-most children. Figure C.2 provides a visual example that further illustrates how incorporating vertex subtraction can effectively reduce the number of vertices required to compute the prefix sum. Intuitively this should buy us another factor 2 improvement, as instead of adding a maximum of $(k - 1)$ vertices per level to produce an estimate, we should now instead combine at most $\lceil (k - 1)/2 \rceil$.

To analyse (and efficiently implement) such a mechanism, the previous integer representation using digits in $[0, k - 1]$ is no longer the natural choice. Instead, when we open up the possibility of *subtracting vertices*, the solution becomes to allow for *negative digits*.

Definition C.10 (Offset k -ary representation of integers). Let k, w and t be positive integers, with k also being odd, satisfying $t \leq (k^w - 1)/2$. We then define $\mathbf{t} = \widehat{\text{rep}}_k(t, w) \in [-\frac{k-1}{2}, \frac{k-1}{2}]^w$ as the unique vector with integer entries in $[-\frac{k-1}{2}, \frac{k-1}{2}]^w$ satisfying $\sum_{i=1}^w k^{i-1} \mathbf{t}_i = t$.

We will return to the fact that Theorem C.10 only considers odd k and first focus on providing intuition for why we are using offset digits. As in the case for k -ary trees using only subtraction, there is a natural connection between $\mathbf{t} = \widehat{\text{rep}}_k(t, h)$ and vertices in $\mathcal{T}_{k,h}$. Put simply: if $\mathbf{t}_\ell > 0$, then to produce the prefix sum for t , we will be adding $\mathbf{t}_\ell \leq (k - 1)/2$ *left-most* children from $\mathcal{T}_{k,h}^\ell$, as in the case without addition except we add fewer vertices in the worst-case. If $\mathbf{t}_\ell < 0$, then we will instead subtract $|\mathbf{t}_\ell| \leq (k - 1)/2$ *right-most* children from $\mathcal{T}_{k,h}^\ell$. In either case, the worst-case number of vertices added or subtracted is reduced to being at most $(k - 1)/2$.

Now, we only consider the case of odd k for a few reasons: (1) if k is odd, then we have symmetry in the digits, meaning we add at worst as many things as we may subtract. This makes the analysis easier. (2) even k is not optimal, see Section C.C for more details. (3) if k is odd, then there is a simple relation

Algorithm 5 k -ary Tree Mechanism with Subtraction

```

1: Input: A stream  $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$ , odd  $k \geq 3$ , privacy parameter  $\varepsilon$ 
2:  $h \leftarrow \lceil \log_k 2T \rceil$ 
3: Lazily draw  $\mathbf{z} \sim \text{Lap}(h/\varepsilon)^{(k^h-1)/2}$ 
4: for  $t = 1$  to  $T$  do
5:    $\text{noise} \leftarrow 0$ 
6:    $\mathbf{t} \leftarrow \widehat{\text{rep}}_k(t, h)$ 
7:    $p \leftarrow 0$ 
8:   for  $\ell = h$  to  $1$  do
9:     for  $j = 1$  to  $|\mathbf{t}_\ell|$  do
10:       $p \leftarrow p + \text{sgn}(\mathbf{t}_\ell) \cdot k^{\ell-1}$ 
11:       $\text{noise} \leftarrow \text{noise} + \mathbf{z}_p$ 
12:     end for
13:   end for
14:   At time  $t$ , output  $\sum_{j=1}^t \mathbf{x}_j + \text{noise}$ 
15: end for

```

for when the root vertex of a tree is used to produce an output: the root vertex is used if t is in the right half of the tree. This gives an intuitive condition on the height to support T inputs: $h = \lceil \log_k(2T) \rceil$ — set the height such that there is a sufficient number of left-half leaves. This can be compared to standard k -ary trees, where the same condition is to use every leaf of the tree except the right-most one.

We are now ready to introduce our main contribution: Algorithm 5. Structurally the algorithm releases prefix sums with a correlation in the noise that depends on the representation $\widehat{\text{rep}}_k(t, h)$ of each time step t .

C.4 Proving Theorem C.1

In this section, we will prove the properties of Algorithm 5, as summarized in Theorem C.1, starting with privacy.

Privacy. Instead of arguing about the privacy of Algorithm 5, we will argue it for another algorithm with the same output distribution, Algorithm 6, whose connection to trees is more immediate. We proceed by establishing the equivalence of the output distributions in Algorithms 5 and 6. The following proposition will be helpful in establishing that the outputs are identically distributed.

Proposition C.11. *When iterating over t , let count_n and p_n be the value of count and p after $1 \leq n \leq \|\mathbf{t}\|_1$ updates in Algorithm 6. Then count_n stores a noisy prefix sum up to $p_n \in [1, (k^h - 1)/2]$.*

Algorithm 6 More tree-like version of Algorithm 5 for analysis

```

1: Input:  $\mathbf{x} \in \{0, 1\}^T$ ,  $k \geq 3$ , privacy parameter  $\varepsilon$ 
2:  $h \leftarrow \lceil \log_k 2T \rceil$ 
3: Construct  $k$ -ary tree  $\mathcal{T}_{k,h}$ .
4: For all  $I \in \mathcal{T}_{k,h}$  compute  $\widehat{\Sigma}(I) = \Sigma(I) + z$  for i.i.d.  $z \sim \text{Lap}(h/\varepsilon)$ .
5: for  $t = 1$  to  $T$  do
6:    $count \leftarrow 0$ 
7:    $\mathbf{t} \leftarrow \widehat{\text{rep}}_k(t, h)$ 
8:    $p \leftarrow 0$ 
9:   for  $\ell = h$  to  $1$  do
10:    for  $j = 1$  to  $|\mathbf{t}_\ell|$  do
11:      if  $\text{sgn}(\mathbf{t}_\ell) > 0$  then
12:         $count \leftarrow count + \widehat{\Sigma}([p + 1, p + k^{\ell-1}])$ 
13:         $p \leftarrow p + k^{\ell-1}$ 
14:      else
15:         $count \leftarrow count - \widehat{\Sigma}([p - k^{\ell-1} - 1, p])$ 
16:         $p \leftarrow p - k^{\ell-1}$ 
17:      end if
18:    end for
19:  end for
20:  At time  $t$ , output  $count$ 
21: end for

```

Proof. We show this by induction on n . Observe that $p_n \in [1, (k^h - 1)/2]$ for all valid n since the first non-zero digit in $\mathbf{t}$ has to be positive, and since the greatest possible integer we can represent in these digits is $(k^h - 1)/2$. We therefore have that $p_1 \geq 1$ and $count_1 = \widehat{\Sigma}[1, p]$ is a noisy prefix sum up to $\mathbf{x}_p$. Assuming that after n updates $count$ represents a noisy prefix sum up to p , we will argue that this still holds after the $n + 1^{\text{st}}$ update to p . Just before $count$ is updated, observe that $p = p_n \stackrel{(\text{mod } k^{\ell-1})}{=} 0$, thus implying that the corresponding partial sums added or subtracted on lines 12 and 13 exist in the tree, and that the result will be a noisy prefix sum up to p_{n+1} . $\square$

We can now prove that the two algorithms produce identically distributed outputs.

Proposition C.12. *The output distribution of Algorithms 5 and Algorithm 6 are equal.*

Proof. First, observe that Algorithm 6 like Algorithm 5 outputs the correct prefix sum on expectation which follows from Theorem C.11 for $n = \|\mathbf{t}\|_1$. We thus only have to argue for the equivalence of the noise added. With this target in mind, note that each time a vertex is used on lines 12 and 15 in

Algorithm 6, it can be uniquely identified by the p after it is updated on the next line. The uniqueness can be inferred from the uniqueness of expressing the corresponding $\mathbf{p} = \widehat{\text{rep}}_k(p, h)$, where $p \in [1, (k^h - 1)/2]$ can be interpreted as a time step. Following up, note that for any time step t , the sequence of p 's that are computed on line 10 for Algorithm 5 are the same as the sequence of p 's obtained after lines 13 and 16 for Algorithm 6. Since Algorithm 5 explicitly indexes the noise terms by the same p , and since each term is identically distributed across the algorithms, it follows that each individual noise term that is added to the output at each step is identical up to their sign.

What remains to account for is the lack of a factor “ $\text{sgn}(\mathbf{e}_\ell)$ ” on line 11 in Algorithm 5. The reason why it is not needed comes from the fact that a given vertex in Algorithm 6 always contributes with the same sign: we always subtract right-most children and always add left-most children to produce prefix sum estimates. Since Laplace noise is symmetric, subtracting or adding noise has the same effect, and we can therefore drop the sign on line 11 in Algorithm 5 and still have the same output distribution, proving the statement. $\square$

We are now ready to prove that Algorithm 5 satisfies ε -DP.

Lemma C.13. *Algorithm 5 satisfies ε -DP.*

Proof. We proceed by proving the privacy of Algorithm 6, the privacy of which implies the privacy of Algorithm 5 in the offline setting via Theorem C.12. The privacy proof is standard and follows from identifying that if all $\widehat{\Sigma}(I)$ on line 4 are released privately, then the remaining computations are a post-processing of that release. For neighboring inputs $\mathbf{x}' \sim \mathbf{x}$, let i be the index at which they differ. Let $\mathbf{y} \in \mathbb{R}^{|\mathcal{T}_{k,h}|}$ be a vector where each entry is identified by a unique interval from $I \in \mathcal{T}_{k,h}$ equal to $\Sigma(I)$ given $\mathbf{x}$. Define $\mathbf{y}'$ analogously for $\mathbf{x}'$. Since any $i \in [1, T]$ is included in exactly h intervals $I \in \mathcal{T}$, intentionally excluding the root vertex which is not used, it follows that the sensitivity for releasing $\mathbf{y}$ is $\Delta_1 = \|\mathbf{y} - \mathbf{y}'\|_1 = h$. As $\mathbf{y}$ is released on line 4 using the Laplace mechanism with appropriate scale, and all computations afterward are post-processing of y , Algorithm 6 satisfies ε -DP.

To extend the privacy analysis to the online case where $\mathbf{x}$ is streamed (non-adaptively), note that the output distribution of Algorithm 5 can be expressed as $A\mathbf{x} + L\mathbf{z}$ for suitable matrices A, L . On receiving $\mathbf{x}_t$, $(A\mathbf{x})_t + (L\mathbf{z})_t$ is output. Since $L\mathbf{z}$ is generated independently of the input, the output distribution of the algorithm is not impacted by having $\mathbf{x}$ gradually revealed. It follows that the algorithm is also ε -DP under continual observation. $\square$

The online argument in the proof is explicit about $\mathbf{x}$ being chosen before any outputs are produced. We note that every non-adaptive ε -DP algorithm for continual counting is also adaptive (see Proposition 2.1 in [DMR⁺22]). Privacy in the online setting can also be proved directly using a “shifting” argument for the Laplace variables $\mathbf{z}$, as done in for example [DNP⁺10b].

Utility. As intuited, allowing for the subtraction of vertices does indeed improve utility. Before stating the improvement, note that, analogous to the case without subtraction, the variance of the output from Algorithm 5 at time t will be equal to $(2h^2/\varepsilon^2) \cdot \|\widehat{\text{rep}}_k(t, h)\|_1$, since line 11 is executed $\|\widehat{\text{rep}}_k(t, h)\|_1$ times.

Lemma C.14. *Algorithm 5 achieves a mean squared error of $\frac{k(1-1/k^2)h^3}{2\varepsilon^2(1-1/k^h)}$ over $T = (k^h - 1)/2$ outputs.*

Proof. First note that the mean squared error over T steps is equal to the average number of noise terms added together to produce an output, call this number $a_{[1, T]}$, times the variance of the noise in each vertex. We thus want to compute $a_{[1, T]} = \frac{1}{T} \sum_{t=1}^T \|\widehat{\text{rep}}_k(t, h)\|_1$. Note that T is the greatest positive integer that can be expressed using h digits in $\{-\frac{k-1}{2}, \dots, \frac{k-1}{2}\}$, and $-T$ the most negative integer. For any representation $\mathbf{t} = \widehat{\text{rep}}_k(t, h)$ where $t \in [1, T]$, note that we have a bijection where $-\mathbf{t}$ encodes a unique negative integer $\geq -T$. We therefore have that $a_{[1, T]} = a_{[-T, -1]}$, and $a_{[-T, T]} = (1 - 1/k^h) \cdot a_{[1, T]}$, where the extra factor comes from not including 0. Since $a_{[-T, T]} = h \cdot \frac{1}{k} \sum_{d=-(k-1)/2}^{(k-1)/2} |d| = \frac{h(k^2-1)}{4k}$, by noting that every digit occurs the same amount of times in each position, we also have that $a_{[1, T]} = \frac{kh}{4} \frac{1-1/k^2}{1-1/k^h}$. Since $\text{Var}[\mathbf{z}_p] = 2h^2/\varepsilon^2$, we arrive at a mean squared error of $\frac{kh}{4} \frac{1-1/k^2}{1-1/k^h} \cdot \frac{2h^2}{\varepsilon^2} = \frac{kh^3(1-1/k^2)}{2\varepsilon^2(1-1/k^h)}$. $\square$

We follow up with a corollary that states how the error scales and what constant the optimal k gives.

Corollary C.15. *Algorithm 5 achieves a mean squared error that scales as $\frac{k(1-1/k^2)}{2\varepsilon^2 \log(k)^3} \log(T)^3 + o(\log(T)^3)$, and which is minimized for $k = 19$ yielding $(0.1236/\varepsilon^2) \log(T)^3 + o(\log(T)^3)$.*

In relation to Honaker Online, this is a constant improvement by more than a factor 4.

Space and time complexity Finally, it turns out that extending to k -ary trees with subtraction does not impact the efficiency of tree-based aggregation. We will need the following proposition to prove both the space and time complexity.

Proposition C.16. *Each noise term $\mathbf{z}_p$ used by Algorithm 5 will only be used for outputs in a time interval $[t_1, t_2] \subseteq [1, T]$.*

Proof. Let the value of the iterate ℓ be q when $\mathbf{z}_p$ is added on line 11. Consider first the case when $q < h$. Note that for any time step t with representation $\mathbf{t} = \widehat{\text{rep}}_k(t, h)$, $\mathbf{z}_p$ is only used if the $h - q$ most-significant digits are identical

across $\mathbf{p} = \widehat{\text{rep}}_k(p, h)$ and $\mathbf{t}$. Interpreting any fixed number of most significant digits of $\widehat{\text{rep}}_k(t, h)$ as an integer, as t is incremented, will produce a sequence of monotonically increasing integers, implying that any noise value $\mathbf{z}_p$ will start to get used at some t_1 , and then after some time step $t_2 \geq t_1$ will never be used again, proving the claim for $q < h$. If $q = h$, then once $\mathbf{z}_p$ is added for the first time at t_1 , it will be added at every future time step including $t_2 = T$, proving the claim fully. $\square$

Lemma C.17. *Algorithm 5 runs in $O(k \log_k T)$ space.*

Proof. Note that at any time step t , we need to store noise values used for the current time step t , of which at most $h(k-1)/2$ are at most needed. Let $\mathbf{z}_p$ be an arbitrary noise value kept in memory at time t , and then no longer used starting from $t' \geq t$. By Theorem C.16, $\mathbf{z}_p$ will never be used again after t' , and it can therefore be removed from memory. It follows that no noise values besides those used at a current time step need to be stored in memory, thus we need to store at most $h(k-1)/2 = O(k \log_k T)$ noise values. $\square$

Theorem C.18 states the time complexity of Algorithm 5, which assumes that (1) Laplace noise can be generated, and (2) $\widehat{\text{rep}}_k(t, h)$ can be computed, both in $O(k)$ time. Note that the cost of (1) is unavoidable, and that (2) can be efficiently computed by incrementing the digits at each step, where an argument analogous to the analysis for incrementing a binary counter implies a time of $O(kT)$ for incrementing the counter T times as measured in digit flips.

Lemma C.18. *Algorithm 5 can be implemented to run in $O(kT)$ time.*

Proof. First let $T = (k^h - 1)/2$, meaning we support as many inputs as possible using h digits. Note that there are exactly T unique noise variables used by Algorithm 5, since p will take on all values in $[1, T]$ when iterating over t . Rather than resetting *noise* on each iteration of t , we can instead update it, removing noise terms that are no longer used and adding new noise as needed. Each of them will be added exactly once and subtracted at most once (Theorem C.16), therefore we run in time $O(T)$ for this case. For $(k^{h-1} + 1)/2 \leq T < (k^h - 1)/2 = T'$, note that $\frac{T'}{T} \leq \frac{(k^h - 1)/2}{(k^{h-1} + 1)/2} \leq k$, and therefore if we estimate the runtime of T by that for T' , we get a runtime of $O(kT')$ in this case. $\square$

Finally, combining Lemmas C.13, C.14, C.17 and C.18 gives us Theorem C.1.

C.5 Laplace vs Gaussian noise

Having designed an efficient ε -DP algorithm for continual counting with a state-of-the-art bound on the error, a natural question arises: how does it compare to (ε, δ) -DP mechanisms?

Since the noise of the Gaussian mechanism grows as δ approaches zero, it is clear that for any given bounds on ℓ_1 and ℓ_2 sensitivity, using the Gaussian mechanism with sufficiently small δ will result in more noise than the Laplace mechanism. We will see that for current continual counting methods this change happens for surprisingly large values of δ , with the exact threshold depending on the number of time steps T .

Furthermore, we adapt the Laplace mechanism to work with ℓ_2 sensitivity and show that it provides a meaningful (ε, δ) -DP guarantee with variance close to that of the Gaussian mechanism.

Laplace vs Gaussian noise for continual counting. We start by stating a lemma that compares the currently best continual counting mechanisms under pure and approximate differential privacy.

Lemma C.19. *For constants $B_\varepsilon, B_{\varepsilon, \delta} > 0$, let $(B_\varepsilon/\varepsilon^2) \log(T)^3 + o(\log(T)^3)$ be a bound on the MeanSE of an ε -DP mechanism and $(B_{\varepsilon, \delta}/\varepsilon^2) \log(T)^2 \log(1/\delta) + o(\log(T)^2 \log(1/\delta))$ be a bound on the mean squared error of an (ε, δ) -DP mechanism, both for continual counting. Then the ε -DP mechanism achieves equivalent or better mean squared error, up to lower-order asymptotically vanishing terms, for $\delta = O(1/T^{B_\varepsilon/B_{\varepsilon, \delta}})$.*

Proof. Comparing the leading terms of the errors and solving for δ yields the statement. $\square$

Proof of Theorem C.2. Algorithm 5 has a leading constant of $B_\varepsilon < 0.124$ for $k = 19$ under ε -DP, and the corresponding constant in [HUU23] is $B_{\varepsilon, \delta} = 4/(\pi^2 \log(e)^3) > 0.1349$, so we have $B_\varepsilon/B_{\varepsilon, \delta} < 0.92$. Theorem C.19 thus implies that the ε -DP mechanism has a smaller bound on the noise for $\delta = O(T^{-0.92})$. $\square$

Laplace noise scaled to ℓ_2 sensitivity

We next consider how we may extend the Laplace mechanism with an *approximate* DP guarantee similar to the Gaussian mechanism, i.e., with noise scaled to the ℓ_2 sensitivity. Since the proof of this is straightforward using known results we suspect this fact may be known, but we are not aware of it appearing in the literature. The technique for extending the Laplace mechanism to approximate DP relies on the following heterogeneous composition theorem [KOV15].

Lemma C.20 (Simplified version of Theorem 3.5 in [KOV15]). *For any $\varepsilon_j > 0$ for $j \in \{1, \dots, k\}$, and $\delta \in (0, 1)$, the class of ε_j -differentially private mechanisms satisfy $(\tilde{\varepsilon}, \delta)$ -differential privacy under k -fold adaptive composition for*

$$\tilde{\varepsilon} = \min \left\{ \sum_{j=1}^k \varepsilon_j, \sum_{j=1}^k \frac{(e^{\varepsilon_j} - 1)\varepsilon_j}{e^{\varepsilon_j} + 1} + \sqrt{2 \ln(1/\delta) \sum_{j=1}^k \varepsilon_j^2} \right\}.$$

Using Theorem C.20, we can derive the following approximate DP guarantee for the ℓ_2 Laplace mechanism.

Lemma C.21 (ℓ_2 Laplace Mechanism). *Let $f : \mathcal{X}^n \rightarrow \mathbb{R}^d$ be a function with ℓ_2 -sensitivity $\Delta_2 := \max_{\mathcal{D} \sim \mathcal{D}'} \|f(\mathcal{D}) - f(\mathcal{D}')\|_2$. For a given dataset $\mathcal{D} \in \mathcal{X}^n$ and $\varepsilon, \delta \in (0, 1)$, the output $f(\mathcal{D}) + \text{Lap}(\Delta_2/a_{\varepsilon, \delta})^d$, where $a_{\varepsilon, \delta} = \sqrt{2 \ln(1/\delta)}(\sqrt{1 + \varepsilon/\ln(1/\delta)} - 1)$, satisfies (ε, δ) -DP.*

Proof. For $x_1, \dots, x_j \in (0, 1)$, since $\frac{e^{x_i}-1}{e^{x_i}+1} \leq x_i/2$ it follows from Theorem C.20 that $(\tilde{\varepsilon}, \delta)$ -differential privacy is also satisfied for

$$\tilde{\varepsilon} = \min \left\{ \sum_{j=1}^k \varepsilon_j, \frac{1}{2} \sum_{j=1}^k \varepsilon_j^2 + \sqrt{2 \ln(1/\delta) \sum_{j=1}^k \varepsilon_j^2} \right\}. \quad (\text{C.1})$$

It suffices to establish the privacy of the Laplace mechanism when restricted to arbitrary neighboring inputs x, x' . We will reason about the privacy loss for each coordinate of $\mathbf{y} = f(x)$ separately relative to the neighboring output $\mathbf{y}' = f(x')$. Focusing on one coordinate $\mathbf{y}_j$ and its private release, we have that its release must satisfy ε_j -differential privacy where $\varepsilon_j = (a_{\varepsilon, \delta}/\Delta_2) \cdot |\mathbf{y}_j - \mathbf{y}'_j|$. Composing the d private releases of each coordinate using the bound (C.1) and noting that $\sum_{j=1}^d \varepsilon_j^2 \leq (a_{\varepsilon, \delta}/\Delta_2)^2 \sum_{j=1}^d (\mathbf{y}_j - \mathbf{y}'_j)^2 \leq a_{\varepsilon, \delta}^2$ therefore gives an (ε, δ) -guarantee where $\varepsilon = a_{\varepsilon, \delta}(\frac{a_{\varepsilon, \delta}}{2} + \sqrt{2 \ln(1/\delta)})$. Proceeding to solve for (positive) $a_{\varepsilon, \delta}$ gives the statement. $\square$

The variance needed for ℓ_2 -scaled Laplace noise compared to Gaussian noise for (ε, δ) -DP is only a constant factor greater whenever $\varepsilon \leq \log(1/\delta)$, and the constant approaches 2 as ε approaches zero.

Corollary C.22. *Let $\mathbf{y}^{\text{lap}}$ and $\mathbf{y}^{\text{gauss}}$ be outputs based on the ℓ_2 Laplace mechanism (Theorem C.21) and the Gaussian mechanism (Theorem C.27) respectively, both to achieve the same (ε, δ) -DP guarantee where $\varepsilon \leq \ln(1/\delta)$. Then $\text{Var}[\mathbf{y}_j^{\text{lap}}] \leq \frac{(\varepsilon/\ln(1/\delta))^2}{2(\sqrt{1+\varepsilon/\ln(1/\delta)}-1)^2} \cdot \text{Var}[\mathbf{y}_j^{\text{gauss}}]$. Also, $\lim_{\varepsilon \rightarrow 0} \frac{(\varepsilon/\ln(1/\delta))^2}{2(\sqrt{1+\varepsilon/\ln(1/\delta)}-1)^2} = 2$.*

Proof. Using Theorem C.21 and noting that the variance of the noise added is twice the square of the scale parameter for Laplace noise, we get that

$$\frac{\text{Var}[\mathbf{y}_j^{\text{lap}}]}{\text{Var}[\mathbf{y}_j^{\text{gauss}}]} = \frac{2\Delta_2^2/(2 \ln(1/\delta)(\sqrt{1 + \varepsilon/\ln(1/\delta)} - 1)^2)}{2 \ln(1.25/\delta)\Delta_2^2/\varepsilon^2} \leq \frac{(\varepsilon/\ln(1/\delta))^2}{2(\sqrt{1 + \varepsilon/\ln(1/\delta)} - 1)^2}.$$

To show the limit, note that for positive $w \leq 1$ we have that $\frac{w}{\sqrt{1+w}-1} \leq \frac{w}{w/2-w^2/8} = \frac{2}{1-w/4} \rightarrow 2$ as $w \rightarrow 0$, and $\frac{w}{\sqrt{1+w}-1} \geq 2$, both based on truncated Taylor expansions of $\sqrt{1+w}$. $\square$

Finally we prove Theorem C.3 from the introduction.

Proof of Theorem C.3. From the proof of Theorem C.21, we have that adding noise $\text{Lap}(\Delta_2/a)$ gives an (ε', δ) -guarantee where $\varepsilon' = a(\frac{a}{2} + \sqrt{2\ln(1/\delta)})$ if $\varepsilon', \delta \in (0, 1)$. Note that our setup is the same if we set $a = \frac{\Delta_2^2}{\lambda}$, and that adding noise $\text{Lap}(\lambda)$ gives an $\frac{\Delta_1}{\lambda}$ -DP guarantee. We can therefore always guarantee (ε, δ) -DP where $\varepsilon = \min\{\frac{\Delta_1}{\lambda}, \varepsilon'\} < 1$ if we enforce $\lambda > \Delta_1$. $\square$

Note that while functions with low ℓ_1 sensitivity also have low ℓ_2 sensitivity, the converse does not hold. In principle we can replace Gaussian noise with Laplace noise at a limited loss in utility for approximate DP (Theorem C.22), but the corresponding ε -DP guarantee afforded by the noise will be weak unless Δ_1 is small. Theorem C.3 can alternatively be seen as adding an approximate DP guarantee to pure DP mechanisms built on top of the Laplace mechanism.

C.6 Lower bound over random inputs

In this section we consider continual observation in the “offline” setting where a mechanism has access to the entire input and has to output a differentially private approximation of all prefix sums. This of course implies a lower bound for the “online” setting. We show Theorem C.4 which achieves the same lower bound that was previously known for worst-case inputs, in the setting where the input is a random string.

Proof of Theorem C.4. We will prove the theorem via a packing argument applied to a carefully constructed collection of random strings. Without loss of generality, let $B = \sqrt{T}$ be an integer divisible by 4, $k \leq T^{1/5}$ be an even integer, and T be large enough such that $k \leq B/4$. Also divide the timeline $[1, T]$ into blocks of length B : $B_1 = [1, B], B_2 = [B + 1, 2B], \dots, B_m = [T - B + 1, T]$, $m = T/B$. For an arbitrary string $\mathbf{y} \in \{0, 1\}^T$, we use the notation $\mathbf{y}_{B_i} \in \{0, 1\}^B$ for the substring of $\mathbf{y}$ formed from the indices in B_i . We use $\text{err}(\cdot)$, taking as input the output of a mechanism, to denote the (random) maximum additive error over all the outputs. The argument is outlined below:

- Construct random strings $\mathbf{x}^{(0)}, \dots, \mathbf{x}^{(m)}$ that are near-neighboring and have small total variation distance to random strings from $\mathcal{U}$.
- Argue that $\mathcal{M}$ assumes additive error $\leq \alpha$ on these inputs with constant probability by virtue of the small total variation distance.
- Show that for $i \neq i'$ $\mathcal{M}$ can be used to distinguish $\mathbf{x}^{(i)}$ from $\mathbf{x}^{(i')}$ unless $\alpha = \Omega(\varepsilon^{-1} \log T)$.

The intuition behind the argument is that an accurate mechanism $\mathcal{M}$ on $\mathcal{U}$ will also be accurate on our random (but highly correlated) strings $\mathbf{x}^{(0)}, \dots, \mathbf{x}^{(m)}$. Being accurate on these strings implies the ability to distinguish between them.

However, since $\mathcal{M}$ satisfies ε -DP, distinguishing between the inputs with good confidence is impossible, implying a lower bound on the additive error α .

To this end, we define the random variables $X_j^{(i)} = \|\mathbf{x}_{B_j}^{(i)}\|_1 = \sum_{\ell \in B_j} \mathbf{x}_\ell^{(i)}$, i.e., as the sum of 1s in the block B_j of $\mathbf{x}^{(i)}$. With the argument outlined, consider the random strings $\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}$ formed by the following process:

1. Draw $\mathbf{x}^{(0)}$ from $\mathcal{U}$ conditioned on $X_j^{(0)} \in [\frac{B}{4}, \frac{3B}{4}]$ for all $j \in [m]$. Call this distribution $\mathcal{D}^{(0)}$.
2. For $i \in [m]$ derive $\mathbf{x}^{(i)}$ from $\mathbf{x}^{(0)}$ by choosing k 0s in B_i uniformly at random and flipping them to 1s. Call these distributions $\mathcal{D}^{(i)}$.

We next define the algorithm $\mathcal{A}(\mathbf{y}, \mathbf{y}')$ outputting the difference between private sums on two arbitrary inputs $\mathbf{y}, \mathbf{y}' \in \{0, 1\}^T$. Formally, on receiving $\mathbf{y}$ and $\mathbf{y}'$, it runs $\mathcal{M}$ on each, producing outputs $\mathbf{a}_1, \dots, \mathbf{a}_T$ and $\mathbf{a}'_1, \dots, \mathbf{a}'_T$ respectively, and at time t , the algorithm outputs $\mathcal{A}(\mathbf{y}, \mathbf{y}')_t = \mathbf{a}_t - \mathbf{a}'_t$. We will show, step-by-step, that the accuracy guarantee of $\mathcal{M}$ on $\mathcal{U}$ will imply distinguishing between our random strings $\mathbf{x}^{(0)}, \dots, \mathbf{x}^{(i)}$.

Consider $\mathcal{A}(\mathbf{x}^{(i)}, \mathbf{x}^{(0)})$ for $i \in [1, m]$. Note that the true prefix sums of $\mathbf{x}^{(i)}$ and $\mathbf{x}^{(0)}$ at the end of each block will be identical up until block B_i , at the end of which they will differ by exactly k . We leverage this observation by introducing the following disjoint events E_j . For $j \in [1, m]$ and arbitrary input strings $\mathbf{y}, \mathbf{y}'$, we define E_j as the event that the following two statements are true: (1) for all $\ell \in [1, j-1]$: $\mathcal{A}(\mathbf{y}, \mathbf{y}')_{\ell B} \leq k/2$, and (2) $\mathcal{A}(\mathbf{y}, \mathbf{y}')_{jB} > k/2$. In other words, E_j is the event that, when tracking only the outputs at the end of blocks, $\mathcal{A}(\mathbf{y}, \mathbf{y}')$ outputs a value greater than $k/2$ for the first time at the end of block B_j . Set $\alpha = k/4$. By our construction, if $\mathcal{M}(\mathbf{x}^{(0)})$ and $\mathcal{M}(\mathbf{x}^{(i)})$ each assumes additive error at most α when running $\mathcal{A}(\mathbf{x}^{(i)}, \mathbf{x}^{(0)})$, then $\mathcal{A}(\mathbf{x}^{(i)}, \mathbf{x}^{(0)})$ has additive error at most $2\alpha = k/2$, and E_i happens with probability 1. We thus have

$$\Pr[\mathcal{A}(\mathbf{x}^{(i)}, \mathbf{x}^{(0)}) \in E_i] \geq \Pr[\mathcal{M}(\mathbf{x}^{(i)}) \leq \alpha] \cdot \Pr[\mathcal{M}(\mathbf{x}^{(0)}) \leq \alpha] .$$

We want to argue that $\mathcal{M}$ is accurate on $\mathbf{x}^{(0)}, \dots, \mathbf{x}^{(m)}$, with constant probability for each input taken separately, and we will argue via the total variation distance between each of these random strings, and the random string $\mathbf{x} \sim \mathcal{U}$. In general, for a (possibly randomized) algorithm $\mathcal{A} : S \rightarrow \mathbb{R}$, distributions $\mathcal{D}, \mathcal{D}'$ over S with total variation distance $\text{TV}(\mathcal{D}, \mathcal{D}') \leq \Delta$, and an event E over the output, we have that if $\Pr_{\mathcal{A}, x \sim \mathcal{D}}[\mathcal{A}(x) \in E] \geq p \geq 0$, then $\Pr_{\mathcal{A}, x \sim \mathcal{D}'}[\mathcal{A}(x) \in E] \geq p - \Delta$. This follows from the observation that, in the extreme case, Δ probability mass can be shifted from inputs $x \in S$ where $\Pr_{\mathcal{A}}[\mathcal{A}(x) \in E] = 1$ to inputs $x' \in S$ where $\Pr_{\mathcal{A}}[\mathcal{A}(x') \in E] = 0$. If we manage to show that, for $i \in [0, m]$, $\text{TV}(\mathbf{x}, \mathbf{x}^{(i)}) = o(1)$, then we can lower bound $\Pr[\mathcal{A}(\mathbf{x}^{(i)}, \mathbf{x}^{(0)}) \in E_i]$ by a positive constant. We derive bounds on these total variation distances in the next two lemmas.

Lemma C.23. *Let $\mathbf{x} \sim \mathcal{U}$ and $\mathbf{x}^{(0)} \sim \mathcal{D}^{(0)}$. Then we have $\text{TV}(\mathbf{x}, \mathbf{x}^{(0)}) \leq \Delta$, where $\Delta = 2\sqrt{T} \exp(-\sqrt{T}/8)$*

Proof. Let S be the support of $\mathcal{U}$ and $S^{(0)} \subseteq S$ the support of $\mathcal{D}^{(0)}$. We have that $X \sim \text{Bin}(B, 1/2)$ describes the number of 1s in an arbitrary block B_i of $\mathbf{x}$. Hoeffding's inequality gives $\Pr[\frac{B}{4} \leq X \leq \frac{3B}{4}] \geq 1 - 2\exp(-B/8)$ whereby it follows that $\Pr[\mathbf{x} \in S^{(0)}] \geq (1 - 2\exp(-B/8))^{T/B}$. Observing that $\mathbf{x}^{(0)}$ is uniformly distributed over $S^{(0)} \subseteq S$, we proceed to directly compute the total variation distance:

$$\begin{aligned} \text{TV}(\mathbf{x}, \mathbf{x}^{(0)}) &= \frac{1}{2} \sum_{\mathbf{s} \in S} |\Pr[\mathbf{x} = \mathbf{s}] - \Pr[\mathbf{x}^{(0)} = \mathbf{s}]| \\ &= \frac{1}{2} |S^{(0)}| \left(\frac{1}{|S^{(0)}|} - \frac{1}{|S|} \right) + \frac{1}{2} (|S| - |S^{(0)}|) \cdot \frac{1}{|S|} = 1 - \Pr[\mathbf{x} \in S^{(0)}] \\ &\leq 1 - (1 - 2\exp(-B/8))^{T/B} \\ &\leq \frac{2T \exp(-B/8)}{B} = 2\sqrt{T} \exp(-\sqrt{T}/8) , \end{aligned}$$

where the last inequality is using Bernoulli's inequality and holds for T larger than an absolute constant. $\square$

We prove that the remaining random strings $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}$ are closely distributed to strings in $\mathcal{U}$ next.

Lemma C.24. *Let $\mathbf{x} \sim \mathcal{U}$ and $\mathbf{x}^{(i)} \sim \mathcal{D}^{(i)}$ for $i \in [m]$. Then $\text{TV}(\mathbf{x}, \mathbf{x}^{(i)}) \leq \Delta'$ where $\Delta' = 10 \cdot T^{-1/20}$.*

Proof. As the total variation distance satisfies the triangle inequality, we can write

$$\text{TV}(\mathbf{x}, \mathbf{x}^{(i)}) \leq \text{TV}(\mathbf{x}, \mathbf{x}^{(0)}) + \text{TV}(\mathbf{x}^{(0)}, \mathbf{x}^{(i)}) .$$

The first term we have already computed in Theorem C.23. To bound the second term, we will use a property of total variation distance: it cannot increase if one applies the same transformation to both arguments. Let $f : [m] \times [0, B] \rightarrow \mathcal{F}$ be a mapping to a family of distributions where $f(i, \ell)$ denotes the distribution described by a process where (1) a string $\mathbf{y} \sim \mathcal{D}^{(0)}$ is sampled and then (2) $\mathbf{y}_{B_i}$ is overwritten by a uniformly sampled string in $\{0, 1\}^B$, conditioned on it having ℓ 1s. For $i \in [m]$, we claim that $(f(i, X_i^{(i)}), \mathcal{D}^{(i)})$ and $(f(i, X_i^{(0)}), \mathcal{D}^{(0)})$ are pairs of identical distributions. To see this for the first pair of distributions, consider $\mathbf{y} \sim f(i, X_i^{(i)})$. By definition $\mathbf{y}_{B_j}$ is identically distributed to $\mathbf{x}_{B_j}^{(i)}$ for $j \neq i$, and so the only special case to consider is $j = i$. For this case, we note that $X_i^{(i)}$ is defined as the number of 1s in $\mathbf{x}_{B_i}^{(i)}$, and that symmetry implies that any substring with ℓ 1s is equally likely, thus $\mathbf{y}_{B_i}$ must be identically distributed to $\mathbf{x}_{B_i}^{(i)}$ as well, and we are done. An analogous argument can be used for showing that $f(i, X_i^{(0)})$ and $\mathcal{D}^{(0)}$ are identically distributed.

We thus have $\text{TV}(\mathbf{x}^{(0)}, \mathbf{x}^{(i)}) = \text{TV}(f(i, X_i^{(0)}), f(i, X_i^{(i)})) \leq \text{TV}(X_i^{(0)}, X_i^{(i)})$, and so our problem reduces to bounding this quantity.

$$\begin{aligned}
\text{TV}(X_i^{(0)}, X_i^{(i)}) &= \frac{1}{2} \sum_{\ell=0}^B |\Pr[X_i^{(0)} = \ell] - \Pr[X_i^{(i)} = \ell]| \\
&= \frac{1}{2} \sum_{\ell=0}^B |\Pr[X_i^{(0)} = \ell] - \Pr[X_i^{(0)} = \ell - k]| \\
&= \frac{1}{2} \left(\sum_{\ell=B/4}^{B/4+k-1} \Pr[X_i^{(0)} = \ell] + \sum_{\ell=3B/4+1-k}^{3B/4} \Pr[X_i^{(0)} = \ell] \right. \\
&\quad \left. + \sum_{\ell=B/4+k}^{3B/4} |\Pr[X_i^{(0)} = \ell] - \Pr[X_i^{(i)} = \ell]| \right) \\
&\leq k \cdot \Pr[X_i^{(0)} = B/4 + k] \\
&\quad + \frac{1}{2} \sum_{\ell=B/4+k}^{3B/4} |\Pr[X_i^{(0)} = \ell] - \Pr[X_i^{(0)} = \ell - k]|
\end{aligned}$$

The last step follows from the symmetry of $X_i^{(0)}$: $\Pr[X_i^{(0)} = B/2 + \ell] = \Pr[X_i^{(0)} = B/2 - \ell]$, and that the probability is greater for values closer to $B/2$. Observe that the first $B/4 - k/2$ terms in the second sum of the last step are equal to the last $B/4 - k/2$ terms by the symmetry of the distribution around $B/2$ (for $\ell = B/2 + k/2$, the term in the sum is equal to zero). Continuing the derivation and leveraging this symmetry, we get

$$\begin{aligned}
\text{TV}(X_i^{(0)}, X_i^{(i)}) &\leq k \cdot \Pr[X_i^{(0)} = B/4 + k] \\
&\quad + \sum_{\ell=B/4+k}^{B/2+k/2-1} (\Pr[X_i^{(0)} = \ell] - \Pr[X_i^{(0)} = \ell - k]) \\
&= k \cdot \Pr[X_i^{(0)} = B/4 + k] \\
&\quad + \sum_{\ell=0}^{k-1} (\Pr[X_i^{(0)} = B/2 - k/2 + \ell] - \Pr[X_i^{(0)} = B/4 + \ell]) \\
&\leq 2k \cdot \Pr[X_i^{(0)} = B/2]
\end{aligned}$$

where the first equality follows from the sum telescoping and the second inequality follows from bounding each term in the sum. Continuing, letting

$X \sim \text{Bin}(B, 1/2)$, we have that

$$\begin{aligned} 2k \cdot \Pr[X_i^{(0)} = B/2] &= 2k \cdot \Pr\left[X = B/2 \mid \frac{B}{4} \leq X \leq \frac{3B}{4}\right] = \frac{2k \cdot \Pr[X = B/2]}{1 - 2\Pr[X < B/4]} \\ &\leq \frac{2k \cdot \Pr[X = B/2]}{1 - 2\exp(-B/8)} \leq 6k \cdot \Pr[X = B/2] = 6k \cdot 2^{-B} \binom{B}{B/2} \\ &\leq 6k \cdot 2^{-B} \cdot \frac{\sqrt{2} \cdot 2^B}{\sqrt{\pi B}} = \frac{6\sqrt{2}}{\sqrt{\pi}} \cdot \frac{k}{\sqrt{B}} = \frac{6\sqrt{2}}{\sqrt{\pi}} \cdot T^{-1/20} , \end{aligned}$$

where all inequalities are valid for T greater than some absolute constant, and the last inequality is a variant of Stirling's inequality. We finally get

$$\begin{aligned} \text{TV}(\mathbf{x}, \mathbf{x}^{(i)}) &\leq \text{TV}(\mathbf{x}, \mathbf{x}^{(0)}) + \text{TV}(\mathbf{x}^{(0)}, \mathbf{x}^{(i)}) \\ &\leq 2\sqrt{T} \exp(-\sqrt{T}/8) + \frac{6\sqrt{2}}{\sqrt{\pi}} \cdot T^{-1/20} \leq 10 \cdot T^{-1/20} , \end{aligned}$$

where the last inequality is true for $T \geq 400$, thus proving the lemma. $\square$

Using Theorem C.23 and Theorem C.24, we arrive at

$$\begin{aligned} \Pr[\mathcal{A}(\mathbf{x}^{(i)}, \mathbf{x}^{(0)}) \in E_i] &\geq \Pr[\text{err}(\mathcal{M}(\mathbf{x}^{(i)})) \leq \alpha] \cdot \Pr[\text{err}(\mathcal{M}(\mathbf{x}^{(0)})) \leq \alpha] \\ &\geq (\Pr[\text{err}(\mathcal{M}(\mathbf{x})) \leq \alpha] - \text{TV}(\mathbf{x}^{(i)}, \mathbf{x})) \\ &\quad \times (\Pr[\text{err}(\mathcal{M}(\mathbf{x})) \leq \alpha] - \text{TV}(\mathbf{x}^{(0)}, \mathbf{x})) \\ &\geq (3/4 - \Delta')(3/4 - \Delta) \geq 1/2 . \end{aligned}$$

where the last inequality holds for T greater than an absolute constant. Next observe that $\mathbf{x}^{(i)}$ and $\mathbf{x}^{(0)}$ are k -neighboring with probability 1, and so we have $\Pr[\mathcal{A}(\mathbf{x}^{(i)}, \mathbf{x}^{(0)}) \in E_i] \leq e^{k\varepsilon} \Pr[\mathcal{A}(\mathbf{x}^{(0)}, \mathbf{x}^{(0)}) \in E_i]$, and therefore $\Pr[\mathcal{A}(\mathbf{x}^{(0)}, \mathbf{x}^{(0)}) \in E_i] \geq e^{-k\varepsilon}/2$. Since all E_j are disjoint, we get

$$1 \geq \sum_{j=1}^m \Pr[\mathcal{A}(\mathbf{x}^{(0)}, \mathbf{x}^{(0)}) \in E_j] \geq m \cdot e^{-k\varepsilon}/2 ,$$

and therefore

$$k \geq \varepsilon^{-1} \log(m/2) .$$

Recalling that $m = T/B = \sqrt{T}$, we thus have $k = \Omega(\varepsilon^{-1} \log T)$ and therefore $\alpha = k/4 = \Omega(\varepsilon^{-1} \log T)$. $\square$

C.7 Discussion

While there are no known fundamental reasons ruling out asymptotic improvements in the mean squared error for ε -DP—it might be possible to match the

$\Omega(\log(T)^2)$ lower bound of [HUU23]—there have been no asymptotic improvements since the binary tree mechanism [DNP⁺10b; CLS⁺12]. This state of affairs motivates exploring how much leading constants can be improved using the factorization mechanism framework. Algorithm 5 falls into the framework, is easy to implement, efficient, and offers the best known error-scaling to date.

Our paper also reveals that using Laplace noise improves the best possible (ε, δ) -differentially private factorization mechanisms based on Gaussian noise whenever δ is sufficiently small. An interesting future direction is to investigate, for fixed ε, δ and a given factorization: which noise distribution achieves optimal error? It seems unlikely that Laplace or Gaussian noise is always best—indeed, Vinterbo [Vin22] has results of this flavor in more restricted settings.

Finally, we show a lower bound for random inputs that matches the best lower bound known for worst-case inputs. It is still possible that random inputs are easier and allow lower error than what can be achieved for worst-case inputs.

Acknowledgments. We are grateful to Monika Henzinger and Jalaj Upadhyay for discussions on continual counting that inspired part of this work. Joel Daniel Andersson: Supported by a Data Science Distinguished Investigator grant from the Novo Nordisk Fonden and VILLUM FONDEN grant 54451. Rasmus Pagh: Supported by a Data Science Distinguished Investigator grant from the Novo Nordisk Fonden and VILLUM FONDEN grant 54451. Teresa Anna Steiner: Supported by a research grant (VIL51463) from VILLUM FONDEN.

Appendix C.A Definitions for Differential Privacy

Here are the standard definitions for differential privacy which we have use for in the paper.

Definition C.25 ((ε, δ) -Differential Privacy [DR14]). A randomized algorithm $\mathcal{M} : \mathcal{X}^n \rightarrow \mathcal{Y}$ is (ε, δ) -differentially private if for all $S \subseteq \text{Range}(\mathcal{M})$ and all neighboring inputs $\mathcal{D}, \mathcal{D}' \in \mathcal{X}^n$, the neighboring relation written as $\mathcal{D} \sim \mathcal{D}'$, we have that:

$$\Pr[\mathcal{M}(\mathcal{D}) \in S] \leq \exp(\varepsilon) \Pr[\mathcal{M}(\mathcal{D}') \in S] + \delta,$$

where $(\varepsilon, 0)$ -DP is referred to as ε -DP.

Lemma C.26 (Laplacian Mechanism [DR14]). Let $f : \mathcal{X}^n \rightarrow \mathbb{R}^d$ be a function with ℓ_1 -sensitivity $\Delta_1 := \max_{\mathcal{D} \sim \mathcal{D}'} \|f(\mathcal{D}) - f(\mathcal{D}')\|_1$. For a given data set $\mathcal{D} \in \mathcal{X}^n$ and $\varepsilon > 0$, the mechanism that releases $f(\mathcal{D}) + \text{Lap}(\Delta_1/\varepsilon)^d$ satisfies ε -DP.

Lemma C.27 (Gaussian Mechanism [DR14]). Let $f : \mathcal{X}^n \rightarrow \mathbb{R}^d$ be a function with ℓ_2 -sensitivity $\Delta_2 := \max_{\mathcal{D} \sim \mathcal{D}'} \|f(\mathcal{D}) - f(\mathcal{D}')\|_2$. For a given data set $\mathcal{D} \in \mathcal{X}^n$ and $\varepsilon, \delta \in (0, 1)$, the mechanism that releases $f(\mathcal{D}) + \mathcal{N}(0, 2 \ln(1.25/\delta) \Delta_2^2 / \varepsilon^2)^d$ satisfies (ε, δ) -DP.

Appendix C.B k -ary Tree Mechanisms: Proofs

In this subsection, we will briefly discuss the proof sketch for Theorem C.9. It is important to note that while this proof is included, it is not considered a main contribution of this paper.

Proof sketch of Theorem C.9. The utility and privacy follow from Lemmas C.7 and C.8, and the online nature of the mechanism follows from the partial sums being added on Line 11 involving inputs $\mathbf{x}_i$ with $i \leq p \leq t$. For the argument about space usage, a standard argument [CSS11; AP23] can be extended from the binary tree mechanism to k -ary tree mechanism to yield $O(k \log_k T)$. Similarly, the time needed to release T prefix sum will be $O(kT)$ (assuming noise can be generated in constant time) based on extending the analysis of incrementing a binary counter to incrementing a k -ary counter. $\square$

Appendix C.C Leveraging subtraction for even k

In the main part of the paper we only considered subtraction for k -ary trees when k was odd. This part of the appendix is intended for motivating that choice.

First note that for even k we lose the symmetry in the offset digits, so we need to make a decision if the digits in the offset representation are $\{-k/2+1, \dots, k/2\}$ or $\{-k/2, \dots, k/2-1\}$. We pick the first one as that allows for supporting more inputs, giving Theorem C.28.

Definition C.28 (Offset (even) k -ary representation of integers). Let k , w and t be positive integers, with k also being even, satisfying $t \leq \frac{k(k^h-1)}{2(k-1)}$. We then define $\mathbf{t} = \widetilde{\text{rep}}_k(t, w) \in [-\frac{k}{2} + 1, \frac{k}{2}]^w$ as the unique vector with integer entries in $[-\frac{k}{2} + 1, \frac{k}{2}]^w$ satisfying $\sum_{i=1}^w k^{i-1} \mathbf{t}_i = t$.

Substituting the representation used in Algorithm 5 by the one in Theorem C.28 and setting the height appropriately gives a valid algorithm using a k -ary tree. However, it performs worse than using odd k .

Claim C.29. For even $k \geq 4$, $h \geq 1$, and $T = \frac{k(k^h-1)}{2(k-1)}$, Algorithm 5, with h correctly set, achieves a mean squared error of $\frac{kh^3}{2\varepsilon^2} \frac{1}{1-1/k^h} + o(h^2)$ when producing T outputs.

Proof sketch. The same proof as for Theorem C.14 does not work due to the asymmetry in the digits. Instead, we can express a recursion for the number of terms needed. Let c_h be the total number of vertices combined to produce all prefixes that a tree of height h can support. Then we have

$$c_h = \left(\frac{k+2}{4} + \frac{k(h-1)}{4} \right) \frac{k}{2} k^{h-1} + c_{h-1},$$

where $c_0 = 0$. The idea behind the recursion is as follows: the first term corresponds to all the integer representations with positive leading digit, of which there are $k/2 \cdot k^h$, and the leading digit contributes $(k+2)/4$, the remaining $k(h-1)/4$ on average. The second term is the case where the first digit is zero, in which case the contribution is c_{h-1} . Solving the recursion and dividing by T yields the average number of vertices added, $c_h/T = \frac{kh}{4} \frac{1}{1-1/k^h} + o(1)$, which when multiplied by the variance of a vertex, $2h^2/\varepsilon^2$, gives the statement. $\square$

Ignoring the lower order terms, this is worse by a factor $1 - 1/k^2$ over the odd case, and we can also find the optimal k here.

Corollary C.30. Algorithm 5, with modifications for even k , achieves a mean squared error that scales as $\frac{k}{2\varepsilon^2 \log(k)^3} \log(T)^3 + o(\log(T)^3)$, and which is minimized for $k = 20$ yielding $(0.1238/\varepsilon^2) \log(T)^3 + o(\log(T)^3)$.

As the analysis is less elegant, we get more lower order terms and the scaling is worse, we focus on odd k in the paper.

Paper D

Continual Counting with Gradual Privacy Expiration

Abstract. Differential privacy with gradual expiration models the setting where data items arrive in a stream and at a given time t the privacy loss guaranteed for a data item seen at time $(t - d)$ is $\varepsilon g(d)$, where g is a monotonically non-decreasing function. We study the fundamental *continual (binary) counting* problem where each data item consists of a bit, and the algorithm needs to output at each time step the sum of all the bits streamed so far. For a stream of length T and privacy *without* expiration continual counting is possible with maximum (over all time steps) additive error $O(\log^2(T)/\varepsilon)$ and the best known lower bound is $\Omega(\log(T)/\varepsilon)$; closing this gap is a challenging open problem.

We show that the situation is very different for privacy with gradual expiration by giving upper and lower bounds for a large set of expiration functions g . Specifically, our algorithm achieves an additive error of $O(\log(T)/\varepsilon)$ for a large set of privacy expiration functions. We also give a lower bound that shows that if C is the additive error of any ε -DP algorithm for this problem, then the product of C and the privacy expiration function after $2C$ steps must be $\Omega(\log(T)/\varepsilon)$. Our algorithm matches this lower bound as its additive error is $O(\log(T)/\varepsilon)$, even when $g(2C) = O(1)$.

Our empirical evaluation shows that we achieve a slowly growing privacy loss with significantly smaller empirical privacy loss for large values of d than a natural baseline algorithm.

* Full title: *Continual Counting with Gradual Privacy Expiration*, Joel Daniel Andersson (University of Copenhagen, BARC), Monika Henzinger (ISTA), Rasmus Pagh (University of Copenhagen, BARC), Teresa Anna Steiner (ISTA), Jalaj Upadhyay (Rutgers University), NeurIPS'24.

D.1 Introduction

Differential privacy under continual observation [DNP⁺10b; CSS11] has seen a renewed interest recently [Hon15; AP23; FHU23; HUU23; DMP⁺24; HUU24; HU25] due to its application in private learning [DMR⁺22; MT22; CDP⁺23; CMR⁺23; CGH⁺25] and statistics [CLS⁺12; STU17; Upa19; FHO21; KMS⁺21; UUA21; CR22; EMM⁺23; HSS23; THS24]. In this model, the curator gets the database in the form of a stream and is required to output a given statistic continually. Dwork, Naor, Pitassi, and Rothblum [DNP⁺10b] and Chan, Shi, and Song [CSS11] introduced the *binary (tree) mechanism* which allows us to estimate the running count of a binary stream of length T with additive error $O(\log^2(T)/\varepsilon)$ under ε -differential privacy.

The traditional definition of continual observation considers every single entry in the stream equally important for analysis and has equal confidentiality. However, in many applications of continual observation, the data becomes less sensitive with time. For example, consider the case where the stream tracks visits to a certain location or website: it being visited a minute ago may constitute more sensitive information than if it was visited a week ago. To capture such scenarios, Bolot, Fawaz, Muthukrishnan, Nikolov, and Taft [BFM⁺13] defined *privacy with expiration*, where the privacy of a streamed data item decreases as a function of the time elapsed since it was streamed. However, known algorithms for privacy with expiration work only in the setting when we expect no privacy after a certain time has elapsed [BFM⁺13, Section 6].

This lack of algorithms for privacy with expiration influences some real-world design choices [AKV24]. In particular, real-world deployments either allocate every user a “privacy budget” that is diminished every time their data is used, such that their data should not be used once the privacy budget reaches zero, or they account for privacy loss over the time. However, since the data can still be useful, another common approach in these deployments use the heuristic of “refreshing the privacy budget”, i.e., the privacy budget is reset to a positive default value after a prescribed time period, irrespective of how much privacy budget has been used so far. This, for example, was pointed out by Tang, Korolova, Bai, Wang, and Wang [TKB⁺17] in Apple’s first large-scale deployment. However, refreshing the privacy budget is very problematic as the privacy loss is, in the worst case, multiplied by the number of refreshes, for example, if the old data is reused (i.e., the privacy expiration is *linear*).

In this paper, we study continual counting with gradual privacy expiration, generalizing the result in Bolot et al. [BFM⁺13]. Our main contributions are algorithms with the following assets:

- *Improved accuracy.* We achieve an additive error of $O(\log(T)/\varepsilon)$ for a large class of privacy expiration functions and show that this is optimal in a particular sense. This is in contrast to continual counting without

expiration, where there is a gap of a $\log T$ factor [DNP⁺10b]. Our work generalizes the $\Omega(\log(T)/\varepsilon)$ lower bound for continual counting to a wide class of privacy expiration functions and shows that for any additive error C , the product of C and the privacy expiration function after $2C$ steps must be $\Omega(\log T)$. We match this lower bound as our additive error is $O(\log(T)/\varepsilon)$, even when the expiration function after $2C$ steps is a constant.

- *Good scaling.* Our algorithms work for unbounded streams, run in amortized $O(1)$ time per update, $O(\log T)$ space, and offer different trade-offs than conventional continual counting algorithms. In allowing for a growing privacy loss, we show that polylogarithmic privacy expiration is sufficient for optimal additive error, and parameterize the algorithm by the speed of the privacy expiration; as expected, faster privacy expiration yields a smaller error.

Related Works. Before presenting our contributions in detail, we give a brief overview of the most relevant related work. Since Chan, Shi, and Song [CSS11] and Dwork, Naor, Pitassi, and Rothblum [DNP⁺10b], several algorithms have been proposed for privately estimating *prefix-sum under continual observation*, i.e., given a stream of inputs $x_1, x_2, \dots$ from some domain $\mathcal{X}$, output $y_t = \sum_{i \leq t} x_i$ for all $t \geq 1$. *Continual binary counting* is a special case of prefix sum when $\mathcal{X} = \{0, 1\}$ and x_t is provided at time t .

When the input is given as a stream, earlier works improved on the basic binary mechanism under (i) distributional assumptions on data [PAK19], (ii) structural assumptions on data [RN10], and (iii) that the importance of data (both with respect to utility and sensitivity) decreases with elapsed time [BFM⁺13], or (iv) by enforcing certain conditions on the behavior of the output [CMH⁺17]. In recent work, Fichtenberger, Henzinger, and Upadhyay [FHU23] gave algorithms to improve the worst-case non-asymptotic guarantees under continual observation using the *matrix mechanism* [LHR⁺10] and Denisov, McMahan, Rush, Smith, and Guha Thakurta [DMR⁺22] used a similar approach to provide empirical results that minimize the mean squared error. Subsequently, Henzinger, Upadhyay, and Upadhyay [HUU23] showed that the algorithm in [FHU23] achieves almost optimal mean-squared error.

These earlier works are in the traditional definition of privacy under continual observation, i.e., they consider data privacy to be constant throughout the stream. The only exception is the work of Bolot, Fawaz, Muthukrishnan, Nikolov, and Taft [BFM⁺13], which defined differentially private continual release with privacy expiration parameterized by a monotonically non-decreasing function g and gave an algorithm for the special case that the data loses all its confidentiality after a prescribed time. Our work is in this privacy model. There is another line of work motivated by applications in private learning that studies privacy-preserving prefix sum without restricting access to the data points

(such as allowing multiple passes) [CMR⁺23; KMC⁺23] and providing privacy-preserving estimates under various privacy models like shuffling [CGM⁺24]. Since we focus on continual observation, we do not compare our results with this line of work.

The work whose techniques are the most related to ours is the algorithm in Dwork, Naor, Pitassi, and Rothblum [DNP⁺10b] for continual counting satisfying pan-privacy [DNP⁺10a]. Roughly speaking, an algorithm is *pan-private* if it is resilient against intruders who can observe snapshots of the internal states of the algorithms.

Our Contributions

We start by first formally stating the problem. As mentioned above, the focus of this work is privacy with expiration given as Definition 3 in Bolot et al. [BFM⁺13]:

Definition D.1. Let $g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ be a non-decreasing function¹. Let $\mathcal{A}$ be a randomized online algorithm that takes as an input a stream $x_1, x_2, \dots$ and at every time step t outputs $\mathcal{A}(x_1, \dots, x_t)$. $\mathcal{A}$ satisfies ε -differential privacy with expiration (function) g if for all $\tau \geq 1$, for all measurable $S \subseteq \text{range}(\mathcal{A})^*$ ², all possible inputs $x_1, \dots, x_\tau$, all $j \leq \tau$ and all x'_j with $|x_j - x'_j| \in [0, 1]$

$$\Pr[(\mathcal{A}(x_1, \dots, x_j, \dots, x_t))_{t=1}^\tau \in S] \leq e^{g(\tau-j)\varepsilon} \Pr[(\mathcal{A}(x_1, \dots, x'_j, \dots, x_t))_{t=1}^\tau \in S]$$

where the probability is over the coin throws of $\mathcal{A}$. We refer to $g(\tau - j) \cdot \varepsilon$ as the privacy loss.

Letting T be the length of the input stream, the best known bound on the ℓ_∞ -error for continual counting under ε -differential privacy is $O(\log(T)^2/\varepsilon)$, achieved by the algorithms in [DNP⁺10b; CSS11]. Alternatively, the analysis in [DNP⁺10b; CSS11] can be used to show that these algorithms run with $\varepsilon' = \varepsilon \log(T)$ achieve ε -differential privacy with expiration function $g(d) = \log(T)$ for all $d = 1, \dots, T$, and error $O(\log(T)/\varepsilon)$. Our main contribution is to show that better trade-offs are possible: In particular, we can achieve the same error with a *strictly smaller function* g , i.e., we can get an $O(\log(T)/\varepsilon)$ bound on the ℓ_∞ -error with an expiration function of $g(d) \approx \log d$. More generally, our algorithm provides a trade-off between privacy loss and both ℓ_∞ -error and expected ℓ_2^2 -error for all expiration functions $f(d)$ that satisfy (roughly) $f(d) \geq 1 + \log^\lambda(d)$ for any $\lambda > 0$. The exact expiration function g is stated below in Theorem D.2. It also includes a parameter B that allows the privacy loss to be “shifted” by B time steps, i.e., there is no privacy loss in the first B

¹We believe g being *non-increasing* is a typo in Bolot et al. [BFM⁺13].

²“ $*$ ” is the Kleene operator: given a set Σ , Σ^* is the (infinite) set of all finite sequences of elements in Σ .

time steps. If the length T of the stream is unknown, then B is a constant. If T is given to the algorithm, then B can be a function of T .

By Definition D.1, any algorithm satisfying differential privacy with expiration g also fulfills differential privacy with any expiration function that is pointwise at least as large as g . Specifically, for two functions f and g defined on the same domain $\mathcal{D}$, we say $f \succeq g$ if $f(x) \geq g(x)$ for all $x \in \mathcal{D}$. We are now ready to state our main theorem:

Theorem D.2. *Let $\lambda \in \mathbb{R}_{>0} \setminus \{\frac{3}{2}\}$ be a constant, and let parameters $\varepsilon \in \mathbb{R}_{>0}$ and $B \in \mathbb{N}$ be given. There exists an algorithm $\mathcal{A}$ that approximates prefix sums of a (potentially unbounded) input sequence $x_1, x_2, \dots$ with $x_i \in [0, 1]$ satisfying ε -differential privacy with any expiration function f such that $f \succeq g$, where*

$$g(d) = \begin{cases} 0 & \text{for } d < B \\ O(1 + \log^\lambda(d - B + 1)) & \text{for } d \geq B \end{cases}$$

Considering all releases up to and including input t , the algorithm $\mathcal{A}$ uses $O(B + \log t)$ space and $O(1)$ amortized time per input/output pair and has the following error guarantees at each individual time step t for $\beta > 0$,

- $\mathbb{E}_{\mathcal{A}}[(\mathcal{A}(x) - \sum_{i=1}^t x_i)^2] = O(B^2 + \log^{3-2\lambda}(t)/\varepsilon^2),$
- $|\mathcal{A}(x) - \sum_{i=1}^t x_i|_w = O(B + \log^q(t) \sqrt{\log(1/\beta)}/\varepsilon)$ with probability $1 - \beta$ where $q = \max(1/2, 3/2 - \lambda)$.

The case when $\lambda \in \{0, 3/2\}$ is covered in Appendix D.B. Note that choosing $\lambda > 3/2$ implies a constant expected squared error at each time step if $B = O(1)$. Parameter λ controls the trade-off between the *asymptotic* growth of the expiration function and the error, while ε controls the trade-off between *initial* privacy (after B time steps which is $\varepsilon \cdot g(B)$) and the error, which is inversely proportional to ε . Also, for releasing T outputs we have the following corollary.

Corollary D.3. *The algorithm $\mathcal{A}$ with $B = O(\log(T)/\varepsilon)$ and $\lambda \geq 1$ incurs a maximum (over all time steps) additive ℓ_∞ -error of $O(\log(T)/\varepsilon)$ when releasing T outputs with probability $1 - 1/T^c$, for constant $c > 0$, and achieves privacy with expiration function g as in Theorem D.2.*

Finally, we complement our upper bound with the following lower bound shown in Section D.5.

Theorem D.4. *Let $\mathcal{A}$ be an algorithm for binary counting for streams of length T which satisfies ε -differential privacy with expiration h . Let C be an integer such that $\mathcal{A}$ incurs a maximum additive error of at most $C < T/2$ over T time steps with a probability of at least $2/3$. Then*

$$2C \cdot h(2C - 1) \geq \frac{\log(T/(6C))}{2\varepsilon}.$$

Note that Theorem D.4 gives a lower bound for $h(j)$ for a specific j , namely $j = 2C - 1$, and as h is non-decreasing by Definition D.1, the lower bound also holds for all $h(j')$ with $j' \geq j$.

Note additionally that Theorem D.4 shows that our algorithm in Corollary D.3 achieves a tight error bound for the expiration functions with $\lambda \geq 1$ and $B = O(\log(T)/\varepsilon)$. Assume $\mathcal{A}'$ is an algorithm that approximates prefix sums in the continual setting and which satisfies differential privacy with expiration function h and maximum error $C \leq B/2 + 1$ at all time steps with probability at least $2/3$ for an even B . When run on a binary input sequence, $\mathcal{A}'$ solves the binary counting problem. Thus, by Theorem D.4, we have that $2C \cdot h(B + 1) \geq 2C \cdot h(2C - 1) \geq \frac{\log(T/(6C))}{2\varepsilon} \geq \frac{\log(T/(3B+6))}{2\varepsilon} = \Omega(\frac{\log T}{\varepsilon})$.

Now consider the algorithm $\mathcal{A}$ given in Corollary D.3 and note that, by definition of the expiration function g , $g(B + 1) = O(1)$ and that Corollary D.3 shows that $C = O(\log(T)/\varepsilon)$. This is tight as Theorem D.4 shows that for such an expiration function $C = \Omega(\log(T)/\varepsilon)$.

Technical Overview

Central to our work is the event-level pan-private algorithm for continual counting by Dwork et al. [DNP⁺10b]. Similarly to the binary tree algorithm of Chan et al. [CSS11], a noise variable z_I is assigned to every *dyadic interval* I (see Section D.3 for a formal definition) contained in $[0, T - 1]$. Let this set of dyadic intervals be called $\mathcal{I}$. In the version of the binary tree algorithm of Chan et al., the noise added to the sum of the values so far (i.e. the non-private output) at any time step $1 \leq t \leq T$ is equal to $\sum_{I \in D_{[0, t-1]}} z_I$, where $D_{[0, t-1]} \subseteq \mathcal{I}$ is the dyadic interval decomposition of the interval $[0, t - 1]$. The pan-private algorithm adds different noise to the output: it adds at time t the *noises for all intervals containing $t - 1$* , i.e., $\sum_{I \in \{I \in \mathcal{I} : t-1 \in I\}} z_I$. This pan-private way of adding noise helps us bound the privacy loss under expiration. For two neighboring streams differing at time step j , we can get the same output at $\tau \geq j$ by shifting the values of the noises of a set of disjoint intervals covering $[j, \tau]$ each by at most 1. Using that $|D_{[j, \tau]}| = O(\log(\tau - j + 1))$, we show that the algorithm satisfies a logarithmic privacy expiration.

In our algorithm, we make four changes to the above construction (i.e., the pan-private construction in Dwork et al. [DNP⁺10b]): **(i)** We do not initialize the counter with noise separately from that introduced by the intervals. **(ii)** We split the privacy budget unevenly across the levels of the dyadic interval set instead of uniformly allocating it. This allows us to control the asymptotic growth of the expiration function, and the error. This change, however, requires a more careful privacy analysis. **(iii)** At time t we add noise identified by intervals containing t , not $t - 1$. While this is a subtle difference, it allows us to exclude intervals starting at 0 from $\mathcal{I}$, leading to our algorithm running on *unbounded* streams with utility that depends on the current time step t . Said differently, our algorithm does not need to know the stream length in advance.

This is in contrast to Dwork et al. [DNP⁺10b] where the construction requires an upper bound T on the length of the stream so that the utility guarantee at each step is fixed and a function of T . (iv) We allow for a delay of B , meaning we output 0 for the initial B steps. This gives perfect privacy for the first B steps, and, since each element of the stream is in $[0, 1]$, the delayed start leads only to an additive error of $O(B)$.

D.2 Preliminaries

Let $\mathbb{N}_{>0}$ denote the set $\{1, 2, \dots\}$ and $\mathbb{R}_{\geq 0}$ the set of non-negative real numbers. We use the symbol g to denote the function that defines the privacy expiration, i.e., $g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. We fix the symbol $\mathcal{A}(x)$ to denote the randomized streaming algorithm that, given an input $x = x_1, x_2, \dots$ as a stream, provides ε -differential privacy with expiration g . All algorithms *lazily draw noise*, meaning that a “noise” random variable is only drawn when first used and is re-used and *not* re-drawn when referenced again. For a random variable, Z , we use $\text{support}(Z)$ to denote its support. For a sequence of random variables $Z = Z_1, Z_2, \dots$, we use $\text{support}(Z)$ to denote $\text{support}(Z_1) \times \text{support}(Z_2) \times \dots$.

Helpful lemmas. We now collect some helpful lemmas shown formally in Appendix D.B. To show privacy with expiration in the following sections, we repeatedly use the following observation: a similar lemma has been used to show the standard definition of differential privacy, e.g., in the proof of Theorem 2.1 of Dwork et al. [DNR⁺15]. Informally, it says that if there exists a map q between random choices made by algorithm $\mathcal{A}$ such that for any input x and fixed sequence z of random choices, the map returns a sequence $q(z)$ such that (1) the output $\mathcal{A}(x, z)$ equals the output $\mathcal{A}(x', q(z))$ and (2) the probability of picking z is similar to the probability of picking $q(z)$, then $\mathcal{A}$ is private. The notion of “similar probability” is adapted to the definition of differential privacy with expiration and depends on the function g . All the results in this section are shown formally in Appendix D.B:

Fact D.5. *Consider an algorithm $\mathcal{A}$ that uses a sequence of random variables $Z = Z_1, Z_2, \dots$ as the only source of randomness. We can model $\mathcal{A} : \chi \times \text{support}(Z)$ as a (deterministic) function of its actual input from the universe χ and the sequence of its random variables Z . Suppose that for all $\tau \in \mathbb{N}_{>0}$, $j \leq \tau$ and all neighboring pairs of input streams $x = x_1, \dots, x_j, \dots, x_\tau$ and $x' = x_1, \dots, x'_j, \dots, x_\tau$, there exists a function $q : \text{support}(Z) \rightarrow \text{support}(Z)$ such that $\mathcal{A}(x; z) = \mathcal{A}(x'; q(z))$ and*

$$\Pr_{z \sim Z}[z \in \mathcal{N}] \leq e^{\varepsilon g(\tau-j)} \Pr_{z' \sim Z}[z' \in q(\mathcal{N})] \quad \text{for all } \mathcal{N} \subseteq \text{support}(Z).$$

Then $\mathcal{A}$ satisfies ε -differential privacy with expiration g .

Algorithm 7 $\mathcal{A}_{\text{simple}}$: Continual counting under linear privacy expiration

```

1: Input: A stream  $x_1, x_2, \dots$ , privacy parameter  $\varepsilon$ 
2: Lazily3 draw  $Z_{t-1} \sim \text{Lap}(\frac{1}{\varepsilon})$ 
3: At time  $t = 1$ , output 0
4: for  $t = 2$  to  $\infty$  do
5:   At time  $t$ , output  $\sum_{i=1}^{t-1} x_i + Z_{t-1}$ 
6: end for

```

Lemma D.6. *Let $Z = Z_1, Z_2, \dots, Z_k$ be a sequence of independent Laplace random variables, such that $Z_i \sim \text{Lap}(b_i)$ for $b_i > 0$, for all $i \in [k]$. Let q be a bijection $q : \text{support}(Z) \rightarrow \text{support}(Z)$ of the following form: For all $\Delta := (\Delta_1 \Delta_2 \dots \Delta_k) \in \mathbb{R}^k$, and for all $z \in \text{support}(Z)$, we have $q(z) = z + \Delta \in \text{support}(Z)$. Then for all $\mathcal{N} \subseteq \text{support}(Z)$ we have*

$$\Pr_{z \in Z}[z \in \mathcal{N}] \leq e^s \Pr_{z \in Z}[z \in q(\mathcal{N})], \quad \text{where} \quad s = \sum_{i=1}^k \frac{|\Delta_i|}{b_i}.$$

D.3 Warmup

As a warm-up, we give two simple algorithms for two obvious choices of the expiration function: the linear expiration function $g(d) = d$ and the logarithmic expiration function $g(d) = 2 \log(d + 1) + 2$.

A Simple Algorithm with Linear Privacy Expiration

First, we consider a simple algorithm which gives ε -differential privacy with expiration $g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$, where $g(d) = d$. The maximum error of this algorithm over T time steps is bounded by $O(\varepsilon^{-1} \log(T/\beta))$, with probability at least $1 - \beta$. The algorithm $\mathcal{A}_{\text{simple}}$ is given in Algorithm 7. It adds fresh Laplace noise to any output sum. Note that this is the same algorithm as the Simple Counting Mechanism I from [CSS11]. However, we show that for the weaker notion of differential privacy with linear expiration, Laplace noise with *constant* scale suffices, even though the sensitivity of $\mathcal{A}_{\text{simple}}$ running on a stream of length T is T . To prove this, we show that for two neighbouring streams differing at time step j , we obtain the same output by “shifting” the values of the Laplace noises for all outputs after step j by at most 1. We defer the proof of the following lemma to Appendix D.B.

Lemma D.7. *The algorithm $\mathcal{A}_{\text{simple}}$, given in Algorithm 7, is ε -differentially private with expiration g , where $g : \mathbb{N} \rightarrow \mathbb{N}$ is the identity function $g(d) = d$ for all $d \in \mathbb{N}$. It incurs a maximum additive error of $O(\varepsilon^{-1} \log(T/\beta))$ over all T time steps simultaneously with probability at least $1 - \beta$.*

³By “lazily”, we mean we draw a random variable the first time it is used by the algorithm.

Algorithm 8 $\mathcal{A}_{\log}$: Continual counting under logarithmic privacy expiration

-
- 1: **Input:** A stream $x_1, x_2, \dots$, privacy param ε k
 - 2: Let $\mathcal{I}$ be the dyadic interval set on $[1, \infty)$
 - 3: $\forall I \in \mathcal{I}$, lazily draw i.i.d. $Z_I \sim \text{Lap}(\frac{1}{\varepsilon})$
 - 4: Set $\mathcal{I}_t = \{I \in \mathcal{I} : t \in I\}$
 - 5: **for** $t = 1$ **to** ∞ **do**
 - 6: At time t , output $\sum_{i=1}^t x_i + \sum_{I \in \mathcal{I}_t} Z_I$
 - 7: **end for**
-

An Algorithm with Logarithmic Privacy Expiration

Next, we show how an algorithm similar to the binary tree algorithm [DNP⁺10b] gives ε -differential privacy with expiration $g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$, where $g(d) = 2 \log(d + 1) + 2$. This result can also be derived from Theorem D.2 by setting $\lambda = 1$ and $B = 0$. As in the case when $g(d) = d$, the maximum error of this algorithm over T time steps is again bounded by $O(\varepsilon^{-1} \log(T/\beta))$, with probability at least $1 - \beta$. Similarly to the binary tree algorithm, we define a noise variable for every node in the tree, but we do this in the terminology of *dyadic intervals*. We consider the *dyadic interval set* $\mathcal{I}$ on $[1, \infty)$ (formally defined shortly), associate a noise variable z_I with each interval $I \in \mathcal{I}$, and at time step t add noise z_I for each $I \in \mathcal{I}$ that contains t . This is similar to the construction in Dwork et al. [DNP⁺10b], with the exception that they instead consider the dyadic interval set on $[0, T - 1]$, add noise z_I at time t if $t - 1 \in I$, and initialize their counter with noise from the same distribution. Our choice of $\mathcal{I}$ allows the algorithm to run on unbounded streams, and leads to adding up $1 + \lfloor \log(t) \rfloor$ noise terms at step t rather than $1 + \lfloor \log(T) \rfloor$. For privacy, we will argue that if two streams differ at time j , then we get the same outputs up to time $\tau \geq j$ by considering a subset of disjoint intervals in $\mathcal{I}$ covering $[j, \tau]$, and shifting the associated Laplace random variables appropriately. In the following, we describe this idea in detail. We start by describing the dyadic interval decomposition of an interval.

Dyadic interval decomposition. For every non-negative integer ℓ , we divide $[1, \infty)$ into disjoint intervals of length 2^ℓ : $\mathcal{I}^\ell = \{[k \cdot 2^\ell, (k+1) \cdot 2^\ell - 1], k \in \mathbb{N}_{>0}\}$. We call $\mathcal{I} = \bigcup_{\ell=0}^{\infty} \mathcal{I}^\ell$ the *dyadic interval set* on $[1, \infty)$, and $\mathcal{I}^\ell$ the ℓ -th level of the dyadic interval set. We show the following two facts in Appendix D.B.

Fact D.8. *Let $\mathcal{I}$ be the dyadic interval set on $[1, \infty)$. For any interval $[a, b]$, $1 \leq a \leq b$, there exists a set of intervals $D_{[a,b]} \subseteq \mathcal{I}$, referred to as the dyadic interval decomposition of $[a, b]$, such that (i) the sets in $D_{[a,b]}$ are disjoint; (ii) $\bigcup_{I \in D_{[a,b]}} I = [a, b]$; and (iii) $D_{[a,b]}$ contains at most 2 intervals per level, and the highest level ℓ of an interval satisfies $\ell \leq \log(b - a + 1)$*

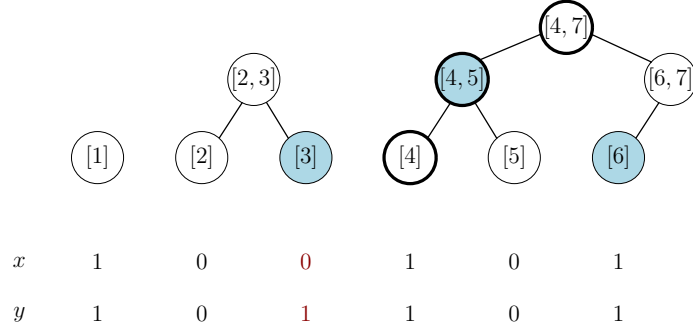


Figure D.1: An example of the noise structure for Algorithm 8 and Algorithm 9 for $B = 0$ on two neighbouring streams x and y differing in position 3. The nodes correspond to the dyadic intervals. The filled nodes mark the intervals I for which the noise Z_I is shifted by one between x and y to get the same outputs for $\tau = 6$. The fat nodes mark the intervals I corresponding to the Z_I which are used in the computation of the fourth prefix sum s_4 .

Fact D.9. Let $\mathcal{I}$ be the dyadic interval set on $[1, \infty)$, and for $t \in \mathbb{N}_{>0}$ define $\mathcal{I}_t = \{I \in \mathcal{I} : t \in I\}$ as the intersection of t with $\mathcal{I}$. Then $|\mathcal{I}_t| = \lfloor \log t \rfloor + 1$.

Lemma D.10. The algorithm $\mathcal{A}_{\log}$ given in Algorithm 8 satisfies ε -differential privacy with expiration g , where $g : \mathbb{N} \rightarrow \mathbb{R}$ is defined as $g(x) = 2 \log(x+1) + 2$. It incurs a maximum additive error of $O(\varepsilon^{-1} \log(T/\beta))$ over all T time steps simultaneously with probability at least $1 - \beta$.

Privacy. We use Fact D.5 and Lemma D.6 to argue privacy of $\mathcal{A}_{\log}$: Let x and x' differ at time j . Note that the prefix sums fulfill the following properties: (i) $\sum_{i=1}^t x_i = \sum_{i=1}^t x'_i$ for all $t < j$ and (ii) $\sum_{i=1}^t x'_i = \sum_{i=1}^t x_i + y$ for all $t \geq j$, where $y = x'_j - x_j \in [-1, 1]$.

In the following, we refer to the output of the algorithm run on input x and with values of the random variables z as $\mathcal{A}_{\log}(x; z)$. Let $\tau \geq j$ be given, and consider $S \subseteq \text{range}(\mathcal{A}_{\log})^*$. Let $Z = (Z_I)_{I \in \mathcal{I}}$ be the sequence of Laplace random variables used by the algorithm. For any fixed output sequence $s \in S$, let $z = (z_I)_{I \in \mathcal{I}}$ be a sequence of values that the Laplace random variables need to assume to get output sequence s for input x . That is $\sum_{i=1}^t x_i + \sum_{I \in \mathcal{I}_t} z_I = s_t$ for $t \geq 1$. Let $D_{[j, \tau]}$ be the decomposition of $[j, \tau]$ as defined in Fact D.8. We define a bijection q satisfying the properties of Fact D.5 as follows: $q(z) = z' = (z'_I)_{I \in \mathcal{I}}$ such that

$$z'_I = \begin{cases} z'_I = z_I & \forall I \notin D_{[j, \tau]} \\ z'_I = z_I + y & \forall I \in D_{[j, \tau]} \end{cases}.$$

We show the two properties needed to apply Fact D.5:

(1) Note that $(\mathcal{A}_{\log}(x; z))_t = \sum_{i=1}^t x_i + \sum_{I \in \mathcal{I}_t} z_I$. For $t < j$, we have $t \notin [j, \tau]$ and therefore $t \notin I$ for any $I \in D_{[j, \tau]}$. Therefore, we have

$$\sum_{i=1}^t x_i + \sum_{I \in \mathcal{I}_t} z_I = \sum_{i=1}^t x'_i + \sum_{I \in \mathcal{I}_t} z_I = \sum_{i=1}^t x'_i + \sum_{I \in \mathcal{I}_t} z'_I.$$

For $j \leq t \leq \tau$, we have that t is contained in exactly one $I \in D_{[j, \tau]}$. Thus, $z'_I = z_I + y$ for exactly one $I \in \mathcal{I}_t$, and $z'_{I'} = z_{I'}$ for all $I' \in \mathcal{I}_t \setminus \{I\}$. Further, since $t \geq j$, we have that $\sum_{i=1}^t x_i = \sum_{i=1}^t x'_i - y$. Together, this shows the first property of Fact D.5 as

$$\sum_{i=1}^t x_i + \sum_{I \in \mathcal{I}_k} z_I = \sum_{i=1}^t x'_i - y + \sum_{I \in \mathcal{I}_t} z'_I + y = \sum_{i=1}^t x'_i + \sum_{I \in \mathcal{I}_t} z'_I,$$

(2) By Fact D.8, $|D_{[j, \tau]}| \leq 2(\log(\tau - j + 1) + 1)$. Thus, by Lemma D.6 for any $\mathcal{N} \in \text{support}(Z)$,

$$\Pr_{z \in Z}[z \in \mathcal{N}] \leq e^{\sum_{I \in D_{[j, \tau]}} |y| \varepsilon} \Pr_{z \in Z}[z \in q(\mathcal{N})] \leq e^{2(\log(\tau - j + 1) + 1) \varepsilon} \Pr_{z \in Z}[z \in q(\mathcal{N})],$$

so the second property of Fact D.5 is fulfilled with $g(x) = 2\log(x + 1) + 2$. By Fact D.5, we have differential privacy with privacy expiration $g(x) = 2\log(x + 1) + 2$.

Accuracy. To show accuracy at step t , let $Y_t = \sum_{I \in \mathcal{I}_t} Z_I$, i.e. the noise added at time step t . By Fact D.9 we add $k = |\mathcal{I}_t| = \lfloor \log t \rfloor + 1$ Laplace noises with scale $\frac{1}{\varepsilon}$. Let $M_{t, \beta} = \max \left\{ \sqrt{\lfloor \log t \rfloor + 1}, \sqrt{\ln(2/\beta)} \right\}$. By Corollary D.16, we have that $\Pr \left[|Y_t| > \frac{2}{\varepsilon} \sqrt{2 \ln(2/\beta)} M_{t, \beta} \right] \leq \beta$, for any $\beta < 1$. Setting $\beta = \beta'/T$, it follows that with probability at least $1 - \beta'$, $|Y_t| = O(\varepsilon^{-1} \log(T/\beta'))$ for all time steps $t \leq T$ simultaneously.

D.4 Proof of Theorem D.2 and Corollary D.3

Section D.3 shows that we can obtain an error smaller than the binary mechanism by using differential privacy with a logarithmic expiration function. Here we show a general trade-off between the expiration function's growth and the error's growth. Two techniques are needed to show the theorem:

Delay. All outputs are delayed for B steps. That is, in the first B steps, the mechanism outputs 0, thereafter, it outputs a private approximation of $\sum_{i=1}^{j-B} x_i$. Delay introduces an extra additive error of up to B , but ensures perfect privacy for the first B time steps after receiving an input.

Budgeting across levels. The privacy budget is split unevenly across levels of the dyadic interval set in order to control the asymptotic growth of the expiration

Algorithm 9 Continual counting, gradual privacy expiration

```

1: Input: A stream  $x_1, x_2 \dots$ , privacy parameter  $\varepsilon$ ,
   parameters  $B \in \mathbb{N}$ ,  $\lambda \in \mathbb{R}_{>0} \setminus \{3/2\}$ 
2: Let  $\mathcal{I}$  be the dyadic interval set on  $[1, \infty)$  and  $\mathcal{I}_t = \{I \in \mathcal{I} : t - B \in I\}$ 
3:  $\forall \ell, I \in \mathcal{I}^\ell$ , lazily draw i.i.d.  $Z_I \sim \text{Lap}\left(\frac{(1+\ell)^{1-\lambda}}{\varepsilon}\right)$ 
4: for  $t = 1$  to  $B$  do
5:   At time  $t$ , output 0
6: end for
7: for  $t = B + 1$  to  $\infty$  do
8:   At time  $t$ , output  $s_{t-B} = \sum_{i=1}^{t-B} x_i + \sum_{I \in \mathcal{I}_t} Z_I$ 
9: end for

```

function. Specifically, the budget at level ℓ is chosen to be proportional to $(\ell + 1)^{\lambda-1}$. The case $\lambda = 1$ corresponds to the even distribution used in the construction of Section D.3.

Our algorithm is shown as Algorithm 9. In the following, we refer to the output of the algorithm run on input x and with values of the random variables z as $\mathcal{A}(x; z)$.

Privacy. We now show that $\mathcal{A}$ satisfies Definition D.1. For x and x' that differ (only) at time $j \leq \tau$, the prefix sums fulfill the following properties:

- $\sum_{i=1}^t x_i = \sum_{i=1}^t x'_i$ for all $t < j$ and
- $\sum_{i=1}^t x'_i = \sum_{i=1}^t x_i + y$ for all $j \leq t \leq \tau$, where $y = x'_j - x_j \in [-1, 1]$.

Let j and τ be defined as in Definition D.1. Due to the delay, if $\tau' = \tau - B < j$ (corresponding to $d < B$) the privacy claim is immediate since the output distributions of $\mathcal{A}$ up to step τ are identical on the two inputs. Otherwise, for $\tau' = \tau - B \geq j$, i.e., for $d \geq B$, we wish to use Fact D.5 and Lemma D.6. Let $Z = (Z_I)_{I \in \mathcal{I}}$ be the sequence of Laplace random variables used by $\mathcal{A}$. For input x consider a fixed length- τ output sequence consisting of B zeros followed by $s_1, \dots, s_{\tau'}$. Let $z = (z_I)_{I \in \mathcal{I}}$ be a sequence of values for the Laplace random variables in order to produce this output sequence with input x . That is, $s_t = \sum_{i=1}^t x_i + \sum_{I \in \mathcal{I}_t} z_I$ for $t \geq 1$. Let $D_{[j, \tau']}$ be the decomposition of $[j, \tau']$ as defined in Fact D.8. We define a bijection q satisfying the properties of Fact D.5 as follows: $q(z) = z' = (z'_I)_{I \in \mathcal{I}}$ such that

$$z'_I = \begin{cases} z'_I = z_I & \forall I \notin D_{[j, \tau']} \\ z'_I = z_I + y & \forall I \in D_{[j, \tau']} \end{cases}.$$

We show the two properties needed to apply Fact D.5:

Lemma D.11. *The function $q(z)$ satisfies $\mathcal{A}(x; z) = \mathcal{A}(x'; q(z))$ and for $g(d) = O(1 + \frac{1}{\lambda} \log^\lambda(d - B + 1))$ we have $\Pr_{z \sim Z}[z \in \mathcal{N}] \leq e^{\varepsilon g(\tau-j)} \Pr[z' \in \mathcal{N}]$ for all $\mathcal{N} \subseteq \text{support}(Z)$.*

Space and time. Algorithm 9 can update the sums $\sum_{i=1}^{t-B} x_i$ and $\sum_{I \in \mathcal{I}_t} Z_I$ in each time step t using the following idea: The B most recent inputs are kept in a buffer to allow calculation of prefix sums with delay and also the $\lfloor \log(t) \rfloor + 1$ random variables of those values that were added to the most recent output. At a given step, each random value that is no longer used is subtracted from the most recent output, and each new random value is added. An amortization argument as in the analysis of the number of bit flips in a binary counter yields the $O(1)$ amortized bound.

Accuracy. To show the accuracy guarantee, we need to account for the error due to delay as well as the noise required for privacy. It is easy to see that the delay causes an error of at most B , since the sum of any B inputs is bounded by B . Thus, for both error bounds it remains to account for the error due to noise. At every time step t after B , the output is the delayed prefix sum plus a sum of Laplace distributed noise terms as indicated by $\mathcal{I}_{t-B}$ with parameters $b_\ell = (\ell + 1)^{1-\lambda}/\varepsilon$, where $\ell = 0, \dots, \lfloor \log(t - B) \rfloor$. To bound the variance of the noise, $2 \sum_\ell b_\ell^2$ we compute:

$$\begin{aligned} \sum_{\ell=0}^{\lfloor \log(t-B) \rfloor} b_\ell^2 &= \frac{1}{\varepsilon^2} \sum_{\ell=0}^{\lfloor \log(t-B) \rfloor} (\ell + 1)^{2(1-\lambda)} \leq \frac{1}{\varepsilon^2} \left(1 + \int_1^{\log(t)+2} x^{2(1-\lambda)} dx \right) \\ &\leq \frac{1}{\varepsilon^2} \left(1 + \left[\frac{1}{3-2\lambda} x^{3-2\lambda} \right]_{x=1}^{x=\log(t)+2} \right) = O\left(\frac{1 + \log(t)^{3-2\lambda}}{\varepsilon^2} \right). \end{aligned}$$

This calculation assumes $\lambda \neq 3/2$, proving the statement on the squared error in Theorem D.2. For the high probability bound we invoke Lemma D.15 with $b_M = \max_\ell(b_\ell) = \max(1, \log(2t)^{1-\lambda})/\varepsilon = O((1 + \log(t)^{1-\lambda})/\varepsilon)$. For $\nu = \sqrt{\sum_\ell b_\ell^2} + b_M \sqrt{\log t} = O(b_M \sqrt{\log t})$, applying Lemma D.15 says that the error from the noise is $O(\nu \sqrt{\log(1/\beta)}) = O(\sqrt{\log(1/\beta)} \log(t)^{\max(0.5, 1.5-\lambda)})$ with probability $1 - \beta$, proving Theorem D.2.

Proof of Corollary D.3. For releasing T outputs, choosing $\beta = 1/T^{c+1}$, $c > 0$ being a constant, and using a union bound over all outputs gives a bound on the maximum noise equal to $O(\log(T)^{\max(1, 2-\lambda)}/\varepsilon)$ with probability $1 - 1/T^c$, proving Corollary D.3.

D.5 Lower Bound on the Privacy Expiration

The lower bound follows from a careful packing argument.

Theorem D.12. *Let $\mathcal{A}$ be an algorithm for binary counting for streams of length T which satisfies ε -differential privacy with expiration g . Let C be an integer and assume that the additive error of $\mathcal{A}$ is bounded by $C < T/2$ at all time steps with a probability of at least $2/3$. Then*

$$\sum_{j=0}^{2C-1} g(j) \geq \log(T/6C)/\varepsilon.$$

Proof. We define $x^{(0)} = 0^T$ to be the 0 stream of length T . Let $T' \leq T$ such that $2C$ divides T' , and $T - T' < 2C$. Since $T \geq 2C$ we have $T' \geq 2C$ and $T' \geq T/2$. We define $T'/(2C)$ data sets $x^{(1)}, \dots, x^{(T'/(2C))}$ as follows:

$$x_t^{(i)} := \begin{cases} 1 & t \in [2C(i-1) + 1, 2Ci] \\ 0 & \text{otherwise} \end{cases}.$$

For $i \in [T'/(2C)]$, let S_i be the set of all output sequences $a_1, \dots, a_T \in \text{range}(\mathcal{A})$ satisfying $a_{2Ci} > C$, and $a_{2Ck} < C$ for all $k < i$. Note that $S_i \cap S_j = \emptyset$ for $i \neq j$ where $i, j \in [T'/(2C)]$. Further, since by assumption $\mathcal{A}$ has error at most C at all times steps with probability at least $2/3$, we have that

$$\Pr[\mathcal{A}(x_1^{(i)}, \dots, x_t^{(i)})_{t=1}^{2Ci} \in S_i] \geq 2/3.$$

Recall that $x^{(0)}$ and $x^{(i)}$ differ in exactly the positions $t \in [2C(i-1) + 1, 2Ci]$. By the assumption that $\mathcal{A}$ satisfies ε -differential privacy with expiration g ,

$$\begin{aligned} 2/3 &\leq \Pr[\mathcal{A}(x_1^{(i)}, \dots, x_t^{(i)})_{t=1}^{2Ci} \in S_i] \\ &\leq e^{\sum_{j=0}^{2C-1} g(j)\varepsilon} \Pr[\mathcal{A}(x_1^{(0)}, \dots, x_t^{(0)})_{t=1}^{2Ci} \in S_i] \\ &= e^{\sum_{j=0}^{2C-1} g(j)\varepsilon} \Pr[\mathcal{A}(x_1^{(0)}, \dots, x_t^{(0)})_{t=1}^{T'} \in S_i]. \end{aligned}$$

Now, since the S_i 's are disjoint, we have

$$\begin{aligned} 1 &\geq \sum_{i=1}^{T'/(2C)} \Pr[\mathcal{A}(x_1^{(0)}, \dots, x_t^{(0)})_{t=1}^{T'} \in S_i] \\ &\geq \sum_{i=1}^{T'/(2C)} (2/3) e^{-\sum_{j=0}^{2C-1} g(j)\varepsilon} \\ &= (T'/(2C))(2/3) e^{-\sum_{j=0}^{2C-1} g(j)\varepsilon}. \end{aligned}$$

It follows that

$$\sum_{j=0}^{2C-1} g(j)\varepsilon \geq \log(2T'/(6C)) \geq \log(T/6C)$$

completing the proof. $\square$

By Definition D.1, $g(j)$ is non-decreasing in j . This immediately gives Theorem D.4.

D.6 Conclusion

In this work, we give the first algorithm for the continual counting problem for privacy with expiration for a wide range of expiration functions and characterize for which expiration functions it is possible to get an ℓ_∞ -error of $O(\log(T)/\varepsilon)$. We also give a general lower bound for any such algorithms and show that ours is tight for certain expiration functions. It would be interesting to study this model further, e.g., with slower-growing expiration functions, and also algorithms for other problems in continual observation, such as maintaining histograms and frequency-based statistics over changing data. Specifically, it would be an interesting direction for future research to study problems in this model, where a polynomial error gap between the batch model and the continual release model is known to exist (for example max sum [JRS⁺23] and counting distinct elements [JKR⁺23; HSS24]), and to see if this model allows for new trade-offs. Further, one of the main applications of continual counting algorithms is in privacy-preserving federated learning algorithms, specifically in stochastic gradient descent (see e.g. [KMS⁺21; DMR⁺22; MT22]). It would be interesting to explore how our algorithm can be deployed in this setting.

Though the concept of privacy expiration has not been defined for approximate differential privacy, it is natural to wonder if there exist analogous results in this setting, which, in general, allows better privacy-utility trade-offs. We note that for ρ -zero-concentrated differential privacy [BS16] there is a natural analog of Definition D.1 for which it seems possible to prove results analogous to those shown here for pure differential privacy. In that context it would also be interesting to see whether the matrix mechanism can be used to improve the constants in the error, similar to [FHU23; HU25].

Acknowledgements

Monika Henzinger: This project has received funding from the European Research Council (ERC) under the European Union’s Horizon 2020 research and innovation programme (Grant agreement No. 101019564) and the Austrian Science Fund (FWF) grant DOI 10.55776/Z422, grant DOI 10.55776/I5982, and grant DOI 10.55776/P33775 with additional funding from the netidee SCIENCE Stiftung, 2020–2024.



Joel Daniel Andersson and Rasmus Pagh are affiliated with Basic Algorithms Research Copenhagen (BARC), supported by the VILLUM Foundation grant 16582, and are also supported by Providentia, a Data Science Distinguished Investigator grant from Novo Nordisk Fonden.

Teresa Anna Steiner is supported by a research grant (VIL51463) from VILLUM FONDEN. This work was done while Teresa Anna Steiner was a Postdoc at the Technical University of Denmark.

Jalaj Upadhyay's research was funded by the Rutgers Decanal Grant no. 302918 and an unrestricted gift from Google.

Appendix D.A Background on Random Variables

Definition D.13 (Laplace Distribution). The *Laplace distribution* centered at 0 with scale b is the distribution with probability density function

$$f_{\text{Lap}}(b)(x) = \frac{1}{2b} \exp\left(\frac{-|x|}{b}\right).$$

We use $X \sim \text{Lap}(b)$ or just $\text{Lap}(b)$ to denote a random variable X distributed according to $f_{\text{Lap}}(b)(x)$.

Fact D.14 (Laplace Tailbound). *If $X \sim \text{Lap}(b)$, then*

$$\Pr[|X| > t \cdot b] \leq e^{-t}.$$

Lemma D.15 (Measure Concentration Lemma in [CSS11]). *Let $Y_1, \dots, Y_k$ be independent variables with $Y_i \sim \text{Lap}(b_i)$ for all $i \in [k]$. Denote $b_M = \max_{i \in [k]} b_i$ and $Y = \sum_{i=1}^k Y_i$. Let $0 < \beta < 1$ and $\nu > \max\left\{\sqrt{\sum_{i=1}^k b_i^2}, b_M \sqrt{\ln(2/\beta)}\right\}$. Then,*

$$\Pr\left[|Y| > \nu \sqrt{8 \ln(2/\beta)}\right] \leq \beta.$$

Corollary D.16. *Let $Y_1, \dots, Y_k$ be independent variables with distribution $\text{Lap}(b)$ and let $Y = \sum_{i=1}^k Y_i$. Let $0 < \beta < 1$. Then*

$$\Pr\left[|Y| > 2b\sqrt{2 \ln(2/\beta)} \max\left\{\sqrt{k}, \sqrt{\ln(2/\beta)}\right\}\right] \leq \beta.$$

Proof. Apply Lemma D.15 to $b_1 = \dots = b_k = b$. □

Appendix D.B Omitted proofs

Omitted proofs from Section D.2

Proof of Fact D.5. Given x, x' and any set of output sequences set S , let $\mathcal{N}_S$ be the set of all choices of random variables z such that $\mathcal{A}(x; z) \in S$. Note that for all $z \in \mathcal{N}_S$, $\mathcal{A}(x'; q(z)) \in S$, i.e., for all $z' \in q(\mathcal{N}_S)$, $\mathcal{A}(x'; z') \in S$. Then

$$\begin{aligned} \Pr_{z \sim Z}[\mathcal{A}(x; z) \in S] &= \Pr_{z \sim Z}[z \in \mathcal{N}_S] \leq e^{\varepsilon g(\tau-j)} \Pr_{z' \sim Z}[z' \in q(\mathcal{N}_S)] \\ &\leq e^{\varepsilon g(\tau-j)} \Pr_{z' \sim Z}[\mathcal{A}(x'; z') \in S]. \end{aligned}$$

This completes the proof of Theorem D.5. □

Proof of Lemma D.6. Let $f_{\text{Lap}}(b)$ denote the density function of $\text{Lap}(b)$. For all $x, y \in \mathbb{R}$:

$$f_{\text{Lap}}(b)(x) = \frac{1}{2b} e^{-|x|/b} \leq \frac{1}{2b} e^{(|y|-|x+y|)/b} = e^{|y|/b} \frac{1}{2b} e^{-(|x+y|)/b} = e^{|y|/b} f_{\text{Lap}}(b)(x+y),$$

where the inequality follows from the triangle inequality $|x+y| \leq |x| + |y|$. In the following integral, let $z' = q(z)$. Let s be as defined in the lemma statement. Then we have

$$\begin{aligned} \Pr_{z \in Z}[z \in \mathcal{N}] &= \int_{z \in \mathcal{N}} \prod_{i=1}^k f_{\text{Lap}}(b_i)(z_i) dz \leq \int_{z \in \mathcal{N}} \prod_{i=1}^k e^{|z'_i - z_i|/b_i} f_{\text{Lap}}(b_i)(z'_i) dz \\ &= e^s \int_{z \in \mathcal{N}} \prod_{i=1}^k f_{\text{Lap}}(b_i)(z'_i) dz = e^s \int_{z \in q(\mathcal{N})} \prod_{i=1}^k f_{\text{Lap}}(b_i)(z_i) dz \\ &= e^s \Pr_{z \sim Z}[z \in q(\mathcal{N})] \end{aligned}$$

This completes the proof of Theorem D.6. $\square$

Omitted proofs from Section D.3

Proof of Lemma D.7. Privacy. We use Fact D.5 and Lemma D.6 to argue the privacy of our simple algorithm, $\mathcal{A}_{\text{simple}}$: Let x and x' differ at time j . Note that the prefix sums fulfill the following properties:

- $\sum_{i=1}^t x_i = \sum_{i=1}^t x'_i$ for all $t < j$ and
- $\sum_{i=1}^t x'_i = \sum_{i=1}^t x_i + y$ for all $t \geq j$, where $y = x'_j - x_j \in [-1, 1]$.

Let $S \subseteq \text{range}(\mathcal{A}_{\text{simple}})^*$. For any $\tau \geq j$, we consider the output distribution of length- τ prefixes. There are two cases: For $j = \tau$ and any $t \leq \tau$, the output on x is given by $\sum_{i=1}^{t-1} x_i + Z_{t-1}$ with $t-1 < j$, thus we get equal output distributions on x and x' . That is,

$$\begin{aligned} \Pr[\mathcal{A}(x_1, \dots, x_t)_{t=1}^\tau \in S] &= \Pr[\mathcal{A}(x'_1, \dots, x'_t)_{t=1}^\tau \in S] \\ &= e^{g(0)\varepsilon} \Pr[\mathcal{A}(x'_1, \dots, x'_t)_{t=1}^\tau \in S]. \end{aligned}$$

For $j < \tau$, let $Z = Z_1, \dots, Z_{\tau-1}$ be the sequence of Laplace random variables used by the algorithm. For any fixed output sequence $s \in S$, note that the algorithm guarantees that $s_1 = 0$ and let $z = z_1, z_2, \dots, z_{\tau-1}$ be the values that the Laplace random variables need to assume to get output sequence s with input x . That is, $\sum_{i=1}^t x_i + z_t = s_{t+1}$ for $t \geq 1$. We define a bijection q satisfying the properties of Fact D.5 as follows. Define $q(z) = z'$ such that

$$z'_i = \begin{cases} z_i & i \leq j \\ z_i + y & i > j \end{cases}.$$

This gives $\sum_{i=1}^t x'_i + z'_t = s_{t+1}$. All Laplace noises have the same distribution $\text{Lap}(1/\varepsilon)$. By Lemma D.6, we have for any $\mathcal{N} \in \text{support}(Z)$,

$$\Pr_{z \in Z}[z \in \mathcal{N}] \leq e^{(\tau-j)\varepsilon} \Pr_{z \in Z}[z \in q(\mathcal{N})].$$

Thus, q fulfills the properties of Fact D.5, and therefore our algorithm satisfies ε -differential privacy with expiration $g(t) = t$.

Accuracy. The error at time step t is given by $x_t + Z_{t-1}$. By the Laplace tail bound (Fact D.14), we have that

$$\Pr_{z_i \sim \text{Lap}(1/\varepsilon)}[|z_i| > \varepsilon^{-1} \log(T/\beta)] \leq \beta/T$$

for any $i \in [T-1]$.

Therefore, by a union bound, $|Z_i| \leq \log(T/\beta)$ simultaneously for all $i \in [T-1]$ with probability at least $1 - \beta$. This implies that, with probability at least $1 - \beta$, the maximum error over the entire stream is bounded by $\varepsilon^{-1} \log(T/\beta)$. $\square$

Proof of Fact D.8. For $a = b$, the claim is immediate and so we prove it for $a < b$. First, consider the case where $a = 2^\ell$ and $b \leq 2 \cdot 2^\ell - 1$. We show that in this case, there exists a set $D_{[a,b]}$ with the properties above, only it contains at most *one* interval per level. If $b = 2 \cdot 2^\ell - 1$, then $D_{[a,b]} = \{[a, b]\}$. Else, we have $b - a + 1 < 2^\ell$, thus there exists a binary representation $(q_0, \dots, q_{\ell-1}) \in \{0, 1\}^\ell$ such that $b - a + 1 = q_0 + 2q_1 + 2^2q_2 + \dots + 2^{\ell-1}q_{\ell-1}$. Note that for any $j \leq \ell$, there is an interval in $\mathcal{I}$ of level j starting at $a = 2^\ell$. We now show how to construct the set $D_{[a,b]}$. In the first step, choose the largest j such that $q_j = 1$ and add the interval $[2^\ell, 2^\ell + 2^j - 1]$ to $D_{[a,b]}$. Then, set $q_j = 0$. Next, assume we already cover the interval $[a, s]$ for some $b \geq s > a$. We then again pick the largest remaining j such that $q_j = 1$ and add the interval $[s, s + 2^j - 1]$ to $D_{[a,b]}$. By an inductive argument, this interval will be in $\mathcal{I}$, since s is the ending position of an interval of a higher level in $\mathcal{I}$. We do this until no more q_j with $q_j = 1$ remains, at which point we have covered $[a, b]$. The same argument can be repeated for the case where $b = 2^\ell - 1$ and $a \geq 1$. Now, for the general case, pick $\ell = \lfloor \log(b) \rfloor$ and $c = 2^\ell - 1$ and define $D_{[a,b]} = D_{[a,c]} \cup D_{[c+1,b]}$. $\square$

Proof of Fact D.9. First note that in our definition of $\mathcal{I}$, we exclude all intervals starting at 0. For a given t , let $\ell \in \mathbb{N}$ be the greatest interval level satisfying $2^\ell - 1 < t$. It follows that for every $j > \ell$, $t \in [0, 2^j - 1] \notin \mathcal{I}$. Analogously, for every $j \leq \ell$ we have $t \notin [0, 2^j - 1]$, and therefore there must exist a unique $I \in \mathcal{I}^j$ such that $t \in I$. Taken together, we have that $|\mathcal{I}_t| = \ell + 1$ where $\ell = \lfloor \log t \rfloor$. $\square$

Omitted Proofs from Section D.4

In this section, we give the detail proof for privacy analysis for all $\lambda \in \mathbb{R}_{\geq 0}$ and accuracy analysis for $\lambda = 3/2$, which is not covered in Section D.4.

Privacy proof for all $\lambda \in \mathbb{R}_{\geq 0}$

Proof of Lemma D.11. We prove a slightly more general result; i.e., for all $\lambda \in \mathbb{R}_{\geq 0}$, we have the following bound on the privacy expiration function:

$$g(d) \leq \begin{cases} 0 & \text{for } d < B \\ 2 \left(1 + \left\lceil \frac{1}{\lambda} ((\log(d - B + 1) + 1)^\lambda - 1) \right\rceil \right) & \text{for } B \leq d \text{ and } \lambda \neq 0 \\ 2(1 + \log \log(d - B + 1)) & \text{for } B \leq d \text{ and } \lambda = 0 \end{cases}$$

We wish to use Theorem D.5. For that, we first show that both the requirements in Theorem D.5 are satisfied.

1. By definition, $\mathcal{A}(x_1 x_2 \dots x_t; z) = \sum_{i=1}^t x_i + \sum_{I \in \mathcal{I}_t} z_I$. For $t < j$, we have $t \notin [j, \tau']$ and therefore $t \notin I$ for all $I \in D_{[j, \tau']}$. It follows that for all $I \in \mathcal{I}_t$ it holds that $I \notin D_{[j, \tau']}$. Therefore, we have

$$\sum_{i=1}^t x_i + \sum_{I \in \mathcal{I}_t} z_I = \sum_{i=1}^t x'_i + \sum_{I \in \mathcal{I}_t} z_I = \sum_{i=1}^t x'_i + \sum_{I \in \mathcal{I}_t} z'_I.$$

For $t \geq j$, we have that t is contained in exactly one $I \in D_{[j, \tau']}$. Thus, $z'_I = z_I + y$ for exactly one $I \in \mathcal{I}_t$, and $z'_{I'} = z_{I'}$ for all $I' \in \mathcal{I}_t \setminus \{I\}$. Further, since $t \geq j$, we have that $\sum_{i=1}^t x_i = \sum_{i=1}^t x'_i - y$. Together, we have

$$\sum_{i=1}^t x_i + \sum_{I \in \mathcal{I}_k} z_I = \sum_{i=1}^t x'_i - y + \sum_{I \in \mathcal{I}_t} z'_I + y = \sum_{i=1}^t x'_i + \sum_{I \in \mathcal{I}_t} z'_I,$$

so the first property of Fact D.5 is fulfilled.

2. By Fact D.8, $D_{[j, \tau']}$ consists of at most two intervals from each of $\mathcal{I}^0, \dots, \mathcal{I}^{\ell^*}$ where $\ell^* = \lfloor \log(\tau' - j + 1) \rfloor$. Thus, by Lemma D.6 we have for any $\mathcal{N} \in \text{support}(Z)$, then

$$\begin{aligned} \Pr_{z \in Z}[z \in \mathcal{N}] &\leq \exp \left(\sum_{\ell=0}^{\log T} \sum_{I \in D_{[j, \tau']} \cap \mathcal{I}^\ell} \frac{|y| \varepsilon}{(1 + \ell)^{1-\lambda}} \right) \Pr_{z \in Z}[z \in q(\mathcal{N})] \\ &\leq \exp \left(2\varepsilon \sum_{\ell=0}^{\ell^*} (1 + \ell)^{\lambda-1} \right) \Pr_{z \in Z}[z \in q(\mathcal{N})] . \end{aligned} \tag{D.1}$$

Using Fact D.5 with $d = \tau - j = B + \tau' - j$ we conclude differential privacy with privacy expiration $g(d) = 2 \sum_{\ell=0}^{\lfloor \log(d-B+1) \rfloor} (1 + \ell)^{\lambda-1}$. Next we bound:

$$\begin{aligned} g(d) &= 2 \sum_{\ell=0}^{\lfloor \log(d-B+1) \rfloor} (1 + \ell)^{\lambda-1} \\ &\leq 2 \left(1 + \int_1^{\log(d-B+1)+1} x^{\lambda-1} dx \right) \\ &= 2 \left(1 + \left[\frac{1}{\lambda} x^\lambda \right]_{x=1}^{x=\log(d-B+1)+1} \right) \\ &= O(1 + \log^\lambda(d - B + 1)) . \end{aligned}$$

Note that we use the assumption $\lambda > 0$.

In the case $\lambda = 0$ it instead holds that $g(d) = O(\log \log(d - B + 1))$ by setting the value of $\lambda = 0$ in the first inequality and then using the bound on the Harmonic sum.

This is not a point of discontinuity as the $O(\cdot)$ notation would imply. We treated λ as a constant and derived the last equation in the above. If instead, we treat λ as a parameter, we have the following

$$g(d) \leq 2 \left(1 + \underbrace{\left[\frac{1}{\lambda} \left((\log(d - B + 1) + 1)^\lambda - 1 \right) \right]}_{f(\lambda)} \right)$$

We can now evaluate the $f(\lambda)$ as $\lambda \rightarrow 0$ as follows: Since $f(\lambda)$ as an indeterminate form and noting that it satisfies the condition required to apply the L'Hôpital rule. Therefore, an application of L'Hôpital rule gives us

$$\begin{aligned} \lim_{\lambda \rightarrow 0} f(\lambda) &= \lim_{\lambda \rightarrow 0} \frac{\frac{d}{d\lambda} ((\log(d - B + 1) + 1)^\lambda - 1)}{\frac{d}{d\lambda} \lambda} \\ &= \lim_{\lambda \rightarrow 0} \log(\log(d - B + 1)) \log^\lambda(d - B + 1) \\ &= \log(\log(d - B + 1)). \end{aligned}$$

That is, we have the desired bound on the privacy expiration. This completes the proof of Theorem D.11. $\square$

Accuracy proof when $\lambda = 3/2$

Recall that in the main text, we stated that for $\lambda = 3/2$, we achieve

$$\sum_{\ell=0}^{\lfloor \log(t-B) \rfloor} b_\ell^2 \leq \sum_{\ell=0}^{\log t} b_\ell^2 = O\left(\frac{\log \log(t)}{\varepsilon^2}\right).$$

This follows from straightforward computation by setting $\lambda = 3/2$:

$$\sum_{\ell=0}^{\log t} b_{\ell}^2 = \sum_{\ell=0}^{\log t} \frac{(\ell+1)^{2(1-\lambda)}}{\varepsilon^2} = \sum_{\ell=0}^{\log t} \frac{1}{\varepsilon^2(\ell+1)} = O\left(\frac{\log \log(t)}{\varepsilon^2}\right).$$

While it seems like a point of discontinuity based on what we showed in Section D.4, we now show that it is not the case. Recall the following computation in Section D.4

$$\begin{aligned} \sum_{\ell=0}^{\log t} b_{\ell}^2 &= \sum_{\ell=0}^{\log t} \frac{(\ell+1)^{2(1-\lambda)}}{\varepsilon^2} \\ &\leq \frac{1}{\varepsilon^2} \left(1 + \int_1^{\log(t)+2} x^{2(1-\lambda)} dx \right) \\ &\leq \frac{1}{\varepsilon^2} \left(1 + \left[\frac{1}{3-2\lambda} x^{3-2\lambda} \right]_{x=1}^{x=\log(t)+2} \right). \end{aligned}$$

Note that we did not include the last step. Now, setting the limits, we get

$$\sum_{\ell=0}^{\log t} b_{\ell}^2 \leq \frac{1}{\varepsilon^2} \left(1 + \frac{(\log(t)+2)^{3-2\lambda} - 1}{3-2\lambda} \right)$$

Just as in the case of the privacy proof, we can compute the limit by applying L'Hôpital's rule to get the desired expression.

Paper E

Space Complexity of Online Convolution

Abstract. We study a discrete convolution streaming problem. An input arrives as a stream of numbers $z = (z_0, z_1, z_2, \dots)$, and at time t our goal is to output $(Tz)_t$ where T is a lower-triangular Toeplitz matrix. We focus on space complexity; the algorithm can store a buffer of $\beta(t)$ numbers in order to achieve this goal.

We characterize space complexity when algorithms perform continuous operations. The matrix T corresponds to a generating function $G(x)$. If $G(x)$ is rational of degree d , then it is known that the space complexity is at most $O(d)$. We prove a corresponding lower bound; the space complexity is at least $\Omega(d)$. In addition, for irrational $G(x)$, we prove that the space complexity is infinite. We also provide finite-time guarantees. For example, for the generating function $\frac{1}{\sqrt{1-x}}$ that was studied in various previous works in the context of differentially private continual counting, we prove a sharp lower bound on the space complexity; at time t , it is at least $\Omega(t)$.

* Full title: *On the Space Complexity of Online Convolution*, Joel Daniel Andersson (BARC, University of Copenhagen), Amir Yehudayoff (University of Copenhagen, Technion IIT). Preprint.

E.1 Introduction

Streaming algorithms process data as it arrives while maintaining only a limited buffer of past information. A fundamental problem in this context is the efficient computation of structured linear transformations. For a linear transformation T , given the stream $z = (z_0, z_1, \dots)$, the goal at time t is to compute $(Tz)_t$. We focus on the case that T is a lower-triangular Toeplitz matrix. Such matrices correspond to discrete convolutions, they appear in a variety of applications, including signal processing, time series analysis, and differential privacy (see e.g. [Bre99; CS03; DNP⁺10b; HUU24] and references therein).

We suggest the following streaming model (a formal definition follows). The state $s^{(t)}$ of the algorithm at time t is a buffer of $\beta(t)$ numbers. When the algorithm receives z_{t+1} , it updates the state $(s^{(t)}, z_{t+1}) \mapsto s^{(t+1)}$. The algorithm is correct at time t if there is a way to recover $(Tz)_t$ from $s^{(t)}$. Our objective is to characterize the minimum space (buffer size) required to solve this task for a given T .

The complexity of this problem is naturally linked to the structure of the generating function G associated with T . It is known [DMP⁺24] that when $G(x)$ is a rational function of degree d , the space complexity is at most $O(d)$. Prior to our work, no corresponding lower bound was known. We close the gap and establish a matching lower bound of $\Omega(d)$. Furthermore, we analyze the case of irrational $G(x)$ and prove that for this case, the space complexity is infinite. Our results classify the space complexity for infinite streams based on the rationality-degree of $G(x)$.

What about finite streams? For a finite stream of length t , we prove that certain generating functions, such as $G_{1/2}(x) = 1/\sqrt{1-x}$, require space at least $\Omega(t)$. This resolves an open question in differentially private (DP) continual counting [DNP⁺10b; CSS11], a central primitive in DP deep learning where space efficiency is critical [KMS⁺21; DMR⁺22]. A leading-constant optimal algorithm for the problem [FHU23; HUU23] relies on solving the streaming task for the corresponding $T = T[G_{1/2}]$. Unsuccessful attempts at solving $T[G_{1/2}]$ with small buffer size led to further research on space versus accuracy tradeoffs [AP23; DMP⁺24; AP25]. Our results vindicate this line of research, and carry insights for future work on streaming DP linear queries.

The model

For a sequence $a = (a_0, a_1, \dots) \in \mathbb{R}^{\mathbb{N}}$ of real numbers,¹ denote by $T = T[a]$ the lower-triangular Toeplitz matrix

$$T_{i,j} = \begin{cases} a_{i-j} & \text{if } i \geq j, \\ 0 & \text{if } i < j. \end{cases}$$

¹In this text $\mathbb{N} = \{0, 1, 2, \dots\}$; matrices and vectors are zero-indexed.

Given the stream $z = (z_0, z_1, z_2, \dots) \in \mathbb{R}^{\mathbb{N}}$, at time $t \in \mathbb{N}$ we want to compute

$$(Tz)_t$$

which depends only on the history $z_0, \dots, z_t$. For example, for $a = (1, 1, 1, \dots)$, we get the problem of releasing all prefix-sums on the input, which is also known as *continual counting*.

This work focuses on the space complexity of this streaming problem. Because we are working over the real numbers, we suggest the following computational model. The model makes sense for all matrices M (and not just for lower-triangular Toeplitz matrices).

Definition E.1 (streaming model). A streaming algorithm is a sequence of maps $m^{(t)}$ and $u^{(t)}$ as follows. Let $\beta : \{-1\} \cup \mathbb{N} \rightarrow \mathbb{N}$ be a buffer size function with $\beta(-1) = 1$. Denote the state at time t by $s^{(t)} \in \mathbb{R}^{\beta(t)}$. Initially, the state is $s^{(-1)} = 0 \in \mathbb{R}$. Upon receiving z_t , the algorithm updates its internal state:

$$s^{(t)} = u^{(t)}(s^{(t-1)}, z_t),$$

where $u^{(t)} : \mathbb{R}^{\beta(t-1)} \times \mathbb{R} \rightarrow \mathbb{R}^{\beta(t)}$. The algorithms then outputs

$$m^{(t)}(s^{(t)})$$

where $m^{(t)} : \mathbb{R}^{\beta(t)} \rightarrow \mathbb{R}$. We assume that the update rules are continuous; for each t , the map $u^{(t)}$ is assumed to be continuous. The algorithm solves the streaming problem defined by $M \in \mathbb{R}^{\mathbb{N} \times \mathbb{N}}$ if for every z and every t ,

$$m^{(t)}(s^{(t)}) = (Mz)_t.$$

The model above assumes that the update rules are continuous (the output maps $m^{(t)}$ are not assumed to be continuous). For computations over $\mathbb{R}$, this assumption is natural and allows to use the algebraic operations $+$, $\times$ as well as any other continuous function like trigonometric functions, max operations and exponential functions. Continuous computations, however, are restricted and can not perform “arbitrary operations”.

Remark E.2. A continuity restriction on the update rules is necessary because without it every problem can be solved with a buffer size of two. The reason is that we can encode $z = (z_0, \dots, z_t) \in \mathbb{R}^{t+1}$ using a pair $(r, x) \in \mathbb{R} \times [0, 1]$ as follows. Let $\varphi : [0, 1] \rightarrow [0, 1]^{t+1}$ be a continuous space-filling curve. For each non-zero $z \in \mathbb{R}^{t+1}$, let $r = \|z\|_2$ and let $x \in [0, 1]$ be so that $\varphi(x) = \frac{z}{r}$ (for $z = 0$, define $r = 0$ and $x = 0$). The map $z \mapsto (r, x)$ is not continuous, but we can reconstruct z from (r, x) using the continuous map $(r, x) \mapsto r \cdot \varphi(x)$.

Remark E.3. The model we consider requires exact computation over $\mathbb{R}$. This is partly motivated by applications in differential privacy (see Section E.1 for more details). But, in fact, our lower bound mechanism can be extended to approximate computation as long as the maps $m^{(t)}, u^{(t)}$ are assumed to be Lipschitz.

Remark E.4. Our lower bounds stated below extend to a stronger model in which, at time t , the state update function takes as input all the future part of z (that is, $(z_{t'})_{t' \geq t}$) and not just z_t . This stronger model is relevant, for example, for the privacy setting where (z_t) is a noise vector that can be sampled as the algorithm sees fit.

We think of the buffer size β as the memory cost of the algorithm. As is standard in computational complexity theory, algorithms have costs and problems have complexities.

Definition E.5. For time t and a time bound $\tau \geq t$, define $\text{space}_M(t; \tau)$ as the minimum value b such that there is a streaming algorithm with buffer size β that computes M correctly up to time τ and $b \geq \max_{t' \leq \tau} \beta(t')$. For infinite streams, the space complexity $\text{space}_M(t)$ is defined as $\text{space}_M(t) = \text{space}_M(t; \infty)$. For a sequence a that defines the Toeplitz matrix $T = T[a]$, we use the notation $\text{space}_a(t, \tau) := \text{space}_{T[a]}(t, \tau)$.

There are two natural ways to decompose the target matrix M : we can write M as $M = LR$ and we can write M as $M = A + B$. The former corresponds to breaking the streaming problem into two problems sequentially, and the latter into two problems in parallel. The following lemma shows that such a decomposition does not allow to save on resources.

Lemma E.6. (*decompositions do not help*) *Let M be a lower-triangular matrix and let $t, I \in \mathbb{N}$. If $M = LR$ for lower-triangular matrices L, R then*

$$\text{space}_L(t; t + I) + \text{space}_R(t; t + I) \geq \text{space}_M(t; t + I).$$

If $M = A + B$ for lower-triangular matrices A, B then

$$\text{space}_A(t; t + I) + \text{space}_B(t; t + I) \geq \text{space}_M(t; t + I).$$

Remark E.7. Theorem E.6 does not hold for the stronger model that can read $(z_{t'})_{t' \geq t}$ at time t .

We also observe the following basic property of algebraic algorithms (we do not rely on it, but we find it informative). An algorithm is called algebraic if for all t , the maps $m^{(t)}$ and $u^{(t)}$ are polynomial maps.

Lemma E.8 (algebraic algorithms are linear). *If the algorithm defined by the maps $\{(u^{(t)}, m^{(t)}) : t \in \mathbb{N}\}$ always outputs a linear function of z , then there is an algorithm that uses only linear maps that computes the same function of z .*

Generating functions

It is convenient and useful to think of the sequence a as a generating function. The sequence a corresponds to the (ordinary) generating function $G = G[a]$

defined by

$$G(x) = \sum_{k=0}^{\infty} a_k x^k.$$

This expression is just a formal infinite sum. For more background on generating functions, see e.g. [Lan03; Wil05].

Let us consider some examples. The first example is the continual counting generating function

$$G_1(x) = \sum_{k=0}^{\infty} x^k.$$

The space complexity of G_1 is 1 because the algorithm can just update the memory by adding the observed z_t . A second example is the “exponential counting” function

$$G_e(x) = \sum_{k=0}^{\infty} 2^k x^k.$$

The space complexity of G_e is 1 as well (multiply the memory by two, and then add the observed z_t). The third example is

$$G_{1/2}(x) = \sum_{k=0}^{\infty} 2^{-2k} \binom{2k}{k} x^k;$$

motivation for studying this function comes from continual counting in the DP framework [FHU23; HUU23]. The fourth example is

$$G_C(x) = \sum_{k=0}^{\infty} C_k x^k,$$

where C_k is the k 'th Catalan number. The trivial upper bound for the space complexities of both $G_{1/2}$ and G_C is $\text{space}(t) \leq t + 1$. Is there a clever way to significantly reduce the buffer size? It turns out that the property that qualitatively governs the space complexity is rationality.

Definition E.9. A polynomial $Q(x)$ of degree d is an expression of the form $Q(x) = \sum_{k=0}^d q_k x^k$ where $q_d \neq 0$. The polynomial $Q(x)$ is invertible if $q_0 \neq 0$. The generating function G is a rational function of degree at most d if there are polynomials P, Q with Q invertible so that $Q(x) \cdot G(x) = P(x)$ where $\deg(Q) \leq d$ and $\deg(P) \leq d - 1$. We say that G has degree d if it is of degree at most d but not of degree at most $d - 1$. We say that G is irrational if it is not rational (of any degree).

Let us go back to the four examples we introduced:

$$\begin{aligned} G_1(x) &= \frac{1}{1-x}, \\ G_e(x) &= \frac{1}{1-2x}, \\ G_{1/2}(x) &= \frac{1}{\sqrt{1-x}}, \\ G_C(x) &= \frac{1-\sqrt{1-4x}}{2x}. \end{aligned}$$

We see that G_1, G_e are degree one rational functions; as simple as it gets. What about G_C and $G_{1/2}$?

Claim. *Both $G_{1/2}$ and G_C are irrational.*

Results

Our main results are that the degree of rationality of G determines the space complexity of the corresponding streaming problem.

Theorem E.10. *If $G[a]$ is a rational function of degree d , then for $t \geq d$,*

$$\text{space}_a(t) = d$$

and for $t < d$,

$$\text{space}_a(t; t + d - 1) = t + 1.$$

Theorem E.11. *If $G[a]$ is irrational, then for all $t \in \mathbb{N}$,*

$$\text{space}_a(t) = t + 1.$$

Theorem E.11 holds just for infinite streams. Irrationality is insufficient to derive lower bounds for finite streams because for every τ , there exists² an irrational $G[a]$ so that $\text{space}_a(t; \tau) = 1$. The theorem does not provide any information on space complexity for a finite time horizon ($\tau < \infty$).

The analysis of finite time horizons is done by studying the corresponding Hankel matrix (see Section E.3). Hankel matrices were studied in combinatorics and in the analytic theory of continued fractions [Wal18]. The properties of Hankel matrices allow to prove that $G_{1/2}$ is essentially as hard as it gets even for finite times.

Theorem E.12. *For all $t \in \mathbb{N}$,*

$$\text{space}_{G_{1/2}}(t; 2t) = t + 1.$$

²Consider for example $G(x) = G_C(x) + \sum_{k=0}^{\tau} (1 - C_k)x^k$.

The theorems reveal “Diophantine approximation like” properties that are related to the space complexity of streaming algorithms. Diophantine approximation studies “how well can a real number α be approximated by rationals?” If α is rational, then it has a perfect representation. If α is irrational, it can only be approximated. There are standard ways to measure the cost of such approximations. The “hardest to approximate” real numbers turn out to be irrational algebraic numbers (e.g. the Thue–Siegel–Roth theorem). For example, the golden ratio $\frac{1+\sqrt{5}}{2}$ has “high complexity” in this model (notice the similarity to G_C). In our context, space complexity corresponds to “distance from rationality”. Rational functions have constant complexity, and irrational ones have infinite complexity. In addition, like the golden ratio, the algebraic irrationals $G_{1/2}$ and G_C are “as far as possible from the rationals”.

Applications

The above results also carry algorithmic implications. Continual counting is about computing prefix-sums of the stream. In our setting, it corresponds to $T_1 = T[(1, 1, 1, \dots)]$. *Differentially private (DP) continual counting* [DNP⁺10b; CSS11] solves the same problem, but also requires the streaming algorithm $\mathcal{A}$ to satisfy differential privacy [DMN⁺06; DR14].

Throughout this section, we only consider the *approximate* DP version of the problem, which sees the most practical use. We also only consider the case where an upper bound on the length of the stream is known.³

Most DP algorithms for continual counting are based on adding *correlated noise* to the exact output and have the form [DMR⁺22]:

$$\mathcal{A}(z) = T_1 z + Ly,$$

where L is a lower-triangular matrix and y is a random Gaussian vector sampled from $\mathcal{N}(0, \sigma^2 I)$. Equivalent forms that correspond to other factorizations of T_1 (such as $T_1(z + T_1^{-1}Ly)$) can be analyzed using Theorem E.6.

The noise parameter σ is determined by the privacy demands and by the matrix L . Observe that $\mathcal{A}(z)$ is essentially running two instances of our streaming problem: upon receiving z_t , it generates a sample y_t and outputs $(T_1 z)_t + (Ly)_t$. The space complexity of T_1 is 1, and so the space complexity of $\mathcal{A}(z)$ is determined by L .

The streaming model we consider requires exact computation of $(Mz)_t$. This is motivated by the DP setting because to provide the DP guarantee, the computation of Ly should be exact. The noise scale σ is chosen so that the covariance $\Sigma = \sigma^2 LL^T$ of the added Gaussian noise precisely masks $T_1(z - z')$ over all neighboring inputs z, z' we seek to protect. If Ly is only approximated

³A technique in [CSS11] allows for removing the assumption of a bounded-length stream at the cost of worse guarantees, but complicates the exposition.

then the resulting distribution may no longer be Gaussian, which is key to our understanding of the privacy guarantees of $\mathcal{A}$.

Space complexity matters for this problem. Part of the recent interest in DP continual counting is motivated by private machine learning [KMS⁺21; DMR⁺22; MT22; CMR⁺23; CGM⁺24]. The main observation is that unconstrained stochastic gradient descent updates with constant learning rate can be viewed as computing prefix sums on adaptively chosen gradient vectors. In other words, the training of a model reduces to running one instance of continual counting per model dimension. Because the model dimension often dominates the number of weight updates [DMP⁺24], a linear space complexity is prohibitive in practice.

The choice of the matrix L has a crucial impact on the DP properties of $\mathcal{A}$. Choosing $L = \sqrt{T_1} = T_{1/2}$ leads to near-optimal guarantees [FHU23; HUU23].⁴ However, attempts to implement $T_{1/2}$ in sublinear space were unsuccessful. This drove further research into other matrices L and into accuracy versus space tradeoffs [AP23; DMP⁺24; AP25; HKU25]. The best known bounds for this trade-off [DMP⁺24] is achieved for $L = T[\tilde{G}_{1/2}] \approx T_{1/2}$, where $\tilde{G}_{1/2}$ is a rational-function approximation of $G_{1/2}$ implemented with memory cost equal to the degree of $\tilde{G}_{1/2}$. Theorem E.10 implies that this implementation of $T[\tilde{G}_{1/2}]$ is tight, and Theorem E.12 motivates why matrices other than $T_{1/2}$ had to be considered for the problem. In a nutshell, there are no sublinear space algorithms for $T_{1/2}$. Any implementation of $\mathcal{A}$ with $L = T_{1/2}$ for n steps, restricted to continuous operations, requires space $\Omega(n)$.

Additional motivation from private machine learning led to studying the problem of DP continual counting with momentum and weight decay [KL24]. Translating this problem to our setting, there is a momentum parameter $0 \leq \mu < 1$ and a weight decay parameter and $0 < \lambda \leq 1$. To avoid degenerate cases, we can assume that $0 \leq \mu < \lambda \leq 1$. The streaming problem now corresponds to the generating function

$$G_{\lambda,\mu} = \frac{1}{(1 - \lambda x)(1 - \mu x)}.$$

In [KL24], a DP streaming algorithm of the form

$$\mathcal{A}_{\lambda,\mu}(z) = T_{\lambda,\mu}z + Ly$$

was presented, achieving asymptotically optimal error for $L = T[\sqrt{G_{\lambda,\mu}}]$. Our methods yield optimal lower bounds for the space complexity of this task as well.

Theorem E.13. *For $0 \leq \mu < \lambda \leq 1$ and for all $t \in \mathbb{N}$,*

$$\text{space}_{\sqrt{G_{\lambda,\mu}}}(t; 2t) \geq t - 4.$$

⁴Recently, an explicit construction of a non-Toeplitz L improving lower order terms was given in [HU25]. No space-efficient implementation for this L is known.

There is additional literature on DP continual counting with arbitrary weights [BFM⁺13; HUU24; HU25]. Most algorithms in this domain are also based on adding correlated noise Ly , and whenever L is Toeplitz our techniques and results can be leveraged.

Remark E.14. In a couple of places below we reuse notation. For example, μ denotes the momentum parameter above and in one of the proofs below it denotes a monomial. The meaning is clear from the context.

E.2 Basic properties

Algebraic decompositions do not help

Proof of Theorem E.6. We prove the statement for factorizations—the statement for sums follows similarly. The observation is that algorithms for solving the streaming problem on L and R yield an algorithm for solving M . Assume we are given the sequences of maps $u_L^{(t)}, m_L^{(t)}$ for L and $u_R^{(t)}, m_R^{(t)}$ for R . On input z , denote the corresponding states by $s_L^{(t)}(z) \in \mathbb{R}^{\beta_L(t)}$ and $s_R^{(t)}(z) \in \mathbb{R}^{\beta_R(t)}$. By the correctness for R , we know $m_R^{(t)}(s_R^{(t)}) = (Rz)_t$. We get an algorithm for M via

$$s_M^{(t)}(z) = (s_L^{(t)}(Rz), s_R^{(t)}(z)) \in \mathbb{R}^{\beta_L(t) + \beta_R(t)}.$$

where

$$u_M^{(t)}(s_M^{(t-1)}, z_t) := \left(u_L^{(t)}\left(s_L^{(t-1)}, m_R^{(t)}(s_R^{(t)}(z))\right), s_R^{(t)}(z) \right)$$

and

$$m_M^{(t)}(s_M^{(t)}) = m_L^{(t)}(s_L^{(t)}(Rz)).$$

The only property of the algorithm that warrants checking is the continuity of $u_M^{(t)}$. This follows from the linearity of $m_R^{(t)}(s_R^{(t)}(z)) = (Rz)_t$. $\square$

Algebraic algorithms are linear

An algebraic streaming algorithm is defined by a sequence of polynomial maps $u^{(t)}$ and $m^{(t)}$. We now explain why we can assume that in this case, the maps $u^{(t)}$ and $m^{(t)}$ are linear. For a multivariate polynomial p and $d \in \mathbb{N}$, denote by $\mathcal{H}_d(p)$ the homogeneous part of degree d in p (it is a linear functional).

Lemma E.15. *Let $f : \mathbb{R}^n \rightarrow \mathbb{R}^k$ and $g : \mathbb{R}^k \rightarrow \mathbb{R}$ be two polynomial maps. Write $f = (f_1, \dots, f_k)$, where each $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$ is a polynomial map. Then, there are linear maps $L_f : \mathbb{R}^n \rightarrow \mathbb{R}^k$ and $L_g : \mathbb{R}^k \rightarrow \mathbb{R}$ so that*

$$L_g \circ L_f = \mathcal{H}_1(g \circ f).$$

In addition, $(L_f)_i = \mathcal{H}_1(f_i)$ for all $i \in [k]$.

Proof. For polynomials over $\mathbb{R}^n$, equality as functions is equivalent to equality as formal polynomials. Define the affine map $A_f : \mathbb{R}^n \rightarrow \mathbb{R}^k$ by

$$(A_f)_i = \mathcal{H}_0(f_i) + \mathcal{H}_1(f_i).$$

Every monomial in $f_i(x) - (A_f)_i(x)$ has degree at least two. So, for every k -variate monomial μ , it holds that

$$\mathcal{H}_1(\mu(f(x))) = \mathcal{H}_1(\mu(f(x) - A_f(x) + A_f(x))) = \mathcal{H}_1(\mu(A_f(x))).$$

It follows that

$$\mathcal{H}_1(g(f(x))) = \mathcal{H}_1(g(A_f(x))).$$

For every k -variate monomial μ , there is a polynomial $q = q_{f,\mu}$ so that

$$\mathcal{H}_1(\mu(A_f(x))) = \mathcal{H}_1(q(L_f(x)))$$

where $(L_f)_i := \mathcal{H}_1(f_i)$. By linearity, there is a polynomial $r = r_{f,g}$ so that

$$\mathcal{H}_1(g(A_f(x))) = \mathcal{H}_1(r(L_f(x))).$$

If μ is a monomial of (total) degree larger than one, then $\mu(L_f(x))$ has degree larger than one (unless it is zero). It follows that

$$\mathcal{H}_1(r(L_f(x))) = L_g(L_f(x))$$

where $L_g = \mathcal{H}_1(r)$. □

Proof of Theorem E.8. The update of the state at time t is

$$s^{(t)} = u^{(t)}(s^{(t-1)}, z_t).$$

Let $L_u^{(t)}$ be the linear map obtained with $g = u^{(t)}$ in Theorem E.15. Inductively define states $\sigma^{(t)}$ by $\sigma^{(-1)} = 0$ and

$$\sigma^{(t)} = L_u^{(t)}(\sigma^{(t-1)}, z_t).$$

It follows by induction and Theorem E.15 that

$$\mathcal{H}_1(s^{(t)}) = L_u^{(t)}(\mathcal{H}_1(s^{(t-1)}), z_t) = L_u^{(t)}(\sigma^{(t-1)}, z_t) = \sigma^{(t)}. \quad (\text{E.1})$$

By assumption, the output at time t is a linear function:

$$m^{(t)}(s^{(t)}) = \mathcal{H}_1(m^{(t)}(s^{(t)})).$$

By Theorem E.15 with $g = m^{(t)}$ and using (E.1), there is a linear map $L_m^{(t)}$ so that

$$\mathcal{H}_1(m^{(t)}(s^{(t)})) = L_m^{(t)}(\sigma^{(t)}). \quad \square$$

E.3 A lower bound mechanism

In this section, we develop a mechanism for proving lower bounds on the space complexity of a sequence a or a Toeplitz matrix $T = T[a]$. For the lower bound, we shall also employ the Hankel matrix $H = H[a]$ defined by

$$H_{i,j} = a_{i+j}.$$

For $I, J \geq 0$, we denote by $H^{(I,J)}$ the $(I+1) \times (J+1)$ matrix that corresponds to positions $0 \leq i \leq I$ and $0 \leq j \leq J$ in H .

Lemma E.16 (small buffer, small rank). *For all $t, I \in \mathbb{N}$,*

$$\text{space}_a(t; t+I) \geq \text{rank}(H^{(I,t)})$$

where $H = H[a]$.

The lemma relates space complexity, which is a “complicated notion”, to the rank of a matrix, which is a notion that we understand pretty well.

Remark E.17. A variant of the lemma holds in a more general framework than the one we consider. It holds for general lower-triangular matrices and not just for lower-triangular Toeplitz matrices. For every matrix A , the buffer size can be bounded from below by the rank of some submatrix of A .

The lower bound is based on a basic topological property; the Borsuk-Ulam theorem. Denote by $\mathbb{S}^{r-1}$ the unit sphere in $\mathbb{R}^r$:

$$\mathbb{S}^{r-1} = \{x \in \mathbb{R}^r : \|x\|_2 = 1\}.$$

Theorem (Borsuk-Ulam). If $f : \mathbb{S}^{r-1} \rightarrow \mathbb{R}^{r-1}$ is continuous then there is $x \in \mathbb{S}^{r-1}$ so that $f(x) = f(-x)$.

Proof of Theorem E.16. Consider the restricted setting where the incoming stream z is so that $z_{t'} = 0$ for all $t' > t$, and we are only interested in the outputs $(Tz)_{t'}$ for $t \leq t' \leq t+I$. Accordingly, for the rest of the proof, assume that z is zero after position t , or equivalently that $z \in \mathbb{R}^{t+1}$. Denote by T' the first $t+1$ columns of T so that $Tz = T'z$.

Denote by $f(z) \in \mathbb{R}^{\beta(t)}$ the state of the memory at time t when the input is $z \in \mathbb{R}^{t+1}$. The map f is continuous by assumption on the algorithm. Denote by M the submatrix of T' formed from row indices $t, t+1, \dots, t+I$; it defines a linear map from $\mathbb{R}^{t+1}$ to $\mathbb{R}^{I+1}$. By definition of rank, there is a vector space $V \subset \mathbb{R}^{t+1}$ of dimension $r = \text{rank}(M)$ so that $Mv \neq 0$ for every non-zero $v \in V$. The space V is isomorphic to $\mathbb{R}^r$. Let S be a copy of the sphere $\mathbb{S}^{r-1}$ in V . For every $v \in S$ we have $Mv \neq 0$.

Denote by $g(z) \in \mathbb{R}^{I+1}$ the output of the algorithm at times $t, t+1, \dots, t+I$ on input z . By the structure of the algorithm, g can be factored as $g = h \circ f$ where $h : \mathbb{R}^{\beta(t)} \rightarrow \mathbb{R}^{I+1}$ (we do not know that h is continuous). Assume towards

a contradiction that $\beta(t) < r$. By the Borsuk-Ulam theorem, there is a $v \in S$ so that $f(v) = f(-v)$ and consequently $g(v) = g(-v)$. By the correctness criterion,

$$Mv = g(v) = g(-v) = -Mv$$

so $Mv = 0$ which is a contradiction.

Because $H^{(I,t)}$ is a column permutation of M , the proof is complete. $\square$

E.4 Asymptotic complexity

In this section, we show that the space complexity of infinite streams is characterized by the rational function degree of the generating function.

Compactness

Lemma E.18. *Let M be a $d \times \mathbb{N}$ real matrix with row rank d . For each $j \in \mathbb{N}$, let $M[j]$ be the $d \times j + 1$ matrix of the first $j + 1$ columns of M . Then, there is some $j \in \mathbb{N}$ so that $M[j]$ has rank d .*

Proof. Assume towards a contradiction that for all j , the matrix $M[j]$ is singular. That is, there is a $v[j]$ in the unit sphere of $\mathbb{R}^d$ so that $v[j]M[j] = 0$; in words, it is in the left kernel of $M[j]$. The unit sphere is compact so by going to a subsequence we can assume that $v[j]$ tends to a limit $v_* \neq 0$.

Fix $j_0 \in \mathbb{N}$, and denote by $m \in \mathbb{R}^d$ the j_0 'th column in M . For all $j > j_0$, we have $\langle v[j], m \rangle = 0$. As $j \rightarrow \infty$, we have $\langle v_*, m \rangle = \langle v_* - v[j], m \rangle \rightarrow 0$. So, $\langle v_*, m \rangle = 0$.

It follows that v_* is in the left kernel of M , which is a contradiction. $\square$

Hankel matrices

We recall the following theorem proved by Kronecker; see e.g. [Pel04; Alp17] and references therein.

Theorem E.19. *Let $a \in \mathbb{R}^{\mathbb{N}}$. The generating function $G = G[a]$ is rational of degree d iff $H = H[a]$ has rank d and $H^{(d-1, d-1)}$ has rank d . The function G is irrational iff the rank of H is infinite.*

Claim E.20. Let $a \in \mathbb{R}^{\mathbb{N}}$ and let $H = H[a]$. If the rank of H is d , then for every $t \leq d$, the first t columns in H are linearly independent.

Proof. If the first t columns of H are linearly dependent, then some column $i \leq t$ is a linear combination of columns $< i$. Because H is an infinite Hankel matrix, all columns after column i are also spanned by columns $< i$ (with the same coefficients). $\square$

Rational functions

Proof of Theorem E.10. Assume that $G = G[a]$ is a rational function of degree d and let $H = H[a]$ be its Hankel matrix. It is known that because $G[a]$ is rational of degree d , it holds that $\text{space}_a(t) \leq d$ for all t ; see (1.8) and Lemma 3.2 in [DMP⁺24]. Their algorithm as stated requires $\beta(0) = d$, but it is possible to implement it with $\beta(t) = \min\{t + 1, d\}$. For $t < d$, let $s^{(t)}$ be a copy of the input stream $s^{(t)} = (z_0, \dots, z_t)$ with the obvious output map $m^{(t)}$. For $t \geq d$, let $s^{(d)}$ be the corresponding state from [DMP⁺24] and use their (fixed) maps $\{(u^{(t)}, m^{(t)}) : t \geq d\}$ for all future computations. It follows that $\text{space}_a(t) \leq \min\{t + 1, d\}$.

Now, let $t < d$. By Theorem E.19, the matrix $H^{(d-1, d-1)}$ has full rank. In particular, $\text{rank}(H^{(d-1, t)}) = t + 1$. Invoking Theorem E.16, we get $\text{space}_a(t; t + d - 1) \geq t + 1$. $\square$

Irrational functions

Proof of Theorem E.11. Because $G[a]$ is irrational, the Hankel matrix $H[a]$ has infinite rank. By Theorem E.20, for all $t \in \mathbb{N}$, the first $t + 1$ columns in H are linearly independent. Fix t and let M be the first $t + 1$ columns of H . By compactness, there is some j so that $H^{(j, t)}$ has rank $t + 1$. By Theorem E.16, we know that $\text{space}_a(t) \geq t + 1$. The upper bound $\text{space}_a(t) \leq t + 1$ is trivial. $\square$

E.5 Finite time horizons

Asymptotic space complexity can be understood via degrees as rational functions. The finite time horizon case for irrational functions can be understood via deeper properties of the corresponding Hankel matrices.

Preliminaries

In this section, we explain some methods for comparing the ranks of different Hankel matrices. We shall use the following simple properties. If $f(x), g(x)$ are generating functions and $T[f], T[g]$ are the corresponding Toeplitz matrices then

$$T[f \cdot g] = T[f] \cdot T[g]$$

and

$$T[f + g] = T[f] + T[g].$$

Lemma E.21. *Let f, g be generating functions and let $\alpha, \beta, \gamma \in \mathbb{R}$ be so that $f_0 \neq \alpha$, so that $g_0 \neq 0$ and so that*

$$fg = \alpha g + \beta xg + \gamma.$$

Then, for every $d \in \mathbb{N}$, the rank of $(H[f])^{(d,d)}$ is at least the rank of $(H[g])^{(d,d)}$ minus three. In addition, when $\beta = 0$, the rank of $(H[f])^{(d,d)}$ is at least the rank of $(H[g])^{(d,d)}$ minus two.

Proof. Fix d , and denote by $T'[h]$ the principle $2d \times 2d$ submatrix of $T[h]$. Because the “upper half” of $T[h]$ is zero,

$$T'[f \cdot g] = T'[f] \cdot T'[g].$$

Therefore, by assumption,

$$T'[f - \alpha - \beta x] \cdot T'[g] = T'[\gamma].$$

The matrix $T'[h]$ can be partitioned into four equal block of size $d \times d$ as

$$T'[h] = \begin{bmatrix} T_{11}[h] & T_{12}[h] \\ T_{21}[h] & T_{22}[h] \end{bmatrix}.$$

By looking at the lower-left block, T_{21} , we see that

$$T_{21}[f - \alpha - \beta x] \cdot T_{11}[g] + T_{22}[f - \alpha - \beta x] \cdot T_{21}[g] = T_{21}[\gamma] = 0,$$

because $T_{21}[\gamma]$ is zero. The matrices $T_{11}[g]$ and $T_{22}[f - \alpha - \beta x]$ have full rank because $f_0 \neq \alpha$ and $g_0 \neq 0$. The matrix

$$M := -T_{22}[f - \alpha - \beta x] \cdot T_{21}[g] \cdot (T_{11}[g])^{-1}$$

has the same rank as $T_{21}[g]$ and

$$T_{21}[f - \alpha - \beta x] = M.$$

So,

$$T_{21}[f] = M + T_{21}[\alpha + \beta x] = M + T_{21}[\beta x].$$

The matrix $T_{21}[\beta x]$ has at most one non-zero entry (so its rank is at most one). The matrix $T_{21}[h]$ is obtained from $(H[h])^{(d,d)}$ by first deleting the first row and first column, and then reversing the rows. The rank of $(H[f])^{(d,d)}$ is hence at least the rank of $T_{21}[f]$. The rank of M , which is equal to the rank of $T_{21}[g]$, is at least the rank of $(H[g])^{(d,d)}$ minus two. $\square$

Examples

Let us start with the Catalan generating function G_C . Denote by H_C the Hankel matrix that corresponds to the Catalan sequence. We note the following folklore lemma (see e.g. [Aig99]).

Lemma E.22. *For every $d \in \mathbb{N}$,*

$$\det(H_C^{(d,d)}) = 1.$$

In particular, $H_C^{(d,d)}$ has full rank.

Denote by $H_{1/2}$ the Hankel matrix that corresponds to $G_{1/2}$. An $n \times n$ matrix has co-rank r if its rank is $n - r$. The full rank of $(H_C)^{(d,d)}$ can be used with Theorem E.21 to show that $(H_{1/2})^{(d,d)}$ has a co-rank of at most four. The rank of $(H_{1/2})^{(d,d)}$ is, in fact, known to be full.

Lemma E.23. *For every $d \in \mathbb{N}$, $(H_{1/2})^{(d,d)}$ has full rank.*

Proof. Proposition 8 in [Aig99] states that $\det(H[1/\sqrt{1-4x}]^{(d,d)}) = 2^d$ implying the full rank of $(H[1/\sqrt{1-4x}])^{(d,d)}$. It follows that $(H[1/\sqrt{1-x}])^{(d,d)}$ also has full rank. $\square$

Proof of Theorem E.12. The lower bound follows from Theorem E.23 and E.16. The upper bound $\text{space}_a(t; 2t) \leq t + 1$ is trivial. $\square$

Finally, recall the generating function

$$G_{\lambda,\mu} = \frac{1}{(1-\lambda x)(1-\mu x)}$$

for $0 \leq \mu < \lambda \leq 1$. Our main lemma here is the following.

Lemma E.24. *Let $b \neq c$ be non-zero complex numbers and let*

$$G(x) = \frac{8}{(b-c)^2} \frac{1 - \frac{b+c}{2}x - \sqrt{(1-bx)(1-cx)}}{x^2}.$$

For all $d \in \mathbb{N}$, the matrix $(H[G])^{(d,d)}$ has full rank.

Our argument relies on the following theorem by Junod (Theorem 2 in [Jun03]).

Theorem E.25. *Consider a generating function*

$$G(x) = \frac{1}{1 - W(x)}$$

with $W(0) = 0$. Suppose that $w(x) = \frac{W(x)}{x} - W'(0)$ satisfies

$$w(x) = x(\alpha + \beta w(x) + \gamma w(x)^2)$$

for some parameters $\alpha \neq 0, \beta$ and γ in $\mathbb{C}$. Then for any $d \in \mathbb{N}$:

$$\det((H[G])^{(d,d)}) = \alpha^{n(n+1)/2} \gamma^{n(n-1)/2}.$$

Junod also remarks that $w(x)$ can be written as

$$w(x) = \frac{F(x) - 1}{xF(x)} - F'(0)$$

and as

$$w(x) = \frac{1 - \beta x - \sqrt{(1 - \beta x)^2 - 4\gamma\alpha x^2}}{2\gamma x},$$

when $\gamma \neq 0$.

Proof of Theorem E.24. To invoke Theorem E.25, we will first prove that $W(x) = 1 - 1/G(x)$ satisfies $W(0) = 0$. We proceed:

$$W(x) = 1 - \frac{(b-c)^2}{8} \frac{x^2}{1 - \frac{b+c}{2}x - \sqrt{(1-bx)(1-cx)}}.$$

For some generating functions R_1, R_2, R_3 , we can write

$$\begin{aligned} & \sqrt{(1-bx)(1-cx)} \\ &= \sqrt{1 - (b+c)x + bcx^2} \\ &= 1 - \frac{(b+c)x}{2} + \frac{bcx^2}{2} - \frac{x^2(b+c-bcx)^2}{8} - \frac{x^3(b+c-bcx)^3}{16} + x^4 R_1 \\ &= 1 - \frac{(b+c)x}{2} + x^2 \left(\frac{bc}{2} - \frac{(b+c-bcx)^2}{8} - \frac{x(b+c-bcx)^3}{16} + x^2 R_1 \right) \\ &= 1 - \frac{(b+c)x}{2} + x^2 \left(\frac{bc}{2} - \frac{b^2+c^2+2bc}{8} + \frac{2bc(b+c)}{8}x - \frac{(b+c)^3}{16}x + x^2 R_2 \right) \\ &= 1 - \frac{(b+c)x}{2} - \frac{x^2(b-c)^2}{8} \left(1 + \frac{b+c}{2}x + x^2 R_3 \right). \end{aligned}$$

So,

$$W(x) = 1 - \frac{1}{1 + \frac{b+c}{2}x + x^2 R_3},$$

and we can conclude that $W(0) = 0$. It also follows that $G'(0) = \frac{b+c}{2}$ and by the remark above we can conclude

$$\begin{aligned} w(x) &= \frac{G(x) - 1}{xG(x)} - G'(0) \\ &= \frac{\frac{8}{(b-c)^2} \frac{1 - \frac{b+c}{2}x - \sqrt{(1-bx)(1-cx)}}{x^2} - 1}{\frac{8}{(b-c)^2} \frac{1 - \frac{b+c}{2}x - \sqrt{(1-bx)(1-cx)}}{x}} - \frac{b+c}{2} \\ &= \frac{8(1 - \frac{b+c}{2}x - \sqrt{(1-bx)(1-cx)}) - (b-c)^2 x^2}{8x(1 - \frac{b+c}{2}x - \sqrt{(1-bx)(1-cx)})} - \frac{b+c}{2}. \end{aligned}$$

We claim that

$$\begin{aligned} & \frac{8(1 - \frac{b+c}{2}x - \sqrt{(1-bx)(1-cx)}) - (b-c)^2 x^2}{8x(1 - \frac{b+c}{2}x - \sqrt{(1-bx)(1-cx)})} \\ &= \frac{x - \frac{x}{2}(1 - \frac{b+c}{2}x + \sqrt{(1-bx)(1-cx)})}{x^2}. \end{aligned}$$

Indeed, this is equivalent to

$$\begin{aligned} & x^2 \left(8 \left(1 - \frac{b+c}{2}x - \sqrt{(1-bx)(1-cx)} \right) - (b-c)^2 x^2 \right) \\ &= \left(8x \left(1 - \frac{b+c}{2}x - \sqrt{(1-bx)(1-cx)} \right) \right) \\ & \quad \cdot \left(x - \frac{x}{2} \left(1 - \frac{b+c}{2}x + \sqrt{(1-bx)(1-cx)} \right) \right) \end{aligned}$$

which is correct because

$$\begin{aligned} & \left(8x \left(1 - \frac{b+c}{2}x - \sqrt{(1-bx)(1-cx)} \right) \right) \frac{x}{2} \left(1 - \frac{b+c}{2}x + \sqrt{(1-bx)(1-cx)} \right) \\ &= 4x^2 \left(1 - \frac{b+c}{2}x - \sqrt{(1-bx)(1-cx)} \right) \left(1 - \frac{b+c}{2}x + \sqrt{(1-bx)(1-cx)} \right) \\ &= 4x^2 \left(\left(1 - \frac{b+c}{2}x \right)^2 - (1-bx)(1-cx) \right) \\ &= 4x^2 \left(\left(\frac{b+c}{2}x \right)^2 - bcx^2 \right) \\ &= 4x^2 \left(\frac{b^2 + c^2 + 2bc}{4} x^2 - bcx^2 \right) \\ &= x^2 (b-c)^2 x^2. \end{aligned}$$

So,

$$\begin{aligned} w(x) &= \frac{x - \frac{x}{2} \left(1 - \frac{b+c}{2}x + \sqrt{(1-bx)(1-cx)} \right)}{x^2} - \frac{b+c}{2} \\ &= \frac{1 - \frac{1}{2} \left(1 - \frac{b+c}{2}x + \sqrt{(1-bx)(1-cx)} \right) - \frac{(b+c)}{2}x}{x} \\ &= \frac{1 - \frac{b+c}{2}x - \sqrt{(1-bx)(1-cx)}}{2x}. \end{aligned}$$

As in the remark above,

$$w(x) = \frac{1 - \beta x - \sqrt{(1 - \beta x)^2 - 4\gamma\alpha x^2}}{2\gamma x},$$

with $\gamma = 1$, with $\beta = \frac{b+c}{2}$ and with $\alpha = \frac{(b-c)^2}{16}$. It follows that the determinant of $(H[F])^{(d,d)}$ is non-zero. $\square$

Denote by $H_{\lambda,\mu}$ the Hankel matrix that corresponds to $G_{\lambda,\mu}$.

Corollary E.26. *For all $0 \leq \mu < \lambda \leq 1$ and for all $d \in \mathbb{N}$, the matrix $(H_{\lambda,\mu})^{(d,d)}$ has co-rank at most five.*

Proof. Start with $f(x) = \sqrt{(1-\lambda x)(1-\mu x)}$ and with

$$G(x) = \frac{1 - \frac{\lambda+\mu}{2}x - f}{x^2}.$$

Because

$$\begin{aligned}
f \cdot G &= \frac{(1 - \frac{\lambda+\mu}{2}x)f - (1 - \lambda x)(1 - \mu x)}{x^2} \\
&= -G - \lambda\mu + \frac{(\lambda + \mu)(1 - f)}{2x} \\
&= -G - \lambda\mu + \frac{(\lambda + \mu)(1 - \frac{\lambda+\mu}{2}x - f)}{2x} + \frac{(\lambda + \mu)\frac{\lambda+\mu}{2}x}{2x} \\
&= \left(\frac{\lambda + \mu}{2}x - 1\right)G + \frac{(\lambda - \mu)^2}{4}.
\end{aligned}$$

Theorem E.24 and Theorem E.21 imply that $(H[f])^{(d,d)}$ has co-rank at most three. Theorem E.21 implies that $(H_{\lambda,\mu})^{(d,d)}$ has co-rank at most five. $\square$

Proof of Theorem E.13. The theorem follows from Theorem E.26 together with Theorem E.16. $\square$

Acknowledgments. We thank Nikita Kalinin for valuable feedback on a preliminary version of this text. J.D.A. is supported by Providentia, a Data Science Distinguished Investigator grant from Novo Nordisk Fonden. A.Y. is supported by a DNRf chair grant. Both are partially supported by BARC.

Bibliography

- [Aig99] Martin Aigner. “Catalan-like Numbers and Determinants”. In: *Journal of Combinatorial Theory, Series A* 87.1 (1999), pp. 33–51.
- [Alp17] Yu Al’pin. “The Hankel matrix rank theorem revisited”. In: *Linear Algebra and its Applications* 534 (2017), pp. 97–101.
- [AKV24] Kareem Amin, Alex Kulesza, and Sergei Vassilvitskii. “Practical Considerations for Differential Privacy”. In: *arXiv preprint arXiv:2408.07614* (2024).
- [AHP⁺24] Joel Daniel Andersson, Monika Henzinger, Rasmus Pagh, Teresa Anna Steiner, and Jalaj Upadhyay. “Continual Counting with Gradual Privacy Expiration”. In: *Advances in Neural Information Processing Systems 37, NeurIPS 2024*. 2024.
- [AP23] Joel Daniel Andersson and Rasmus Pagh. “A Smooth Binary Mechanism for Efficient Private Continual Observation”. In: *Advances in Neural Information Processing Systems 36, NeurIPS 2023*. 2023.
- [AP25] Joel Daniel Andersson and Rasmus Pagh. “Streaming Private Continual Counting via Binning”. In: *IEEE Conference on Secure and Trustworthy Machine Learning, SaTML 2025*. IEEE, 2025, pp. 575–589.
- [APS⁺25] Joel Daniel Andersson, Rasmus Pagh, Teresa Anna Steiner, and Sahel Torkamani. “Count on Your Elders: Laplace vs Gaussian Noise”. In: *6th Symposium on Foundations of Responsible Computing, FORC 2025*. Vol. 329. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025, 10:1–10:24.
- [ARN⁺25] Joel Daniel Andersson, Lukas Retschmeier, Boel Nelson, and Rasmus Pagh. “Private Lossless Multiple Release”. In: *Forty-second International Conference on Machine Learning*. 2025.
- [AY25] Joel Daniel Andersson and Amir Yehudayoff. “On the Space Complexity of Online Convolution”. In: *CoRR* abs/2505.00181 (2025).

- [BW18] Borja Balle and Yu-Xiang Wang. “Improving the Gaussian Mechanism for Differential Privacy: Analytical Calibration and Optimal Denoising”. In: *Proceedings of the 35th International Conference on Machine Learning, ICML 2018*. Vol. 80. Proceedings of Machine Learning Research. PMLR, 2018, pp. 403–412.
- [Ben77] G. Bennett. “Schur multipliers”. In: *Duke Mathematical Journal* 44.3 (1977), pp. 603–639.
- [BFM⁺13] Jean Bolot, Nadia Fawaz, Shanmugavelayutham Muthukrishnan, Aleksandar Nikolov, and Nina Taft. “Private decayed predicate sums on streams”. In: *Proceedings of the 16th International Conference on Database Theory*. 2013, pp. 284–295.
- [Bre99] Richard Brent. “Stability of fast algorithms for structured linear systems”. In: *Fast reliable algorithms for matrices with structure* (1999), pp. 103–116.
- [BS16] Mark Bun and Thomas Steinke. “Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds”. In: *Theory of Cryptography*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2016, pp. 635–658.
- [CKS20] Clément L Canonne, Gautam Kamath, and Thomas Steinke. “The discrete Gaussian for differential privacy”. In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 15676–15688.
- [CR22] Adrian Rivera Cardoso and Ryan Rogers. “Differentially Private Histograms under Continual Observation: Streaming Selection into the Unknown”. In: *International Conference on Artificial Intelligence and Statistics, AISTATS 2022*. Vol. 151. Proceedings of Machine Learning Research. PMLR, 2022, pp. 2397–2419.
- [CLS⁺12] T. -H. Hubert Chan, Mingfei Li, Elaine Shi, and Wenchang Xu. “Differentially Private Continual Monitoring of Heavy Hitters from Distributed Streams”. en. In: *Privacy Enhancing Technologies*. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2012, pp. 140–159.
- [CSS11] T.-H. Hubert Chan, Elaine Shi, and Dawn Song. “Private and Continual Release of Statistics”. In: *ACM Trans. Inf. Syst. Secur.* 14.3 (2011), 26:1–26:24.
- [CMH⁺17] Yan Chen, Ashwin Machanavajjhala, Michael Hay, and Gerome Miklau. “Pegasus: Data-adaptive differentially private stream processing”. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 2017, pp. 1375–1388.

- [CDP⁺23] Christopher A Choquette-Choo, Krishnamurthy Dvijotham, Krishna Pillutla, Arun Ganesh, Thomas Steinke, and Abhradeep Thakurta. “Correlated Noise Provably Beats Independent Noise for Differentially Private Learning”. In: *International Conference on Learning Representation*. 2023.
- [CGM⁺24] Christopher A Choquette-Choo, Arun Ganesh, Ryan McKenna, H Brendan McMahan, John Rush, Abhradeep Guha Thakurta, and Zheng Xu. “(Amplified) Banded Matrix Factorization: A unified approach to private training”. In: *Advances in Neural Information Processing Systems* 36 (2024).
- [CGH⁺25] Christopher A. Choquette-Choo, Arun Ganesh, Saminul Haque, Thomas Steinke, and Abhradeep Guha Thakurta. “Near-Exact Privacy Amplification for Matrix Mechanisms”. In: *The Thirteenth International Conference on Learning Representations, ICLR 2025*. OpenReview.net, 2025.
- [CGS⁺24] Christopher A. Choquette-Choo, Arun Ganesh, Thomas Steinke, and Abhradeep Guha Thakurta. “Privacy Amplification for Matrix Mechanisms”. In: *The Twelfth International Conference on Learning Representations, ICLR 2024*. OpenReview.net, 2024.
- [CMR⁺23] Christopher A. Choquette-Choo, H. Brendan McMahan, Keith Rush, and Abhradeep Thakurta. “Multi-Epoch Matrix Factorization Mechanisms for Private Machine Learning”. In: *Proceedings of the 40th International Conference on Machine Learning*. ICML’23. Honolulu, Hawaii, USA, 2023.
- [CLN⁺24] Edith Cohen, Xin Lyu, Jelani Nelson, Tamás Sarlós, and Uri Stemmer. “Lower Bounds for Differential Privacy Under Continual Observation and Online Threshold Queries”. In: *The Thirty Seventh Annual Conference on Learning Theory, 2023*. Vol. 247. Proceedings of Machine Learning Research. PMLR, 2024, pp. 1200–1222.
- [CS03] Edith Cohen and Martin Strauss. “Maintaining time-decaying stream aggregates”. In: *Proceedings of the Twenty-Second ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, 2003*. ACM, 2003, pp. 223–233.
- [CL22] Thomas H Cormen and Charles E Leiserson. *Introduction to Algorithms, fourth edition*. en. London, England: MIT Press, 2022.
- [CKS19] Graham Cormode, Tejas Kulkarni, and Divesh Srivastava. “Answering range queries under local differential privacy”. In: *Proc. VLDB Endow.* 12.10 (2019), pp. 1126–1138.

- [DK22] Yuval Dagan and Gil Kur. “A bounded-noise mechanism for differential privacy”. In: *Proceedings of Thirty Fifth Conference on Learning Theory*. Vol. 178. Proceedings of Machine Learning Research. PMLR, 2022, pp. 625–661.
- [DMR⁺22] Sergey Denisov, H Brendan McMahan, John Rush, Adam Smith, and Abhradeep Guha Thakurta. “Improved differential privacy for sgd via optimal private linear operators on adaptive streams”. In: *Advances in Neural Information Processing Systems 35* (2022), pp. 5910–5924.
- [DN03] Irit Dinur and Kobbi Nissim. “Revealing information while preserving privacy”. In: *Proceedings of the twenty-second ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. PODS ’03. New York, NY, USA: Association for Computing Machinery, 2003, pp. 202–210.
- [DRS22] Jinshuo Dong, Aaron Roth, and Weijie J. Su. “Gaussian Differential Privacy”. In: *Journal of the Royal Statistical Society Series B: Statistical Methodology* 84.1 (2022), pp. 3–37.
- [DSZ21] Jinshuo Dong, Weijie J. Su, and Linjun Zhang. “A Central Limit Theorem for Differentially Private Query Answering”. In: *Advances in Neural Information Processing Systems 34, NeurIPS 2021*. 2021, pp. 14759–14770.
- [DMP⁺24] Krishnamurthy (Dj) Dvijotham, H. Brendan McMahan, Krishna Pillutla, Thomas Steinke, and Abhradeep Thakurta. “Efficient and Near-Optimal Noise Generation for Streaming Differential Privacy”. In: *Foundations of Computer Science*. 2024.
- [DMN⁺06] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. “Calibrating Noise to Sensitivity in Private Data Analysis”. In: *TCC*. 2006, pp. 265–284.
- [DNP⁺10a] Cynthia Dwork, Moni Naor, Toniann Pitassi, Guy N Rothblum, and Sergey Yekhanin. “Pan-Private Streaming Algorithms.” In: *ics*. 2010, pp. 66–80.
- [DNP⁺10b] Cynthia Dwork, Moni Naor, Toniann Pitassi, and Guy N. Rothblum. “Differential privacy under continual observation”. In: *Proceedings of the forty-second ACM symposium on Theory of computing*. STOC ’10. New York, NY, USA: Association for Computing Machinery, 2010, pp. 715–724.
- [DNR⁺15] Cynthia Dwork, Moni Naor, Omer Reingold, and Guy N Rothblum. “Pure differential privacy for rectangle queries via private partitions”. In: *International Conference on the Theory and Application of Cryptology and Information Security*. Springer. 2015, pp. 735–751.

- [DR14] Cynthia Dwork and Aaron Roth. “The Algorithmic Foundations of Differential Privacy”. In: *Found. Trends Theor. Comput. Sci.* 9.3-4 (2014), pp. 211–407.
- [ELM⁺24] Alessandro Epasto, Quanquan C. Liu, Tamalika Mukherjee, and Felix Zhou. “The Power of Graph Sparsification in the Continual Release Model”. In: *CoRR* abs/2407.17619 (2024).
- [EMM⁺23] Alessandro Epasto, Jieming Mao, Andres Munoz Medina, Vahab Mirrokni, Sergei Vassilvitskii, and Peilin Zhong. “Differentially Private Continual Releases of Streaming Frequency Moment Estimations”. In: *14th Innovations in Theoretical Computer Science Conference (ITCS 2023)*. Vol. 251. 2023, 48:1–48:24.
- [FHO21] Hendrik Fichtenberger, Monika Henzinger, and Lara Ost. “Differentially Private Algorithms for Graphs Under Continual Observation”. In: *29th Annual European Symposium on Algorithms, ESA 2021*. 2021.
- [FHU23] Hendrik Fichtenberger, Monika Henzinger, and Jalaj Upadhyay. “Constant Matters: Fine-grained Error Bound on Differentially Private Continual Observation”. In: *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research. PMLR, 2023, pp. 10072–10092.
- [GWX10] J. Gehrke, G. Wang, and X. Xiao. “Differential privacy via wavelet transforms”. In: *2010 IEEE 26th International Conference on Data Engineering (ICDE 2010)*. Los Alamitos, CA, USA: IEEE Computer Society, 2010, pp. 225–236.
- [GDG⁺19] Quan Geng, Wei Ding, Ruiqi Guo, and Sanjiv Kumar. “Optimal Noise-Adding Mechanism in Additive Differential Privacy”. In: *The 22nd International Conference on Artificial Intelligence and Statistics, AISTATS 2019*. Vol. 89. Proceedings of Machine Learning Research. PMLR, 2019, pp. 11–20.
- [GDG⁺20] Quan Geng, Wei Ding, Ruiqi Guo, and Sanjiv Kumar. “Tight Analysis of Privacy and Utility Tradeoff in Approximate Differential Privacy”. In: *The 23rd International Conference on Artificial Intelligence and Statistics, AISTATS 2020*. Vol. 108. Proceedings of Machine Learning Research. PMLR, 2020, pp. 89–99.
- [HLL⁺22] Yuxuan Han, Zhicong Liang, Zhipeng Liang, Yang Wang, Yuan Yao, and Jiheng Zhang. “Private Streaming SCO in ℓ_p geometry with Applications in High Dimensional Online Decision Making”. In: *Proceedings of the 39th International Conference on Machine Learning*. ISSN: 2640-3498. PMLR, 2022, pp. 8249–8279.

- [HRM⁺10] Michael Hay, Vibhor Rastogi, Gerome Miklau, and Dan Suciu. “Boosting the Accuracy of Differentially Private Histograms through Consistency”. In: *Proc. VLDB Endow.* 3.1–2 (2010), pp. 1021–1032.
- [HKU25] Monika Henzinger, Nikita P. Kalinin, and Jalaj Upadhyay. *Binned Group Algebra Factorization for Differentially Private Continual Counting*. 2025.
- [HLS22] Monika Henzinger, Andrea Lincoln, and Barna Saha. “The Complexity of Average-Case Dynamic Subgraph Counting”. In: *Proceedings of the 2022 ACM-SIAM Symposium on Discrete Algorithms, SODA 2022*. SIAM, 2022, pp. 459–498.
- [HSS23] Monika Henzinger, A. R. Sricharan, and Teresa Anna Steiner. “Differentially Private Data Structures under Continual Observation for Histograms and Related Queries”. In: *CoRR* abs/2302.11341 (2023).
- [HSS24] Monika Henzinger, A. R. Sricharan, and Teresa Anna Steiner. “Private Counting of Distinct Elements in the Turnstile Model and Extensions”. In: *Proc. APPROX/RANDOM 2024*. 2024, 40:1–40:21.
- [HU25] Monika Henzinger and Jalaj Upadhyay. “Improved Differentially Private Continual Observation Using Group Algebra”. In: *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM. 2025.
- [HUU24] Monika Henzinger, Jalaj Upadhyay, and Sarvagya Upadhyay. “A unifying framework for differentially private sums under continual observation”. In: *Proceedings of Symposium on Discrete Algorithms (SODA)*. SIAM. 2024, pp. 995–1018.
- [HUU23] Monika Henzinger, Jalaj Upadhyay, and Sarvagya Upadhyay. “Almost tight error bounds on differentially private continual counting”. In: *Proceedings of the 2023 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM. 2023, pp. 5003–5039.
- [HAB⁺20] Naoise Holohan, Spiros Antonatos, Stefano Braghin, and Pól Mac Aonghusa. “The Bounded Laplace Mechanism in Differential Privacy”. In: *J. Priv. Confidentiality* 10.1 (2020).
- [Hon15] James Honaker. “Efficient use of differentially private binary trees”. In: *Theory and Practice of Differential Privacy (TPDP 2015), London, UK 2* (2015), pp. 26–27.

- [HGY⁺22] Ziyue Huang, Yuan Qiu, Ke Yi, and Graham Cormode. “Frequency estimation under multiparty differential privacy: one-shot and streaming”. In: *Proceedings of the VLDB Endowment* 15.10 (2022), pp. 2058–2070.
- [JKR⁺23] Palak Jain, Iden Kalemaj, Sofya Raskhodnikova, Satchit Sivakumar, and Adam D. Smith. “Counting Distinct Elements in the Turnstile Model with Differential Privacy under Continual Observation”. In: *Proc. 37th NeurIPS*. 2023.
- [JRS⁺23] Palak Jain, Sofya Raskhodnikova, Satchit Sivakumar, and Adam Smith. “The Price of Differential Privacy under Continual Observation”. In: *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research. PMLR, 2023, pp. 14654–14678.
- [JSW24] Palak Jain, Adam Smith, and Connor Wagaman. “Time-Aware Projections: Truly Node-Private Graph Statistics under Continual Observation”. In: *IEEE Symposium on Security and Privacy, SP 2024*. IEEE, 2024, pp. 127–145.
- [Jun03] Alexandre Junod. “Hankel determinants and orthogonal polynomials”. In: *Expositiones Mathematicae* 21.1 (2003), pp. 63–74.
- [KLS21] Peter Kairouz, Ziyu Liu, and Thomas Steinke. “The Distributed Discrete Gaussian Mechanism for Federated Learning with Secure Aggregation”. In: *Proceedings of the 38th International Conference on Machine Learning, ICML 2021*. Vol. 139. Proceedings of Machine Learning Research. PMLR, 2021, pp. 5201–5212.
- [KMS⁺21] Peter Kairouz, Brendan McMahan, Shuang Song, Om Thakkar, Abhradeep Thakurta, and Zheng Xu. “Practical and private (deep) learning without sampling or shuffling”. In: *International Conference on Machine Learning*. PMLR. 2021, pp. 5213–5225.
- [KOV15] Peter Kairouz, Sewoong Oh, and Pramod Viswanath. “The Composition Theorem for Differential Privacy”. In: *Proceedings of the 32nd International Conference on Machine Learning*. Vol. 37. Proceedings of Machine Learning Research. Lille, France: PMLR, 2015, pp. 1376–1385.
- [KL24] Nikita Kalinin and Christoph H. Lampert. “Banded Square Root Matrix Factorization for Differentially Private Model Training”. In: *Advances in Neural Information Processing Systems 38, NeurIPS 2024*. 2024.
- [KMU⁺25] Nikita P. Kalinin, Ryan McKenna, Jalaj Upadhyay, and Christoph H. Lampert. “Back to Square Roots: An Optimal Bound on the Matrix Factorization Error for Multi-Epoch Differentially Private SGD”. In: *CoRR* abs/2505.12128 (2025).

- [KMC⁺23] Anastasiia Koloskova, Ryan McKenna, Zachary Charles, John Rush, and H Brendan McMahan. “Gradient Descent with Linearly Correlated Noise: Theory and Applications to Differential Privacy”. In: *Advances in Neural Information Processing Systems* 36 (2023).
- [KHP16] Fragkiskos Koufogiannis, Shuo Han, and George J. Pappas. “Gradual Release of Sensitive Data under Differential Privacy”. In: *J. Priv. Confidentiality* 7.2 (2016).
- [Lan03] Sergei Lando. *Lectures on generating functions*. Vol. 23. American Mathematical Soc., 2003.
- [LSS08] Troy Lee, Adi Shraibman, and Robert Spalek. “A Direct Product Theorem for Discrepancy”. In: *Proceedings of the 23rd Annual IEEE Conference on Computational Complexity, CCC 2008*. IEEE Computer Society, 2008, pp. 71–80.
- [LHR⁺10] Chao Li, Michael Hay, Vibhor Rastogi, Gerome Miklau, and Andrew McGregor. “Optimizing linear counting queries under differential privacy”. In: *Proceedings of the twenty-ninth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. ACM, 2010, pp. 123–134.
- [LMH⁺15] Chao Li, Gerome Miklau, Michael Hay, Andrew McGregor, and Vibhor Rastogi. “The matrix mechanism: optimizing linear counting queries under differential privacy”. en. In: *The VLDB Journal* 24.6 (2015), pp. 757–781.
- [Liu19] Fang Liu. “Generalized Gaussian Mechanism for Differential Privacy”. In: *IEEE Trans. Knowl. Data Eng.* 31.4 (2019), pp. 747–756.
- [MNT20] Jiří Matoušek, Aleksandar Nikolov, and Kunal Talwar. “Factorization norms and hereditary discrepancy”. In: *International Mathematics Research Notices* 2020.3 (2020), pp. 751–780.
- [MMH⁺23] Ryan McKenna, Gerome Miklau, Michael Hay, and Ashwin Machanavajjhala. “Optimizing Error of High-Dimensional Statistical Queries Under Differential Privacy”. In: *J. Priv. Confidentiality* 13.1 (2023).
- [MT22] H. Brendan McMahan and Abhradeep Thakurta. *Federated Learning with Formal Differential Privacy Guarantees*. en. Accessed online 2023-05-15. 2022.
- [Mir12] Ilya Mironov. “On significance of the least significant bits for differential privacy”. In: *the ACM Conference on Computer and Communications Security, CCS’12*. ACM, 2012, pp. 650–661.

- [Mir17] Ilya Mironov. “Rényi Differential Privacy”. In: *30th IEEE Computer Security Foundations Symposium, CSF 2017*. IEEE Computer Society, 2017, pp. 263–275.
- [Pel04] Vladimir Peller. “Hankel operators and their applications”. In: *BULLETIN-AMS* 41.3 (2004), pp. 401–407.
- [PAK19] Victor Perrier, Hassan Jameel Asghar, and Dali Kaafar. “Private continual release of real-valued data streams”. In: *Network and Distributed Systems Security (NDSS) Symposium*. 2019.
- [PUC⁺25] Krishna Pillutla, Jalaj Upadhyay, Christopher A. Choquette-Choo, Krishnamurthy Dvijotham, Arun Ganesh, Monika Henzinger, Jonathan Katz, Ryan McKenna, H. Brendan McMahan, Keith Rush, Thomas Steinke, and Abhradeep Thakurta. “Correlated Noise Mechanisms for Differentially Private Learning”. In: *CoRR* abs/2506.08201 (2025).
- [QYL13] Wahbeh Qardaji, Weining Yang, and Ninghui Li. “Understanding hierarchical methods for differentially private histograms”. In: *Proc. VLDB Endow.* 6.14 (2013), pp. 1954–1965.
- [RS25] Sofya Raskhodnikova and Teresa Anna Steiner. “Fully Dynamic Algorithms for Graph Databases with Edge Differential Privacy”. In: *Proc. ACM Manag. Data* 3.2 (2025), 99:1–99:28.
- [RN10] Vibhor Rastogi and Suman Nath. “Differentially private aggregation of distributed time-series with transformation and encryption”. In: *Proceedings of SIGMOD International Conference on Management of data*. 2010, pp. 735–746.
- [STU17] Adam Smith, Abhradeep Thakurta, and Jalaj Upadhyay. “Is interaction necessary for distributed private learning?” In: *IEEE Symposium on Security and Privacy*. IEEE. 2017, pp. 58–77.
- [SMM19] David M. Sommer, Sebastian Meiser, and Esfandiar Mohammadi. “Privacy Loss Classes: The Central Limit Theorem in Differential Privacy”. In: *Proc. Priv. Enhancing Technol.* 2019.2 (2019), pp. 245–269.
- [SLM⁺18] Shuang Song, Susan Little, Sanjay Mehta, Staal Vinterbo, and Kamalika Chaudhuri. “Differentially Private Continual Release of Graph Statistics”. In: *arXiv preprint arXiv:1809.02575* (2018).
- [Ste22] Thomas Steinke. “Composition of Differential Privacy & Privacy Amplification by Subsampling”. In: *CoRR* abs/2210.00597 (2022).
- [SU16] Thomas Steinke and Jonathan R. Ullman. “Between Pure and Approximate Differential Privacy”. In: *J. Priv. Confidentiality* 7.2 (2016).

- [Str23] Gilbert Strang. *Introduction to Linear Algebra*. 6th ed. Wellesley, MA: Wellesley-Cambridge Press, 2023.
- [TKB⁺17] Jun Tang, Aleksandra Korolova, Xiaolong Bai, Xueqiang Wang, and Xiaofeng Wang. “Privacy loss in Apple’s implementation of differential privacy on MacOS 10.12”. In: *preprint arXiv:1709.02753* (2017).
- [TS13] Abhradeep Guha Thakurta and Adam D. Smith. “(Nearly) Optimal Algorithms for Private Online Learning in Full-information and Bandit Settings”. In: *Advances in Neural Information Processing Systems 26: NeurIPS 2013*. 2013, pp. 2733–2741.
- [THS24] Max Dupré la Tour, Monika Henzinger, and David Saulpic. “Making Old Things New: A Unified Algorithm for Differentially Private Clustering”. In: *Forty-first International Conference on Machine Learning, ICML 2024*. OpenReview.net, 2024.
- [Upa19] Jalaj Upadhyay. “Sublinear Space Private Algorithms Under the Sliding Window Model”. In: *International Conference on Machine Learning*. 2019, pp. 6363–6372.
- [UUA21] Jalaj Upadhyay, Sarvagya Upadhyay, and Raman Arora. “Differentially Private Analysis on Graph Streams”. In: *International Conference on Artificial Intelligence and Statistics*. PMLR. 2021, pp. 1171–1179.
- [Vad17] Salil P. Vadhan. “The Complexity of Differential Privacy”. In: *Tutorials on the Foundations of Cryptography*. Springer International Publishing, 2017, pp. 347–450.
- [Vin22] Staal Amund Vinterbo. “Differential privacy for symmetric log-concave mechanisms”. In: *International Conference on Artificial Intelligence and Statistics, AISTATS 2022*. Vol. 151. Proceedings of Machine Learning Research. PMLR, 2022, pp. 6270–6291.
- [Wal18] Hubert Stanley Wall. *Analytic theory of continued fractions*. Courier Dover Publications, 2018.
- [Wil05] Herbert Wilf. *generatingfunctionology*. CRC press, 2005.